Knowledge-Based Editor

Volume 1

Programmer's Apprentice Group

MIT AI Lab

January, 1985

Table of Contents: Knowledge Based Editor

Volume 1

OZ:<KBE.DOC>358.TEXT
OZ:<KBE.TEST>358.LISP

OZ:<KBE>KBE.INDEX

OZ:<KBE>LISPM.INIT
OZ:<KBE>SYSDEF.LISP

OZ:<KBE.SOURCE>BASIC.LISP
OZ:<KBE.SOURCE>NOISE.LISP
OZ:<KBE.SOURCE>SETS.LISP
OZ:<KBE.SOURCE>CONDITIONS.LISP
OZ:<KBE.SOURCE>ENGLISH.LISP
OZ:<KBE.SOURCE>SYSTEM-MAGIC.LISP
OZ:<KBE.SOURCE>FLAVOR-MAGIC.LISP
OZ:<KBE.SOURCE>MACRO.LISP
OZ:<KBE.SOURCE>KVARS.LISP
OZ:<KBE.SOURCE>KMAC.LISP
OZ:<KBE.SOURCE>INTERFACE-MACROS.LISP
OZ:<KBE.SOURCE>MBOX.LISP
OZ:<KBE.SOURCE>POST-OFFICE.LISP
OZ:<KBE.SOURCE>BEHAVIOR.LISP
OZ:<KBE.SOURCE>EVENTS.LISP
OZ:<KBE.SOURCE>ACTOR.LISP
OZ:<KBE.SOURCE>COMPLETE.LISP
OZ:<KBE.SOURCE>COMMENT-MACRO.LISP
OZ:<KBE.SOURCE>READMACRO.LISP
OZ:<KBE.SOURCE>OUTIL.LISP
OZ:<KBE.SOURCE>DTRACE.LISP
OZ:<KBE.SOURCE>DUTIL.LISP
OZ:<KBE.SOURCE>DMAC.LISP
OZ:<KBE.SOURCE>DVARS.LISP
OZ:<KBE.SOURCE>DB.LISP
OZ:<KBE.SOURCE>GC.LISP
OZ:<KBE.SOURCE>DDEBUG.LISP

Volume 2

OZ:<KBE.SOURCE>SURFACE.LISP
OZ:<KBE.SOURCE>EXPAND.LISP
OZ:<KBE.SOURCE>PRIMOP.LISP
OZ:<KBE.SOURCE>DFLOW.LISP
OZ:<KBE.SOURCE>UTIL.LISP
OZ:<KBE.SOURCE>SEGTRN.LISP
OZ:<KBE.SOURCE>ANALYZE.LISP
OZ:<KBE.SOURCE>CODER.LISP

Volume 3

OZ:<KBE.SOURCE>PSEUDO.LISP
OZ:<KBE.SOURCE>DICT.LISP
OZ:<KBE.SOURCE>GRAM.LISP
OZ:<KBE.SOURCE>REFER.LISP
OZ:<KBE.SOURCE>VERBS.LISP
OZ:<KBE.SOURCE>IMPLEMENT.LISP
OZ:<KBE.SOURCE>ADA-EVAL.LISP
OZ:<KBE.SOURCE>ADA-ANAL.LISP
OZ:<KBE.SOURCE>ADA-PARSE.LISP
OZ:<KBE.SOURCE>ADA-UNPARSE.LISP
OZ:<KBE.SOURCE>VIEW.LISP
OZ:<KBE.SOURCE>PLI.LISP
OZ:<KBE.SOURCE>DEFLIB.LISP
OZ:<KBE.SOURCE>SYSLIB.LISP
OZ:<KBE.SOURCE>KBELIB.LISP
OZ:<KBE.SOURCE>DEMO-LIB.LISP
OZ:<KBE.SOURCE>DEMO-ADA-LIB.LISP

Volume 4

OZ:<KBE.SOURCE>BRIDGE.LISP
OZ:<KBE.SOURCE>KAOS.LISP
OZ:<KBE.SOURCE>SOCIETY.LISP
OZ:<KBE.SOURCE>STANDARD-ACTOR.LISP
OZ:<KBE.SOURCE>REMOTE.LISP
OZ:<KBE.SOURCE>OVERLAPPING-REGIONS.LISP
OZ:<KBE.SOURCE>MOUSE-SENSITIVE-REGIONS.LISP
OZ:<KBE.SOURCE>OPTIONS.LISP
OZ:<KBE.SOURCE>SIMPLE.LISP
OZ:<KBE.SOURCE>SIMPLE-ACTOR.LISP
OZ:<KBE.SOURCE>MAGIC-ACTOR.LISP
OZ:<KBE.SOURCE>TV-MAGIC.LISP
OZ:<KBE.SOURCE>ZWEI-MAGIC.LISP
OZ:<KBE.SOURCE>POSSIBILITIES.LISP
OZ:<KBE.SOURCE>BELIEF.LISP
OZ:<KBE.SOURCE>EDIT.LISP
OZ:<KBE.SOURCE>EDIT-ACTOR.LISP
OZ:<KBE.SOURCE>CONSISTENCY.LISP
OZ:<KBE.SOURCE>EDIT-COMMANDS.LISP
OZ:<KBE.SOURCE>EDIT-MISC.LISP
OZ:<KBE.SOURCE>EDIT-SIMPLE.LISP
OZ:<KBE.SOURCE>EDIT-DEBUG.LISP
OZ:<KBE.SOURCE>KBE-MODE.LISP
OZ:<KBE.SOURCE>FOCUS.LISP
OZ:<KBE.SOURCE>TEST.LISP
OZ:<KBE.SOURCE>DOC.LISP
OZ:<KBE.SOURCE>PARSE.LISP
OZ:<KBE.SOURCE>EXPLAIN.LISP
OZ:<KBE.SOURCE>DOCUMENT.LISP
OZ:<KBE.SOURCE>PBM-HANDLERS.LISP
OZ:<KBE.SOURCE>INIT.LISP
OZ:<KBE.SOURCE>DEBUG.LISP

-*- Mode:Text; Fonts:CPTFONT; -*-

Documentation for KBE 358.0x, produced 12/14/84 13:43:37

The Knowledge-Based-Editor (KBE) is an extension to the
normal editing environment developed by the Programmer's
Apprentice group at MIT.

Interaction with the KBE may occur in different ways:

  Some commands are directly available as normal editor
  commands which can be invoked by a single keystroke.

  Some commands are extended text commands, which can be
  done either immediately (an execute request), or in
  background (a submit request).

Commands related to immediate action tend to be named
Super-something. Commands which relate to the background
KBE's background server process tend to be called
Hyper-something.

In particular,...

  Super-X requests immediate execution of a text request
  Hyper-X submits a request without waiting for it to be
     executed. The status of such requests can be checked
     with Hyper-S, and their results can later be
     picked up with Hyper-Resume.

Here is a full list of KBE-related Editor commands...

m-X Mark This Definition ...
Mark the definition at the cursor.

m-X Evaluate This Definition ...
Evaluate the definition at the cursor.

m-X Eval And Compile This Definition ...
Eval the definition at the cursor.
If the definition name is fbound when done, compile it as well.

Super-X and Super-Meta-X (KBE Execute Anything) ...
Enters a mini-buffer reading text for the KBE.
When mini-buffer is exited, executes the KBE commands it contains.

Hyper-X and Hyper-Meta-X (KBE Submit Anything) ...
Like Super-X, but to submit rather than to execute.

Super-C (KBE Execute Make Cliche) ...
Like Super-X, but the mini-buffer initially contains:
  Define a cliche

Hyper-C (KBE Submit Make Cliche) ...
Like Super-C, but to submit rather than to execute.

Super-D (KBE Execute Make Defun) ...
Like Super-X, but the mini-buffer initially contains:
  Define a program

Hyper-D (KBE Submit Make Defun) ...
Like Super-D, but to submit rather than to execute.

Super-P (KBE Execute Append Parameter) ...
Like Super-X, but the mini-buffer initially contains:
  Append a parameter

Hyper-P (KBE Submit Append Parameter) ...
Like Super-P, but to submit rather than to execute.

Super-W (KBE Execute What Needs To Be Done) ...
Immediately executes KBE command: What needs to be done?
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-W (KBE Submit What Needs To Be Done) ...
Like Super-W, but to submit rather than to execute.

Super-End (KBE Execute Finish) ...
Like Super-X, but the mini-buffer initially contains:
  Finish editing {this definition}

Hyper-End (KBE Submit Finish) ...
Like Super-End, but to submit rather than to execute.

c-m-X KBE Execute Reap Database ...
Immediately executes KBE command: Reap database.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-Rubout (KBE Submit Reap Database) ...
Like c-m-X KBE Execute Reap Database, but to submit rather than to execute.

Super-Clear-Input (KBE Execute Reinitialize Database) ...
Immediately executes KBE command: Reinitialize database.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-Clear-Input (KBE Submit Reinitialize Database) ...
Like Super-Clear-Input, but to submit rather than to execute.

Super-Page (KBE Execute Clear Display) ...
Immediately executes KBE command: Clear display.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-Page (KBE Submit Clear Display) ...
Like Super-Page, but to submit rather than to execute.

Super-I (KBE Execute Implement) ...
Like Super-X, but the mini-buffer initially contains:
  Implement {next unfilled role} as

Hyper-I (KBE Submit Implement) ...
Like Super-I, but to submit rather than to execute.

Super-. and Super-Meta-. (KBE Execute Move To) ...
Like Super-X, but the mini-buffer initially contains:
  Move to

Hyper-. and Hyper-Meta-. (KBE Submit Move To) ...
Like Super-., but to submit rather than to execute.

Super-; (KBE Execute Comment) ...
Immediately executes KBE command: Comment {this definition}.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-; (KBE Submit Comment) ...
Like Super-;, but to submit rather than to execute.

Super-= and Super-Control-= (KBE Execute Describe) ...
Immediately executes KBE command: Describe this.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-= and Hyper-Control-= (KBE Submit Describe) ...
Like Super-=, but to submit rather than to execute.

Super-E (KBE Execute Explain) ...
Immediately executes KBE command: Explain this.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-E (KBE Submit Explain) ...

Like Super-E, but to submit rather than to execute.

Super-U (KBE Execute Undo) ...
Immediately executes KBE command: Undo.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-U (KBE Submit Undo) ...
Like Super-U, but to submit rather than to execute.

Super-L (KBE Execute Use Language) ...
Like Super-X, but the mini-buffer initially contains:
  Use language

Hyper-L (KBE Submit Use Language) ...
Like Super-L, but to submit rather than to execute.

Super-S (KBE Execute Share) ...
Like Super-X, but the mini-buffer initially contains:
  Share

c-m-X KBE Submit Share ...
Like Super-S, but to submit rather than to execute.

Super-R (KBE Execute Remove) ...
Like Super-X, but the mini-buffer initially contains:
  Remove {next unfilled role}

Hyper-R (KBE Submit Remove) ...
Like Super-R, but to submit rather than to execute.

Super-T (KBE Execute Test Yourself) ...
Immediately executes KBE command: Test yourself.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-T (KBE Submit Test Yourself) ...
Like Super-T, but to submit rather than to execute.

Super-V (KBE Execute View) ...
Immediately executes KBE command: View {this definition} in {another language}.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-V (KBE Submit View) ...
Like Super-V, but to submit rather than to execute.

Super-A (KBE Execute Add) ...
Like Super-X, but the mini-buffer initially contains:
  Add

Hyper-A (KBE Submit Add) ...
Like Super-A, but to submit rather than to execute.

Super-Z (KBE Execute Analyze) ...
Immediately executes KBE command: Analyze ${text of current definition}.
With an explicit arg, allows confirmation/editing in mini-buffer.

Hyper-Z (KBE Submit Analyze) ...
Like Super-Z, but to submit rather than to execute.

Hyper-Resume (Resume KBE Edits) ...
Complete any requests which were done in background.

Hyper-S (KBE Status) ...
Inquires about the status of the KBE requests.

m-X Edit KBE Options ...
Edit a table of KBE-related control variables.
With an argument, edit KBE internal variables as well.

m-X Bug KBE ...

Enter a buffer which is set up to send a KBE bug report.

Super-Control-Y and Hyper-Control-Y (Yank KBE Bug Message) ...
Inserts the text of the (nth) most-recent KBE bug message

Super-F and Super-Control-F (Forward Marked KBE Role) ...
Goes forward one KBE role. Searches forward for a KBE role marker.
With an argument, N, goes forward N roles.
The argument may be negative (to search backward).

Super-B and Super-Control-B (Backward Marked KBE Role) ...
Goes back one KBE role. Searches backward for a KBE role marker.
With an argument, N, goes backward N roles.
The argument may be negative (to search forward).

Super-Rubout (Trim KBE Reference) ...
Delete the first "the ... of" in most recent definite reference.
eg, changes "the x of the y" to just "the y".

m-X Kill KBE Requests ...
Kills text in buffer (aborts submitting it).

m-X Submit And Kill KBE Requests ...
Submits text in buffer.

m-X Yank Pop KBE Request ...
Inserts the text of the (nth) most-recent KBE request, killing
previously yanked text if any.

Hyper-Super-Control-Meta-A (Automatic Ada Mode) ...
Enters Ada Mode and updates the Attribute List

Hyper-Super-Control-Meta-L (Automatic Lisp Mode) ...
Enters Lisp Mode and updates the Attribute List

Hyper-Super-Control-Meta-N (Name This Expression) ...
Name the s-expression at the cursor.

Hyper-Super-Control-Meta-Y (Yank Last Several KBE Requests) ...
Yanks the text for KBE requests since this command was last used.
Text of recent requests are yanked onto separate lines.
If this is not done on a blank line, anything before the cursor is
repeated before every request yanked. Anything after the cursor is
repeated after every request yanked.

Hyper-Super-Control-Meta-C (Change This Expression) ...
Change the style of the enclosing form.

Hyper-Super-Control-Meta-D (Toggle KBE Debug Mode) ...
Enable debug mode, or with an argument, disable it.

Hyper-Super-Control-Meta-Abort (Reset Apprentice) ...
Resets the KBE process. Works in Debug Mode only.

Hyper-Super-Control-Meta-System (Select Apprentice) ...
Selects the KBE process. Works in Debug Mode only.

Hyper-Super-Control-Meta-Resume (Make Buffer Consistent For KBE) ...
Tries to recover from some kinds of KBE interface lossage.
Makes definitions in buffer consistent with definitions known
to the interface. With an arg, queries for each definition.

Hyper-Super-Control-Meta-; (Compile Cliche Comments) ...
Compile cliche comment for current form.

Super-Altmode and Hyper-Altmode (Complete KBE Word) ...
Complete current word

Super-Space and Hyper-Space (Auto Complete KBE Word) ...

Complete and self-insert

Super-Help and Hyper-Help (Document KBE Completions) ...
Document what the completion would do at this buffer point.

Super-Control-Help and Hyper-Control-Help (Document KBE) ...
Gives help information about KBE commands.
With an arg, shows KBE dictionary information instead.

KBE Dictionary (Grouped)

Adverb

HERE, adverb
 In "What needs to be done here" means like "in the top".

Article

A, article
 Designates an indefinite reference.
AN, article
 Synonym for "a".
A<N>, article
 Synonym for "a" and "an".
THE, article
 Designates a definite reference.

Descriptor

IMPLEMENTED, descriptor
 Declares a definite reference to already exist.
UNIMPLEMENTED, descriptor
 Declares a definite reference to not already exist.
ROLE, descriptor
 Denotes the named roles of a program fragment.

Ordinal

FIRST, ordinal
 Selects the first of several of things.
SECOND, ordinal
 Selects the second of several of things.
THIRD, ordinal
 Selects the third of several of things.
FOURTH, ordinal
 Selects the fourth of several of things.
FIFTH, ordinal
 Selects the fifth of several of things.
SIXTH, ordinal
 Selects the sixth of several of things.
SEVENTH, ordinal
 Selects the seventh of several of things.
LAST, ordinal
 Selects the last of several of things.

Port Descriptor

PARAMETER, port-descriptor
 Used to designate a parameter in a port reference.
ARGUMENT, port-descriptor
 Used to designate an argument in a port reference.
FREE-VARIABLE, port-descriptor
 Used to designate a free-variable in a port reference.
VALUE, port-descriptor
 Used to designate a value in a port reference.
SIDE-EFFECT, port-descriptor
 Used to designate a side-effect in a port reference.
INPUT, port-descriptor
 Used to designate an input in a port reference.
OUTPUT, port-descriptor
 Used to designate an output in a port reference.
TEMPORAL, port-descriptor
 Used to designate a temporal in a port reference.
NON-TEMPORAL, port-descriptor
 Used to designate a non-temporal in a port reference.
PORT, port-descriptor
 Used to designate a port in a port reference.

Preposition

IN, preposition
 Used as a relational or guide word.
OF, preposition
 Used as a relational or guide word.

Pronoun

IT, pronoun
 Used as a context-dependent definite reference.
THIS, pronoun
 Used as a context-dependent definite reference.
THAT, pronoun
 Used as a context-dependent definite reference.

Verb

```
TYPEOUT, verb
 Syntax: "Typeout <text>."
 Requests that the given <text> be typed out.
REMARK, verb
 Syntax: "Remark <text>."
 Requests that the given <text> be remarked about.
CLEAR, verb
 Syntax: "Clear display."
 Requests that the display area be cleared.
HOW, verb
 Syntax: "How is <cliche-or-program> defined."
 Asks what how <cliche-or-program> is defined.
ENABLE, verb
 Syntax: "Enable <switch> mode."
 Enables the KBE feature named <switch>.
DISABLE, verb
 Syntax: "Disable <switch> mode."
 Disables the KBE feature named <switch>.
LOAD, verb
 Syntax: "Load <thing-to-load>."
 Loads the <thing-to-load>. Normally, this is a filename. If
 it is the word "patches" then KBE patches will be loaded
 instead.
COMPILE, verb
 Syntax: "Compile <thing-to-compile>."
 Compiles the <thing-to-compile>, which should be a filename.
LOCATE, verb
 Syntax: "Locate <position>."
 Verbally marks a shift of presentational focus to
 <position>. Does not shift the presentational focus. The
 <position> is an arbitrary plist containing data about the
 position.
ANALYZE, verb
 Syntax: "Analyze <reference>."
 Makes database consistent with the given <reference>.
ADD, verb
 Syntax: "Add <reference> <where>."
 Adds <reference> at the cursor position. The word "here" may
 optionally follow for euphony.
COMPUTE, verb
 Syntax: "Compute <reference>."
 Computes referent of the given <reference>.
EVAL, verb
 Syntax: "Eval <code>."
 Calls EVAL on <code>.
EXPLAIN, verb
 Syntax: "Explain <reference>."
 Explains the segment designated by <reference>.
DOCUMENT, verb
 Syntax: "Document <reference>."
 Documents the segment designated by <reference>.
COMMENT, verb
 Syntax: "Comment <reference>."
 Comments the program designated by <reference>.
MOVE, verb
 Syntax: "Move to <reference>."
 Moves to the segment designated by <reference>.
DISPLAY, verb
 Syntax: "Display <reference>."
 Displays the segment designated by <reference>.
REINITIALIZE, verb
 Syntax: "Reinitialize database."
 Reinitializes the KBE database (for debugging).
USE, verb
 Syntax: "Use language <language>."
```

```
 Sets the language of reference to <language>.
VIEW, verb
 Syntax: "View <toplevel-reference> in <language>."
 Requests the code named by <toplevel-reference> be displayed
 in the given <language>.
DESCRIBE, verb
 Syntax: "Describe <reference>."
 Describes the segment designated by <reference>.
WHAT, verb
 Syntax: "What needs to be done <where>."
 Requests a list of things that need to be done. An optional
 context may be supplied as in "What needs to be done in the
 program FOO."
REAP, verb
 Syntax: "Reap database."
 Causes the database to be reaped and GC'd.
REMOVE, verb
 Syntax: "Remove <definite-reference>."
 Remove the thing referred to by <definite-reference>.
UNDO, verb
 Syntax: "Undo."
 Undoes the last request.
DEFINE, verb
 Syntax: "Define a<n> <optional-cliche-name> <spec> <optional-roles-or-parameters>."
 Defines a given program/cliche. Parameters/roles may
 optionally also be specified as in "... with a
 parameter/role X." or "... with parameters/roles X and Y."
APPEND, verb
 Syntax: "Append a parameter <var> <where>."
 Appends the specified parameter to the parameter list of the
 current program (or to some other program, whose name may
 be introduced by "to").
SHARE, verb
 Syntax: "Share <definite-reference-1> and <definite-reference-2>."
 Shares <definite-reference-1> and <definite-reference-2>.
 eg, "Share the x and the y."
FINISH, verb
 Syntax: "Finish editing <definition-name>."
 Cleans up after editing <definition-name>.
COPY, verb
 Syntax: "Copy to <seg-reference> <reference>."
 Copies the cliche <reference> to the cliche <seg-reference>
IMPLEMENT, verb
 Syntax: "Implement <seg-reference> <as-or-like> <reference>."
 Implements the segment given by <seg-reference> <as/like>
 the code fragment given by <reference>.
REIMPLEMENT, verb
 Syntax: "Reimplement <seg-reference> <as-or-like> <reference>."
 Re-implements the segment given by <seg-reference> as the
 code fragment given by <reference>.
REPORT, verb
 Syntax: "Report status."
 Requests a status report.
TEST, verb
 Syntax: "Test yourself."
 Requests that the apprentice test itself.
TELL, verb
 Syntax: "Tell <actor> <request>."
 Asks an actor to tell another actor to do something.
ASK, verb
 Syntax: "Ask <actor> <request>."
 Asks an actor to ask another actor to do something.
```

Cliche

PRINT_REPAIR, cliche
Role: repair.
No available documentation.

PRINT_REPAIR_HEADINGS, cliche.
No available documentation.

PRINT_SUMMARY, cliche
Roles: summary, summarized_file, and summarized_file_name.
No available documentation.

PRINT_SUMMARY_HEADINGS, cliche.
No available documentation.

FILE_SUMMARIZATION, cliche
Roles: source_file, source_file_name, and summaries.
No available documentation.

FILE_SELECTION, cliche
Roles: source_file, selection_test, record_key_accessor, selection_file, and enumerator.
No available documentation.

SIMPLE_REPORT, cliche
Roles: enumerator, print_item, epilog, file_name, title, line_limit, and column_headings.
No available documentation.

ALL_CHAINS_ENUMERATION, cliche
Roles: main_file, chain_file, chain_file_name, main_file_name, main_file_chain_field, chain_record, and chain_
No available documentation.

CHAIN_ENUMERATION, cliche
Roles: main_file, chain_file, main_file_key, chain_file_name, main_file_name, main_file_chain_field, chain_rec
No available documentation.

FILE_ENUMERATION, cliche
Roles: file, file_name, and data_record.
No available documentation.

QUERY_USER_FOR_KEY, cliche
Roles: file and file_name.
No available documentation.

READ, cliche
Roles: file, key, file_name, and data_record.
No available documentation.

KEYED_FILE_DEFINITION, cliche
Role: file_name.
No available documentation.

CHAIN_FILE_DEFINITION, cliche
Role: file_name.
No available documentation.

PRE_LOOP_COMPUTATION, cliche
Roles: item and datum.
No available documentation.

LEADING-BOUNDARY-DETECTOR, cliche
Roles: item, boundary-test, and boundary-indicator.
No available documentation.

COUNT, cliche
Role: item.
No available documentation.

TABULARIZED-PRINT-OUT, cliche
Roles: format-string, item, and maximum-charpos.
No available documentation.

PRINT-OUT, cliche
Roles: format-string, item, and size-in-lines.
No available documentation.

SIMPLE-REPORT, cliche
Roles: enumerator, print-item, epilog, file-name, title, line-limit, and column-headings.
No available documentation.

PRODUCT, cliche
Role: number.
No available documentation.

SUM, cliche
Role: number.
No available documentation.

LIST-GENERATION, cliche
Roles: list and element.
No available documentation.

VECTOR-ENUMERATION, cliche
Roles: vector and element.
No available documentation.

SUBLIST-ENUMERATION, cliche
Roles: list and sublist.
No available documentation.

LIST-ENUMERATION, cliche
Roles: list and element.
Enumerates the successive elements of list.

SELECT, cliche
Roles: item, selection-test, and selected-item.
No available documentation.

ACCUMULATION, cliche
Roles: item, zero-for-operator, and operator.
No available documentation.

GENERATION, cliche
Roles: structure, element-accessor, element, and step.
No available documentation.

ENUMERATION, cliche
Roles: structure, empty-test, element-accessor, element, and step.
No available documentation.

SPLICE-IN, cliche
Roles: position and new-element.
Splices new element into a list. A new cell is created with head new element and tail the same as the tail of previous sublist; previous sublist has its tail modified to be new element.

TRAILING-POINTER-SUBLIST-ENUMERATION, cliche
Roles: list and sublist.
Enumerates the sublsits in the tail of the list returned by initial value computation maintaining a backpointer to successive values of sublist under examination.

SPLICE-OUT, cliche
Role: position.
Splices an element out of a list; previous sublist has its tail modified to be its tail's tail.

TRAILING-POINTER-LIST-ENUMERATION, cliche
Roles: list and element.
Enumerates the elements in the tail of the list returned by initial value computation maintaining a backpointer to successive values of current.

LINEAR-SEARCH, cliche
Roles: enumerator, search-test, and action.
Searches enumerator for an element which satisfies search-test. If found, returns action. Otherwise, returns NIL.

EQUALITY-WITHIN-EPSILON, cliche
Roles: first-number, second-number, and epsilon.
Succeeds when value being tested and value being tested are within epsilon of each other.

SQUARE, cliche
Role: number.
No available documentation.

AVERAGE, cliche
Roles: first-number and second-number.
Averages thing to be averaged and thing to be averaged by summing them and dividing by two.

SUCCESSIVE-APPROXIMATION, cliche

Roles: initial-approximation, criterion, and improvement-step.
Successive approximation. Beginning with an initial
approximation of initial-approximation, tries to find a
value for which predicate expression succeeds. If that test
fails, a new approximation is generated from the old by
improvement-step. This process is repeated until the test
succeeds.

USE, cliche
Role: thing.
No available documentation.

MAP, cliche
Roles: item, function, and result.
No available documentation.

SIMPLE-SUBSEG, cliche
Role: subseg.
Used to nest one seg inside another.

BINARY-FUNCTION, cliche
Roles: x and y.
Dummy one argument function cliche

UNARY-FUNCTION, cliche
Role: x.
Dummy one argument function cliche

ZEROARY-FUNCTION, cliche.
Dummy zero argument function cliche

Send bugs, queries, etc. to BUG-KBE@MIT-OZ.

```
;;; -*- Mode: LISP; Package: KBE; Base: 10; -*-
;;;
;;; Knowledge Based Editor.   Automatic self-test results.
;;;
;;; For KBE, Friday the fourteenth of December, 1984; 2:25:23 pm.
;;;
;;; In >World1.load on PA Lisp Machine Merlin
;;; using KBE 358.0x in Release 4.5.
;;;
;;; Special options:
;;;  * KBE debug mode (local): No.
;;;  * Garbage collector enabled: No.

;;;; Summary
;;;  The BUCKET sequence failed:
;;;   Bad code generated.
;;;  The DELETE sequence failed:
;;;   Bad code generated.
;;;  The HASH sequence was successful.
;;;  The MEMQ sequence failed:
;;;   Bad code generated.
;;;  The SQRT sequence was successful.
;;;  The SUMLIST sequence was successful.
```

```
;;;; The BUCKET Sequence

;;; >Reinitialize database.
;;; >Define a<n> program BUCKET with parameters OBJ and L.
;;; >Implement the program as a linear-search of a sublist-enumeration of [L] .
;;; >Implement the search-test as [(EQ (CAR LIST) OBJ)] .
;;; >Implement the action as [LIST] .
;;; >Reimplement the search-test as an equality-within-epsilon.
;;; >Implement the first-number as [(CAR LIST)] .
;;; >Implement the second-number as [OBJ] .
;;; >Append a parameter EPS.
;;; >Implement the epsilon as [EPS] .

;;; Test failed!
;;; Bad code generated.

;;; (BUCKET OBJ L EPS)
;;; The program BUCKET is a linear-search.
;;;   The enumerator is a sublist-enumeration.
;;;      It enumerates the sublists of L.
;;;   The search-test is an equality-within-epsilon.
;;;      It determines whether (CAR LIST) and OBJ differ by less than EPS.
;;;   The action is LIST.

 ;; Faulty code produced for BUCKET...

(DEFUN BUCKET (OBJ L EPS)
  (LET* ((LIST L))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (IF (< (ABS (- (CAR LIST) OBJ)) EPS) (RETURN LIST))
      #{LIST, the output sublist}
      (SETQ LIST (CDR LIST)))))

 ;; Correct code would have been...

(DEFUN BUCKET (OBJ L EPS)
  (LET* ((LIST L))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (IF (< (ABS (- (CAR #{LIST, the output item}) OBJ)) EPS) (RETURN LIST))
      (SETQ LIST (CDR LIST)))))
```

```
;;;; The DELETE Sequence

;;; >Reinitialize database.
;;; >Define a<n> program MYDELETE with a parameter SYMBOL.
;;; >Implement it as a linear-search.
;;; >Implement the enumerator as a trailing-pointer-list-enumeration of [(MYHASH SYMBOL)] .
;;; >Implement the search-test as [(EQ {a use of the element} SYMBOL)] .
;;; >Implement the action as a splice-out of [PREVIOUS-SUBLIST] .
;;; >Define a<n> program MYOTHERDELETE with a parameter SYMBOL.
;;; >Implement it as a linear-search.
;;; >Implement the enumerator as a trailing-pointer-sublist-enumeration of [(MYHASH SYMBOL)] .
;;; >Implement the search-test as [(EQ (CAR LIST) SYMBOL)] .
;;; >Implement the action as a splice-out of [PREVIOUS-SUBLIST] .

;;; Test failed!
;;; Bad code generated.

;;; (MYDELETE SYMBOL)
;;; The program MYDELETE is a linear-search.
;;;   The enumerator is a trailing-pointer-list-enumeration.
;;;      It enumerates the elements of the tail of (MYHASH SYMBOL)
;;;      maintaining a trailing pointer.
;;;   The search-test is (EQ #{(CAR LIST), the output element} SYMBOL).
;;;   The action is a splice-out.
;;;      It splices an element out of a list after PREVIOUS-SUBLIST.

(DEFUN MYDELETE (SYMBOL)
  (LET* ((PREVIOUS-SUBLIST (MYHASH SYMBOL))
         (LIST (CDR PREVIOUS-SUBLIST)))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (WHEN (EQ #{(CAR LIST), the output element} SYMBOL)
        (RPLACD PREVIOUS-SUBLIST (CDDR PREVIOUS-SUBLIST))
        (RETURN NIL))
      (SETQ PREVIOUS-SUBLIST LIST)
      (SETQ LIST (CDR LIST)))))

;;; (MYOTHERDELETE SYMBOL)
;;; The program MYOTHERDELETE is a linear-search.
;;;   The enumerator is a trailing-pointer-sublist-enumeration.
;;;      It enumerates the sublists of the tail of (MYHASH SYMBOL)
;;;      maintaining a trailing pointer.
;;;   The search-test is (EQ (CAR LIST) SYMBOL).
;;;   The action is a splice-out.
;;;      It splices an element out of a list after PREVIOUS-SUBLIST.

 ;; Faulty code produced for MYOTHERDELETE...

(DEFUN MYOTHERDELETE (SYMBOL)
  (LET* ((PREVIOUS-SUBLIST (MYHASH SYMBOL))
         (LIST (CDR PREVIOUS-SUBLIST)))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (WHEN (EQ (CAR LIST) SYMBOL)
        (RPLACD PREVIOUS-SUBLIST (CDDR PREVIOUS-SUBLIST))
        (RETURN NIL))
      #{LIST, the output sublist}
      (SETQ PREVIOUS-SUBLIST LIST)
      (SETQ LIST (CDR LIST)))))

 ;; Correct code would have been...

(DEFUN MYOTHERDELETE (SYMBOL)
  (LET* ((PREVIOUS-SUBLIST (MYHASH SYMBOL))
         (LIST (CDR PREVIOUS-SUBLIST)))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (WHEN (EQ (CAR #{LIST, the output sublist}) SYMBOL)
```

```
      (RPLACD PREVIOUS-SUBLIST (CDDR PREVIOUS-SUBLIST))
      (RETURN NIL))
    (SETQ PREVIOUS-SUBLIST LIST)
    (SETQ LIST (CDR LIST)))))
```

```
;;;; The HASH Sequence

;;; >Reinitialize database.
;;; >Define a<n> program MYHASH with a parameter SYMBOL.
;;; >Implement it as [(ARRAYFETCH TBL (ABS (REMAINDER (MAKNUM SYMBOL) TBLSIZE)))] .

;;; No coding errors in this sequence.

;;; (MYHASH SYMBOL)
;;;  Note: No explanation generated by KBE.

(DEFUN MYHASH (SYMBOL) (ARRAYFETCH TBL (ABS (REMAINDER (MAKNUM SYMBOL) TBLSIZE))))
```

```
;;;; The MEMQ Sequence

;;; >Reinitialize database.
;;; >Define a<n> program MYMEMQ with parameters OBJ and SEQUENCE.
;;; >Implement it as a linear-search.
;;; >Implement the enumerator as a list-enumeration of [SEQUENCE] .
;;; >Implement the search-test as an EQ of the element and [OBJ] .
;;; >Implement the action as [LIST] .
;;; >Define a<n> program MYOTHERMEMQ with parameters OBJ and SEQUENCE.
;;; >Implement it as a linear-search.
;;; >Implement the enumerator as a sublist-enumeration of [SEQUENCE] .
;;; >Implement the search-test as [(EQ (CAR LIST) OBJ)] .
;;; >Implement the action as [LIST] .

;;; Test failed!
;;; Bad code generated.

;;; (MYMEMQ OBJ SEQUENCE)
;;; The program MYMEMQ is a linear-search.
;;;   The enumerator is a list-enumeration.
;;;      It enumerates the elements of SEQUENCE.
;;;   The search-test is a binary-function.  It nil.
;;;   The action is LIST.

(DEFUN MYMEMQ (OBJ SEQUENCE)
  (LET* ((LIST SEQUENCE))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (IF (EQ #{(CAR LIST), the output element} OBJ) (RETURN LIST))
      (SETQ LIST (CDR LIST)))))

;;; (MYOTHERMEMQ OBJ SEQUENCE)
;;; The program MYOTHERMEMQ is a linear-search.
;;;   The enumerator is a sublist-enumeration.
;;;      It enumerates the sublists of SEQUENCE.
;;;   The search-test is (EQ (CAR LIST) OBJ).
;;;   The action is LIST.

 ;; Faulty code produced for MYOTHERMEMQ...

(DEFUN MYOTHERMEMQ (OBJ SEQUENCE)
  (LET* ((LIST SEQUENCE))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (IF (EQ (CAR LIST) OBJ) (RETURN LIST))
      #{LIST, the output sublist}
      (SETQ LIST (CDR LIST)))))

 ;; Correct code would have been...

(DEFUN MYOTHERMEMQ (OBJ SEQUENCE)
  (LET* ((LIST SEQUENCE))
    (LOOP DO
      (IF (NULL LIST) (RETURN NIL))
      (IF (EQ (CAR #{LIST, the output sublist}) OBJ) (RETURN LIST))
      (SETQ LIST (CDR LIST)))))
```

```
;;;; The SQRT Sequence

;;; >Reinitialize database.
;;; >Define a<n> program MYSQRT with a parameter NUM.
;;; >Implement it as a successive-approximation.
;;; >Implement the initial-approximation as [1.] .
;;; >Implement the improvement-step as an average of [APPROXIMATION] and [(// NUM APPROXIMATION)] .
;;; >Implement the criterion as an equality-within-epsilon of [(* APPROXIMATION APPROXIMATION)] and [NUM] .
;;; >Implement the epsilon as [0.0001] .
;;; >Define a<n> program MYOTHERSQRT with a parameter NUM.
;;; >Implement it as a successive-approximation.
;;; >Implement the initial-approximation as [1.] .
;;; >Implement the improvement-step as an average of [APPROXIMATION] and [(// NUM APPROXIMATION)] .
;;; >Implement the criterion as an equality-within-epsilon of [(* APPROXIMATION APPROXIMATION)] and [NUM] .
;;; >Append a parameter CRITERION.
;;; >Implement the epsilon as [CRITERION] .

;;; No coding errors in this sequence.

;;; (MYSQRT NUM)
;;; The program MYSQRT is a successive-approximation.
;;;   The initial-approximation is 1.
;;;   The criterion is an equality-within-epsilon.
;;;      It determines whether a first-number and NUM differ by less than
;;;      0.0001.
;;;   The improvement-step is an average.
;;;      It computes the average of APPROXIMATION and a second-number.

(DEFUN MYSQRT (NUM)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) 0.0001) (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))

;;; (MYOTHERSQRT NUM CRITERION)
;;; The program MYOTHERSQRT is a successive-approximation.
;;;   The initial-approximation is 1.
;;;   The criterion is an equality-within-epsilon.
;;;      It determines whether a first-number and NUM differ by less than
;;;      CRITERION.
;;;   The improvement-step is an average.
;;;      It computes the average of APPROXIMATION and a second-number.

(DEFUN MYOTHERSQRT (NUM CRITERION)
  (LET* ((APPROXIMATION 1))
    (LOOP DO
      (IF (< (ABS (- (* APPROXIMATION APPROXIMATION) NUM)) CRITERION) (RETURN APPROXIMATION))
      (SETQ APPROXIMATION (// (+ APPROXIMATION (// NUM APPROXIMATION)) 2.0)))))
```

```
;;;; The SUMLIST Sequence

;;; >Reinitialize database.
;;; >Define a<n> program SUMLIST with a parameter LIST.
;;; >Implement it as a sum of a list-enumeration of [LIST] .

;;; No coding errors in this sequence.

;;; (SUMLIST LIST)
;;; The program SUMLIST is a sum.
;;;   The number is a list-enumeration.
;;;      It enumerates the elements of LIST.

(DEFUN SUMLIST (LIST)
  (LET* ((SUM 0))
    (LOOP DO
      (IF (NULL LIST) (RETURN SUM))
      (SETQ SUM (+ SUM #{(CAR LIST), the output element}))
      (SETQ LIST (CDR LIST)))))
```

- #$	- DEMO-ADA-LIB
#$	DEMO-ADA-LIB
#$	DEMO-ADA-LIB
- #$	- DEMO-ADA-LIB
#$	DEMO-ADA-LIB
#$	DEMO-ADA-LIB
- #$	- DEMO-ADA-LIB
#$	DEMO-ADA-LIB
#$	DEMO-ADA-LIB
- #$	- DEMO-ADA-LIB
#$	DEMO-ADA-LIB
#$	DEMO-ADA-LIB
- #$	- DEMO-ADA-LIB
#$	DEMO-ADA-LIB
#$	DEMO-ADA-LIB
- &*	- DMAC
&field0	DMAC
&field1	DMAC
- &field2	- DMAC
&field3	DMAC
&fieldk	DMAC
- &~case	- DMAC
&~seg~deep	DMAC
&~seg~roleseg	DMAC
- &~seg~segcase	- DMAC
&~seg~subseg	DMAC
&~seg	DMAC
- &	- DMAC
(&)	ADA-EVAL
(*)	ADA-EVAL
- (**)	- ADA-EVAL
(+)	ADA-EVAL
(-)	ADA-EVAL
- (//)	- ADA-EVAL
(//=)	ADA-EVAL
(//=)	ADA-EVAL
- (//=)	- ADA-EVAL
(//=)	ADA-EVAL
(/[macro)	READMACRO
- (6noop 6proto-seg)	- PRIMOP
(:property and k*expander)	EXPAND
(:property cond k*expander)	EXPAND
- (:property defsubst k*expander)	- EXPAND
(:property defun k*expander)	EXPAND
(:property go k*expander)	EXPAND
- (:property k*slot 6fexpr)	- PRIMOP
(:property kbe-/#-comment-macro k*expander)	EXPAND
(:property kbe-prog-named k*expander)	EXPAND
- (:property loop k*expander)	- EXPAND
(:property or k*expander)	EXPAND
(:property prog1 k*expander)	EXPAND
- (:property progn k*expander)	- EXPAND
(:property return k*expander)	EXPAND
(:property return-from k*expander)	EXPAND
- (:property setq k*expander)	- EXPAND
(:property values k*expander)	EXPAND
(<)	ADA-EVAL
- (<)	- ADA-EVAL
(<)	ADA-EVAL
(<=)	ADA-EVAL
- (<=)	- ADA-EVAL
(<=)	ADA-EVAL
(=)	ADA-EVAL
- (=)	- ADA-EVAL
(=)	ADA-EVAL
(=)	ADA-EVAL
- (>)	- ADA-EVAL
(>)	ADA-EVAL
(>)	ADA-EVAL

- (>= calendar)
  (>=)
  (>=)
- (>=)
  (ABS)
  (actor-background-process-mixin :after :init)
- (actor-background-process-mixin :process)
  (actor-background-process-mixin :process-state)
  (actor-background-process-mixin :reset)
- (actor-background-process-mixin :select)
  (actor-background-process-mixin :whostate)
  (actor-default-behavior-mixin :event)
- (actor-default-behavior-mixin :otherwise :behavior)
  (actor-io-mixin :after :init)
  (actor-io-mixin :prepare-remark-window)
- (actor-io-mixin :prepare-typeout-window)
  (actor-io-mixin :remark)
  (actor-io-mixin :remark)
- (actor-io-mixin :typeout)
  (actor-io-mixin :typeout)
  (actor-io-mixin :with-io-lock)
- (actor-io-mixin remark)
  (actor-io-mixin remark)
  (actor-io-mixin typeout)
- (actor-io-mixin typeout)
  (actor-message-passing-mixin :after :init)
  (actor-message-passing-mixin :ask)
- (actor-message-passing-mixin :ask-internal)
  (actor-message-passing-mixin :current-client)
  (actor-message-passing-mixin :current-message)
- (actor-message-passing-mixin :delegate)
  (actor-message-passing-mixin :delegate-internal)
  (actor-message-passing-mixin :event)
- (actor-message-passing-mixin :new-msg-id)
  (actor-message-passing-mixin :process-message)
  (actor-message-passing-mixin :queue-message)
- (actor-message-passing-mixin :read-messages)
  (actor-message-passing-mixin :reply)
  (actor-message-passing-mixin :reply-to)
- (actor-message-passing-mixin :send-locked)
  (actor-message-passing-mixin :tell)
  (actor-name-mixin :after :init)
- (actor-name-mixin :print-self)
  (actor-window :name-for-selection)
  (actor-window :process-toplevel)
- (actor-window :reset)
  (actor-window :run-reason)
  (actor-window :selectable-windows)
- (ada-attribute k*expander)
  (ada-begin k*expander)
  (ada-cond k*expander)
- (ada-constant-dcl k*expander)
  (ada-construct k*expander)
  (ada-convert k*expander)
- (ada-dcl k*expander)
  (ada-end_of_file)
  (ada-exit k*expander)
- (ada-file-close)
  (ada-file-def k*expander)
  (ada-file-index)
- (ada-file-open)
  (ada-file-read)
  (ada-file-reset)
- (ada-file-set-index)
  (ada-file-size)
  (ada-file-write)
- (ada-fn k*expander)
  (ada-fn~create 6proto-seg)
  (ada-fn~get 6proto-seg)

- ADA-EVAL
  ADA-EVAL
  ADA-EVAL
- ADA-EVAL
  ADA-EVAL
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  BEHAVIOR
- BEHAVIOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  ACTOR
  ACTOR
- ACTOR
  ACTOR
  ACTOR
- ACTOR
  ACTOR
  ACTOR
- ACTOR
  ACTOR
  ACTOR
- ACTOR
  ACTOR
  ACTOR
- ACTOR
  ACTOR
  ACTOR
- ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- STANDARD-ACTOR
  STANDARD-ACTOR
  STANDARD-ACTOR
- ADA-ANAL
  ADA-ANAL
  ADA-ANAL
- ADA-ANAL
  ADA-ANAL
  ADA-ANAL
- ADA-ANAL
  ADA-EVAL
  ADA-ANAL
- ADA-EVAL
  ADA-ANAL
  ADA-EVAL
- ADA-EVAL
  ADA-EVAL
  ADA-EVAL
- ADA-EVAL
  ADA-EVAL
  ADA-EVAL
- ADA-ANAL
  ADA-ANAL
  ADA-ANAL

- (ada-fn~new_line 6proto-seg)	- ADA-ANAL
(ada-fn~new_page 6proto-seg)	ADA-ANAL
(ada-fn~put 6proto-seg)	ADA-ANAL
- (ada-fn~set_col 6proto-seg)	- ADA-ANAL
(ada-fn~write 6proto-seg)	ADA-ANAL
(ada-init-dcl k*expander)	ADA-ANAL
- (ada-listener :process-toplevel)	- DEBUG
(ada-listener :process-toplevel)	DEBUG
(ada-loop k*expander)	ADA-ANAL
- (ada-new-fn k*expander)	- ADA-ANAL
(ada-new-package k*expander)	ADA-ANAL
(ada-raise k*expander)	ADA-ANAL
- (ada-return k*expander)	- ADA-ANAL
(ada-select k*expander)	ADA-ANAL
(ada-setq k*expander)	ADA-ANAL
- (ada-subtype-dcl k*expander)	- ADA-ANAL
(ada-top-fn k*expander)	ADA-ANAL
(ada-top-generic k*expander)	ADA-ANAL
- (ada-top-package k*expander)	- ADA-ANAL
(ada-top-package-body k*expander)	ADA-ANAL
(ada-type-dcl k*expander)	ADA-ANAL
- (ada-use k*expander)	- ADA-ANAL
(ada-while k*expander)	ADA-ANAL
(ada_int_get)	ADA-EVAL
- (ada_int_put)	- ADA-EVAL
(ada_int_put_str)	ADA-EVAL
(AND)	ADA-EVAL
- (apprentice :current-client-focus)	- MAGIC-ACTOR
(apprentice :event)	MAGIC-ACTOR
(apprentice :prepare-remark-window)	MAGIC-ACTOR
- (apprentice :prepare-typeout-window)	- MAGIC-ACTOR
(apprentice :set-current-client-focus)	MAGIC-ACTOR
(apprentice task)	MAGIC-ACTOR
- (aref 6proto-seg)	- PRIMOP
(arrayfetch 6proto-seg)	PRIMOP
(arraystore 6proto-seg)	PRIMOP
- (aset 6proto-seg)	- PRIMOP
(assoc 6proto-seg)	PRIMOP
(associated-process-mixin :default :process-toplevel)	TV-MAGIC
- (basic-actor answer)	- STANDARD-ACTOR
(basic-actor complaint)	STANDARD-ACTOR
(basic-actor report)	STANDARD-ACTOR
- (basic-actor suggest-code-home)	- STANDARD-ACTOR
(basic-actor suggested-code-home)	STANDARD-ACTOR
(belief-structure :belief)	BELIEF
- (belief-structure :believe)	- BELIEF
(belief-structure :disbelieve)	BELIEF
(belief-structure :map-over-beliefs)	BELIEF
- (belief-structure :map-over-beliefs-about)	- BELIEF
(belief-structure :map-over-beliefs-of-type)	BELIEF
(believer-mixin :belief)	BELIEF
- (believer-mixin :believe)	- BELIEF
(believer-mixin :disbelieve)	BELIEF
(believer-mixin :map-over-beliefs)	BELIEF
- (believer-mixin :map-over-beliefs-about)	- BELIEF
(believer-mixin :map-over-beliefs-of-type)	BELIEF
(believer-mixin belief)	BELIEF
- (believer-mixin believe)	- BELIEF
(bounds-of-definition :ada)	ZWEI-MAGIC
(bounds-of-definition :lisp)	ZWEI-MAGIC
- (box-drawing-mixin :complement-box)	- TV-MAGIC
(box-drawing-mixin :outline-box)	TV-MAGIC
(caar 6proto-seg)	PRIMOP
- (cadr 6proto-seg)	- PRIMOP
(car 6proto-seg)	PRIMOP
(Cclose 6proto-seg)	PRIMOP
- (cdar 6proto-seg)	- PRIMOP
(cddr 6proto-seg)	PRIMOP
(cdr 6proto-seg)	PRIMOP

- (chain_field) - DEMO-ADA-LIB
(cliche-instance action) IMPLEMENT
(cliche-instance unparse) GRAM
- (clock calendar) - ADA-EVAL
(close text_io) ADA-EVAL
(cobol-fn k*expander) EXPAND
- (code action) - VERBS
(code unparse) GRAM
(code-window-mixin :after :refresh) VIEW
- (code-window-mixin :change-focus) - VIEW
(code-window-mixin :display-program) VIEW
(code-window-mixin :home) VIEW
- (code-window-mixin :mouse-click) - VIEW
(code-window-mixin :scroll) VIEW
(code-window-mixin :who-line-documentation-string) VIEW
- (com-mode-break :ada) - DEBUG
(com-mode-break :lisp) ZWEI-MAGIC
(command-menu-mixin :after :mouse-buttons) TV-MAGIC
- (cond style-macro) - EDIT-DEBUG
(condition-mixin :before :bug-report-description) CONDITIONS
(condition-mixin :bug-report-recipient-system) CONDITIONS
- (condition-mixin :proceed) - CONDITIONS
(continuable-error-condition-mixin :case :proceed :continue)
CONDITIONS
(Copeni 6proto-seg) PRIMOP
- (Copenio 6proto-seg) - PRIMOP
(Copeno 6proto-seg) PRIMOP
(corresponding_file_name) DEMO-ADA-LIB
- (corresponding_headings) - DEMO-ADA-LIB
(corresponding_reporting) DEMO-ADA-LIB
(CORRESPONDING_SUMMARIZED_FILE) DEMO-ADA-LIB
- (Cread 6proto-seg) - PRIMOP
(create text_io) ADA-EVAL
(Crewrite 6proto-seg) PRIMOP
- (current-editor-path :ada) - ZWEI-MAGIC
(current-editor-path :ada) ZWEI-MAGIC
(current-editor-path :lisp) ZWEI-MAGIC
- (Cwrite 6proto-seg) - PRIMOP
(day calendar) ADA-EVAL
(debug-pane :after :activate) DEBUG
- (debug-pane :process-toplevel) - DEBUG
(def-ada-cliche k*expander) ADA-ANAL
(def-ada-fn k*expander) ADA-ANAL
- (def-ada-generic k*expander) - ADA-ANAL
(def-ada-package k*expander) ADA-ANAL
(def-ada-package-body k*expander) ADA-ANAL
- (default) - DEMO-ADA-LIB
(deffn macro) OUTIL
(delegate-mixin :after :init) SOCIETY
- (derived) - DEMO-ADA-LIB
(do k*expander) EXPAND
(do-named k*expander) EXPAND
- (edit-frame :name-for-selection) - EDIT
(edit-frame :selectable-windows) EDIT
(edit-frame :typeout-window) EDIT
- (edit-mixin answer) - EDIT-ACTOR
(editing-actor :update-definition-info) EDIT-ACTOR
(editing-actor analyze) EDIT-ACTOR
- (editing-actor coder-confirmation) - EDIT-ACTOR
(editing-actor document-code) EDIT-ACTOR
(editing-actor how) EDIT-ACTOR
- (editing-actor report) - EDIT-ACTOR
(editing-actor suggest-focus) EDIT-ACTOR
(editing-actor suggested-focus) EDIT-ACTOR
- (editing-actor typeout) - EDIT-ACTOR
(editing-actor-secretary :finish-request) EDIT-ACTOR
(editing-actor-secretary :finish-requests) EDIT-ACTOR
- (editing-actor-secretary :next-pending-request) - EDIT-ACTOR
(editing-actor-secretary :pending-requests) EDIT-ACTOR

(editing-actor-secretary :submit-by-proxy)	EDIT-ACTOR
- (editing-actor-secretary :suggest-focus)	- EDIT-ACTOR
(editing-actor-secretary :suggested-focus)	EDIT-ACTOR
(editing-actor-secretary presentational-focus)	EDIT-ACTOR
- (editing-actor-secretary suggest-focus)	- EDIT-ACTOR
(editing-actor-secretary suggested-focus)	EDIT-ACTOR
(editing-input-pane :process-toplevel)	SIMPLE
- (editor-pane :mouse-select)	- EDIT
(editor-pane :typeout)	EDIT
(employer-mixin :defer-task)	STANDARD-ACTOR
- (employer-mixin :find-apprentice)	- STANDARD-ACTOR
(employer-mixin :remark)	STANDARD-ACTOR
(employer-mixin :submit-by-proxy)	STANDARD-ACTOR
- (employer-mixin :submit-by-proxy-internal)	- STANDARD-ACTOR
(employer-mixin :typeout)	STANDARD-ACTOR
(employer-mixin :warn)	STANDARD-ACTOR
- (employer-mixin associated-apprentice)	- STANDARD-ACTOR
(employer-mixin task)	STANDARD-ACTOR
(enum-lit k*expander)	ADA-ANAL
- (file :conc-name (:type :named-list))	- ADA-EVAL
(FILE_NAME_ROOT)	DEMO-ADA-LIB
(format 6proto-seg)	PRIMOP
- (get text_io)	- ADA-EVAL
(get text_io)	ADA-EVAL
(get-definition :ada)	ZWEI-MAGIC
- (get-definition :lisp)	- ZWEI-MAGIC
(goback k*expander)	EXPAND
(here unparse)	GRAM
- (if style-macro)	- EDIT-DEBUG
(int-lit k*expander)	ADA-ANAL
(internal-condition :case :proceed :handled)	CONDITIONS
- (it action)	- REFER
(it unparse)	GRAM
(k*go 6fexpr)	PRIMOP
- (k*if 6fexpr)	- PRIMOP
(k*ifnull 6fexpr)	PRIMOP
(k*infinite-loop-pseudo-exit 6fexpr)	PRIMOP
- (k*infinite-loop-pseudo-exit 6proto-seg)	- PRIMOP
(k*lambda 6fexpr)	PRIMOP
(k*prog 6fexpr)	PRIMOP
- (k*psetq 6fexpr)	- PRIMOP
(k*quote 6fexpr)	PRIMOP
(k*setf 6proto-seg)	PRIMOP
- (k*tag 6fexpr)	- PRIMOP
(language :named-array :conc-name)	PARSE
(language :named-structure-invoke)	PARSE
- (let k*expander)	- EXPAND
(let* k*expander)	EXPAND
(let-if k*expander)	EXPAND
- (line text_io)	- ADA-EVAL
(line-truncating-mixin :after :init)	TV-MAGIC
(mailbox :after :init)	MBOX
- (mailbox :assure-read-lock)	- MBOX
(mailbox :await-message)	MBOX
(mailbox :describe-storage)	MBOX
- (mailbox :extract-message)	- MBOX
(mailbox :find-message)	MBOX
(mailbox :insert-message)	MBOX
- (mailbox :next-message)	- MBOX
(mailbox :next-message)	MBOX
(mailbox :next-reply)	MBOX
- (mailbox :print-self)	- MBOX
(mailbox :send-locked)	MBOX
(make-dummy-definition :ada)	ZWEI-MAGIC
- (make-dummy-definition :lisp)	- ZWEI-MAGIC
(margin-scroll-bar-mixin :after :activate)	TV-MAGIC
(match macro)	OUTIL
- (message :display)	- MBOX
(message :print-self)	MBOX

(MOD) ADA-EVAL
- (month calendar) - ADA-EVAL
(morpheme :named-structure-invoke) PARSE
(mouse-sensitive-regions-mixin :choose) MOUSE-SENSITIVE-REGIONS
- (mouse-sensitive-regions-mixin :mouse-sensitive-region) - MOUSE-SENSITIVE-REGIONS
(multiple-value k*expander) EXPAND
(multiple-value-bind k*expander) EXPAND
- (name unparse) - GRAM
(name-mixin :print-self) FLAVOR-MAGIC
(new_line text_io) ADA-EVAL
- (new_line text_io) - ADA-EVAL
(new_page text_io) ADA-EVAL
(non-null 6proto-seg) PRIMOP
- (NOT) - ADA-EVAL
(nterpri 6proto-seg) PRIMOP
(options-pane :after :activate) OPTIONS
- (options-window :after :refresh) - OPTIONS
(options-window :process-toplevel) OPTIONS
(OR) ADA-EVAL
- (overlapping-regions-mixin :before :clear-screen) - OVERLAPPING-REGIONS
(overlapping-regions-mixin :handle-mouse) OVERLAPPING-REGIONS
(overlapping-regions-mixin :mouse-click) OVERLAPPING-REGIONS
- (overlapping-regions-mixin :mouse-moves) - OVERLAPPING-REGIONS
(overlapping-regions-mixin :mouse-sensitive-region) OVERLAPPING-REGIONS
(overlapping-regions-mixin :move-mouse-region) OVERLAPPING-REGIONS
- (overlapping-regions-mixin :select-region) - OVERLAPPING-REGIONS
(page text_io) ADA-EVAL
(parameter unparse) VERBS
- (parameters unparse) - VERBS
(pif k*expander) EXPAND
(port-reference action) REFER
- (port-reference unparse) - GRAM
(post-office :add) POST-OFFICE
(post-office :remove) POST-OFFICE
- (post-office :remove-from) - POST-OFFICE
(post-office :resolve) POST-OFFICE
(post-office :resolve-remote) POST-OFFICE
- (prog k*expander) - EXPAND
(prog* k*expander) EXPAND
(prog2 k*expander) EXPAND
- (progv k*expander) - EXPAND
(progw k*expander) EXPAND
(property-list-mixin :get-property) FLAVOR-MAGIC
- (property-list-mixin :property-list-pointer) - FLAVOR-MAGIC
(property-list-mixin :put-property) FLAVOR-MAGIC
(property-list-mixin :remove-property) FLAVOR-MAGIC
- (put text_io) - ADA-EVAL
(put text_io) ADA-EVAL
(quote k*expander) EXPAND
- (RECORD_KEY_ROOT) - DEMO-ADA-LIB
(relative-reference action) REFER
(relative-reference unparse) GRAM
- (rem) - ADA-EVAL
(remote-actor :process-message) REMOTE
(remote-actor :process-message) REMOTE
- (rename) - DEMO-ADA-LIB
(return-list k*expander) EXPAND
(role unparse) VERBS
- (roles unparse) - VERBS
(root-reference action) REFER
(root-reference unparse) GRAM
- (rplaca 6proto-seg) - PRIMOP
(rplacd 6proto-seg) PRIMOP
(setf k*expander) EXPAND
- (set_col text_io) - ADA-EVAL
(set_output text_io) ADA-EVAL
(simple-actor clear) SIMPLE-ACTOR
- (simple-actor coder-confirmation) - SIMPLE-ACTOR
(simple-actor suggest-focus) SIMPLE-ACTOR

(simple-frame :after :init) SIMPLE
- (simple-frame :name-for-selection) - SIMPLE
(simple-frame :remark-window) SIMPLE
(simple-frame :typeout-window) SIMPLE
- (simple-pane-mixin :mouse-select) - TV-MAGIC
(simple-pane-mixin :name-for-selection) TV-MAGIC
(size_in_lines) DEMO-ADA-LIB
- (skip 6proto-seg) - PRIMOP
(society :delete) SOCIETY
(society :insert) SOCIETY
- (society :map) - SOCIETY
(society-member-mixin :after :init) STANDARD-ACTOR
(society-member-mixin :name?) STANDARD-ACTOR
- (standard_output text_io) - ADA-EVAL
(stop k*expander) EXPAND
(stop-run k*expander) EXPAND
- (str-lit k*expander) - ADA-ANAL
(stream-with-charpos :fresh-line) DEMO-LIB
(stream-with-charpos :operation-handled-p) DEMO-LIB
- (stream-with-charpos :read-cursorpos) - DEMO-LIB
(stream-with-charpos :set-cursorpos) DEMO-LIB
(stream-with-charpos :tyo) DEMO-LIB
- (stream-with-charpos :unclaimed-message) - DEMO-LIB
(substring 6proto-seg) PRIMOP
(SUGGEST_VARIABLE_NAME) DEMO-ADA-LIB
- (that action) - REFER
(that unparse) GRAM
(this action) REFER
- (this unparse) - GRAM
(time_of calendar) ADA-EVAL
(top action) REFER
- (top unparse) - GRAM
(u+) ADA-EVAL
(u-) ADA-EVAL
- (unfinished-edits-buffer :add-node) - POSSIBILITIES
(unfinished-edits-buffer :description) POSSIBILITIES
(unfinished-edits-buffer :document) POSSIBILITIES
- (unfinished-edits-buffer :header-description-1) - POSSIBILITIES
(unfinished-edits-buffer :list-subnodes) POSSIBILITIES
(unfinished-edits-buffer :reusable) POSSIBILITIES
- (unfinished-edits-buffer :revert) - POSSIBILITIES
(unless style-macro) EDIT-DEBUG
(values-list k*expander) EXPAND
- (visible-cursor-location :ada) - EDIT-COMMANDS
(visible-cursor-location :lisp) EDIT-COMMANDS
(vocabulary :otherwise :interpret) PSEUDO
- (when style-macro) - EDIT-DEBUG
(with-open-file k*expander) EXPAND
(XOR) ADA-EVAL
- (year calendar) - ADA-EVAL
(zwei:get-section-name :ada) ZWEI-MAGIC
(zwei:top-level-editor :change-mode-line-list) ZWEI-MAGIC
- (zwei:zmacs-frame :delegate) - ZWEI-MAGIC
(_eq 6fexpr) PRIMOP
(_eq 6proto-seg) PRIMOP
- (_gt 6fexpr) - PRIMOP
(_gt 6proto-seg) PRIMOP
(_lt 6fexpr) PRIMOP
- (_lt 6proto-seg) - PRIMOP
(|...| :gformat) PLI
2focus-seg IMPLEMENT
- *abstract-code-references* - EDIT-COMMANDS
ada-break-chars ADA-PARSE
ada-input-marker DEBUG
- *ada-result-marker* - DEBUG
allow-KBE-remote REMOTE
almost-this REFER
- *alphabet* - ENGLISH
alphanumeric GRAM

any
- *ask
at-end-noticers
auto-submit
- *Automatic-KBE-Package*
Automatic-Lisp-Mode
automatic-reanalysis-when-needed
- *awaiting*
beliefs
blah-names
- *buffer*
buffer-storage
bug
- *bug-history*
bug-mail-address
casification-hook
- *character-set-size*
check-code-consistency
cmarkers
- *command-prompt*
completion-table
current-editor-path
- *current-editor-path-bp*
current-editor-path-time
db-area-count
- *db-areas*
db-audit
db-check-fact-ports
- *db-check-facts*
db-context
db-context-after-reap
- *db-context-count*
db-context-count-after-reap
db-context-names
- *db-fields*
db-free-areas
db-keywords
- *db-last-pattern-found*
db-properties
db-top-fn
- *db-top-fn-idx*
db-vars
debug
- *debug-parser*
debug-zwei-errors
default-apprentice
- *define-cliche
*define-completion
*define-ordinal
- *define-role
*delegate
demo-file-pathname-defaults
- *dictionary-list*
*disp-dump
display-language
- *disp
*equal
EXPLAIN-debug
- *explainer*
expression
focus-style
- *force-slots-up*
gc-enabled
gc-enabled
- *gc-last-address*
going-to-submit
here
- *implement
in-command-mini-buffer

PARSE
- ACTOR
VERBS
CONSISTENCY
- ZWEI-MAGIC
ZWEI-MAGIC
CONSISTENCY
- EDIT-ACTOR
BELIEF
CODER
- PARSE
PARSE
CONDITIONS
- CONDITIONS
KMAC
PARSE
- PARSE
CONSISTENCY
DVARS
- PSEUDO
PSEUDO
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
DMAC
- DMAC
DVARS
DVARS
- DVARS
DVARS
GC
- DVARS
GC
DVARS
- DMAC
DMAC
DMAC
- DVARS
DMAC
DVARS
- DB
DMAC
KMAC
- ZWEI-MAGIC
ZWEI-MAGIC
STANDARD-ACTOR
- PSEUDO
PSEUDO
PSEUDO
- PSEUDO
ACTOR
TEST
- PSEUDO
UTIL
VERBS
- UTIL
UTIL
EXPLAIN
- VERBS
PARSE
EDIT-ACTOR
- SEGTRN
GC
GC
- GC
EDIT-COMMANDS
REFER
- IMPLEMENT
EDIT-COMMANDS

initial-db-context
- *initial-db-context-number*
initial-editing-actor
initial-editing-actor-secretary
- *internal-names*
kaos-debug
kaos-prompt
- *kbe-input-window-comtab*
kbe-mini-buffer-comtab
kbe-mini-buffer-multi-line-comtab
- *kbe-mode-key-bindings*
kbe-role-marker
KBE-switches
- *KBE-user-package*
kbe-user-role-marker
key-order
- *keyword-package*
known-modes
last-form-parsed
- *loop-code-template*
mark-post-finish-hook
mark-pre-finish-hook
- *max-buffer-line-length*
message-being-processed
message-being-processed-terminated
- *multi-mode-info*
nest-loop-epilog
nested-ada-fn-spec
- *notification-data*
null-cmarker
old-definition
- *options*
ordinals
pattern-vars
- *pending-dflows*
pgms-filename
plan-def-segcases
- *pli-gformat-atoms*
PO
pointer
- *port-descriptors*
post-office
*pread-1
- *pread
*print-test
quote-chars
- *recycle-old-db-areas*
*reimplement
reply-to
- *reply
separate-gens-and-terms
server-window
- *simple-mini-buffer-command*
single-mode-info
*snarf
- *SO*
society
special-text_io-fns
- *statement*
SUGGESTED-NAME-FROM-SLOT
supress-constraints
- *system-initializations*
*tell
test-file-pathname-defaults
- *that*
this
token-buffer
- *token-in-stream*
token-type

DVARS
- DVARS
EDIT-ACTOR
EDIT-ACTOR
- REFER
KAOS
KAOS
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
COMMENT-MACRO
KMAC
- KVARS
ZWEI-MAGIC
DB
- KVARS
ZWEI-MAGIC
ADA-PARSE
- CODER
EDIT-ACTOR
EDIT-ACTOR
- ZWEI-MAGIC
ACTOR
ACTOR
- ZWEI-MAGIC
CODER
ADA-UNPARSE
- REMOTE
DVARS
EDIT-ACTOR
- OPTIONS
PSEUDO
ADA-PARSE
- IMPLEMENT
UTIL
DB
- PLI
DB
PARSE
- PSEUDO
POST-OFFICE
UTIL
- UTIL
UTIL
GRAM
- GC
IMPLEMENT
INTERFACE-MACROS
- ACTOR
ANALYZE
KAOS
- EDIT-COMMANDS
ZWEI-MAGIC
UTIL
- DB
SOCIETY
ADA-ANAL
- PARSE
PRIMOP
IMPLEMENT
- KMAC
ACTOR
TEST
- REFER
REFER
PSEUDO
- ADA-PARSE
PSEUDO

toplevel-chars	GRAM
- *trap-db-inconsistency*	- VERBS
*undefine-cliche	PSEUDO
*undefine-role	PSEUDO
- *undo-stack*	- VERBS
unknown-process-states	STANDARD-ACTOR
unquestionable-vars	CODER
- *user-window*	- KAOS
when-error-debug	MACRO
zmacs-default-mode-line-name	ZWEI-MAGIC
- *zwei-commands*	- ZWEI-MAGIC
*	PBM-HANDLERS
*	PBM-HANDLERS
- *	- PLI
+&	DMAC
+cflow	DB
- +dflow	- DB
+recursive	DB
+role	DB
- +subseg	- DB
+	PBM-HANDLERS
+	PBM-HANDLERS
- +	- PLI
-&	DMAC
--&	DMAC
- --input	- DB
--output	DB
--roles	DB
- --role	- DB
--subseg	DB
-plantype	DB
- -role	- DB
-segtype	DB
-subseg	DB
- -supseg	- DB
-	PBM-HANDLERS
-	PBM-HANDLERS
- -	- PLI
//	PBM-HANDLERS
//	PBM-HANDLERS
- //	- PLI
2ancestors	IMPLEMENT
2brothers-before	IMPLEMENT
- 2check-constraints	- IMPLEMENT
2check-constraint	IMPLEMENT
2check-pending-dflows	IMPLEMENT
- 2check-should-eval	- IMPLEMENT
2comes-from-output-slot	IMPLEMENT
2creation-time-kbe-comment	COMMENT-MACRO
- 2creation-time-plan	- IMPLEMENT
2fed-input	IMPLEMENT
2find-safe-name	IMPLEMENT
- 2find-side-effect-out	- IMPLEMENT
2find-var-source1	IMPLEMENT
2find-var-source	IMPLEMENT
- 2focus-seg-kbe-comment	- COMMENT-MACRO
2fvs-test	IMPLEMENT
2get-better-from	IMPLEMENT
- 2get-better-to	- IMPLEMENT
2get-cliche	IMPLEMENT
2get-higher-from	IMPLEMENT
- 2get-rename-alist	- IMPLEMENT
2gra0	IMPLEMENT
2gra	IMPLEMENT
- 2id-horizontal	- IMPLEMENT
2id-ingoing	IMPLEMENT
2id-initial-interval	IMPLEMENT
- 2id-make-final-value	- IMPLEMENT
2id-make-initial-value	IMPLEMENT

2id-outgoing	IMPLEMENT
- 2id-temporal-ingoing	- IMPLEMENT
2id-temporal-outgoing	IMPLEMENT
2ifv	IMPLEMENT
- 2imp-input-dflow	- IMPLEMENT
2imp-top	IMPLEMENT
2imp-type	IMPLEMENT
- 2imp	- IMPLEMENT
2input-from-var	IMPLEMENT
2intermediate-segs	IMPLEMENT
- 2intermediate-steps1	- IMPLEMENT
2intermediate-steps	IMPLEMENT
2make-dummy-tinput	IMPLEMENT
- 2make-dummy-toutput	- IMPLEMENT
2remove*assertions	IMPLEMENT
2remove-dflow	IMPLEMENT
- 2remove-input	- IMPLEMENT
2remove-output	IMPLEMENT
2remove-seg	IMPLEMENT
- 2remove-tinput	- IMPLEMENT
2remove-toutput	IMPLEMENT
2rename-var-names-seg	IMPLEMENT
- 2rename-var-names	- IMPLEMENT
2rename-vars-avoiding	IMPLEMENT
2rename-vars-for-imp	IMPLEMENT
- 2replace-port	- IMPLEMENT
2ROLE-FILLERS	IMPLEMENT
2seg-inside-seg	IMPLEMENT
- 2side-effect-out	- IMPLEMENT
2splice-into-parallel-dflows	IMPLEMENT
2top->simple	IMPLEMENT
- 2var-for-port	- IMPLEMENT
2var-names-used-seg	IMPLEMENT
2var-names-used	IMPLEMENT
- 3AFFECTED	- ANALYZE
3bigloopp	ANALYZE
3check-path	ANALYZE
- 3check-step	- ANALYZE
3common-dflow-path	ANALYZE
3commutable-preds	ANALYZE
- 3commute-preds	- ANALYZE
3complexpredp	ANALYZE
3copy-cflow	ANALYZE
- 3copy-ssr-skeleton	- ANALYZE
3dfaffect	ANALYZE
3dflow-simp1	ANALYZE
- 3dflow-simp2	- ANALYZE
3dflow-simp	ANALYZE
3do-dflow-move	ANALYZE
- 3expr-simp-top	- ANALYZE
3expr-simp-ungroup	ANALYZE
3expr-simp1	ANALYZE
- 3expr-simp2	- ANALYZE
3expr-simp	ANALYZE
3exprize	ANALYZE
- 3exprp	- ANALYZE
3fix-infinite-loops	ANALYZE
3fix-slot-assertions	ANALYZE
- 3get-bot-j	- ANALYZE
3get-top-j	ANALYZE
3group-up	ANALYZE
- 3high	- ANALYZE
3is-mobile	ANALYZE
3joinp	ANALYZE
- 3loop-ssrize	- ANALYZE
3make-roles	ANALYZE
3make-up-name-for-returnval	CODER
- 3make-up-name	- CODER
3n2	ANALYZE

3nsegs	ANALYZE
- 3optnext	- ANALYZE
3optprev	ANALYZE
3p1	ANALYZE
- 3p2	- ANALYZE
3pred2p	ANALYZE
3predize-i	ANALYZE
- 3predize	- ANALYZE
3psegs	ANALYZE
3recursive-ssrize	ANALYZE
- 3segfacts	- ANALYZE
3segtype	ANALYZE
3side-effects	ANALYZE
- 3simplepredp	- ANALYZE
3simplep	ANALYZE
3slotname-parse	ANALYZE
- 3ssr-analyze	- ANALYZE
3ssr-find-removable-group	ANALYZE
3ssr-simp	ANALYZE
- 3ssr-subsets1	- ANALYZE
3ssra-build-affected	ANALYZE
3ssra-build-porttrans	ANALYZE
- 3ssra-build-tc	- ANALYZE
3ssra-classify-subsegs	ANALYZE
3ssra-compute-seg-order	ANALYZE
- 3ssra-init-affects	- ANALYZE
3ssra-remove-fragment	ANALYZE
3ssra-ungroup	ANALYZE
- 3ssrp	- ANALYZE
3ssrs1	ANALYZE
3ssrs	ANALYZE
- 3suggest-dmove-path	- ANALYZE
3unfactor-preds	ANALYZE
3up-to-ssr	ANALYZE
- 3update-affected	- ANALYZE
3xor-simp	ANALYZE
3xorize	ANALYZE
- 5abstract-ordering	- CODER
5add-name-class	CODER
5annotate-return-dflow	CODER
- 5annotate-var-dflow	- CODER
5annotate	CODER
5any-follow	CODER
- 5assign-pair	- CODER
5assign1	CODER
5assign	CODER
- 5bd	- CODER
5bit^	CODER
5break-region	CODER
- 5can-be-ret	- CODER
5case-action-sort	CODER
5cio-order-L	CODER
- 5clean-noops1	- CODER
5clean-noops	CODER
5clean@assertions	CODER
- 5cleanup	- CODER
5close1	CODER
5close	CODER
- 5codelambda	- CODER
5codeopencode	CODER
6codepred2	CODER
- 6codepred	- CODER
5coder-post-phase-hacks	CODER
5codesimple	CODER
- 5codessr-loop	- CODER
5codessr-prog	CODER
5codessr	CODER
- 5codetop	- CODER
5codeup-top	CODER

5codeupinner	CODER
- 5codeup	- CODER
5codexor	CODER
5consolodate-cond	CODER
- 5constrain-segs	- CODER
5constrain<	CODER
5determine-name-class	CODER
- 5determine-nesting	- CODER
5estimate-/@info	CODER
5estimate-code	CODER
- 5expr-nesting	- CODER
5fd-1	CODER
5find-io-L	CODER
- 5find-regions	- CODER
5find-region	CODER
5find-successors	CODER
- 5find-tcs	- CODER
5fio-L1	CODER
5fio-L	CODER
- 5fix-@assertions	- CODER
5fix-depending-on	CODER
5get-lisp-constant	CODER
- 5get-lisp-name	- CODER
5get-related-regions1	CODER
5get-related-regions	CODER
- 5goes-to-side-effect	- CODER
5goodname	CODER
5init-order	CODER
- 5kill-dummy	- CODER
5let-hack	CODER
5lh-1	CODER
- 5lh-lessp	- CODER
5lh-order	CODER
5list-car	CODER
- 5make-multiple-pattern	- CODER
5make-precede	CODER
5make-prog2	CODER
- 5make-with-open-file-forms	- CODER
5make1	CODER
5mark-returnval	CODER
- 5ncr-1	- CODER
5nd	CODER
5need-not	CODER
- 5new-id	- CODER
5no-cond-rets	CODER
5noop-related	CODER
- 5NOOP-TRAN	- CODER
5nr	CODER
5null-computation	CODER
- 5order-id	- CODER
5pc-1	CODER
5po1	CODER
- 5port-make-precede	- CODER
5port-precedes?	CODER
5port>order-id	CODER
- 5precedes?	- CODER
5pretty-conds	CODER
5progn-trans	CODER
- 5psetq-order	- CODER
5ptype	CODER
5put-here	CODER
- 5region-classify	- CODER
5regions-overlap	CODER
5remove-from-pattern	CODER
- 5rep1	- CODER
5rep	CODER
5root-sexpr	CODER
- 5sexpr-in	- CODER
5simple-stuff-at-end	CODER

5sor-1	CODER
- 5source	- CODER
5suggest-names	CODER
5suggest-name	CODER
- 5supress-output-roles0	- CODER
5supress-output-roles	CODER
5tc-collapse	CODER
- 5tc-fix-remaining-tports	- CODER
5tc-fix-slots	CODER
5tc-move-frag-body-ports	CODER
- 5tc-move-frag-dj	- CODER
5tc-move-frag-init	CODER
5tc-move-frag-ports	CODER
- 5tc-move-fragkeyseg	- CODER
5tc-move-tinput1	CODER
5tc-move-tinput	CODER
- 5tc-move-toutput1	- CODER
5tc-move-toutput	CODER
5temporal-compose-all	CODER
- 5terminal?	- CODER
5try-make-be-name	CODER
5try-make-be-returnval	CODER
- 5un-nestable-fns	- CODER
5use-name-classes	CODER
5with-open-file-hack	CODER
- 6+df	- PRIMOP
6?df	PRIMOP
6abstractio	SURFACE
- 6aio2	- SURFACE
6and-pred	EXPAND
6apply	PRIMOP
- 6a	- DFLOW
6build-k*psetq	PRIMOP
6cleanup	DFLOW
- 6construct-df-info	- DFLOW
6copy-top-seg	PRIMOP
6create-dflow1	DFLOW
- 6create-dflow	- DFLOW
6data-flow	DFLOW
6df-name	PRIMOP
- 6df-num	- PRIMOP
6dfn->port	PRIMOP
6dfns	PRIMOP
- 6df	- PRIMOP
6do-joining	DFLOW
6d	PRIMOP
- 6eval	- PRIMOP
6fast-defspecs	PRIMOP
6fast-proto-names	PRIMOP
- 6fast-proto-name	- PRIMOP
6fast-proto	PRIMOP
6find-joining	DFLOW
- 6find-live1	- DFLOW
6find-parts1	DFLOW
6find-parts	DFLOW
- 6find-write-estimate-live	- DFLOW
6flow-analyze	SURFACE
6flow-eval	PRIMOP
- 6good-code	- EXPAND
6h	DFLOW
6ignored-fns	DFLOW
- 6instantiate-funcall	- PRIMOP
6is-in	EXPAND
6kill-unneeded-top-seg	SURFACE
- 6lambda-like-fn?	- EXPAND
6make-super-outputs	DFLOW
6mapvars	EXPAND
- 6or-pred	- EXPAND
6process-jhunk	DFLOW

6resultvar	EXPAND
- 6sameio	- SURFACE
6savevar	EXPAND
6set-up-flows	DFLOW
- 6set-up-linkages	- DFLOW
6set-	DFLOW
6setU	DFLOW
- 6set^?	- DFLOW
6set^	DFLOW
6side-effects	DFLOW
- 6symeval	- PRIMOP
6trans-live	DFLOW
6used-temp	EXPAND
- :EVAL	- KAOS
:FUNNY-EVAL	KAOS
:local-KBE-dispatch	REMOTE
- :remote-service	- REMOTE
<-	MACRO
<attribute-name>	ADA-PARSE
- <attribute>	- ADA-PARSE
<basic-dcl>	ADA-PARSE
<basic-exp-unit>	ADA-PARSE
- <basic-mode>	- ADA-PARSE
<basic-stmnt>	ADA-PARSE
<basic-top-use>	ADA-PARSE
- <basic-type-name-root>	- ADA-PARSE
<begin>	ADA-PARSE
<box>	ADA-PARSE
- <cliche-comment-comment>	- ADA-PARSE
<cliche-comments>	ADA-PARSE
<cliche-constraints-comment>	ADA-PARSE
- <cliche-described-roles-comment>	- ADA-PARSE
<cliche-primary-roles-comment>	ADA-PARSE
<cliche>	ADA-PARSE
- <complex-role>	- ADA-PARSE
<cond>	ADA-PARSE
<constant-dcl>	ADA-PARSE
- <contour-name-root>	- ADA-PARSE
<contour-name>	ADA-PARSE
<contour-names+>	ADA-PARSE
- <converter>	- ADA-PARSE
<dcl-annotation>	ADA-PARSE
<dcl-cliche>	ADA-PARSE
- <dcl>	- ADA-PARSE
<dcls>	ADA-PARSE
<def-form>	ADA-PARSE
- <dummy>	- ADA-PARSE
<else>	ADA-PARSE
<elsif>	ADA-PARSE
- <empty-cliche>	- ADA-PARSE
<enumeration-element-name>	ADA-PARSE
<enumeration-element-root>	ADA-PARSE
- <enumeration-literal>	- ADA-PARSE
<enumeration>	ADA-PARSE
<exception-clause>	ADA-PARSE
- <exception-clauses>	- ADA-PARSE
<exception-name-root>	ADA-PARSE
<exception-name>	ADA-PARSE
- <exception>	- ADA-PARSE
<exit>	ADA-PARSE
<exp-annotation>	ADA-PARSE
- <exp>	- ADA-PARSE
<expr-cliche>	ADA-PARSE
<exps+>	ADA-PARSE
- <exps>	- ADA-PARSE
<field-name>	ADA-PARSE
<field-selector>	ADA-PARSE
- <field>	- ADA-PARSE
<fields+>	ADA-PARSE

<fn-call> ADA-PARSE
- <fn-designator> - ADA-PARSE
<fn-name-root> ADA-PARSE
<fn-name> ADA-PARSE
- <fn-spec> - ADA-PARSE
<fn> ADA-PARSE
<full-cliche> ADA-PARSE
- <generic-arg> - ADA-PARSE
<generic-args> ADA-PARSE
<generic-name-root> ADA-PARSE
- <generic-name> - ADA-PARSE
<generic> ADA-PARSE
<if> ADA-PARSE
- <in-out> - ADA-PARSE
<init-dcl> ADA-PARSE
<instance> ADA-PARSE
- <integer-literal> - ADA-PARSE
<integer> ADA-PARSE
<limited-private> ADA-PARSE
- <literal> - ADA-PARSE
<loop> ADA-PARSE
<maybe-modified-exp> ADA-PARSE
- <maybe-modified-exps+> - ADA-PARSE
<mode> ADA-PARSE
<modified-exp> ADA-PARSE
- <modified> - ADA-PARSE
<new-fn> ADA-PARSE
<new-package> ADA-PARSE
- <op-exp> - ADA-PARSE
<operator-string> ADA-PARSE
<operator> ADA-PARSE
- <package-body> - ADA-PARSE
<package-name-root> ADA-PARSE
<package-name> ADA-PARSE
- <package> - ADA-PARSE
<param> ADA-PARSE
<params> ADA-PARSE
- <parenthesised-exp> - ADA-PARSE
<qualifications> ADA-PARSE
<raise> ADA-PARSE
- <range-exp> - ADA-PARSE
<range-pair> ADA-PARSE
<range> ADA-PARSE
- <record-constructor> - ADA-PARSE
<record> ADA-PARSE
<return> ADA-PARSE
- <role-name-string> - ADA-PARSE
<role> ADA-PARSE
<sentence> ADA-PARSE
- <setq> - ADA-PARSE
<simple-role> ADA-PARSE
<stmnt-annotation> ADA-PARSE
- <stmnt> - ADA-PARSE
<stmnts+> ADA-PARSE
<string-literal> ADA-PARSE
- <string-type> - ADA-PARSE
<string> ADA-PARSE
<subtype-dcl> ADA-PARSE
- <symbol> - ADA-PARSE
<top-level-form> ADA-PARSE
<top-use> ADA-PARSE
- <true-field> - ADA-PARSE
<type-dcl> ADA-PARSE
<type-name-root> ADA-PARSE
- <type-name> - ADA-PARSE
<type> ADA-PARSE
<use> ADA-PARSE
- <variable-dcl> - ADA-PARSE
<variable> ADA-PARSE

<while>
- <with>
<word>
<
- =dflow-from
=dflow-to
=port-seg
- =
=
=
- >&
>@seg-code
>plantype
- >segtype
>supseg
>
- ?&
?1role
??&
- ??dflow-use
??input
??output
- ??roles
??role
??subseg
- ?@io-code
?@io-kind
?@io-name
- ?@seg-code
?@seg-kind
?@seg-name
- ?dflow-source
?empty-slot
?lambda-exp
- ?plantype
?recurrence
?recursive-supseg
- ?role
?segtype
?supseg
- @-fact?
@createvar-fact?
@io-fact?
- @seg-fact?
a*seg
a-&
- a-or-an
a<n>
a?&
- ACCUMULATION
acknowledge-having-queued-stream
active-default-lbp
- active-default-rbp
active-language
active-syntax-table
- active-whitespace-chars
actor-background-process-mixin
actor-default-behavior-mixin
- actor-name-mixin
actor-window
actor
- ada-add-prefix
ada-add-use
ada-attribute
- ada-attribute
ada-begin
ada-begin
- ada-binary-op-info
ada-bind-level

ADA-PARSE
- ADA-PARSE
ADA-PARSE
PLI
- SEGTRN
SEGTRN
SEGTRN
- PBM-HANDLERS
PBM-HANDLERS
PLI
- DMAC
DB
DB
- DB
DB
PLI
- DMAC
DB
DMAC
- DB
DB
DB
- DB
DB
DB
- DB
DB
DB
- DB
DB
DB
- DB
DB
DB
- DB
DB
DB
- DB
DB
DB
- DMAC
DMAC
DMAC
- DMAC
IMPLEMENT
DMAC
- ENGLISH
DICT
DMAC
- KBELIB
EDIT-COMMANDS
PARSE
- PARSE
PARSE
PARSE
- PARSE
STANDARD-ACTOR
BEHAVIOR
- ACTOR
STANDARD-ACTOR
ACTOR
- ADA-EVAL
ADA-EVAL
ADA-EVAL
- ADA-UNPARSE
ADA-EVAL
ADA-UNPARSE
- ADA-UNPARSE
ADA-UNPARSE

ada-bind	ADA-UNPARSE
- ada-bool	- ADA-EVAL
ada-break-repl	DEBUG
ada-cliche-plist	ADA-UNPARSE
- ada-cond	- ADA-EVAL
ada-cond	ADA-UNPARSE
ada-constant-dcl	ADA-EVAL
- ada-constant-dcl	- ADA-UNPARSE
ada-construct-int	ADA-EVAL
ada-construct	ADA-EVAL
- ada-construct	- ADA-UNPARSE
ada-convert	ADA-EVAL
ada-convert	ADA-UNPARSE
- ada-current-input	- ADA-EVAL
ada-current-output	ADA-EVAL
ada-data-storage	ADA-EVAL
- ada-dcls	- ADA-UNPARSE
ada-dcl	ADA-EVAL
ada-dcl	ADA-UNPARSE
- ada-definition-type-name	- ADA-UNPARSE
ada-determine-subs	ADA-EVAL
ada-determine-sub	ADA-EVAL
- ada-do-dcl	- ADA-EVAL
ada-do-subtype-dcl	ADA-EVAL
ada-ends-file-def	ADA-UNPARSE
- ada-end_of_file	- ADA-EVAL
ada-enumeration	ADA-UNPARSE
ada-exceptions	ADA-UNPARSE
- ada-exception	- ADA-UNPARSE
ada-exit	ADA-EVAL
ada-exit	ADA-UNPARSE
- ada-field	- ADA-UNPARSE
ada-find-1	ADA-EVAL
ada-find-fast	ADA-EVAL
- ada-find-right-fn	- ADA-EVAL
ada-find-true-type	ADA-EVAL
ada-find	ADA-EVAL
- ada-fn-spec	- ADA-UNPARSE
ada-fn	ADA-EVAL
ada-fn	ADA-UNPARSE
- ada-functions	- ADA-UNPARSE
ada-gap-between-dcls	ADA-UNPARSE
ada-gather-lisp-tokens	ADA-PARSE
- ada-gensym-counter	- ADA-EVAL
ada-get-exp-thing	ADA-PARSE
ada-info	ADA-EVAL
- ada-init-dcl	- ADA-EVAL
ada-init-dcl	ADA-UNPARSE
ada-init	ADA-EVAL
- ada-instance	- ADA-EVAL
ada-interesting-def	ADA-UNPARSE
ada-kbe-/#-comment-macro-printer	ADA-UNPARSE
- ada-last-exception	- ADA-EVAL
ada-line-size-0	DEMO-ADA-LIB
ada-line-size	DEMO-ADA-LIB
- ada-listener	- DEBUG
ada-listener	DEBUG
ada-literal-class	ADA-EVAL
- ada-loop	- ADA-EVAL
ada-loop	ADA-UNPARSE
ada-lower-case-0	ADA-UNPARSE
- ada-lower-case-1	- ADA-UNPARSE
ada-lower-case	ADA-UNPARSE
ada-mode	ZWEI-MAGIC
- ada-new-fn	- ADA-EVAL
ada-new-fn	ADA-UNPARSE
ada-new-package	ADA-EVAL
- ada-new-package	- ADA-UNPARSE
ada-old-files	ADA-EVAL

ada-op-?	ADA-UNPARSE
- ada-op-name	- ADA-UNPARSE
ada-param-gf	ADA-UNPARSE
ada-parse-to-end	ADA-PARSE
- ada-parse	- ADA-PARSE
ada-prefix-it	ADA-EVAL
ada-prefix-stack	ADA-EVAL
- ada-process-arg1	- ADA-UNPARSE
ada-qname	ADA-UNPARSE
ada-raise	ADA-EVAL
- ada-raise	- ADA-UNPARSE
ada-range	ADA-UNPARSE
ada-rec	ADA-UNPARSE
- ada-remove-comments	- ADA-PARSE
ada-repl-prompt	DEBUG
ada-repl-read	DEBUG
- ada-repl	- DEBUG
ada-reraise	ADA-EVAL
ada-resume	DEBUG
- ada-return	- ADA-EVAL
ada-return	ADA-UNPARSE
ada-select	ADA-EVAL
- ada-select	- ADA-UNPARSE
ada-setq	ADA-EVAL
ada-setq	ADA-UNPARSE
- ada-standard-input	- ADA-EVAL
ada-standard-output	ADA-EVAL
ada-starts-file-def	ADA-UNPARSE
- ada-statement-classify	- ADA-UNPARSE
ada-statements	ADA-UNPARSE
ada-store-data	ADA-EVAL
- ada-store-fast	- ADA-EVAL
ada-store-fn	ADA-EVAL
ada-stored-data	ADA-EVAL
- ada-string-length	- ADA-EVAL
ada-string	ADA-UNPARSE
ada-sublis-1	ADA-EVAL
- ada-sublis-alist	- ADA-EVAL
ada-sublis	ADA-EVAL
ada-subtype-dcl	ADA-EVAL
- ada-subtype-dcl	- ADA-UNPARSE
ada-supress	ADA-UNPARSE
ada-symbol-class	ADA-PARSE
- ada-symbol-type	- ADA-PARSE
ada-test-true	ADA-EVAL
ada-test-when	ADA-EVAL
- ada-text-breakup	- EDIT-ACTOR
ada-tokenize	ADA-PARSE
ada-tokens-parse	ADA-PARSE
- ada-type-dcl	- ADA-EVAL
ada-type-dcl	ADA-UNPARSE
ada-type-match	ADA-EVAL
- ada-types-match	- ADA-EVAL
ada-type	ADA-UNPARSE
ada-unary-op-info	ADA-UNPARSE
- ada-unbool	- ADA-EVAL
ada-use-packages	ADA-EVAL
ada-use	ADA-EVAL
- ada-use	- ADA-UNPARSE
ada-value-build	ADA-EVAL
ada-when-clause	ADA-UNPARSE
- ada-while	- ADA-EVAL
ada-while	ADA-UNPARSE
ada-with	ADA-UNPARSE
- ADA?	- ADA-UNPARSE
ada	PARSE
ada	PARSE
- add-at-end-noticer	- VERBS
add-bucky-bits	BASIC

add-descriptor
- add-dictionary-entry
add-fact
add-layer
- add-parameter
add-pending-dflow
add-system-key
- add-to-mailing-list
Add
add
- algol
algol
alist-of-strings-to-do-in-function
- alist-of-strings-to-do-in
alist-of-strings-to-do
all-demo-names
- all-demo-sequence-names
all-fact-cases
all-subroles-aux
- all-subroles
all-subsegs
alpha-numeric
- alphabetp
alphalessp-dcl
analyze-for-recognition
- analyze
Analyze
Analyze
- AND
AND
AND
- and
Anything
an
- append-delimiter-if
append-delimiter
Append-Parameter
- APPEND
APPEND
Append
- apprentice-eval-command
apprentice-reset
apprentice-select
- apprentice-status
apprentice
arg-order
- argument
arraydefs
arrayfetch
- arrayfetch
arraystore
arraystore
- ask
ask
assert-fact
- associated-apprentice
associated-process-mixin
assq-carefully
- assure-buffered-definitions-consistent
assure-db-consistency
assure-real-editor-buffer
- audit-fact
audit-segcase
auto-fill-stream
- AVERAGE
a
a
- backward-char
backward-defun

PSEUDO
- PSEUDO
DB
DMAC
- VERBS
IMPLEMENT
BASIC
- POST-OFFICE
EDIT-COMMANDS
VERBS
- PARSE
PARSE
VERBS
- VERBS
VERBS
TEST
- TEST
DB
REFER
- REFER
DB
EDIT-ACTOR
- PARSE
ADA-ANAL
ANALYZE
- ANALYZE
EDIT-COMMANDS
VERBS
- OUTIL
OUTIL
OUTIL
- PLI
EDIT-COMMANDS
DICT
- GRAM
GRAM
EDIT-COMMANDS
- OUTIL
OUTIL
VERBS
- MAGIC-ACTOR
OPTIONS
OPTIONS
- OPTIONS
MAGIC-ACTOR
VERBS
- DICT
MACRO
PLI
- UTIL
PLI
UTIL
- DEBUG
INTERFACE-MACROS
DB
- EVENTS
TV-MAGIC
BASIC
- EDIT-COMMANDS
VERBS
ZWEI-MAGIC
- DB
DB
ENGLISH
- KBELIB
DICT
DICT
- ZWEI-MAGIC
ZWEI-MAGIC

backward-line
- backward-sexp
backward-up-list
backward-word
- BAG
BAG
barf
- barf
basic-actor
basic-qfn
- bdb
beginning-of-line
begin
- belief
believer-mixin
believe
- binary-function
bind-editor-buffer
bind-focus
- blah-name?
blank-stuff
bound-token
- bounds-of-definition
box-drawing-mixin
break-up-join
- breakchar-p
bucky-char-name
buffer-command-prescan
- buffer-editing-type
buffer-end
buffer-font
- buffer-name-if-relevant
buffer-name
buffer-package
- buffer-pathname
buffer-radix
buffer-start
- buffer-stream
build-*DB-TOP-FN*-aux
build-*DB-TOP-FN*
- build-cmarker
build-fact
build-layers-list
- caar
cadr
calendar
- call-kaos-handler
car
case-action
- casify
cbarf
cbarf
- cdar
cdddddr
cddr
- cdr
cflow-fact?
cflow-splice-in
- cflow-splice-out
cgraph
chain-field
- change-focus-to
char-after
char-before
- char-peek
char-read
char-size-0
- char-size-1
charpos

ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- OUTIL
OUTIL
CONDITIONS
- CONDITIONS
STANDARD-ACTOR
DB
- UTIL
ZWEI-MAGIC
OUTIL
- EVENTS
BELIEF
EVENTS
- SYSLIB
ZWEI-MAGIC
REFER
- CODER
EDIT-ACTOR
ZWEI-MAGIC
- ZWEI-MAGIC
TV-MAGIC
SEGTRN
- ZWEI-MAGIC
BASIC
EDIT-COMMANDS
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
DB
DB
- DMAC
DMAC
DMAC
- PLI
PLI
ADA-EVAL
- KAOS
PLI
CODER
- PARSE
CONDITIONS
CONDITIONS
- PLI
MACRO
PLI
- PLI
DMAC
SEGTRN
- SEGTRN
DB
DEMO-ADA-LIB
- VERBS
ZWEI-MAGIC
ZWEI-MAGIC
- PARSE
PARSE
IMPLEMENT
- IMPLEMENT
DEMO-LIB

check-date
- check-for-implements1
check-for-implements
check-for-new-files
- check-isnt-init
check-name
check-sectionization
- choose-KBE-test
CLASSIFYQ
CLASSIFYQ
- CLASSIFY
CLASSIFY
clean-ada-cliche
- clean-ada-cond
clean-ada-defun
clean-ada-loop
- clean-ada-quote
clean-ada-return
clean-ada-setq
- clean-ada-~fn
clean-aggregate-file-defs
clean-dcl-class
- clean-dcl-info-fn
clean-dcl-info-setq
clean-dcl-info
- clean-dcl-sort
clean-dcl-var-type
clean-get-closes
- clean-get-dcls
clean-get-exceptions
clean-get-opens
- clean-get-params
clean-kill-call
clean-open-info
- clean-refers-to
clean-sort-dcls
clean-string-size
- clean-type-info
cleanup-ada-code
cleanup-ada-fn-call
- Clear-Display
clear-display
clear-marks
- clear
cliche-instance-action
cliche-name?
- cliche-name
cliche?
cmarker-is-in?
- cmarker?
cmarker
code->facts
- code->language
code->parsed
code->path-1
- code-and-display-it
code-char-matches
code-char?
- code-chars
code-equal
code-for-defun
- CODE-FOR
code-is-consistent?
code-is-consistent
- code-mode
code-pane
code-predicted
- code-ready
code-window
ADA-EVAL
- VERBS
VERBS
ADA-ANAL
- IMPLEMENT
REFER
ZWEI-MAGIC
- TEST
OUTIL
OUTIL
- OUTIL
OUTIL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- ADA-ANAL
ADA-ANAL
ADA-ANAL
- EDIT-COMMANDS
EVENTS
REFER
- DICT
IMPLEMENT
PSEUDO
- EXPLAIN
EXPLAIN
DMAC
- DMAC
DMAC
BRIDGE
- CONSISTENCY
BRIDGE
FOCUS
- VERBS
PSEUDO
PSEUDO
- PSEUDO
CONSISTENCY
DB
- CODER
CONSISTENCY
CONSISTENCY
- CODER
VIEW
PSEUDO
- EDIT-ACTOR
VIEW

code1
- coder-confirmation
code
coerce-parsed-form
- collect
com-backward-marked-KBE-role
com-change-this-expression
- com-compile-cliche-comments
com-document-KBE-completions
com-Document-KBE
- com-eval-and-compile-this-definition
com-evaluate-this-definition
com-forward-marked-KBE-role
- com-KBE-status
com-kill-KBE-requests
com-Make-buffer-consistent-for-KBE
- com-mark-this-definition
com-mode-break
com-name-this-expression
- com-not-implemented-for-this-mode
com-reset-apprentice
com-resume-KBE-edits
- com-select-apprentice
com-submit-and-kill-KBE-requests
com-toggle-kbe-debug-mode
- com-trim-KBE-reference
com-yank-KBE-bug-message
com-yank-last-several-KBE-requests
- com-yank-pop-KBE-request
command-key-names
command-key-name
- command-keys
command-menu-mixin
comment-char?
- comment-chars
comment-definition
Comment
- comment
compare-args
compare-code-1
- compare-code
compare-sexps-for-mark
compare-texts-for-mark
- compiler-add
Compile
complain
- complain
complete-current-token
compress-code-list
- compress-code
compress-facts
compress-fact
- compress-port
compress-segcase
computation-in
- computation-out
compute-offset
compute
- cond-k*expander-clause
cond-k*expander1
condition-restart-case-if
- condition-restart-case
cond
constant-handler?
- CONSTANT
CONSTANT
construct-DEFFACTS
- construct-DEFPLAN
construct-DEFSPECS
CODER
- EVENTS
CODER
PARSE
- MACRO
EDIT-MISC
EDIT-DEBUG
- EDIT-DEBUG
KBE-MODE
DOC
- ZWEI-MAGIC
ZWEI-MAGIC
EDIT-MISC
- EDIT-COMMANDS
EDIT-SIMPLE
EDIT-DEBUG
- ZWEI-MAGIC
ZWEI-MAGIC
EDIT-DEBUG
- ZWEI-MAGIC
EDIT-DEBUG
EDIT-COMMANDS
- EDIT-DEBUG
EDIT-SIMPLE
EDIT-DEBUG
- EDIT-MISC
EDIT-MISC
EDIT-DEBUG
- EDIT-SIMPLE
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
TV-MAGIC
PSEUDO
- PSEUDO
FOCUS
EDIT-COMMANDS
- VERBS
CONSISTENCY
CONSISTENCY
- CONSISTENCY
EDIT-ACTOR
EDIT-ACTOR
- GC
DICT
CONDITIONS
- CONDITIONS
ZWEI-MAGIC
DB
- DB
DB
DB
- DB
DB
ADA-PARSE
- ADA-PARSE
FOCUS
VERBS
- EXPAND
EXPAND
MACRO
- MACRO
PLI
EXPLAIN
- PBM-HANDLERS
PBM-HANDLERS
DB
- DB
DB

construct-plan-def
- construct-verb-bindings
cons
containing-buffer
- containing-tc
continuable-error-condition-mixin
copy-buffer-pointer
- copy-from-buffer-between
copyseg
copy
- copy
corr-code
corresponding-file-name
- corresponding-headings
corresponding-reporting
corresponding_summarized_file-stub
- could-possibly-be-op-exp
COUNT
create-buffer-pointer
- create-cliche-for-function
create-dummy-definition
create-plan-for-code1
- create-plan-for-code
create_test_records
cr
- current-editor-buffer
current-editor-definition-name
current-editor-path
- cursor-location
data-field
days-in-month
- db-audit
db-cons
db-context-is
- db-context-was
db-copylist
db-display-trace-info
- db-expunge
db-gc
DB-INDEX
- db-init-sym
db-init-vars
db-list*
- db-list
db-make-list
DB-NAMES
- db-ncons
db-property?
db-push
- db-reap-and-gc
db-reap
db-stats
- db-trace
db-untrace
db-xcons
- dbhunk-name
dbx
dcl-for
- de-hyphenate
debug-frame-system-key
debug-frame
- debug-options-window
debug-pane
debug
- decompose-cliche-comment
def-ada-cliche
def-ada-cliche
- def-ada-entry
def-ada-fn-internal
DB
- PSEUDO
MACRO
ZWEI-MAGIC
- IMPLEMENT
CONDITIONS
ZWEI-MAGIC
- ZWEI-MAGIC
DB
BASIC
- IMPLEMENT
REFER
DEMO-ADA-LIB
- DEMO-ADA-LIB
DEMO-ADA-LIB
DEMO-ADA-LIB
- ADA-PARSE
DEMO-LIB
ZWEI-MAGIC
- IMPLEMENT
ZWEI-MAGIC
VERBS
- VERBS
ADA-EVAL
MACRO
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- EDIT-COMMANDS
PARSE
ADA-EVAL
- DB
DMAC
DMAC
- DMAC
DMAC
DTRACE
- GC
GC
DMAC
- DMAC
DVARS
DMAC
- DMAC
DMAC
DMAC
- DMAC
DMAC
DMAC
- VERBS
GC
DDEBUG
- DTRACE
DTRACE
DMAC
- DMAC
DMAC
ADA-ANAL
- BASIC
DEBUG
DEBUG
- OPTIONS
DEBUG
DEBUG
- DEFLIB
ADA-UNPARSE
DEFLIB
- ADA-UNPARSE
ADA-EVAL

def-ada-fn	ADA-EVAL
- def-ada-fn	- ADA-UNPARSE
def-ada-generic	ADA-EVAL
def-ada-generic	ADA-UNPARSE
- def-ada-gformat	- ADA-UNPARSE
def-ada-gf	ADA-UNPARSE
def-ada-package-body	ADA-EVAL
- def-ada-package-body	- ADA-UNPARSE
def-ada-package	ADA-EVAL
def-ada-package	ADA-UNPARSE
- def-ada-text-cliche	- DEMO-ADA-LIB
def-ada-top-internal	ADA-EVAL
def-ada-top	ADA-EVAL
- def-ada-top	- ADA-UNPARSE
DEF-ADA-TOP	DEFLIB
def-ada	ADA-EVAL
- def-alistq	- MACRO
def-alist	MACRO
def-coercion	PARSE
- def-db-const	- DMAC
def-db-property	DMAC
def-db-struct	DMAC
- def-db-var	- DMAC
def-def-alist	MACRO
def-fact-predicate	DMAC
- def-lambda-like-k*expander	- EXPAND
def-language	PARSE
def-map-k*expander	EXPAND
- def-operator	- PARSE
def-over-init	OUTIL
def-over-return	OUTIL
- def-over-step	- OUTIL
def-pli-expression-format	PLI
def-pli-gformat	PLI
- def-rule	- ADA-PARSE
def-syntax	PARSE
default-delegate	STANDARD-ACTOR
- default-mark-post-hook	- EDIT-ACTOR
default-mark-pre-hook	EDIT-ACTOR
default-segcase	REFER
- default-vocabulary	- PSEUDO
default	IMPLEMENT
defdef-option	KMAC
- defdefmacro	- MACRO
defdefmacro	MACRO
deffacts	DB
- define-action	- PSEUDO
define-action	PSEUDO
define-additional-interpretation	PSEUDO
- define-additional-interpretation	- PSEUDO
define-adverb	PSEUDO
define-article	PSEUDO
- define-behavior	- BEHAVIOR
define-behavior	BEHAVIOR
define-condition	KMAC
- define-consolidated	- SURFACE
define-db-action	REFER
define-db-interpretation	REFER
- define-descriptor	- PSEUDO
define-editor-command	ZWEI-MAGIC
define-editor-definition-type	ZWEI-MAGIC
- define-editor-request	- EDIT-COMMANDS
define-event	BEHAVIOR
define-internal-option	KMAC
- define-interpretation	- PSEUDO
define-kaos-handler	KAOS
define-KBE-option	KMAC
- define-major-mode	- ZWEI-MAGIC
define-noise	MACRO

define-option-function
- define-option-line
define-ordinal
define-part-of-speech
- define-port-descriptor
define-preposition
define-pronoun
- define-setf-handler
define-synonyms
define-system-initialization
- define-trace-option
define-verb-action
define-verb-parse
- define-verb-unparse
define-verb
define-verb
- define-vocabulary
define-zwei-command
define
- define
definition-last-modified-time
definition-type-name
- defplan
defprototype
defspecs
- defun-locked
defun
del-layer
- delegate-mixin
delegate
delete-chars-in-set
- delete-from-buffer-between
deliver
demo-file-pathname
- demo-sequence
demo-unsetup
derived
- describe-reference
Describe
describe
- descriptor?
dest-is-term
DETAIL
- determine-ada-file-info
determine-record-enumerated
determine-ref-sharing
- DF-NAME
dffrom
dflow-fact?
- dflow-source
dflows-by-id-obj
digitp
- direct_io
disable-debug-mode
disable-interface
- Disable
disaggregate-file-def
discard-close-info
- discard-some-dcls
disp-dump
disp-print
- display-bug-message
display-options-sample
display-options
- display-pane
display-ready
display-report
- display
disp

OPTIONS
- OPTIONS
PSEUDO
PSEUDO
- PSEUDO
PSEUDO
PSEUDO
- MACRO
PSEUDO
KMAC
- KMAC
PSEUDO
PSEUDO
- PSEUDO
PSEUDO
PSEUDO
- PSEUDO
ZWEI-MAGIC
SURFACE
- VERBS
ZWEI-MAGIC
EDIT-COMMANDS
- DB
DEFLIB
DB
- MACRO
PLI
DMAC
- SOCIETY
INTERFACE-MACROS
ZWEI-MAGIC
- ZWEI-MAGIC
POST-OFFICE
TEST
- TEST
DEMO-LIB
IMPLEMENT
- VERBS
EDIT-COMMANDS
VERBS
- PSEUDO
EXPLAIN
PBM-HANDLERS
- ADA-ANAL
DEMO-ADA-LIB
VERBS
- PRIMOP
SEGTRN
DMAC
- VERBS
SEGTRN
PARSE
- ADA-EVAL
KMAC
KMAC
- DICT
ADA-ANAL
ADA-ANAL
- ADA-ANAL
UTIL
UTIL
- CONDITIONS
OPTIONS
OPTIONS
- SIMPLE
EDIT-ACTOR
DEMO-LIB
- VERBS
UTIL

dn
- do-editor-commands
do-society
doc-decode-code
- document-code
document-completions
document-editor-commands
- document-KBE-to-stream
document-KBE
document-mini-buffer-execute-command
- document-mini-buffer-submit-command
document-mini-buffer
document-one-editor-command
- document-reference
document
dr
- dsc-unparse1
dsc-unparse
dummy-ada-role
- dummy-port?
dump-plans
edit-KBE-options
- edit-mixin
edit-window-input-commands-mixin
edit-window
- editing-actor-secretary
editing-actor
editing-actor
- editing-input-pane
editor-awaiting?
editor-barf
- editor-barf
editor-buffer-get
editor-buffer-get
- editor-buffer-put
editor-command-name
editor-comment-begin
- editor-comment-end
editor-current-time
editor-delegate
- editor-echo-output
editor-insert-last-command-char
editor-last-command-char
- editor-main-output
editor-mark
editor-pane
- editor-point
editor-prompt-and-execute
editor-prompt-and-submit
- editor-state
editor-window-must-redisplay-rest-of-line
editor-window-redisplay
- employer-mixin
employer-vocabulary
enable-interface
- Enable
end-of-command
end-of-definition-comments
- end-of-line
enter-lisp-mode
entered-code-environment
- enum-lit
enum-lit
enumerate-flows-aux
- enumerate-flows
ENUMERATION
EQUALITY-WITHIN-EPSILON
- EQ
EQ
CODER
- ZWEI-MAGIC
SOCIETY
DOCUMENT
- EVENTS
ZWEI-MAGIC
ZWEI-MAGIC
- DOC
DOC
EDIT-COMMANDS
- EDIT-COMMANDS
EDIT-COMMANDS
ZWEI-MAGIC
- VERBS
VERBS
CODER
- GRAM
GRAM
ADA-ANAL
- IMPLEMENT
DB
KMAC
- EDIT-ACTOR
ZWEI-MAGIC
ZWEI-MAGIC
- EDIT-ACTOR
EDIT-ACTOR
EDIT-ACTOR
- SIMPLE
EDIT-ACTOR
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
EDIT-ACTOR
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
EDIT
- ZWEI-MAGIC
EDIT-COMMANDS
EDIT-COMMANDS
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- STANDARD-ACTOR
STANDARD-ACTOR
KMAC
- DICT
ACTOR
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
PSEUDO
- ADA-EVAL
ADA-UNPARSE
DB
- DB
KBELIB
KBELIB
- PBM-HANDLERS
PBM-HANDLERS

eq
- erase-facts
erase-fact
error-condition-mixin
- essential-actor
establish-region
eval-commands
- eval-command
eval
execute-stream-contents
- expand-code-list
expand-code
expand-define-cliche
- expand-facts
expand-fact
expand-port
- expand-segcase
explain-fn
explain-how-code-is-superseded
- explain-input1
explain-input
explain-output
- explain-parts
explain-part
explain-reference
- explain-termination
explain-wff
explain1
- Explain
explain
explain
- explanation
EXPR
expt
- extended-command-p
external-slot-name
f-by-k
- fact->parsed
fact->seg
fact-is-in?
- fact-key-dispatch
fact?
factdef
- facts-by-cm
facts-by-idx
facts-by-k
- FASTAND
FASTAND
FASTAND
- FASTOR
FASTOR
field0
- field1
field2
field3
- fieldk
fifth
file-default-fn2
- file-name-root
fill-buffer-between
filter-join-cor
- find-buffer
find-facts
find-input1
- find-inputs
find-input
find-path-1
- find-path
find-single-definition-node
PLI
- DB
DB
CONDITIONS
- STANDARD-ACTOR
ZWEI-MAGIC
VERBS
- REFER
VERBS
EDIT-COMMANDS
- DB
DB
DEFLIB
- DB
DB
DB
- DB
EXPLAIN
CONSISTENCY
- EXPLAIN
EXPLAIN
EXPLAIN
- EXPLAIN
EXPLAIN
VERBS
- EXPLAIN
EXPLAIN
EXPLAIN
- EDIT-COMMANDS
EXPLAIN
VERBS
- VERBS
PBM-HANDLERS
PLI
- ZWEI-MAGIC
CODER
DB
- BRIDGE
BRIDGE
DMAC
- DMAC
DMAC
DMAC
- DDEBUG
DDEBUG
DDEBUG
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
DMAC
- DMAC
DMAC
DMAC
- DMAC
DICT
MACRO
- IMPLEMENT
ZWEI-MAGIC
ANALYZE
- ZWEI-MAGIC
DB
EXPLAIN
- EXPLAIN
EXPLAIN
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC

find-single-definition
- find
finish-edit
Finish
- Finish
first-sexp-difference1
first-sexp-difference
- first-symbol
first-text-difference1
first-text-difference
- fix-if-KBE-command-name
fix-role-name-string
fix-up-semi-kbe-/#-comment-macros
- flow-fact?
flow-or-port-fact?
flush-chars
- fonts-any
fonts-large
fonts-medium
- fonts-small
fonts
force-db-consistency
- force-to-final-value
format-explanation-for-comment
format-explanation
- format-KBE-test
format-test-file-header
format-test-result
- format-test-summary
format-trace
format:m
- forward-char
forward-db-properties
forward-db-value
- forward-defun
forward-down-list
forward-line
- forward-sexp
forward-word
fourth
- free-db-area
free-variable
from
- fset-
fset<
fset=
- fsetmem
fsetU
fset^
- full-actor-name?
funcall-in-editor
function-name->parsed
- functions
fuse-joins
f
- g2move-cflows
g2move-dflows
g2move-tports
- garden-variety-fn-apply
genatom-not-in
gencase
- GENERATION
genobj
genseg
- get-ada-file-info
get-ada-function-spec
get-appropriate-definition
- get-cliche-role-tree
get-cliche-roles
ZWEI-MAGIC
- EXPLAIN
VERBS
EDIT-COMMANDS
- VERBS
EDIT-ACTOR
EDIT-ACTOR
- ADA-ANAL
EDIT-ACTOR
EDIT-ACTOR
- ZWEI-MAGIC
ADA-PARSE
COMMENT-MACRO
- DMAC
DMAC
ZWEI-MAGIC
- OPTIONS
OPTIONS
OPTIONS
- OPTIONS
OPTIONS
VERBS
- IMPLEMENT
FOCUS
FOCUS
- TEST
TEST
TEST
- TEST
KMAC
MOUSE-SENSITIVE-REGIONS
- ZWEI-MAGIC
GC
GC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
DICT
- GC
DICT
EXPLAIN
- SETS
SETS
SETS
- SETS
SETS
SETS
- ACTOR
ZWEI-MAGIC
VERBS
- ADA-EVAL
SEGTRN
DB
- SEGTRN
SEGTRN
SEGTRN
- EXPAND
DB
DB
- KBELIB
DB
DB
- ADA-ANAL
ZWEI-MAGIC
EDIT-ACTOR
- DEFLIB
DEFLIB

get-code-for-kbe-comment
- get-completion-from-range
get-completion-from-table
get-current-token-completion
- get-db-area
get-def-name
get-definition-text
- get-definition
get-name-from-seg
get-possibly-cached-definition
- get-predefined-type-len
get-prefix
get-segcase
- get-seg
get-toplevel-name
get-toplevel-seg
- going-to-submit
go
graph
- greaterp
grind-code-for-buffer
grind-code
- group2
group
grumble
- hack-apprentice
handle-&in-out
handle-format
- handle-kaos-error
handle-new-file
handler-format
- handles?
handle
has-plan-type
- has-slot
here
how
- how
idx-port
idx-seg
- if
ignorable-by-setf?
imbedded-references-1
- imbedded-references
immediate-command?
implement-dflow
- implement-in-tc
implement-internal0
implement-internal
- implement-like
implement-seg-as-port-dflow
implement-seg-as-port
- implement-seg
implemented
Implement
- implement
in-chars?
in-dictionary?
- in-layers
in-order?
in-order
- inbox
indent-this-line-for-lisp
index-fact
- infer-definition-type
init-ada-files
init-noise-for
- initialize-editor-window
initialize-system

COMMENT-MACRO
- COMPLETE
COMPLETE
ZWEI-MAGIC
- GC
ADA-PARSE
ZWEI-MAGIC
- ZWEI-MAGIC
BRIDGE
ZWEI-MAGIC
- ADA-ANAL
BASIC
BRIDGE
- DB
BRIDGE
BRIDGE
- EDIT-COMMANDS
PLI
DB
- PLI
ZWEI-MAGIC
VERBS
- SEGTRN
SEGTRN
BASIC
- OPTIONS
COMMENT-MACRO
EXPLAIN
- KAOS
ADA-ANAL
EXPLAIN
- BASIC
EXPLAIN
EXPLAIN
- EXPLAIN
DICT
DICT
- VERBS
DB
DB
- PLI
MACRO
EDIT-COMMANDS
- EDIT-COMMANDS
EDIT-COMMANDS
IMPLEMENT
- IMPLEMENT
IMPLEMENT
IMPLEMENT
- IMPLEMENT
IMPLEMENT
IMPLEMENT
- IMPLEMENT
DICT
EDIT-COMMANDS
- IMPLEMENT
PSEUDO
PSEUDO
- DMAC
DB
DB
- MBOX
ZWEI-MAGIC
DB
- ZWEI-MAGIC
ADA-ANAL
NOISE
- ZWEI-MAGIC
KMAC

init
- inline-lambda-k*expander
input-fact?
input-port-fact?
- input
insert-comment-definition
insert-into-buffer-after-point
- insert-into-buffer-after
insert-into-buffer-before-point
insert-into-buffer-before
- int-lit
int-lit
intern-KBE
- intern-keyword
internal-condition-mixin
internal-condition
- internal-continuable-error
internal-error
internal-warning
- interval-stream
invalidate-new-emarkers
in
- io-fact?
io-or-@-fact?
irritating-noise
- is-&in-out
it
it
- kaos-handler
kaos-lexpr-send
kaos-prompt-for
- kaos-prompt
kaos-send-returning-list
kaos-send
- kaos-server-test
kaos-server
kbe-#-comment-macro
- kbe-/#-comment-complaint
kbe-/#-comment-length
kbe-/#-comment-macro
- kbe-/#-comment-macro
kbe-/#-comment-reader
kbe-/{-reader
- kbe-1
kbe-comment-code
kbe-comment-name->string
- kbe-comment-name
kbe-comment-needs-implementation?
kbe-comment-type
- kbe-mode
KBE-obsolete-loop-macro
kbe-version-string-for-pathname
- kbe
keyword-name
kill-buffer
- kill-port-and-commentary
kill-port
kill-seg-and-its-subsegs
- kill-seg
kill-shared-seg
l-env-sort
- lambda
language-morpheme
language-structure
- last-*pread
last-editor-command-type
last
- LEADING-BOUNDARY-DETECTOR
led

DEBUG
- EXPAND
DMAC
DMAC
- DICT
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ADA-EVAL
ADA-UNPARSE
BASIC
- BASIC
CONDITIONS
CONDITIONS
- CONDITIONS
CONDITIONS
CONDITIONS
- ZWEI-MAGIC
DB
DICT
- DMAC
DMAC
INIT
- PRIMOP
DICT
REFER
- KAOS
KAOS
KAOS
- KAOS
KAOS
KAOS
- KAOS
KAOS
PLI
- COMMENT-MACRO
COMMENT-MACRO
ADA-UNPARSE
- COMMENT-MACRO
COMMENT-MACRO
COMMENT-MACRO
- DEBUG
COMMENT-MACRO
COMMENT-MACRO
- COMMENT-MACRO
COMMENT-MACRO
COMMENT-MACRO
- ZWEI-MAGIC
OUTIL
SYSTEM-MAGIC
- DEBUG
DMAC
ZWEI-MAGIC
- DB
DB
SEGTRN
- DB
VERBS
CODER
- EXPAND
PARSE
PARSE
- UTIL
ZWEI-MAGIC
DICT
- DEMO-LIB
PARSE

lessp
- lexpr-funcall-in-editor
lexpr-send-all-that-handle
lexpr-send-all
- lexpr-send-any-superior-if-handles
lexpr-send-any-superior
lexpr-send-any-that-handles
- lexpr-send-any
lexpr-send-delegate
lexpr-send-if-handles
- line-size-0
line-size-1
line-truncating-mixin
- LINEAR-SEARCH
lisp-cursor-location
LISP
- LISP
list*
LIST-ENUMERATION
- LIST-GENERATION
list-noises
listener-pane
- list
LIST
LIST
- load-dump
Load
locate
- longest-line-length
loop-info
lower-rets
- ls-numeric-arg
macroexpand-1-no-substs
macroexpand-no-substs
- mailbox
maintenance_files
make-a-phrase
- make-actor-full-name
make-ada-fn
make-be-it
- make-be-marked
make-be-this
make-buffer-current
- make-cliche-definition
Make-Cliche
make-code-consistent?
- make-compatable-cflow
make-dbhunk
make-defdefmacro-source-file-wrapper
- make-definition-internal
Make-Defun
make-dummy-definition
- make-external-name-list
make-fact-be-in
make-fact-be-out
- make-field-printer
make-home-suggestion
make-interned-cmarker
- make-kbe-/#-comment
make-message
make-new-cliches
- make-noise-about
make-print-stuff
make-program-definition
- make-ref-extra
make-ref-input
make-ref-output
- make-ref-seg
make-ref-slot
PLI
- ZWEI-MAGIC
SOCIETY
SOCIETY
- BASIC
BASIC
SOCIETY
- SOCIETY
BEHAVIOR
BASIC
- IMPLEMENT
IMPLEMENT
TV-MAGIC
- KBELIB
EDIT-COMMANDS
PARSE
- PARSE
MACRO
KBELIB
- KBELIB
NOISE
SIMPLE
- MACRO
OUTIL
OUTIL
- DB
DICT
REFER
- FOCUS
CODER
CODER
- IMPLEMENT
EXPAND
EXPAND
- MBOX
ADA-EVAL
DOCUMENT
- ACTOR
ADA-UNPARSE
REFER
- REFER
REFER
ZWEI-MAGIC
- VERBS
EDIT-COMMANDS
CONSISTENCY
- SEGTRN
DMAC
BASIC
- VERBS
EDIT-COMMANDS
ZWEI-MAGIC
- CODER
DB
DB
- ADA-ANAL
VERBS
DB
- CODER
MBOX
ADA-ANAL
- NOISE
ADA-ANAL
VERBS
- REFER
REFER
REFER
- REFER
REFER

make-ref-tinput
- make-ref-toutput
make-ref
make-syntax-table
- make-termination-wff
make-time
MAKEFSET
- MAKEFSET
makefset
makenset
- MAKEQSET
MAKEQSET
makeqset
- MAKESET
MAKESET
mapcan
- mapcar
mapcon
mapc
- maplist
map
MAP
- margin-scroll-bar-mixin
mark-check-point
mark-sexp-from-offsets
- mark-sexp-from-path
match-completion
match-pathnames
- match-segcase
matches-fact?
maybe-display-options-sample
- maybe-intern
maybe-parse-command
maybe-send-command
- memoizeq
memoize
mempat
- mini-analyze
mini-buffer-stream
mouse-sensitive-output
- mouse-sensitive-princ
mouse-sensitive-regions-mixin-filter
mouse-sensitive-regions-pane
- mouse-sensitive-regions-window
move-blinker
move-mark
- move-point-to-top-of-window
move-point
move-seg-down
- move-seg-up
move-to-new-db-area
move-to-next-KBE-role
- Move-To
moved-out
move
- mpred-roles
mydelete-plan-file
my
- name->segcase
name->seg
name-symbol
- name-type
named-lambda
named-subst
- nametran
NCONC
NCONC
- ncons
new-cmarker
REFER
- REFER
REFER
PARSE
- EXPLAIN
ADA-EVAL
OUTIL
- OUTIL
SETS
SETS
- OUTIL
OUTIL
SETS
- OUTIL
OUTIL
EXPAND
- EXPAND
EXPAND
EXPAND
- EXPAND
EXPAND
SYSLIB
- TV-MAGIC
ADA-PARSE
FOCUS
- FOCUS
COMPLETE
TEST
- DB
DB
OPTIONS
- BASIC
TEST
EDIT-COMMANDS
- MACRO
MACRO
DB
- CODER
ZWEI-MAGIC
MOUSE-SENSITIVE-REGIONS
- ZWEI-MAGIC
MOUSE-SENSITIVE-REGIONS
MOUSE-SENSITIVE-REGIONS
- MOUSE-SENSITIVE-REGIONS
OVERLAPPING-REGIONS
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
SEGTRN
- SEGTRN
GC
EDIT-MISC
- EDIT-COMMANDS
CODER
VERBS
- CODER
IMPLEMENT
MACRO
- BRIDGE
BRIDGE
REFER
- REFER
EXPAND
EXPAND
- DB
OUTIL
OUTIL
- MACRO
DMAC

new-style-kbe-/#-comment-macro-output
- new-style-kbe-/#-comment-macro-print
NEW-VIEW
NEW-VIEW
- newmap
newportmap
NLIST
- NLIST
NLIST
no-dflow-from
- NO-DFLOW-MOVEMENT
NO-GROUP-IN-TC
noise-for
- non-ada-format
non-temporal
non-terminal-in
- non-terminal-out
non-terminal-root
not-fully-specified
- not-implemented?
not
nset-
- nset<
nset=
nsetmem
- nsetU
nset^
nud
- null-sentence
num-lines
objtran
- odd-KBE-options
odd-options-1
odd-options
- of
OK-to-append-delimiter-directly?
old-style-kbe-/#-comment-macro-output
- old-style-kbe-/#-comment-macro-print
one-of-in1
one-of-in
- one-of-out
optional-in
optional-out
- options-edit-all
options-pane
options-toggle-debug
- options
order-things1
order-things
- ordered-by-references
ordinal-pos
ordinal-select
- ordinal-sort
ordinal-unparse
OR
- OR
or
out-layers
- output-fact?
output-port-fact?
output
- over-in-facts
overlapping-regions-pane
overlapping-regions-window
- over
pairup
parameter
- parse-actor-full-name
parse-actor
COMMENT-MACRO
- PLI
PBM-HANDLERS
PBM-HANDLERS
- ANALYZE
ANALYZE
OUTIL
- OUTIL
OUTIL
DB
- ANALYZE
ANALYZE
NOISE
- ADA-UNPARSE
DICT
ADA-PARSE
- ADA-PARSE
ADA-PARSE
ADA-ANAL
- VERBS
PLI
SETS
- SETS
SETS
SETS
- SETS
SETS
PARSE
- PSEUDO
EDIT-ACTOR
SEGTRN
- KMAC
BASIC
BASIC
- DICT
GRAM
COMMENT-MACRO
- PLI
ADA-PARSE
ADA-PARSE
- ADA-PARSE
ADA-PARSE
ADA-PARSE
- OPTIONS
OPTIONS
OPTIONS
- OPTIONS
REFER
REFER
- ADA-ANAL
REFER
REFER
- REFER
GRAM
OUTIL
- OUTIL
PLI
DMAC
- DMAC
DMAC
DICT
- DMAC
OVERLAPPING-REGIONS
OVERLAPPING-REGIONS
- OUTIL
DEFLIB
DICT
- ACTOR
ACTOR

parse-argument-list	BASIC
- parse-command	- PSEUDO
parse-comp1	ADA-PARSE
parse-definite-reference	PSEUDO
- parse-definition-spec	- PSEUDO
parse-descriptor	PSEUDO
parse-error	PARSE
- parse-error	- PARSE
parse-indefinite-reference	PSEUDO
parse-infix	PARSE
- parse-led-lparen	- PARSE
parse-left	PARSE
parse-matchfix-internal	PARSE
- parse-matchfix	- PARSE
parse-name	PSEUDO
parse-nary-internal	PARSE
- parse-nary	- PARSE
parse-nofix	PARSE
parse-noop	PARSE
- parse-nud-lparen	- PARSE
parse-optional-cliche-name	PSEUDO
parse-ordinal	PSEUDO
- parse-PL/1-=	- PARSE
parse-PL/1-do-internal	PARSE
parse-PL/1-do	PARSE
- parse-PL/1-end	- PARSE
parse-PL/1-if	PARSE
parse-PL/1-pointer	PARSE
- parse-PL/1-statement	- PARSE
parse-plural	PSEUDO
parse-port-descriptor	PSEUDO
- parse-port-reference-minus-the	- PSEUDO
parse-position	PSEUDO
parse-postfix	PARSE
- parse-prefix	- PARSE
parse-pronoun-reference	PSEUDO
parse-reference	PSEUDO
- parse-relative-reference	- PSEUDO
parse-remote-actor	REMOTE
parse-request	DEBUG
- parse-roles-or-parameters-internal	- PSEUDO
parse-roles-or-parameters	PSEUDO
parse-root-or-relative-reference	PSEUDO
- parse-root-reference	- PSEUDO
parse-seg-reference	PSEUDO
parse-sentence	PSEUDO
- parse-the-reference	- PSEUDO
parse-token-class	ADA-PARSE
parse-token-type	ADA-PARSE
- parse-top-reference	- PSEUDO
parse-toplevel-reference	PSEUDO
parse-variable-name	PSEUDO
- parsed->code	- BRIDGE
parsed->facts	BRIDGE
parser-internal	PARSE
- parser	- PARSE
parse	PARSE
part-marker?	COMMENT-MACRO
- partial-order-sort	- CODER
parts	DFLOW
pas	DB
- path->code-1	- FOCUS
path->code	FOCUS
path->string	FOCUS
- peek-token	- PSEUDO
peval	UTIL
phrase-append	ENGLISH
- PL/1	- PARSE
PL/1	PARSE

pl?
- place-to-create-dummy-definition
place-to-create-dummy-definition
plan-def
- plan-fact?
plan-for-source
plan-top
- plan-type
pleasant-noise-1
pleasant-noise-2
- pli-bind
pli-cond-clause
pli-cond-rest
- pli-defun-body
pli-expression-format-bind
pli-fn-format
- pli-gd2
pli-gd
pli-guess-decls
- pli-is-s1
pli-is-subroutine
pli-nil-format
- pli-prepare-for-setup
pli-prog-sexpr
pli-progn
- pli-setup
pli-statement?
pli-statement
- pli-subroutine
pli-t-format
pli-test-nil
- pli-test-t
pli-top-loop
pli-top-prog
- pli-unsetup
pluralize
point-precedes?
- pop-db-context
port-descriptor?
port-fact?
- port-facts
port-reference-adj-parse
port-reference1
- porttrans
port
possible-completions
- possible-embedded-code
pp-in
pp-out
- pp-trace
pp
pread
- pred-action-cor
predicate-6fexpr
pred
- PRED
PRED
presentational-focus
- PREVIOUS-VALUE
print-as-pli
print-dictionary-internal
- print-dictionary
PRINT-OUT
print-test
- process-1
process-comment-string
process-pattern-in
- process-pattern-out
process-rule-body
REFER
- ZWEI-MAGIC
ZWEI-MAGIC
DB
- DMAC
IMPLEMENT
BRIDGE
- EXPLAIN
INIT
INIT
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
PLI
PLI
- PLI
IMPLEMENT
ZWEI-MAGIC
- DB
PSEUDO
DMAC
- DB
REFER
REFER
- ANALYZE
DICT
COMPLETE
- DOCUMENT
ADA-PARSE
ADA-PARSE
- KMAC
DB
UTIL
- ANALYZE
PRIMOP
DB
- PBM-HANDLERS
PBM-HANDLERS
EVENTS
- DEMO-LIB
PLI
PSEUDO
- PSEUDO
DEMO-LIB
UTIL
- UTIL
DOCUMENT
ADA-PARSE
- ADA-PARSE
ADA-PARSE

process-toplevel
- PRODUCT
progn
program-name?
- prompt-and-read-command
pronoun?
property-list-mixin
- punctuation?
push-db-context
push-possibilities-buffer
- put-last
pwp
p
- QF
QPAS
QPP
- QPWP
QP
qset-
- qset<
qset=
qsetU
- qset^
QS
rdb
- read-code
read-command-prompt
read-commands-from-stream-contents
- read-commands-from-string
read-command
read-definition-name
- read-forms-from-stream
read-hacking-/{
read-hacking-dots
- read-kbe-/#-stuff
read-literal-string
read-matched-code
- read-pathname
read-to-matching-/}
read-token-internal
- read-token
readstring
real-editor-buffer?
- really-get-definition
really-read-token
really-scan-word
- Reap-Database
Reap
record_key_root-stub
- recursion-check
recursion-stack
redisplay-all-windows
- refered-to-tc
reference-unparse
region-stream
- REGIONS
reimplement
Reinitialize-Database
- reinitialize
relative-reference1
released-system-version-active?
- remark
remark
remark
- remember-db-context
remote-service-notify
remove-bucky-bits
- remove-markers
remove-misplaced-whitespace

TV-MAGIC
- KBELIB
PLI
PSEUDO
- PSEUDO
PSEUDO
FLAVOR-MAGIC
- PSEUDO
DB
ZWEI-MAGIC
- BASIC
DB
DB
- DVARS
DVARS
DVARS
- DVARS
DVARS
SETS
- SETS
SETS
SETS
- SETS
DVARS
UTIL
- PSEUDO
PSEUDO
EDIT-COMMANDS
- EDIT-COMMANDS
PSEUDO
ZWEI-MAGIC
- TEST
COMMENT-MACRO
PSEUDO
- COMMENT-MACRO
DEMO-ADA-LIB
PSEUDO
- ZWEI-MAGIC
COMMENT-MACRO
PSEUDO
- PSEUDO
BASIC
ZWEI-MAGIC
- ZWEI-MAGIC
PSEUDO
PSEUDO
- EDIT-COMMANDS
VERBS
DEMO-ADA-LIB
- EXPLAIN
EXPLAIN
ZWEI-MAGIC
- IMPLEMENT
GRAM
ZWEI-MAGIC
- CODER
IMPLEMENT
EDIT-COMMANDS
- VERBS
REFER
SYSTEM-MAGIC
- DICT
EVENTS
EVENTS
- DMAC
REMOTE
BASIC
- VERBS
FOCUS

remove-reference
- Remove
Remove
rename-all
- repetition-in
repetition-out
replace-definition
- replace-or-create-definition
reply
request-command-key
- request-display
require-mouse-confirmation
require-one-of
- reset-apprentice
resolve-language
rest-of-buffer-stream
- restore-db-context
resume-KBE-edits
return-if-not
- return-if
return
reverse-order-by-dflow
- REVLIST
REVLIST
role-args
- role-name?
roles
role
- rplaca
rplacd
run-KBE-test
- run-test
s-do-macro
s-old-style-do-macro
- same-fact-key?
scan-word
search-buffer-backward-delimited
- search-buffer-backward
search-buffer-between
search-buffer-forward-delimited
- search-buffer-forward
search-buffer
seg-facts
- segment-of
sel1+
select-a-or-an
- select-apprentice
select-is-or-are
SELECT
- semantics
send-all-that-handle
send-all
- send-any-superior-if-handles
send-any-superior
send-any-that-handles
- send-any
send-delegate
send-if-handles
- send-mail
send-stream-contents
sentence-terminator?
- sentence-terminators
sequence-in
sequence-out
- sequential_io
set-bucky-bits
set-buffer-attribute
- set-buffer-mode
set-cursor-for-ada
VERBS
- EDIT-COMMANDS
VERBS
IMPLEMENT
- ADA-PARSE
ADA-PARSE
ZWEI-MAGIC
- CONSISTENCY
INTERFACE-MACROS
EDIT-COMMANDS
- VERBS
TV-MAGIC
PSEUDO
- STANDARD-ACTOR
VERBS
ZWEI-MAGIC
- DB
EDIT-COMMANDS
MACRO
- MACRO
PLI
IMPLEMENT
- OUTIL
OUTIL
PLI
- PSEUDO
CODER
DICT
- PLI
PLI
TEST
- UTIL
EXPAND
EXPAND
- DMAC
PSEUDO
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
ZWEI-MAGIC
- ZWEI-MAGIC
ZWEI-MAGIC
DB
- EXPLAIN
DUTIL
ENGLISH
- STANDARD-ACTOR
ENGLISH
KBELIB
- PARSE
SOCIETY
SOCIETY
- BASIC
BASIC
SOCIETY
- SOCIETY
BEHAVIOR
BASIC
- ZWEI-MAGIC
EDIT-COMMANDS
PSEUDO
- PSEUDO
ADA-PARSE
ADA-PARSE
- ADA-EVAL
BASIC
ZWEI-MAGIC
- ZWEI-MAGIC
EDIT-ACTOR

set-cursor-for-lisp	EDIT-ACTOR
- set-db-context	- DB
set-editor-command-type	ZWEI-MAGIC
set-editor-list-syntax-from-character	ZWEI-MAGIC
- set-mode-line	- ZWEI-MAGIC
set-noise-for	NOISE
set-switch	KMAC
- set-up-db-names	- DB
set-up-morpheme-structure	PARSE
set-	SETS
- set<	- SETS
setq-param	ADA-EVAL
setq	PLI
- setup-editor	- EDIT-ACTOR
setup-Gprint	INIT
setup-plural	PSEUDO
- setup-pp	- INIT
setU	SETS
set^	SETS
- seventh	- DICT
share-dests	VERBS
share-references	VERBS
- share0	- VERBS
shared-expr	VERBS
Share	EDIT-COMMANDS
- share	- VERBS
side-effect	DICT
simple-buffer-insert	EDIT-COMMANDS
- simple-command-key	- EDIT-COMMANDS
simple-condition-mixin	CONDITIONS
SIMPLE-EXPRS	PRIMOP
- simple-funcall-or-constant	- VERBS
simple-pane-mixin	TV-MAGIC
SIMPLE-REPORT	DEMO-LIB
- simple-request?	- ACTOR
simple-request?	ACTOR
simple-subseg	SYSLIB
- singular-form	- PSEUDO
site-string	SYSTEM-MAGIC
sixth	DICT
- size-in-characters	- IMPLEMENT
size-in-lines	IMPLEMENT
slot-fact?	DMAC
- slot-marker?	- BRIDGE
slot-name	EXPLAIN
snarf	UTIL
- society-member-mixin	- STANDARD-ACTOR
sole-fed-input?	IMPLEMENT
sole-used-output?	IMPLEMENT
- sort-facts	- DB
sort-key-facts	DB
specs-for-new-ada-fn	ADA-EVAL
- splice-code-at-path-1	- VERBS
splice-code-at-path	VERBS
SPLICE-IN	KBELIB
- SPLICE-OUT	- KBELIB
SQUARE	KBELIB
ssr-roles	ANALYZE
- ssr	- DB
standard-actor	STANDARD-ACTOR
status-pane	SIMPLE
- str-lit	- ADA-EVAL
str-lit	ADA-UNPARSE
stream-with-charpos	DEMO-LIB
- string->kbe-comment-name	- COMMENT-MACRO
string-buffer-between	ZWEI-MAGIC
string-capitalize	BASIC
- string-copy	- BASIC
string-for-command	EDIT-COMMANDS

string-for-comment
- string-parser
stuff-or-nil
sub-gather
- SUBLIST-ENUMERATION
submit-stream-contents-to-KBE
submit-stream-contents
- subseg*
subst
suggest-code-home
- suggest-focus
suggest-variable-name
suggested-code-home
- suggested-focus
SUM
SUM
- SUM
SUM
superseg
- symbol-pluralize
symbolconc
symbolize-scanned-word
- syntax-definition
syntax-for-definition
syntax
- system-keys
system-version-string
s
- TABULARIZED-PRINT-OUT
TAGS
task
- tc-sorted-ssrs
tcomp-prototype
td-copy-input-dflow
- td-copy-output-dflow
td-find-dominant
td-maybe-has-become-simple-seg
- td-maybe-should-be-split-seg
td-move-segs
td-move-toutputs
- tell
tell
tempcomp
- temporal-combine
temporal-compose
temporal-decompose
- temporal
term-join-cor
terminal-in
- terminal-out
test-file-pathname
test-KBE-1
- test-KBE
test-parser
Test-Yourself
- test
text_io
that
- that
the-adj-parse
the
- thing-type
third
this-definition-name
- this
this
tinput-fact?
- tio-fact?
token-peek-2

FOCUS
- PARSE
ADA-PARSE
ADA-PARSE
- KBELIB
EDIT-COMMANDS
EDIT-COMMANDS
- EXPLAIN
EXPAND
EVENTS
- EVENTS
DEMO-ADA-LIB
EVENTS
- EVENTS
KBELIB
OUTIL
- OUTIL
OUTIL
OUTIL
BRIDGE
- BASIC
BASIC
PSEUDO
- PARSE
FOCUS
PARSE
- INIT
SYSTEM-MAGIC
DB
- DEMO-LIB
PRIMOP
EVENTS
- CODER
CODER
ANALYZE
- ANALYZE
ANALYZE
ANALYZE
- CODER
ANALYZE
ANALYZE
- DEBUG
INTERFACE-MACROS
DB
- CODER
CODER
ANALYZE
- DICT
ANALYZE
ADA-PARSE
- ADA-PARSE
TEST
TEST
- TEST
PSEUDO
EDIT-COMMANDS
- TEST
ADA-EVAL
DICT
- REFER
REFER
DICT
- REFER
DICT
EDIT-COMMANDS
- DICT
REFER
DMAC
- DMAC
PARSE

token-peek
- token-read
token-string
tokenize-dash
- tokenize-number-internal
tokenize-number
tokenize-single-char
- tokenize-stars
tokenize-symbol
tokenize
- top-segcase
top
total-size
- toutput-fact?
to
TRAILING-POINTER-LIST-ENUMERATION
- TRAILING-POINTER-SUBLIST-ENUMERATION
trim-cursor-location
trim-KBE-reference-internal
- trim-KBE-reference
type-field
typed-parse
- typeout
typeout
typeout
- u1-flatten-like-ops
u1-make-and
u1-make-body
- u1-make-fn
u1-make-lambda
u1-make-or
- u1-make-progn
u1-p-c-cgen
u1-p-c-check
- u1-p-cgen
u1-p-e-cgen
u1-p-e-check
- u1-p-l-check
u1findelligablesetq
u1oversubst
- u1overusedin
u1subst
ugmove-cflow
- ugmove-dflow
ugmove-slots-down
ugmove-slots-up
- ugmove-tports
undo-last-edit
Undo
- undo
ungroup-middle
ungroup
- unimplemented
unparse-parameters-or-roles
unread-token
- update-zmacs-mode-line
uppercase-p
upport
- use-language-internal
Use-Language
user-condition-mixin
- user-error
user-warning
use
- utility-init-db
utility-test
utility
- validate-buffer-mode
validate-buffer-package

PARSE
- PARSE
PSEUDO
PARSE
- PARSE
PARSE
PARSE
- PARSE
PARSE
PARSE
- VERBS
MACRO
IMPLEMENT
- DMAC
EXPLAIN
KBELIB
- KBELIB
EDIT-COMMANDS
EDIT-MISC
- EDIT-MISC
PARSE
PARSE
- DICT
EVENTS
EVENTS
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
OUTIL
- OUTIL
OUTIL
SEGTRN
- SEGTRN
SEGTRN
SEGTRN
- SEGTRN
VERBS
EDIT-COMMANDS
- VERBS
SEGTRN
SEGTRN
- DICT
VERBS
PSEUDO
- ZWEI-MAGIC
ZWEI-MAGIC
ANALYZE
- VERBS
EDIT-COMMANDS
CONDITIONS
- CONDITIONS
CONDITIONS
VERBS
- OPTIONS
OPTIONS
OPTIONS
- ZWEI-MAGIC
ZWEI-MAGIC

validate-zmacs-buffers-data
- validate-zmacs-buffers-datum
value
vanilla-editor-frame
- VECTOR-ENUMERATION
View
view
- visible-cursor-location-1
visible-cursor-location
vocabulary
- warning-condition-mixin
warn
What-Needs-To-Be-Done
- what
when-abnormal-exit
when-command-setup-needed-for
- when-error
when-interface-enabled
when-pop-fails
- when
whitespace-char?
whitespace-chars
- wildcard-fact?
window-delegate
with-automatic-error-recovery-if
- with-automatic-error-recovery
with-buffer-pointer
with-consistent-code
- with-current-editor-definition-name
with-db-locked
with-defaults
- with-delegate
with-fresh-token-buffer
with-interface-disabled
- with-kbe-package
with-local-buffer-pointer
with-lock
- with-mouse-sensitive-region
with-one-open-file
with-open-files
- with-reply-to
with-trace
without-end-of-command
- word-break?
word-separator?
word-separators
- xmailbox
xor
xplan-test-copy
- xplan-test-implement
xplan-test-setup
xplan-test1
- xrole-fillers
yank-into-buffer-at-point
yank-into-buffer-at
- zippy-noise-down
zippy-noise-internal
zippy-noise-up
- zmacs-mode-line
zwei:ada-mode
^dflow-from
- ^dflow-to
^
~01
- ~1st
~case
~id-port
- ~nset
~seg~deep

ZWEI-MAGIC
- ZWEI-MAGIC
DICT
ZWEI-MAGIC
- KBELIB
EDIT-COMMANDS
VERBS
- EDIT-COMMANDS
EDIT-COMMANDS
PSEUDO
- CONDITIONS
CONDITIONS
EDIT-COMMANDS
- VERBS
MACRO
EDIT-COMMANDS
- MACRO
KMAC
MACRO
- PLI
PSEUDO
PSEUDO
- DMAC
BEHAVIOR
KMAC
- KMAC
ZWEI-MAGIC
CONSISTENCY
- ZWEI-MAGIC
DMAC
MACRO
- INTERFACE-MACROS
PSEUDO
KMAC
- INTERFACE-MACROS
ZWEI-MAGIC
MACRO
- MOUSE-SENSITIVE-REGIONS
DEMO-LIB
DEMO-LIB
- INTERFACE-MACROS
KMAC
VERBS
- PSEUDO
PSEUDO
PSEUDO
- MBOX
DB
IMPLEMENT
- IMPLEMENT
IMPLEMENT
IMPLEMENT
- ANALYZE
ZWEI-MAGIC
ZWEI-MAGIC
- INIT
INIT
INIT
- ZWEI-MAGIC
ZWEI-MAGIC
SEGTRN
- SEGTRN
PLI
DMAC
- DMAC
DMAC
DB
- DMAC
DMAC

~seg~roleseg	DMAC
- ~seg~segcase	- DMAC
~seg~subseg	DMAC
~seg	DMAC
- ~set	- DMAC

```
;;; -*-Mode:LISP; Fonts:MEDFNB; -*-

;;; To use this init file, just do (LOGIN 'KBE 'OZ).
;;; You will be prompted with questions about how much of the system
;;; to load, whether to compile things, whether to ask for confirmations,
;;; etc. If you don't select confirmations, reasonable things will
;;; happen based on the answers to the other questions.

;;; Disable incoming net traffic

(si:disable-services)
(format t "~&(Chaos servers disabled)~%")

;;; Load definitions of component packages/systems.
;;;
;;; Defines the KBE package, KBE logical device, and KBE system.
;;; This must be done before the next s-expression can be read,
;;; since it can contain references to KBE: ...

(load "PS:<KBE>SYSDEF")

;;; Support files

(let-globally ((si:gc-report-stream nil))
 (let ((old-more-state (send terminal-io ':more-p)))
  (unwind-protect
   (let* ((full? (fquery '(:choices
                           (((T "Yes.") #/Y #\Space)
                            ((NIL "No.") #/N #\Rubout)
                            ((:COMPILE "Compile and load system.") #/C)
                            ((:LOAD "Load system.") #/L)
                            ((:DUMP "Set up to dump system.") #/D))
                           :help-function
                           (lambda (stream choices ignore)
                             (format stream
                                     "~{~& ~1{~1G~C: ~0G~1{~1G~A~}~}~}"
                                     choices)))
                         "~&Set up full KBE environment? "))
          (confirm?      (selectq full?
                           ((:COMPILE :LOAD :DUMP) nil)
                           (otherwise
                            (y-or-n-p "Ask for confirmations? "))))
          (compile?      (selectq full?
                           ((:COMPILE) t)
                           ((:LOAD :DUMP) nil)
                           (otherwise
                            (y-or-n-p "Compile recent changes? "))))
          (plan-to-dump? (selectq full?
                           ((:COMPILE) nil)
                           ((:DUMP) t)
                           (otherwise
                            (y-or-n-p "Plan to dump? "))))
          (load-patches? (selectq full?
                           ((:COMPILE :LOAD :DUMP) t)
                           (otherwise
                            (y-or-n-p "Load patches? "))))
          (increment?    (and compile?
                              (or (not (status feature 3600.))
                                  (y-or-n-p "Increment patch version? "))))))

     (when (si:get-system-version "KBE")
       (setq confirm? t) ;Paranoia
       (when plan-to-dump?
         (ferror "You can't be serious about wanting to dump this out."))
       (setq plan-to-dump? nil))

     (when (not plan-to-dump?)
       (si:enable-services)
       (format t "~&(Chaos servers enabled)~%"))
```

```
(send terminal-io ':set-more-p nil)

;; Maybe get latest system stuff loaded

(catch-error ;eg, if site is down...
 (if load-patches?
     (cond (confirm? (load-patches))
           (t (load-patches ':noselective)))))

(let ((base 10.)      ;Must output base ten with decimal point
      (*nopoint nil)) ;or MAKE-SYSTEM may sometimes lose. -kmp

  (if (or (not confirm?) (y-or-n-p "Make-System KBE? "))
      (make-system 'KBE
                   (if confirm?   ':noop    ':noconfirm)
                   (if compile?   ':compile ':noop      )
                   (if (and compile? (not increment?))
                       ':no-increment-patch
                       ':noop)
                   ':no-reload-system-declaration))

  (if (and compile? (not increment?))
      (make-system 'KBE ':noconfirm ':no-reload-system-declaration)))

(cond ((or (and (not confirm?) full?)
           (y-or-n-p "Set up default environment? "))
       (cond-every ((or (not confirm?)
                        (y-or-n-p "Default font CPTFONTB? "))
                    (tv:set-default-font fonts:cptfontb)
                    ;; We keep switching around what the above default
                    ;; font is, but it seems generally agreed that in
                    ;; any case, the initial lisp listener probably
                    ;; wants to remain invariant of that.
                    )
                   ((or (not confirm?)
                        (y-or-n-p "Initial Lisp Listener in font MEDFNB? "))
                    (send terminal-io ':set-font-map '(fonts:medfnb))
                    (send (car (send terminal-io ':blinker-list))
                          ':set-size
                          (send terminal-io ':char-width)
                          (send terminal-io ':line-height)))
                   ((or (not confirm?)
                        (y-or-n-p "BASE,IBASE 10, *NOPOINT T? "))
                    (add-initialization "Base Ten Default"
                      '(setq base 10. ibase 10. *nopoint t)
                      '(:warm :now))) ; Saying :WARM implies :COLD
                   ((and (status feature 3600.)
                         (= si:sequence-break-interval 1000000.)
                         (or
                          (not confirm?)
                          (y-or-n-p "Reduce sequence break interval? ")))
                    (setq si:sequence-break-interval 10000.))
                   ((or (not confirm?)
                        (y-or-n-p "Jazzy PA herald on cold boots?"))
                    (add-initialization "PA Herald"
                      '(kbe:print-PA-herald)
                      '(:warm))))) ;Saying :WARM implies :COLD

(kbe:initialize-system) ;Want to do this after setting up fonts, etc.

(catch-error
 (cond ((or (and (not confirm?) full?)
            (y-or-n-p "Load EMouse? "))
        (load "KBE: EMACS; EMOUSE LISP"))))

(catch-error
 (cond ((and (not (status feature 3600.))
             (or (and (not confirm?) full?)
```

```
                (y-or-n-p "Load KMouse? ")))
        (load "KBE: LMLIB; KMOUSE"))))

(catch-error
 (cond ((or (and (not confirm?) full?)
            (y-or-n-p "Load DEMO init file? "))
        (load "KBE: KBE; DEMO-INIT"))))

(catch-error
 (cond ((or (and (not confirm?) full?)
            (y-or-n-p "Load LOGIN advice? "))
        (load "KBE: PA; LOGIN"))))

(catch-error ;eg, in case AI is down...
 (if (and load-patches? (not compile?))
     (cond ((not confirm?)
            (load-patches 'KBE ':noselective))
           ((or full? (y-or-n-p "Load patches? "))
            (load-patches 'KBE)))))

(catch-error
 (cond ((and compile?
             increment?
             (not plan-to-dump?)
             (or (not confirm?)
                 (y-or-n-p "Create documentation file? ")))
        (let ((version-info (kbe:system-version-string "KBE")))
          (format t "~&Producing documentation for ~A..."
                  version-info)
          (format t "~&~A documentation written to ~A.~%"
                  version-info
                  (kbe:document-KBE-to-file))))))

(when (and compile?
           (not plan-to-dump?)
           (or (not confirm?)
               (y-or-n-p "Run standard self-test? ")))
  (format t "~&Running self-test sequence ...~%")
  (let ((data kbe:(with-delegate (parse-actor "Apprentice 1")
                    (eval-command (parse-command "Test yourself."))))))
    (let ((text    (get data ':text))
          (passed? (get data ':success)))
      (format t "~&~A~%" text)
      (when (and passed?
                 increment?
                 (or (not confirm?)
                     (y-or-n-p "Release this KBE version? ")))
        (set-system-status 'KBE ':released)
        (format t "~A is now released."
                (kbe:system-version-string "KBE")))
      (when (and (not passed?)
                 (eq (multiple-value-bind (ignore ignore status)
                         (si:get-system-version "KBE"))
                     ':released)
                 (or (not confirm?)
                     (y-or-n-p "De-Release this KBE version? ")))
        (set-system-status 'KBE ':broken)
        (format t "~A is no longer released."
                (kbe:system-version-string "KBE"))))))

(multiple-value-bind (major minor)
    (si:get-release-version)
  ;; People like RWK at Symbolics keep putting non-numeric
  ;; data into the release info, so we need to be cautious here...
  (if (not (numberp major)) (setq major -1))
  (let ((base 10.) (*nopoint t))
    (setq minor (or (parse-number (format nil "~A" minor)) -1)))
  (when (= major 4)
    (cond ((not (or (= minor 3) (= minor 4) (= minor 5)))
```

```
            (format t
                    "~&Bug fixes NOT being loaded into Release ~D.~D~%"
                    major minor))
                  ((mapcar #'(lambda (x) (load (car x)))
                           (ignore-errors
                             (cdr (fs:directory-list
                                    #-3600 "KBE: BUGFIX; * QBIN >"
                                    #+3600 "KBE: BUGFIX; * BIN >"
                                    ':fast ':noerror))))
                   ;If any files were loaded, note the fact...
                   ;(setq si:system-additional-info "Hacked Magic")
                   (format t "~&KBE private patches loaded.~%")))))


    ;; Acknowledge successful termination.

    (format t "~&~A loaded.~%"
            (kbe:system-version-string "Knowledge-Based-Editor"))

    (when plan-to-dump?
      (let ((ok? t))
        (cond-every (compile?
                     (setq ok? nil)
                     (format t "~&>>> Warning: System freshly compiled."))
                    (chaos:chaos-servers-enabled
                     (setq ok? nil)
                     (format t "~&>>> Warning: Chaos servers enabled."))
                    ((not load-patches?)
                     (setq ok? nil)
                     (format t "~&>>> Warning: Patches not loaded.")))
        (when ok?
          (multiple-value-bind (ignore ignore keyword)
              (si:get-system-version 'system)
            (if (or (null si:system-additional-info)
                    (equal si:system-additional-info "")
                    (equal si:system-additional-info "Vanilla"))
                (setq si:system-additional-info
                      (caseq keyword
                        ((:EXPERIMENTAL) "Black Magic")
                        ((:RELEASED)     "Sorcerer's Apprentice")
                        ((:BROKEN)       "Faulty Magic")
                        ((:OBSOLETE)     "Worn Out Magic")
                        (T               si:system-additional-info)))))
          (format t "~&Ready for dump.~%")))))

    ;; Get user's attention.

    (kbe:pleasant-noise-2)

    ) ;TEL
  (send terminal-io ':set-more-p old-more-state)
  ;TCETORP-DNIWNU
  )
 ;TEL
 )
;YLLABOLG-TEL
)
```

```
;;; -*- Mode:LISP; Package:USER; Base:10; Fonts: MEDFNB; -*-

;;; SYSDEF: System definition for the KBE (Knowledge-Based Editor)

;;; See the LispM init file for KBE to see how this file can be
;;; used for building a KBE system for temporary use or for dumping.

;;; Define a KBE package for our sources to live on.
;;;
;;;   The following symbols must be treated specially:
;;;
;;;   <-              We use DICK's <- macro for now. It is like a
;;;                   multi-arg SETF. It is also in the process of
;;;                   being phased out.
;;;
;;;   LET, LET*       We use destructuring LET and LET* and probably
;;;                   will continue to since they are upward compatible
;;;                   with the system version. We don't want to force
;;;                   destructuring forms on other users of our band,
;;;                   though, so we shadow it as a courtesy.


(package-declare KBE GLOBAL 10000. nil
  (myrefname GLOBAL KBE)
  (shadow <- LET LET*))

(package-declare VANILLA GLOBAL 2000. nil
  (myrefname GLOBAL VANILLA))

;;; Kludge:
;;;
;;;   To avoid clobbering the system version of LET/LET*, we force
;;;   the file LISPM2;LET to load into our package. This causes a
;;;   problem because it expects to load on SI: and find the symbol
;;;   DEFMACRO-SET-INDENTATION-FOR-ZWEI there, which it does not. So here
;;;   we steal that symbol onto our package so that we can get LET loaded
;;;   correctly.

(multiple-value-bind (sym found? ignore)
    (intern-local-soft "DEFMACRO-SET-INDENTATION-FOR-ZWEI" 'SI)
  (cond ((not found?)
         (ferror nil "SI:DEFMACRO-SET-INDENTATION-FOR-ZWEI is gone!"))
        (t
         (intern sym 'KBE))))

;;; Define a logical device KBE: from which we can load our system.

(fs:add-logical-pathname-host "KBE" "OZ"
   '(("DEMO"    "PS:<KBE.DEMO>"  )
     ("DOC"     "PS:<KBE.DOC>"   )
     ("EMACS"   "PS:<EMACS>"     )
     ("KBE"     "PS:<KBE>"       )
     ("LISPM2"  "SRC:<S.SYS2>"   )
     ("LMLIB"   "PS:<LMLIB>"     )
     ("MISC"    "PS:<KBE.MISC>"  )
     ("PA"      "PS:<PA>"        )
     ("BUGFIX"  "PS:<KMP.LISPM.BUGS.FIXES>")
     ("PATCH"   "PS:<KBE.PATCH>" )
     ("SOURCE"  "PS:<KBE.SOURCE>")
     ("TEST"    "PS:<KBE.TEST>"  )))

;;; The KBE system: a Knowledge-Based Editor

(defsystem KBE
  (:name              "Knowledge-Based-Editor")
  (:short-name        "KBE")
  (:pathname-default "KBE: SOURCE;")
  (:patchable         "KBE: PATCH; ")
```

```
(:module LET    "KBE: LISPM2; LET" :package KBE)
(:module LETS   "KBE: LMLIB;  LETS")
(:module GPRINT "KBE: LMLIB;  GPRINT")
(:module PP     "KBE: LMLIB; PP")

(:module BASE   ("BASIC"
                 "NOISE"
                 "SETS"
                 "CONDITIONS"
                 "ENGLISH"
                 "SYSTEM-MAGIC"
                 "FLAVOR-MAGIC"))
(:module MACRO  ("MACRO"))          ; General utility macros
(:module KVARS  "KVARS")            ; General KBE variables
(:module KMAC   "KMAC"  )           ; KBE-specific macros
(:module IMAC   "INTERFACE-MACROS")
(:module MBOX   ("MBOX" "POST-OFFICE"))
(:module BEHAVE "BEHAVIOR")
(:module KUTIL  ("EVENTS" "ACTOR" "COMPLETE"))
(:module COMMNT "COMMENT-MACRO")
(:module RDMAC  "READMACRO")
(:module OUTIL  "OUTIL" )       ; Archaic macros OVER, BEGIN, MATCH, ...
(:module DTRACE "DTRACE")       ; Debugging macro support for DB
(:module DUTIL  "DUTIL" )       ; General utility functions for DB
(:module DMAC   "DMAC"  )       ; Main macro support for DB
(:module DVARS  "DVARS" )       ; Variable set-up for DB
(:module DB     "DB"    )       ; Database
(:module GC     "GC"    )       ; Garbage Collector
(:module DDEBUG "DDEBUG")       ; DB Debugging stuff
(:module SURFACE "SURFACE") ;SURFACE, EXPAND, PRIMOP, DFLOW were FLOWS
(:module EXPAND  "EXPAND" )
(:module PRIMOP  "PRIMOP" )
(:module DFLOW   "DFLOW"  )
(:module FNS    ("UTIL" "SEGTRN" "ANALYZE" "CODER"))
(:module PSEUDO "PSEUDO")
(:module ED0    ("DICT" "GRAM"))
(:module ED1    "REFER" )
(:module ED2    "VERBS" )
(:module ED3    "IMPLEMENT")
(:module ADA    ("ADA-EVAL" "ADA-ANAL" "ADA-PARSE" "ADA-UNPARSE"))
(:module VU     "VIEW"  )
(:module PLI    "PLI"   )
(:module DEFLIB "DEFLIB")
(:module KBELIB ("SYSLIB" "KBELIB"))
(:module DEMLIB ("DEMO-LIB"))
(:module DEMADALIB ("DEMO-ADA-LIB"))
(:module BRIDGE "BRIDGE")
(:module KAOS   "KAOS"  )
(:module SOCIETY "SOCIETY")
(:module STNDRD  "STANDARD-ACTOR")
(:module REMOTE "REMOTE")
(:module SENSE  ("OVERLAPPING-REGIONS" "MOUSE-SENSITIVE-REGIONS"))
(:module SIMPLE ("OPTIONS" "SIMPLE" "SIMPLE-ACTOR"))
(:module MAGIC  "MAGIC-ACTOR")
(:module TV     "TV-MAGIC")
(:module ZMAGIC ("ZWEI-MAGIC" "POSSIBILITIES"))
(:module BELIEF "BELIEF")
(:module ZWEI   ("EDIT" "EDIT-ACTOR" "CONSISTENCY"))
(:module CMDS   ("EDIT-COMMANDS"
                 "EDIT-MISC"
                 "EDIT-SIMPLE"
                 "EDIT-DEBUG"
                 "KBE-MODE"
                 "FOCUS"
                 "TEST"
                 "DOC"))
(:module PARSE  "PARSE" )
(:module EXPLAIN ("EXPLAIN" "DOCUMENT"))
(:module HANDLERS "PBM-HANDLERS")
```

```
(:module INIT   "INIT"  )     ; Various kinds of initializations
(:module DEBUG  "DEBUG" )

(:skip :readfile    OUTIL)

(:fasload           PP)
(:fasload           GPRINT)
(:fasload           LET)
(:fasload           LETS)

(:compile-load-init BASE   (PP)
                           (:fasload PP LET LETS)
                           (:fasload PP))
;; The reason PP is 'required' by BASE is that it globalizes
;; some symbols which will be necessary to the correct loading
;; of other things.

(:compile-load-init MACRO  (PP)
                           (:fasload PP LET LETS BASE)
                           (:fasload BASE))

(:compile-load-init KVARS  (PP MACRO)
                           (:fasload PP LET LETS BASE MACRO)
                           (:fasload BASE))

(:compile-load-init KMAC   (PP MACRO)
                           (:fasload PP LET LETS BASE MACRO KVARS)
                           (:fasload BASE KVARS))

(:compile-load-init IMAC   (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                           (:fasload BASE KVARS KMAC))

(:compile-load-init MBOX   (PP MACRO)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC)
                           (:fasload BASE KVARS IMAC))

(:compile-load-init BEHAVE (PP MACRO IMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC)
                           (:fasload BASE KVARS IMAC MBOX))

(:compile-load-init KUTIL  (PP MACRO IMAC BEHAVE)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     IMAC BEHAVE)
                           (:fasload BASE KVARS IMAC MBOX BEHAVE))

(:compile-load-init TV     (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                           (:fasload BASE KVARS KMAC))

(:compile-load-init PSEUDO (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO
                                     KVARS KMAC KUTIL)
                           (:fasload BASE KVARS KMAC KUTIL))

(:compile-load-init ZMAGIC (PP MACRO IMAC TV)
                           (:fasload PP LET LETS MACRO IMAC TV)
                           (:fasload BASE KVARS IMAC TV))

(:compile-load-init COMMNT (PP MACRO)
                           (:fasload PP LET LETS MACRO KMAC ZMAGIC)
                           (:fasload BASE KVARS ZMAGIC))

(:compile-load-init RDMAC  (PP MACRO)
                           (:fasload PP LET LETS MACRO KMAC ZMAGIC)
                           (:fasload BASE KVARS ZMAGIC))

(:compile-load-init OUTIL  (PP MACRO RDMAC)
                           ((:fasload PP LET LETS BASE MACRO KVARS RDMAC)
```

```
                          (:readfile OUTIL))
                         (:fasload BASE KVARS))

(:compile-load-init DTRACE (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                           (:fasload BASE KVARS KMAC))

(:compile-load-init DMAC   (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                           (:fasload BASE KVARS KMAC))

(:compile-load-init DVARS  (PP MACRO KMAC DMAC DTRACE)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     KMAC DMAC DTRACE)
                           (:fasload BASE KVARS KMAC DMAC))

(:compile-load-init DUTIL  (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     KMAC DMAC DVARS)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init DB     (PP MACRO KMAC RDMAC OUTIL DMAC DTRACE)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                     OUTIL DMAC DVARS DTRACE RDMAC)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init GC     (PP MACRO KMAC OUTIL DMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     KMAC OUTIL DMAC DVARS)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init DDEBUG (PP MACRO KMAC OUTIL DMAC DTRACE)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     KMAC OUTIL DMAC DVARS DTRACE)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init SURFACE (PP MACRO KMAC RDMAC OUTIL DMAC)
                            (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                      RDMAC OUTIL DMAC DVARS DB DUTIL)
                            (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init EXPAND (PP MACRO KMAC RDMAC OUTIL DMAC SURFACE)
                           (:fasload PP LET LETS BASE MACRO KVARS
                                     KMAC RDMAC OUTIL DMAC DVARS DB
                                     DUTIL SURFACE)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init PRIMOP (PP MACRO KMAC RDMAC OUTIL
                            DMAC SURFACE EXPAND)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                     RDMAC OUTIL DMAC DVARS DB
                                     DUTIL SURFACE EXPAND)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init DFLOW  (PP MACRO KMAC RDMAC OUTIL DMAC SURFACE
                               EXPAND PRIMOP)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                     RDMAC OUTIL DMAC DVARS DB
                                     DUTIL SURFACE EXPAND PRIMOP)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init FNS    (PP MACRO KMAC RDMAC OUTIL DMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                     RDMAC OUTIL DMAC DVARS DB DUTIL)
                           (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init BRIDGE  (PP MACRO KMAC DMAC)
                            (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                      DMAC DVARS)
```

```
                                (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init ED0         (PP MACRO KMAC PSEUDO)
                                (:fasload PP LET LETS BASE MACRO
                                          KVARS KMAC PSEUDO BRIDGE)
                                (:fasload BASE KVARS KMAC PSEUDO BRIDGE
                                          COMMNT))

(:compile-load-init ED1         (PP MACRO KMAC OUTIL DMAC RDMAC PSEUDO)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB DUTIL ED0)
                                (:fasload BASE KVARS KMAC DVARS KUTIL ED0))

(:compile-load-init ED2         (PP MACRO KMAC OUTIL DMAC RDMAC PSEUDO ED1)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB
                                          ED1 DUTIL)
                                (:fasload BASE KVARS KMAC DVARS KUTIL ED1))

(:compile-load-init ED3         (PP MACRO KMAC OUTIL DMAC RDMAC
                                 PSEUDO ED1 ED2)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB
                                          PSEUDO ED2 DUTIL)
                                (:fasload BASE KVARS KMAC DVARS KUTIL
                                          PSEUDO ED2))

(:compile-load-init ADA         (GPRINT PP MACRO RDMAC KMAC PSEUDO)
                                (:fasload GPRINT PP LET LETS BASE MACRO RDMAC
                                          KVARS KMAC PSEUDO)
                                (:fasload GPRINT PP BASE KVARS KMAC KUTIL
                                          PSEUDO))

(:compile-load-init VU          (PP MACRO KMAC DMAC PSEUDO)
                                (:fasload PP LET LETS BASE MACRO KVARS
                                          KMAC DMAC DVARS DB DUTIL TV PSEUDO)
                                (:fasload BASE KVARS KMAC DVARS TV PSEUDO))

(:compile-load-init PLI         (GPRINT PP MACRO KMAC DMAC VU)
                                (:fasload GPRINT PP LET LETS BASE MACRO KVARS
                                          KMAC DMAC DVARS DB DUTIL VU)
                                (:fasload GPRINT BASE KVARS KMAC DVARS))

(:compile-load-init DEFLIB      (PP MACRO KMAC OUTIL DMAC RDMAC ED1)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB DUTIL
                                          SURFACE EXPAND PRIMOP DFLOW
                                          ED1 ED2)
                                (:fasload BASE KVARS KMAC DVARS))

(:compile-load-init KBELIB      (PP MACRO KMAC OUTIL DMAC RDMAC ED1
                                 DEFLIB COMMNT)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB DUTIL FNS
                                          SURFACE EXPAND PRIMOP DFLOW
                                          ED1 ED2 ED3 DEFLIB COMMNT)
                                (:fasload BASE KVARS KMAC DVARS ED1))

(:compile-load-init DEMLIB      (PP MACRO KMAC OUTIL DMAC RDMAC ED1
                                 DEFLIB KBELIB  COMMNT)
                                (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                          OUTIL RDMAC DMAC DVARS DB DUTIL FNS
                                          SURFACE EXPAND PRIMOP DFLOW
                                          ED1 ED2 ED3 DEFLIB KBELIB
                                          COMMNT)
                               .(:fasload BASE KVARS KMAC DVARS ED1))

(:compile-load-init DEMADALIB (PP MACRO KMAC OUTIL DMAC RDMAC ED1 ADA
                               DEFLIB KBELIB  COMMNT)
```

```
                        (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                  OUTIL RDMAC DMAC DVARS DB DUTIL FNS
                                  SURFACE EXPAND PRIMOP DFLOW
                                  ED1 ED2 ED3 ADA DEFLIB KBELIB
                                  COMMNT)
                        (:fasload BASE KVARS KMAC DVARS ED1 ADA))

(:compile-load-init KAOS    (PP MACRO KMAC)
                            (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                            (:fasload BASE KVARS KMAC))

(:compile-load-init BELIEF  (PP MACRO IMAC BEHAVE)
                            (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                      BEHAVE KUTIL)
                            (:fasload BASE KVARS IMAC KUTIL))

(:compile-load-init SOCIETY (PP MACRO IMAC BEHAVE)
                            (:fasload PP LET LETS BASE MACRO KVARS
                                      IMAC BEHAVE KUTIL BELIEF)
                            (:fasload BASE KVARS IMAC BEHAVE KUTIL
                                      BELIEF))

(:compile-load-init STNDRD  (PP MACRO IMAC BEHAVE TV PSEUDO)
                            (:fasload PP LET LETS BASE MACRO KVARS
                                      IMAC BEHAVE KUTIL BELIEF
                                      SOCIETY TV PSEUDO)
                            (:fasload BASE KVARS IMAC BEHAVE KUTIL BELIEF
                                      SOCIETY TV PSEUDO))

(:compile-load-init REMOTE  (PP MACRO IMAC KAOS SOCIETY)
                            (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                      KAOS SOCIETY STNDRD)
                            (:fasload BASE KVARS IMAC KAOS SOCIETY
                                      STNDRD))

(:compile-load      SENSE)

(:compile-load-init SIMPLE (PP MACRO IMAC BEHAVE SOCIETY SENSE)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                     BEHAVE KUTIL SOCIETY STNDRD VU TV
                                     ZMAGIC SENSE)
                           (:fasload BASE KVARS IMAC SOCIETY STNDRD
                                     VU TV BEHAVE KUTIL BRIDGE ZMAGIC
                                     SENSE))

(:compile-load-init MAGIC  (PP MACRO IMAC BEHAVE SIMPLE ED1 SOCIETY)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                     BEHAVE KUTIL SIMPLE ED1
                                     SOCIETY STNDRD ZMAGIC)
                           (:fasload BASE KVARS IMAC BEHAVE KUTIL ED1
                                     SOCIETY STNDRD BRIDGE ZMAGIC))

(:compile-load-init ZWEI   (PP MACRO IMAC BEHAVE SIMPLE SOCIETY
                               MAGIC ZMAGIC DMAC STNDRD)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                     SIMPLE SOCIETY STNDRD MAGIC
                                     ZMAGIC DMAC BEHAVE KUTIL)
                           (:fasload BASE KVARS IMAC SOCIETY STNDRD
                                     BRIDGE ZMAGIC BEHAVE KUTIL))

(:compile-load-init CMDS   (PP MACRO IMAC PSEUDO BEHAVE SIMPLE SOCIETY
                               MAGIC ZMAGIC DMAC ZWEI)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC
                                     PSEUDO SIMPLE SOCIETY STNDRD MAGIC
                                     ZMAGIC DMAC  BEHAVE KUTIL ZWEI)
                           (:fasload BASE KVARS IMAC PSEUDO SOCIETY
                                     STNDRD BRIDGE ZMAGIC BEHAVE KUTIL
                                     ZWEI))
```

```
(:compile-load-init PARSE  (PP MACRO IMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS IMAC)
                           (:fasload BASE KVARS IMAC))

(:compile-load-init EXPLAIN (PP MACRO KMAC OUTIL DMAC PSEUDO)
                            (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                      OUTIL DMAC DVARS DB DUTIL PSEUDO)
                            (:fasload BASE KVARS KMAC DVARS PSEUDO))

(:compile-load-init HANDLERS (PP MACRO KMAC OUTIL DMAC)
                             (:fasload PP LET LETS BASE MACRO KVARS KMAC
                                       OUTIL DMAC DVARS DB DUTIL)
                             (:fasload BASE KVARS KMAC DVARS EXPLAIN))

(:compile-load-init INIT   (PP MACRO KMAC)
                           (:fasload PP LET LETS BASE MACRO KVARS KMAC)
                           (:fasload BASE KVARS KMAC))

(:compile-load-init DEBUG  (PP MACRO KMAC SOCIETY)
                           (:fasload PP LET LETS BASE MACRO
                                     KVARS KMAC SOCIETY)
                           (:fasload BASE KVARS KMAC)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; BASIC: Module of basic utility functions which may be used
;;;        by any other module in the KBE system.

;;; (CIRCULAR-LIST* x1 x2 ... xN l)
;;;
;;;  Like CIRCULAR-LIST, but takes LIST* conventions.

(defun circular-list* (&rest l)
  (declare (values circular-list))
  (lexpr-funcall #'lexpr-funcall #'circular-list l))

;;; (TEXT format-string . format-args)
;;;
;;;  Returns the string which would have been printed by the given
;;;  format string and format args.

(defun text (format-string &rest format-args)
  (declare (values text-string))
  (lexpr-funcall #'format nil format-string format-args))

;;; (GRUMBLE format-string . format-args)

(defun grumble (format-string &rest format-args)
  (format error-output "~&")
  (lexpr-funcall #'format error-output format-string format-args))

;;; (MAYBE-INTERN string pkg)
;;;
;;;  Like INTERN but if pkg is NIL, uses MAKE-SYMBOL to make an
;;;  uninterned symbol rather than using INTERN and interning on
;;;  the current package. If the specified package is locked, the
;;;  default package is assumed.

(defun maybe-intern (string pkg)
  (declare (values symbol))
  (setq string (string string))
  (values (cond ((not pkg) (make-symbol string t))
                (t (condition-case ()
                       (intern string pkg)
                     ((sys:package-locked) (intern string)))))))

;;; (SYMBOLCONC x y)
;;;
;;;  Returns a symbol which is the result of concatenating the
;;;  print names of two given objects.
;;;
;;;  The package used will be the package of the first of its
;;;  arguments that is a symbol. If no such argument or that
;;;  symbol is not interned, a gensym is produced.

(defun symbolconc (x y)
  (declare (values sym))
  (let ((base 10.) (*nopoint t))
    (maybe-intern (format nil "~A~A" x y)
                  (cond ((symbolp x) (symbol-package x))
                        ((symbolp y) (symbol-package y))))))

;;; (SYMBOL-PLURALIZE x)
```

```
;;;
;;;  Returns a symbol which is the result of pluralizing its
;;;  argument. The result is in the same package as the argument.

(defun symbol-pluralize (sym)
  (declare (values plural-sym))
  (maybe-intern (string-pluralize sym) (symbol-package sym)))

;;; (ADD-BUCKY-BITS char . bit-names)
;;; (REMOVE-BUCKY-BITS char . bit-names)
;;;
;;;  CHAR should be a fixnum character.
;;;  BIT-NAMES should be a list of bucky-bit names such as
;;;   :CONTROL, :META, :SUPER, or :HYPER.

(defun add-bucky-bits (char &rest bit-names)
  (lexpr-funcall #'set-bucky-bits 1 char bit-names))

(defun remove-bucky-bits (char &rest bit-names)
  (lexpr-funcall #'set-bucky-bits 0 char bit-names))

(defun set-bucky-bits (bit char &rest bit-names)
  (do ((c char (dpb bit
                    (selectq (car n)
                      ((:control) %%kbd-control)
                      ((:meta)    %%kbd-meta)
                      ((:super)   %%kbd-super)
                      ((:hyper)   %%kbd-hyper)
                      (otherwise
                       (ferror "~S is not the name of a bucky bit."
                               (car n))))
                    c))
       (n bit-names (cdr n)))
      ((null n) c)))

(defun bucky-char-name (char)
  (format nil "~:[~;Hyper-~]~:[~;Super-~]~:C"
          (not (zerop (ldb %%kbd-hyper   char)))
          (not (zerop (ldb %%kbd-super   char)))
          (remove-bucky-bits char ':hyper ':super)))

;;; (STRING-COPY string)
;;;
;;; Returns a copy of the given string.

(defun string-copy (string) (string-append string))

;;; (COPY tree)
;;;
;;;  This copies all levels of list structure in a given tree.
;;;
;;;  This is the same as COPYTREE. Why aren't we using COPYTREE? --kmp

(defun copy (tree) (subst nil nil tree))

;;; (GET-PREFIX sym)
;;;
;;;  Takes a symbol as an argument. If the symbol's print name contains
;;;  no trailing digits, the symbol is simply returned. If it does
;;;  simply returns it. If it does contain trailing digits, those digits
;;;  are stripped and the resulting string is interned in whatever package
;;;  sym was interned in (or not interned if sym wasn't).
;;;
;;;  (get-prefix 'foo1)          => FOO
;;;  (get-prefix '5fo3o)         => 5FO3O
;;;  (get-prefix (gensym))       => G ;not interned
;;;  (get-prefix ':foo3)         => :FOO

(defun get-prefix (sym)
```

```
  (declare (values sym-without-trailing-digits))
  (let ((pname (get-pname sym)))
    (let ((pos (1+ (string-reverse-search-not-set "0123456789" pname))))
      (cond ((= pos (string-length pname)) sym)
            (t (maybe-intern (substring pname 0 pos)
                             (symbol-package sym)))))))

;;; (PUT-LAST atom prop value)
;;;
;;;  This is like PUTPROP, but:
;;;   (1) argument order is better
;;;   (2) puts the property at the end of the plist. This allows the
;;;       DB-INDEX property to stay close to the head of the plist,
;;;
;;;  This used to be APPEND. I changed to to NCONC. I bet no one cares
;;;  and it'll probably waste less conses. --kmp

(defun put-last (atom prop value)
  (remprop atom prop)
  (setplist atom (nconc (plist atom)
                        (list*-in-area sys:property-list-area
                                       prop value nil)))
  value)

;;; (ASSQ-CAREFULLY obj list)
;;;
;;;  Like ASSQ, but ignores any elements of list which are atomic.
;;;
;;;  (Historical note: This was the behavior of ASSQ in Maclisp.)

(defun assq-carefully (obj list)
  (do ((l list (cdr l)))
      ((null l) nil)
    (cond ((and (listp (car l))
                (eq (caar l) obj))
           (return (car l))))))


;;; (READSTRING [stream])
;;;   is like readline but reads a string to either and end of file.
;;;
;;;   By default, an <End> char can also behave like an end of file.
;;;   This is changeable by giving an END-CHARS argument of chars,
;;;   if any, which can be treated like eof.

(defun readstring (&optional (stream standard-input)
                             (end-chars '(#\End)))
  (declare (values string))
  (with-output-to-string (string-stream)
    (do ((c (tyi stream -1) (tyi stream -1)))
        ((or (= c -1) (memq c end-chars)))
      (tyo c string-stream))))

;;; (DE-HYPHENATE x)
;;;
;;;  Returns a string which is lowercase the lowercase form of x
;;;  and which has hyphens changed to spaces. eg,
;;;
;;;     (DE-HYPHENATE 'FOO)             => "foo"
;;;     (DE-HYPHENATE 'FOO-OF-THE-BAR)  => "foo of the bar"

(defun de-hyphenate (string-or-symbol)
  (declare (values lowercase-dehyphenated-string))
  (let ((string (string-downcase string-or-symbol)))
    (dotimes (n (string-length string))
      (when (eq (aref string n) #/-)
        (setf (aref string n) #\space)))
    string))
```

```
;;; (STRING-CAPITALIZE x)
;;;
;;;  Returns a string identical to x, but with the first
;;;  character in uppercase. eg,
;;;
;;;   (STRING-CAPITALIZE "FOO")          => "FOO"
;;;   (STRING-CAPITALIZE "Foo")          => "Foo"
;;;   (STRING-CAPITALIZE "foo")          => "Foo"
;;;   (STRING-CAPITALIZE "foo bar")      => "Foo bar"

(defun string-capitalize (x)
  (declare (values capitalized-string))
  (string-append (char-upcase (character x))
                 (substring x 1)))
```

```
;;; These are utilities for the KBE's internal referencing to
;;; particular interning styles.
;;;
;;; (INTERN-KBE string)
;;;
;;;  Returns the interned copy of string on the KBE's internal package
;;;  (which is currently 'KBE but which might change). This function
;;;  should be, but probably isn't always, called to do interning on
;;;  that package.
;;;
;;; (INTERN-KEYWORD string)
;;;
;;;  Returns the interned copy of string on the keyword package. That
;;;  is currently the USER package, but may change if/when Common Lisp
;;;  goes in.

(defun intern-KBE (pname)
  (declare (values a-symbol-in-the-KBE-package))
  (intern (string pname) 'KBE))

(defun intern-keyword (pname)
  (declare (values a-symbol-in-the-KEYWORD-package))
  (intern (string pname) ""))
```

```
;;; (*DEFINE-SYSTEM-KEY key           ; a character
;;;                     doc           ; a string
;;;                     flavor        ; a window flavor
;;;                     query?        ; if T, query about conflicts
;;;                     re-init-form) ; a form to EVAL to make more of them
;;;
;;; Defines a key sequence "System <key>" which will select a window
;;; of the specified flavor.
;;;
;;; This is not intended to be called directly from user code. It is
;;; support for DEFINE-SYSTEM-KEY to expand into.

(defun add-system-key (key query? doc flavor &optional (re-init-form 't))
  (let ((entry (assq key tv:*system-keys*)))
    (when (or (not entry)
              (not query?)
              (equal (caddr entry) doc)
              (fquery '() "Key sequence ~:C ~:C was ~S~
                           ~%Redefine it to be ~S? "
                      #\system key (caddr entry) doc))
      (tv:add-system-key key flavor doc re-init-form))))
```

```
;;;; Utilities for User Option Facility

;;; These functions are useful in conjunction with the
;;; User Options Facility described on page 18 of
;;; "Lisp Machine Choice Facilities".

;;; (ODD-OPTIONS options-alist)
;;;
;;;  The options-alist must have been defined by DEFINE-USER-OPTION-ALIST.
;;;  This returns a list of exactly one string per option variable which
;;;  differs from the default; the string describes the difference.

(defun odd-options (options-alist)
  (do ((oddities nil)
       (options options-alist (cdr options)))
      ((null options) oddities)
    (let ((default (get (caar options) 'tv:default-value)))
      (cond ((neq (symeval (caar options)) default)
             (push (odd-options-1 (car options)) oddities))))))

;;; (ODD-OPTIONS-1 alist-entry)
;;;
;;;  Does formatting of the string to be returned by ODD-OPTIONS.

(defun odd-options-1 (options-alist-entry)
  (let (((option name type type-data) options-alist-entry))
    (let ((val (symeval option)))
      (format nil "~A: ~A"
              (or name option)
              (selectq type
                ((:BOOLEAN) (cond ((not val) "No.")
                                  ((eq val 't) "Yes.")
                                  (t (format nil "~S ;Yes" val))))
                ((:ASSOC)   (let ((name-info
                                    (rassoc val type-data)))
                              (cond (name-info (car name-info))
                                    (t (format nil "~S ;Non-standard value"
                                               val)))))
                (otherwise  (format nil "~S" val)))))))
```

```
;;;; General message passing utilities

;;; (HANDLES? object message)
;;;
;;;  Returns true if object handles message, else false.

(defun handles? (object message)
  (send object ':operation-handled-p message))

;;; (SEND-IF-HANDLES object message . args)
;;;
;;;  If object handles message, works like
;;;       (SEND object message . args).
;;;  Otherwise, returns NIL.

(defun send-if-handles (object message &rest args)
  (lexpr-send object ':send-if-handles message args))

;;; (LEXPR-SEND-IF-HANDLES object message . lexpr-args)
;;;
;;;  If object handles message, works like
;;;      (LEXPR-SEND object message . lexpr-args).
;;;  Otherwise returns NIL.

(defun lexpr-send-if-handles (object message &rest args)
  (cond ((send object ':operation-handled-p message)
         (lexpr-funcall #'lexpr-send object message args))))
```

```
;;;; Parsing argument lists

;;; (PARSE-ARGUMENT-LIST arglist)
;;;
;;;  This parses an argument list, arglist, returning two values:
;;;   1: A list of arguments suitable for FUNCALL or LEXPR-FUNCALL
;;;   2: NIL if the last argument is not spread (ie, the FUNCALL case),
;;;      or T otherwise (ie, the LEXPR-FUNCALL case).
;;;   3: The bvl up to but not including any &AUX info.
;;;   4: The &AUX info (suitable for use in a LET*) or NIL if none.
;;;
;;;  For example,
;;;    (PARSE-ARGUMENT-LIST '(X Y &OPTIONAL (Q 3) &REST Z &AUX W))
;;;      => (X Y Q Z), T, (X Y &OPTIONAL (Q 3) &REST Z), (W)
;;;

(defun parse-argument-list (bvl)
  (declare (values arglist spread-p bvl aux-list))
  (do ((l bvl (cdr l))
       (arglist nil (cons var arglist))
       (spread-p nil)
       var)
      ((or (null l) (eq (car l) '&aux))
       (values (nreverse arglist) spread-p (ldiff bvl l) (cdr l)))
    (let ((current (car l)))
      (do ()
          ((or (not (atom current))
               (not (eq (character current) #/&))))
        (cond ((memq current '(&rest &body))
               (setq spread-p t)))
        (pop l)
        (setq current (car l)))
      (setq var (cond ((atom current) current)
                      (t (car current)))))))
```

```
;;;; Message utilities for hierarchical systems

;;; These utilities are intended primarily to support the window system
;;; where if a message isn't handled by a window, you want to ask the
;;; superior window, but in fact they are general enough to use on any
;;; object which answers a :SUPERIOR message.

;;; (SEND-ANY-SUPERIOR obj message . args)
;;;
;;; Finds the first x in obj and its superiors which handles
;;; the given message and tries sending w the given message,
;;; returning the result.
;;;
;;; Returns NIL, NIL if no such x is found.
;;;
;;; If a x is found, the first return value is the result of sending
;;; that object the given message and the second return value is T.

(defun send-any-superior (obj message &rest args)
  (do ((x obj (send-if-handles x ':superior)))
      ((null x) (values nil nil))
    (cond ((send x ':operation-handled-p message)
           (return (lexpr-send x message args) t)))))

;;; (LEXPR-SEND-ANY-SUPERIOR obj message . args)
;;;
;;; Lexpr version of SEND-ANY-SUPERIOR.

(defun lexpr-send-any-superior (obj message &rest args)
  (lexpr-funcall #'lexpr-funcall #'send-any-superior obj message args))

;;; (SEND-ANY-SUPERIOR-IF-HANDLES obj message . args)
;;;
;;; Tries each x in obj and its superiors with the given message.
;;; If no x handles the message or if all x's which handle the message
;;; return NIL, then NIL is returned. Otherwise, the first non-NIL
;;; result is returned and no succeeding (superior) x's are tried.

(defun send-any-superior-if-handles (obj message &rest args)
  (do ((x obj (send-if-handles x ':superior)))
      ((null x) nil)
    (let ((test (lexpr-send-if-handles x message args)))
      (cond (test (return test))))))

;;; (LEXPR-SEND-ANY-SUPERIOR-IF-HANDLES obj message . args)
;;;
;;; Lexpr version of SEND-ANY-SUPERIOR-IF-HANDLES.

(defun lexpr-send-any-superior-if-handles (obj message &rest args)
  (lexpr-funcall #'lexpr-funcall
                 #'send-any-superior-if-handles obj message args))
```

```
;;;; Support for DEFDEFMACRO

(defun make-defdefmacro-source-file-wrapper (definition-type fn-spec body)
  (let ((canonical-definition-type
          (or (get definition-type 'zwei:definition-function-spec-type)
              definition-type))
        (canonical-function-spec
          (cond ((and (not (atom fn-spec))
                      (= (length fn-spec) 2))
                 '(:property ,@fn-spec))
                (t fn-spec))))
    '(local-declare ((sys:function-parent ,canonical-function-spec
                                          ,canonical-definition-type))
       (record-source-file-name ',canonical-function-spec
                                ',canonical-definition-type)
       ,body)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; The KBE will want to be able to generate noises for various different
;;; events to alert the user to things that he might want to do. This
;;; section provides some tools so that users dis-satisfied with the
;;; noise they get can pick another...
;;;
;;; (INIT-NOISE-FOR name &rest args-info)
;;;
;;;  Sets up a noise given by noise named NAME. The args-info arguments
;;;  should be strings which describe each of the args the given noise
;;;  desires.
;;;
;;; (SET-NOISE-FOR name [function])
;;;
;;;  Sets the noise-making function for a particular named noise.
;;;  Noise-making functions are functions of any number of arguments.
;;;  The first argument will be the noise NAME. The remaining arguments,
;;;  if any, are specific to that kind of noise.
;;;
;;;  If NAME is NIL, sets the default noise making function.
;;;
;;;  If no function is supplied, the default function will be used.
;;;  If the supplied function is NIL, the noise is ignored.
;;;
;;; (MAKE-NOISE-ABOUT name &rest other-args)
;;;
;;;  Provides an abstraction for ringing the bell in various interesting
;;;  ways. In fact, the bell needn't be rung at all if the user wants
;;;  to define handlers that do something completely different with
;;;  the notifications. The noise name must have been previously defined
;;;  by an INIT-NOISE-FOR form.
;;;
;;; (LIST-NOISES)
;;;
;;;  Displays information about the currently defined kinds of KBE
;;;  noises in table format.

(defun default-noise (&optional name &rest name-specific-data)
  name name-specific-data ;ignored
  (beep))

(defvar *noises* (ncons #'default-noise))

(defun noise-for (thing)
  (do ((l (cdr *noises*) (cdr l)))
      ((null l) (ferror nil "Undefined noise ~S" thing))
    (cond ((eq (caar l) thing)
           (return (cond ((eq (cadar l) 't)
                          (car *noises*))
                         (t
                          (cadar l))))))))

(defun make-noise-about (thing &rest other-args)
  (let ((handler (noise-for thing)))
    (cond (handler (lexpr-funcall handler thing other-args)))))

(defun set-noise-for (name &optional (handler 't handler?))
  (cond ((null name)
         (cond ((not handler?)
```

```
              (ferror nil "Default noise handler may not be defaulted.")))
        (setf (car *noises*) handler))
       (t
        (let ((slot (or (assq name (cdr *noises*))
                        (ferror nil "Noise ~S is not defined." name))))
          (setf (cadr slot) handler))))
  handler)

(defun init-noise-for (name &rest args-info)
  (let ((old-handler (assq name (cdr *noises*))))
    (cond ((not old-handler)
           (push (list* name t (copylist args-info)) (cdr *noises*))
           t)
          ((equal (cddr old-handler) args-info)
           nil)
          (t
           (ferror nil "Conflicting noise initialization for ~S."
                   name args-info)))))

(defun list-noises ()
  (cond ((not *noises*)
         (format t "~&No noises defined."))
        ((cdr *noises*)
         (let* ((package (pkg-find-package "GLOBAL")) ;display package info
                (flag nil)
                (column  (+ (apply #'max
                                   (mapcar #'(lambda (x)
                                               (string-length
                                                 (text "~S" (car x))))
                                           (cdr *noises*)))
                            3)))
           (format t "~&Name~VTHandler Function~%" column)
           (dolist (slot (cdr *noises*))
             (caseq (cadr slot)
               ((NIL) (format t "~&~S~VTNo handler.~%"
                              (car slot) column)
                      (setq flag t))
               ((T)   (format t "~&~S~VTDefault handler.~%"
                              (car slot) column))
               (T     (format t "~&~S~VT~S~%"
                              (car slot) column (cadr slot))))
             (cond ((cddr slot)
                    (do ((args (cddr slot) (cdr args))
                         (i 1 (1+ i)))
                        ((null args))
                      (format t "~&~VT Arg ~D: ~A~%"
                              column i (string-capitalize-words
                                         (car args)))))))
           (format t "~&Arg 0 for all handlers is the noise name.~@[~*~
                      ~%Noises without handlers ignored.~]~%"
                   flag))))
  (cond ((car *noises*)
         (format t "~&The default handler is ~S.~%"
                 (car *noises*))))
  t)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; SETS: Useful set operations

;;;; Sets

;;; The following fns define the basic operations on sets.  They come in
;;; 4 versions: sets with membership determined by equal (set), sets
;;; with membership determined by eq (qset), sets with membership
;;; determined by = that are sorted smallest number first (nset), and
;;; sets with membership determined by eq where the elements are ordered
;;; by MAKNUM smallest first (fset).
;;;
;;; Note: In principle, fsets shouldn't work on the LispM because
;;;       the gc might move stuff and pointer values might change.
;;;       Since the gc is never really used, we ignore that problem
;;;       and let things be fast.

;;; The operations are:
;;;
;;;  1. make(n/f/q/ )set -- take a list and DESTRUCTIVELY convert it
;;;                          to a set -- in particular delete duplicates.
;;;  2. (n/f/q/ )set=    -- determine if two sets are the same
;;;  3. (n/f/q/ )setU    -- form the union of two sets
;;;  4. (n/f/q/ )set^    -- form the intersection of two sets
;;;  5. (n/f/q/ )set-    -- the result is the elements of the first
;;;                          set not in the second set
;;;  6. (n/f/q/ )set<    -- tests whether the first set is a subset of
;;;                          the second
;;;  7. (n/f)setmem      -- tests whether an element is in a set (member
;;;                          and memq are used for sets and qsets)

;;; The fsets and nsets (fast sets) are the most interesting. The others
;;;  just slug along slowly.  They preserve the order of the sets for
;;;  convenience.  It is intended that fsets be used whenever you care the
;;;  least bit about speed.
```

```
;;;; EQUAL sets

;;; SETs: Sets based on Lisp equal
;;;
;;; Note: member is one of these operations.

(defun makeset (list)
  (do ((l list (cdr l)))
      ((null (cdr l)) list)
    (setf (cdr l) (delete (car l) (cdr l)))))

(defun set= (setx sety)
  (and (set< setx sety) (set< sety setx)))

(defun set< (setx sety)
  (do ((subsetx setx (cdr subsetx)))
      ((null subsetx) t)
    (cond ((not (member (car subsetx) sety))
           (return nil)))))

(defun setU (setx sety)
  (do ((subsetx setx (cdr subsetx))
       (newset  sety))
      ((null subsetx) newset)
    (cond ((not (member (car subsetx) sety))
           (push (car subsetx) newset)))))

(defun set^ (setx sety)
  (do ((subsetx setx (cdr subsetx))
       (newset nil))
      ((null subsetx) (nreverse newset))
    (cond ((member (car subsetx) sety)
           (push (car subsetx) newset)))))

(defun set- (setx sety)
  (do ((subsetx setx (cdr subsetx))
       (newset nil))
      ((null subsetx) (nreverse newset))
    (cond ((not (member (car subsetx) sety))
           (push (car subsetx) newset)))))
```

```
;;;; Sorted fixnum sets

;;; NSETs: Sorted fixnum sets

(defun makenset (list)
  (setq list (sort list #'<))
  (do ((l list))
      ((null (cdr l)) list)
    (cond ((= (car l) (cadr l)) (rplacd l (cddr l)))
          (t (setq l (cdr l))))))

(defun nset= (nsetx nsety)
  (do ((fx nsetx (cdr fx))
       (fy nsety (cdr fy)))
      ((or (null fx) (null fy))
       (and (null fx) (null fy))) ;return T only if sets are same size
    (cond ((not (= (car fx) (car fy)))
           (return nil)))))

(defun nset< (nsetx nsety)
  (do ((x (car nsetx))
       (y (car nsety)))
      (nil)
    (cond ((null nsetx) (return t))
          ((null nsety) (return nil))
          ((< x y) (return nil))
          ((< y x) (setq nsety (cdr nsety) y (car nsety)))
          (t (setq nsetx (cdr nsetx) x (car nsetx))
             (setq nsety (cdr nsety) y (car nsety))))))

(defun nsetmem (x nsety)
  (do ((yl nsety (cdr yl)))
      ((null yl) nil)
    (cond ((= x (car yl)) (return t))
          ((< x (car yl)) (return nil)))))

(defun nsetU (nsetx nsety)
  (do ((res)
       (x (car nsetx))
       (y (car nsety)))
      (nil)
    (cond ((null nsetx)
           (return (nreconc res (append nsety nil))))
          ((null nsety)
           (return (nreconc res (append nsetx nil))))
          ((< x y)
           (push x res)
           (setq nsetx (cdr nsetx) x (car nsetx)))
          ((< y x)
           (push y res)
           (setq nsety (cdr nsety) y (car nsety)))
          (t
           (push x res)
           (setq nsetx (cdr nsetx) x (car nsetx))
           (setq nsety (cdr nsety) y (car nsety))))))

(defun nset^ (nsetx nsety)
  (do ((res)
       (x (car nsetx))
       (y (car nsety)))
      (nil)
    (cond ((or (null nsetx) (null nsety)) (return (nreverse res)))
          ((< x y) (setq nsetx (cdr nsetx) x (car nsetx)))
          ((< y x) (setq nsety (cdr nsety) y (car nsety)))
          (t (push x res)
             (setq nsetx (cdr nsetx) x (car nsetx))
             (setq nsety (cdr nsety) y (car nsety))))))
```

```
(defun nset- (nsetx nsety)
  (do ((res)
       (x (car nsetx))
       (y (car nsety)))
      (nil)
    (cond ((or (null nsetx) (null nsety))
           (return (nreconc res (append nsetx nil))))
          ((< x y)
           (push x res)
           (setq nsetx (cdr nsetx) x (car nsetx)))
          ((< y x)
           (setq nsety (cdr nsety) y (car nsety)))
          (t
           (setq nsetx (cdr nsetx) x (car nsetx))
           (setq nsety (cdr nsety) y (car nsety))))))
```

```
;;;; Fast sets

;;; FSETs: Fast sets based on exploiting pointer equality

;;; If you know that none of the addresses of things are going to
;;; change then you can still use the sorted sets on the LispM,
;;; but you better be right!
;;;
;;; Note: If the GC gets turned back on, the fset definitions should
;;;       be made back into QSET operations. FSETMEM is MEMQ. There
;;;       is no QSETMEM.

(defun makefset (list)
  (setq list (sort list #'(lambda (x y) (< (%pointer x) (%pointer y)))))
  (do ((l list))
      (nil)
    (cond ((null (cdr l)) (return list))
          ((eq (car l) (cadr l)) (rplacd l (cddr l)))
          (t (setq l (cdr l))))))

(defun fset= (fsetx fsety)
  (do ((fx fsetx (cdr fx))
       (fy fsety (cdr fy)))
      ((or (null fx) (null fy))
       (and (null fx) (null fy))) ;return T only if sets are same size
    (cond ((not (eq (car fx) (car fy)))
           (return nil)))))

(defun fset< (fsetx fsety)
  (do ((x (car fsetx))
       (y (car fsety)))
      (nil)
    (cond ((null fsetx)
           (return t))
          ((null fsety)
           (return nil))
          ((< (%pointer x) (%pointer y))
           (return nil))
          ((< (%pointer y) (%pointer x))
           (setq fsety (cdr fsety) y (car fsety)))
          (t
           (setq fsetx (cdr fsetx) x (car fsetx))
           (setq fsety (cdr fsety) y (car fsety))))))

(defun fsetmem (x fsety)
  (do ((yl fsety (cdr yl)))
      ((null yl) nil)
    (cond ((eq x (car yl)) (return t))
          ((< (%pointer x) (%pointer (car yl))) (return nil)))))

(defun fsetU (fsetx fsety)
  (do ((res)
       (x (car fsetx))
       (y (car fsety)))
      (nil)
    (cond ((null fsetx) (return (nreconc res (append fsety nil))))
          ((null fsety) (return (nreconc res (append fsetx nil))))
          ((< (%pointer x) (%pointer y))
           (push x res)
           (setq fsetx (cdr fsetx) x (car fsetx)))
          ((< (%pointer y) (%pointer x))
           (push y res)
           (setq fsety (cdr fsety) y (car fsety)))
          (t
           (push x res)
           (setq fsetx (cdr fsetx) x (car fsetx))
           (setq fsety (cdr fsety) y (car fsety))))))
```

```
(defun fset^ (fsetx fsety)
  (do ((res)
       (x (car fsetx))
       (y (car fsety)))
      (nil)
    (cond ((or (null fsetx) (null fsety))
           (return (nreverse res)))
          ((< (%pointer x) (%pointer y))
           (setq fsetx (cdr fsetx) x (car fsetx)))
          ((< (%pointer y) (%pointer x))
           (setq fsety (cdr fsety) y (car fsety)))
          (t
           (push x res)
           (setq fsetx (cdr fsetx) x (car fsetx))
           (setq fsety (cdr fsety) y (car fsety))))))

(defun fset- (fsetx fsety)
  (do ((res)
       (x (car fsetx))
       (y (car fsety)))
      (nil)
    (cond ((or (null fsetx) (null fsety))
           (return (nreconc res (append fsetx nil))))
          ((< (%pointer x) (%pointer y))
           (push x res)
           (setq fsetx (cdr fsetx) x (car fsetx)))
          ((< (%pointer y) (%pointer x))
           (setq fsety (cdr fsety) y (car fsety)))
          (t
           (setq fsetx (cdr fsetx) x (car fsetx))
           (setq fsety (cdr fsety) y (car fsety))))))
```

```
;;;; EQ sets

;;; QSETs: Sets based on EQ-ness

;;; The following implement set operations on eq sets
;;;  (Note: MEMQ is one of these operations)

(defun makeqset (list)
  (do ((l list (cdr l)) (set nil))
      ((null l) set)
    (cond ((not (memq (car l) (cdr l)))
           (push (car l) set)))))

(defun qset= (qsetx qsety)
  (and (qset< qsetx qsety) (qset< qsety qsetx)))

(defun qset< (qsetx qsety)
  (do ((ql qsetx (cdr ql)))
      ((null ql) t)
    (cond ((not (memq (car ql) qsety))
           (return nil)))))

(defun qsetU (qsetx qsety)
  (let ((newset qsety))
    (dolist (x qsetx)
      (cond ((not (memq x qsety))
             (push x newset))))
    newset))

(defun qset^ (qsetx qsety)
  (let ((newset nil))
    (dolist (x qsetx)
      (cond ((memq x qsety)
             (push x newset))))
    (nreverse newset))) ; This NREVERSE shouldn't be necessary, should it?

(defun qset- (qsetx qsety)
  (let ((newset nil))
    (dolist (x qsetx)
      (cond ((not (memq x qsety))
             (push x newset))))
    (nreverse newset))) ; Someone seems to be relying on this NREVERSE!
                        ; The following piece of code would not ANALYZE
                        ; (gets a BARF in TD-COPY-OUTPUT-DFLOW):
                        ;   (DEFINE TEST-DO2 (I)
                        ;     (DO ((X 1 (1+ X)))
                        ;         ((= X I))
                        ;       (SETQ X (+ X 2))
                        ;       (PRINT 'FOO)))
                        ; At some point, we should find and fix the
                        ; tab that relies on sets to have order! -kmp
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; CONDITION: Basic condition primitives for the KBE
```

```
;;;; Condition Mixin

;;; CONDITION-MIXIN
;;;
;;;  A base mixin for conditions which is tolerant of unsupplied data.

(defflavor condition-mixin (description data) ()
  :initable-instance-variables
  :gettable-instance-variables
  (:default-init-plist
    :data nil
    :description nil
    :proceed-types '()))

(defmethod (condition-mixin :report) (stream)
  (cond (description (format stream "~A" description))
        (t (format stream "~A was signalled, but had no description."
                   (string-capitalize
                     (a-or-an
                       (string-downcase (de-hyphenate (typep self)))))))))

(defwhopper (condition-mixin :proceed) (proceed-type &rest args)
  (if (not (or (memq proceed-type (send self ':proceed-types))
               (eq proceed-type ':which-operations)))
      (ferror "Attempted illegal (SEND '~S ':PROCEED '~S ...)"
              self proceed-type))
  (lexpr-continue-whopper proceed-type args))

(defmethod (condition-mixin :bug-report-recipient-system) () ':kbe)

(defmethod (condition-mixin :before :bug-report-description) (stream arg)
  arg ;ignored
  (let ((options (odd-KBE-options)))
    (when options
      (format stream "~2&KBE option~P in effect:~{~% * ~A~}~%"
              (length options) options))))
```

```
;;;; Mixable condition types

;;; INTERNAL-CONDITION-MIXIN
;;;     USER-CONDITION-MIXIN
;;;
;;;  Mixing in these types allow one to distinguish conditions which
;;;  should be visible to the interface (USER conditions) from
;;;  conditions which should be visible only internally (INTERNAL
;;;  conditions)

(defflavor internal-condition-mixin () ())
(defflavor     user-condition-mixin () ())

;;;  SIMPLE-CONDITION-MIXIN
;;; WARNING-CONDITION-MIXIN
;;;   ERROR-CONDITION-MIXIN
;;;
;;;  Mixing in these types allows one to distinguish conditions which
;;;  are either used just to implement some control structure (SIMPLE
;;;  conditions); mild, ignorable errors (WARNING conditions); or
;;;  potentially fatal errors (ERROR conditions).

(defflavor  simple-condition-mixin () (condition-mixin sys:condition))
(defflavor warning-condition-mixin () (condition-mixin sys:condition))
(defflavor   error-condition-mixin () (condition-mixin sys:error))

;;; CONTINUABLE-ERROR-CONDITION-MIXIN
;;;
;;;  Like ERROR-CONDITION-MIXIN, but provides for the ability
;;;  to proceed the error with no (user) action.

(defflavor continuable-error-condition-mixin (continue-description)
           (error-condition-mixin)
  :initable-instance-variables
  :gettable-instance-variables)

(defmethod (continuable-error-condition-mixin :case :proceed :continue) ()
  ':continue)

(defmethod (continuable-error-condition-mixin
             :case :document-proceed-type :continue)
           (stream)
  (lexpr-funcall #'format stream (send self ':continue-description)
                                 (send self ':data)))
```

```
;;; Warning/Error Mixin

;;; Internal Error/Warning
;;;  We hope this kind doesn't happen in normal user interactions.

(defflavor internal-continuable-error ()
           (internal-condition-mixin continuable-error-condition-mixin))

(defflavor internal-error ()
           (internal-condition-mixin error-condition-mixin))

(defflavor internal-warning ()
           (internal-condition-mixin warning-condition-mixin))

(defflavor internal-condition ()
           (internal-condition-mixin simple-condition-mixin))

(defmethod (internal-condition :case :proceed :handled) ()
  "Declare this condition handled; stop signalling it."
  ':handled)

;;; User Error
;;;  We expect this kind might occur in normal user interactions.

(defflavor user-error ()
           (user-condition-mixin error-condition-mixin))

(defflavor user-warning ()
           (user-condition-mixin warning-condition-mixin))

(compile-flavor-methods
  internal-error internal-warning internal-condition
  user-error user-warning)
```

```
;;;; Abstractions for Errors and Warnings

;;; (WARN format-string . format-args)
;;;
;;;  This warns about an interesting but non-fatal condition
;;;  probably the fault of the user.

(defun warn (format-string &rest format-args)
  (signal 'user-warning
          ':description
             (lexpr-funcall #'format nil format-string format-args)
          ':data
             (copylist format-args)
          ':proceed-types '(:handled)))

;;; (COMPLAIN format-string . format-args)
;;;
;;;  This signals a fatal error which is likely to have been brought
;;;  on by some fault of the user's rather than a system bug.

(defun complain (format-string &rest format-args)
  (error 'user-error
         ':description
            (lexpr-funcall #'format nil format-string format-args)
         ':data (copylist format-args)))

(defprop complain t :error-reporter)

;;; (BARF format-string . format-args)
;;;
;;;  This signals a fatal error due to a system bug of some sort.

(defvar *bug* nil "Bound to a condition object during KBE error breaks.")
(defvar *bug-history* '() "List of data about previous bugs.")

(defun barf (format-string &rest format-args)
  (let ((*bug* (make-condition 'internal-error
                 ':description
                    (lexpr-funcall #'format nil format-string format-args)
                 ':data (copylist format-args)
                 ;; This :PROCEED-TYPES belongs in the call to ERROR
                 ;; later in this definition, but due to a design
                 ;; misfeature in the way :settable-instance-variables
                 ;; works, it gets ignored, so we fake the effect here.
                 ':proceed-types '())))
    (push *bug* *bug-history*)
    (error *bug*)))

(defprop barf t :error-reporter)

(defun display-bug-message (&optional (n 0) (stream standard-output))
  (let ((n (or n 0))) ;in case user used NIL explicitly
    (let ((bug-report (nth n *bug-history*)))
      (cond ((not bug-report) (format stream "(No ~:R message)" n))
            (t (format stream "~%~A~%"
                       (send bug-report ':report-string)))))))
```

```
(defun cbarf (continue-format-string error-format-string &rest format-args)
  (declare (special *debug*))
  (unless (not *debug*)
    (let ((*bug* (make-condition 'internal-continuable-error
                  ':continue-description
                  (lexpr-funcall #'format nil continue-format-string
                                 format-args)
                  ':description
                  (lexpr-funcall #'format nil error-format-string
                                 format-args)
                  ':data (copylist format-args)
                  ;; This :PROCEED-TYPES belongs in the call to SIGNAL
                  ;; later in this definition, but due to a design
                  ;; misfeature in the way :settable-instance-variables
                  ;; works, it gets ignored, so we fake the effect here.
                  ':proceed-types '(:continue))))
      (push *bug* *bug-history*)
      (let ((vals (multiple-value-list (signal *bug*))))
        (selectq (car vals)
          (:continue (values))
          (otherwise
           (ferror "Tried to proceed with ~{~S~^, ~}." vals)))))))

(defprop cbarf t :error-reporter)
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; ENGLISH: Utilities for manipulating strings containing English text.

;;; This library is part of the explanation facility originally
;;; written by Scott Cyphers. For more info on that, see comments
;;; in the EXPLAIN module.
```

```
;;; (AUTO-FILL-STRING text-string [line-length] [fill-prefix] [whitespace])
;;;
;;;  Returns a version of TEXT-STRING which has been filled to the given
;;;  LINE-LENGTH. FILL-PREFIX is used as a leader on any extra lines
;;;  produced by filling. WHITESPACE can be a list of characters to
;;;  be treated as whitespace.

(defun auto-fill-string (text-string
                         &optional line-length fill-prefix whitespace)
  (with-output-to-string (string-stream)
    (auto-fill-stream string-stream
                      text-string
                      line-length
                      fill-prefix
                      whitespace)))

;;; (AUTO-FILL-STREAM stream text-string
;;;    [line-length] [fill-prefix] [whitespace])
;;;
;;;  Fills TEXT-STRING to a given stream.
;;;
;;;  STREAM is expected to handle string output operations and
;;;   a :READ-CURSORPOS message.
;;;
;;;  LINE-LENGTH defaults to 60. and specifies the target length of
;;;   the filled lines.
;;;
;;;  FILL-PREFIX, if supplied, is used to pad after any carriage returns
;;;   output by filling.
;;;
;;;  WHITESPACE defaults to the value of *AUTO-FILL-CHARS*, and specifies
;;;   which characters are to be treated as whitespace.

(defvar *auto-fill-chars* '(#\return #\tab #\space))

(defun auto-fill-stream (stream text-string
                         &optional line-length fill-prefix whitespace)
  ;; Default our args (allow NIL to mean not-supplied)
  (setq line-length (or line-length 60.))
  (setq whitespace (or whitespace *auto-fill-chars*))
  ;; Main algorithm
  (do ((cursor 0 new-cursor)
       (new-cursor))
      ((not cursor))
    (setq new-cursor (string-search-set whitespace text-string cursor))
    (let ((token (substring text-string cursor new-cursor))
          (pos   (send stream ':read-cursorpos)))
      (cond ((zerop pos))
            ((> (+ pos                    ; current pos
                   1                      ; the space we might output
                   (string-length token)) ; the chars we might output
                line-length)
             (format stream "~%~@[~A~]" fill-prefix))
            (t
             (format stream " ")))
      (format stream "~A" token)
      (when new-cursor
        (setq new-cursor
              (string-search-not-set whitespace
                                     text-string
                                     new-cursor))))))
```

```
;;;; Phrase Manipulation

;;; (PHRASE-APPEND conjunction phrase-list)
;;;
;;;  Appends some number of text phrases (strings) using the given
;;;  conjunction (if more than one phrase given) and appropriate
;;;  numbers of commas (if more than two phrases given).
;;;
;;;  eg,
;;;     (phrase-append "and" '("a"))            => "a"
;;;     (phrase-append "and" '("a" "b"))        => "a and b"
;;;     (phrase-append "and" '("a" "b" "c"))    => "a, b, and c"
;;;
;;;  If any of the phrases contains a "," we use a ";" instead to
;;;  help avoid confusion.
;;;
;;;  eg,
;;;     (phrase-append "or" '("a, b, and c" "d" "e"))
;;;        => "a, b, and c; d; or e"

(defun phrase-append (conjunction phrase-list)
  (let ((reversed (reverse phrase-list))
        (comma (cond ((some phrase-list #'(lambda (phrase)
                                            (string-search "," phrase)))
                      "; ")
                     (t ", "))))
    (selectq (length reversed)
      ((0) "-??-")
      ((1) (car reversed))
      ((2) (format nil "~A ~A ~A"
                   (cadr reversed) conjunction (car reversed)))
      (otherwise
           (let ((phrase nil))
             (push (pop reversed) phrase)
             (push (format nil "~A~A " comma conjunction) phrase)
             (push (pop reversed) phrase)
             (push comma phrase)
             (do ()
                 ((null reversed)
                  (apply 'string-append phrase))
               (push (pop reversed) phrase)
               (if reversed (push comma phrase))))))))
```

```
;;; (A-OR-AN noun-string)
;;;
;;;  Prepends the appropriate one of either "a " or "an " to
;;;  the given noun-string.

(defun a-or-an (noun-string)
  (format nil "~A ~A"
          (select-a-or-an noun-string)
          noun-string))

;;; (SELECT-A-OR-AN noun-string)
;;;
;;;  Selects the appropriate one of either "a" or "an" for the
;;;  given noun-string. Tries to hack as many screw cases as it
;;;  can get in a general way...
;;;
;;;  eg,
;;;     (SELECT-A-OR-AN "foo")                    => "a"
;;;     (SELECT-A-OR-AN "euphemism")              => "a"
;;;     (SELECT-A-OR-AN "example")                => "an"
;;;     (SELECT-A-OR-AN "unicorn")                => "a"
;;;     (SELECT-A-OR-AN "unimportant detail")     => "an"
;;;     (SELECT-A-OR-AN "unique item")            => "a"
;;;
;;;  but,
;;;     (SELECT-A-OR-AN "unidentified loss")      => "a"

(defvar *alphabet* "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz")

(defun select-a-or-an (string)
  (setq string (string string))
  (let ((len (string-length string))
        (idx (string-search-not-set *alphabet* string)))
    (cond ((zerop len) "a") ;random
          ((= len 1)
           (cond ((memq (char-upcase (aref string 0))
                        '(#/A #/E #/F #/H #/I #/L #/M #/N
                          #/O #/R #/S #/X))
                  "an")
                 (t "a")))
          (idx (select-a-or-an (substring string 0 idx)))
          ((or (%string-equal "EU" 0 string 0 2)
               (and (> len 4)
                    (%string-equal "UNI" 0 string 0 3)
                    (or (memq (char-upcase (aref string 3))
                              '(#/A #/E #/I #/O #/U #/Y))
                        (memq (char-upcase (aref string 4))
                              '(#/A #/E #/I #/O #/U #/Y)))))
           "a")
          ((and (> len 2)
                (= (char-upcase (aref string 0)) #/U)
                (= (char-upcase (aref string 2)) #/E))
           "a")
          ((memq (char-upcase (aref string 0))
                 '(#/A #/E #/I #/O #/U))
           "an")
          (t "a"))))

(defun select-is-or-are (symbol)
  (or (ignore-errors
        (if (member (aref (string symbol) (1- (string-length symbol)))
                    '(#/s #/S))
            "are" "is"))
      "is//are"))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; SYSTEM-MAGIC: Miscellaneous LispM System Internals Magic

;;;; System Utilities

;;; Purpose
;;;
;;;   This module contains code which makes strong assumptions about the
;;;   internal workings of the Lisp Machine in order to provide some
;;;   higher level abstractions useful elsewhere in the KBE.
;;;
;;; Dependency Information
;;;
;;;   All code in this module is vanilla. It lives in the KBE package
;;;   because it is part of the KBE system, but it requires no special
;;;   support from the KBE system and can be lifted freely for other
;;;   applications.
```

```
;;; (LOADED-BAND-STRING)
;;;
;;;  Returns a string containing a phrase describing the loaded
;;;  band including the number, comment (if any), and microcode.
;;;  eg, the return value might be something like:
;;;
;;;    "band 9 (Black Magic) with microcode 17"

(defsubst system-additional-info ()
  (if (and (boundp 'si:system-additional-info)
           (plusp (array-active-length si:system-additional-info)))
      si:system-additional-info))

(defun loaded-band-string ()
  #+3600 (format nil "~A~@[ (~A)~]"
                 si:loaded-band-name
                 (system-additional-info))
  #-3600 (format nil "band ~C~@[ (~A)~] with microcode ~D"
                 (ldb #o2010 si:current-loaded-band)
                 (system-additional-info)
                 si:%microcode-version-number))

;;; (SITE-STRING)
;;;
;;;  Returns a string containing the site name. eg, the return value
;;;  might be something like:
;;;
;;;             "Lisp Machine Twenty-Two"

(defun site-string ()
  (values (chaos:host-data)))
```

```
;;; (RELEASED-SYSTEM-VERSION-ACTIVE? system)
;;;
;;;  Returns true if system is the name of a system which is on the
;;;  release list and its current patch version is the version effective
;;;  as of the release.

(defun released-system-version-active? (system)
  (not (assoc system (si:unreleased-patch-systems))))

;;; (KBE-VERSION-STRING-FOR-PATHNAME)
;;;
;;;  Returns a string which is suitable for use as an FN1 in most
;;;  filesystems and which is characteristic of the current KBE system.
;;;  eg, In KBE 103.7, (KBE-VERSION-STRING-FOR-PATHNAME) => 103-07

(defun kbe-version-string-for-pathname ()
  (multiple-value-bind (major minor)
      (si:get-system-version "KBE")
    (cond ((zerop minor) (format nil "~3,'0D" major))
          (t (format nil "~3,'0D-~2,'0D" major minor)))))

;;; (SYSTEM-VERSION-STRING system-name)
;;;
;;;  Returns a string representing the id of a given system name. Uses a
;;;  compact format for system status when possible. eg, here are some
;;;  sample conditions and return values:
;;;
;;;    (SYSTEM-VERSION-STRING "KBE")    => "KBE 27.3x"      ; Experimental
;;;    (SYSTEM-VERSION-STRING "System") => "System 210.128" ; Released
;;;    (SYSTEM-VERSION-STRING "FOO")    => "FOO 0.0 (non-existent)"
;;;
;;;  By special exception, accepts the keyword :RELEASE in place of a
;;;  string and returns a release system version string. eg,
;;;
;;;    (SYSTEM-VERSION-STRING ':RELEASE) => "Release 4.0"

(defun system-version-string (system)  ;used in com-Bug-KBE
  (multiple-value-bind (major minor state)
      (cond ((eq system ':release)
             (setq system "Release")
             (si:get-release-version))
            ((not (stringp system))
             (barf "Bad argument to SYSTEM-VERSION-STRING: ~S" system))
            (t
             (si:get-system-version system)))
    (format nil "~A ~D.~D~A"
            system (or major 0.) (or minor 0.)
            (caseq state
              ((:released) "")
              ((:experimental) "x")
              ((nil) " (non-existent)")
              (t (format nil " (~A)" (string-downcase state)))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10.; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; FLAVOR-MAGIC: Useful flavor utilities

;;;; Flavor Utilities

;;; Purpose
;;;
;;;   This module contains code which may tie heavily into the
;;;   internal workings of the Lisp Machine flavor system in order
;;;   to provide some higher level flavor abstractions useful
;;;   elsewhere in the KBE.
;;;
;;;
;;; Dependency Information
;;;
;;;   All code in this module is vanilla. It lives in the KBE package
;;;   because it is part of the KBE system, but it requires no special
;;;   support from the KBE system and can be lifted freely for other
;;;   applications.
```

```
;;;; Name Mixin

;;; NAME-MIXIN
;;;
;;; This mixin is intended to add flexible print methods to a flavor.
;;;
;;;
;;; :NAME a-name                    (NAME-MIXIN init option)
;;;  Initializes the name of the flavor to a-name. If not supplied,
;;;  it is initialized based on the type of the flavor being instantiated
;;;  and the number of previous anonymous instances created. eg, the
;;;  first anonymous instance of SPACE-MAN would be called "Space Man 1".
;;;
;;; :SET-NAME a-name                (NAME-MIXIN message)
;;;  Sets the name of the flavor to a-name.
;;;
;;; :NAME                           (NAME-MIXIN message)
;;;  Gets the name of the flavor.
;;;
;;; Objects of this flavor handle the :PRINT-SELF message displaying as
;;; #<type name octal-address> for PRIN1, or as just their name for PRINC.
;;;
;;; eg,         (FORMAT T "~S" (MAKE-INSTANCE 'SPACE-MAN))
;;;
;;;             displays:   #<SPACE-MAN "Space Man 1" 123456>
;;;
;;;             (FORMAT NIL "~A" (MAKE-INSTANCE 'SPACE-MAN))
;;;
;;;             displays:   Space Man 1

(defflavor name-mixin (name) ()
  (:documentation :mixin "Adds a :NAME message to a flavor.")
  :gettable-instance-variables
  :settable-instance-variables
  :initable-instance-variables
  (:default-init-plist :name nil))

(defmethod (name-mixin :after :init) (&rest ignore)
  (when (not name)
    (let ((my-type (typep self)))
      (setq name (format nil "~A ~D"
                         (string-capitalize-words my-type)
                         (setf (get my-type 'anonymous-instance-count)
                               (1+ (or (get my-type
                                            'anonymous-instance-count)
                                       0)))))))))

(defmethod (name-mixin :print-self) (stream level prin1? &rest ignore)
  level ;ignored
  (let ((my-name (or (send self ':name) "Anonymous")))
    (cond ((not prin1?) (format stream "~A" my-name))
          (t
           (format stream "#<~S ~S ~O>"
                   (typep self) my-name (%pointer self))))))
```

```
;;;; Property List Mixin

;;; PROPERTY-LIST-MIXIN
;;;
;;;  This mixin allows an object to have its own property list.
;;;  This may be extremely useful for objects that change a lot,
;;;  so there is an easy way to simulate the effect of extra
;;;  instance variables during debugging.
;;;
;;;  :PROPERTY-LIST
;;;    Gets the property list.
;;;
;;;  :PROPERTY-LIST-POINTER
;;;    Gets a locative to the property list.
;;;
;;;  :GET-PROPERTY indicator
;;;    Gets the value of the property named by INDICATOR.
;;;
;;;  :PUT-PROPERTY indicator value
;;;    Places VALUE in the property slot named by INDICATOR.
;;;
;;;  :REMOVE-PROPERTY indicator
;;;    Removes the slot named by INDICATOR from the object's
;;;    property list.

(defflavor property-list-mixin ((property-list '())) ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (property-list-mixin :property-list-pointer) ()
  (locf property-list))

(defmethod (property-list-mixin :get-property) (indicator)
  (get (locf property-list) indicator))

(defmethod (property-list-mixin :put-property)
           (indicator &optional (value (or (get (locf property-list)
                                                indicator)
                                           t)))
  (putprop (locf property-list) value indicator))

(defmethod (property-list-mixin :remove-property) (indicator)
  (remprop (locf property-list) indicator))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; MACRO: This module contains general lisp utility macros for use
;;;        by any other module in the KBE system except BASIC.
;;;
;;;        This module also contains some support functions needed at
;;;        runtime by the macros it defines.
```

```
;;;; Control Structure

;;; (REPEAT let-bvl body)
;;;
;;;  REPEAT is an infinite loop. It takes optional variable bindings
;;;  a la LET. It does not allow assignment specs for the second pass
;;;  through; for that, use DO.
;;;

(defmacro repeat (bvl &body body)
  ;; Interim error check. Can go away after next full system recompilation
  (dolist (statement body)
    (if (atom statement)
        (ferror "GO tags not allowed in REPEAT: ~S" statement)))
  '(let ,bvl (do () (nil) (progn ,@body))))

;;; (REPEAT* let-bvl body)
;;;
;;;  This is the same as REPEAT, but uses destructuring LET*, not LET.
;;;

(defmacro repeat* (bvl &body body)
  ;; Interim error check. Can go away after next full system recompilation
  (dolist (statement body)
    (if (atom statement)
        (ferror "GO tags not allowed in REPEAT*: ~S" statement)))
  '(let* ,bvl (do () (nil) (progn ,@body))))


;;; (RETURN-IF x)      Returns x iff x is non-null.
;;; (RETURN-IF-NOT x)  Returns x iff x is null.

(defmacro return-if (x)
  (once-only (x) '(if ,x (return ,x))))

(defmacro return-if-not (x)
  '(return-if (not ,x)))
```

```
;;;; Binding

(defmacro with-defaults (vars-and-locs &body forms)
  '(let ,vars-and-locs
     (unwind-protect (progn ,@forms)
                     ,@(mapcar #'(lambda (x) '(setf ,(cadr x) ,(car x)))
                               vars-and-locs))))
```

```
;;;; Lisp data accessors

;;; These are intended only for very low-level use in specifying data
;;; storage layouts. Not for normal user code.
;;;
;;; Note: These should rightly be funcallable as well, but there's little
;;;       point. SUBSTs currently don't compile right, so better we should
;;;       get an error if they are funcalled since currently we don't
;;;       intentionally funcall them anyway...

(defmacro cr (x) x)
(defmacro cdddddr (x) '(cdr (cddddr ,x)))
```

```
;;;; Control Structure

;;; These macros are not being phased out and are acceptable for
;;; writing into new code.

;;; (WHEN-ABNORMAL-EXIT exp form1 form2 form3 ...)
;;;
;;;  Executes exp, returning its value.
;;;  If an abnormal exit is done from exp, form1, form2, ... are executed.

(defmacro when-abnormal-exit (exp &body abnormal-exit-forms)
  (let ((var (gensym)))
    `(let ((,var t))
       (unwind-protect (values-list
                         (prog1 (multiple-value-list ,exp)
                                (setq ,var nil)))
                       (cond (,var ,@abnormal-exit-forms))))))

;;; (WHEN-ERROR exp form1 form2 ...)
;;;
;;; If exp doesn't err, returns the value it returns.
;;;
;;; If exp errs, traps the error and executes forms.
;;;    In this case, the value of the last form is returned.

(defvar *when-error-debug* nil)

(defmacro when-error (exp &body forms)
  `(condition-case-if (not *when-error-debug*) ()
       ,exp
     (sys:error ,@forms)))
```

```
;;; (CONDITION-RESTART-CASE-IF condition form
;;;   ((flavor1 format-string1 . format-args1) . action1)
;;;   ((flavor2 format-string2 . format-args2) . action2)
;;;   ...)
;;;
;;; Evaluates CONDITION, then executes FORM.
;;; If the condition evaluated to something true, the FORM is
;;; evaluated in a context where restart handlers are in effect
;;; according to the clauses (first clause is innermost restart
;;; handler). The action for each clause is taken IFF that restart
;;; handler is selected.

(defmacro condition-restart-case-if (cond form &body clauses)
  (cond ((null clauses) form)
        (t
         (let ((tag (gensym)))
           '(condition-restart-case-if ,cond
                (prog ,tag ()
                      (declare (si:invisible-block t))
                      (catch-error-restart-if ,cond
                                              ,(caar clauses)
                        (return-from ,tag ,form))
                      (return-from ,tag (progn ,@(cdar clauses))))
              ,@(cdr clauses))))))

(defmacro condition-restart-case (form &body clauses)
  '(condition-restart-case-if t ,form ,@clauses))
```

```
;;;; File-I/O

;;; (FILE-DEFAULT-FN2 filename type version)
;;;
;;;  Does filename defaulting for the fn2 and/or version number.
;;;  This works in a reasonably file-system independent way.
;;;
;;;  Eg, something like this should happen:
;;;
;;;   (FILE-DEFAULT-FN2 "foo;bar" 'PLAN ':newest)
;;;       => #<ITS-PATHNAME "FOO;BAR PLAN">
;;;
;;;   (FILE-DEFAULT-FN2 "ee:<kmp>foo" 'PLAN ':oldest)
;;;       => #<TOPS20-PATHNAME "EE:PS:<KMP>FOO.PLAN.-2">

(defmacro file-default-fn2 (filename &optional (type    ''':unspecific)
                                               (version ''':unspecific))
  '(fs:merge-pathname-defaults (fs:parse-pathname ,filename)
                               NIL ;no full pathname default
                               ,type
                               ,version))
```

```
;;;; List/String Handling

;;; (COLLECT (var list) predicate-expression)
;;;
;;;  var should be a symbol. list is a list to iterate across.
;;;  on each pass of the iteration, predicate-expression is evaluated
;;;  in a context where var is bound to a new element of the list.
;;;  collects and returns a list of all values of var for which
;;;  predicate-expression returns true.

(defmacro collect ((var list) predicate)
  (let ((lvar (gensym)) (rvar (gensym)))
    '(do-named t ;makes us invisible to any RETURN or GO in the predicate
               ((,lvar ,list (cdr ,lvar))
                (,rvar nil)
                (,var))
               ((null ,lvar) ,rvar)
       (setq ,var (car ,lvar))
       (if ,predicate (push ,var ,rvar)))))
```

```
;;;; Message Passing

;;; (MY symbol)
;;;
;;;  Same as (SEND SELF ':symbol).

(defmacro my (keyword)
  '(send self ',(intern keyword "")))
```

```
;;;; Stacks

;;; These are useful functions to use along with PUSH and POP in
;;; dealing with lists as stacks.

;;; (WHEN-POP-FAILS stack . actions-if-empty)
;;;
;;;  If stack is a list, returns the top of the stack (popping that
;;;  element at the same time).
;;;
;;;  If stack is an atom or is empty, runs actions-if-empty instead.

(defmacro when-pop-fails (stack &body actions-if-empty)
  (cond (actions-if-empty
         '(cond ((atom ,stack) ,@actions-if-empty)
                (t (pop ,stack))))
        (t '(pop ,stack))))

;this is just another mnemonic for CAR except that it will also optionally
;check whether the stack is a valid stack.

(defmacro top (stack . actions-if-empty)
  (cond (actions-if-empty
         '(cond ((atom ,stack) ,@actions-if-empty)
                (t (car ,stack))))
        (t '(car ,stack))))
```

```
;;;; SETF-related extensions

;;; (<- loc expression loc expression ...)
;;;
;;;  Like a multi-arg version of SETF.

(defmacro <- (&rest body)
  (do ((code nil) (l body (cddr l)))
      ((null l)
       (cond ((null (cdr code)) (car code))
             (t '(progn ,@(nreverse code)))))
    (if (null (cdr l))
        (ferror "Odd number of arguments to <-~% ~S" '(<- ,@body)))
    (push '(setf ,(car l) ,(cadr l)) code)))
```

```
;;; (DEFDEFMACRO defining-name (name bvl . body) . forms)
;;;
;;; This is like DEFMACRO but is specially designed for macros that
;;; define other macros. It sets up appropriate declarations so that
;;; ZWEI can later find definitions of things which were the result
;;; of an expansion of that macro.

(defmacro defdefmacro (defining-name
                       defining-type
                       (name-var bvl-var . body-var)
                       &body forms)
  (let ((namespec-var (gensym)))
    (make-defdefmacro-source-file-wrapper
      'defdefmacro
      defining-name
      '(local-declare ()
         ;(defprop ,defining-name defun zwei:definition-function-spec-type)
         (defprop ,defining-name ,defining-type si:definition-type-name)
         (defmacro ,defining-name (,namespec-var ,bvl-var &body ,body-var)
           (let ((,name-var ,namespec-var))
             (make-defdefmacro-source-file-wrapper ',defining-name
                                                   ,namespec-var
                                                   (progn ,@forms)))))))

(defprop defdefmacro defun zwei:definition-function-spec-type)
```

```
;;; (DEFINE-SETF-HANDLER name (form-pattern value) . body)
;;;
;;;  This is an extended version of DEFSETF which also works when
;;;  you want to get your hands on the whole input form, or some tail
;;;  of it. It was mostly necessary in Maclisp where DEFSETF is a pain.
;;;  On the LispM, it's pretty straightforward...

(defmacro define-setf-handler (name (form value) &body body)
  (let ((temp (gensym)))
    '(eval-when (eval load compile) ;This EVAL-WHEN is losing because
                                    ; it side-effects the environment
                                    ; global to the compiler, but we
                                    ; have to have it because otherwise
                                    ; the SETF property won't be defined
                                    ; in time for our compiled macros to
                                    ; use it...
                ;; These convolutions get around warnings about redefining
                ;; existing SETF handlers if there are any...
                (defun (,name :nsetf) (,temp ,value)
                  (let ((,form ,temp)) ,@body))
                (cond ((and (get ',name ':setf) ;warn only first time...
                            (not (get ',name ':nsetf)))
                       (grumble
                         "~%;Replacing system SETF property for ~S ~
                               with an upward compatible definition.~
                         ~%; The old version is saved on the ~
                               :OSETF property."
                         ',name)
                       (putprop ',name (get ',name ':setf) ':osetf)))
                (putprop ',name
                         (get ',name ':nsetf)
                         ':setf))))

;;; This makes it possible to assign to a [] expression
;;; The basic idea is to define SETFs for
;;; the basic list constructors CONS NCONS LIST LIST*.
;;; This will then work to destructure something if you say
;;; (SETF (LIST* A B) X) or (SETF '(,A ,@B) X) or (SETF [A ! B] X)
;;; note that there is no particularly good thing to do with
;;; append, since it is ambiguous.

;;; (IGNORABLE-BY-SETF? obj)
;;;
;;;  Returns T if obj is something our destructuring SETF extensions
;;;  should not bother trying to construct an assignment for. This
;;;  spans things like (FUNCTION ...), (QUOTE ...), T, NIL, numbers, etc.

(defun ignorable-by-setf? (obj)
  (cond ((listp obj)   (memq (car obj) '(quote function)))
        ((symbolp obj) (memq obj '(t nil)))
        (t t)))

;;; (SETF (CONS a b) cons)
;;;
;;;  Assigns a the (CAR cons) and b the (CDR cons).

(define-setf-handler cons ((nil a b) v)
  (let* ((var (gensym)))
    '(let ((,var ,v))
       ,@(cond ((not (ignorable-by-setf? a)) '((setf ,a (car ,var)))))
       ,@(cond ((not (ignorable-by-setf? b)) '((setf ,b (cdr ,var))))))))

;;; (SETF (NCONS a) cons)
;;;
;;;   Assigns a the (CAR cons).
;;;
;;;   Presumably the car is NIL, but no check is actually made.
```

```
(define-setf-handler ncons ((nil a) v)
  (cond ((not (ignorable-by-setf? a)) '(setf ,a (car ,v)))))

;;; (SETF (LIST x y z ..) list)
;;;
;;;  Assigns x, y, z, ... to successive cars of list.

(define-setf-handler list ((nil . args) v)
  (let* ((var (gensym)))
    (do ((tos args (cdr tos)) (step '(setq ,var (cdr ,var))) (result))
        ((null tos) '(let ((,var ,v)) ,@(nreverse result)))
        (cond ((not (eq tos args)) (push step result)))
        (cond ((not (ignorable-by-setf? (car tos)))
               (push '(setf ,(car tos) (car ,var)) result))))))

;;; (SETF (LIST* x y z ... w) list)
;;;
;;;  Like (SETF (LIST x y z ... w) list) except w receives the remaining
;;;  tail, not the car of the tail.

(define-setf-handler list* ((nil . args) v)
  (let* ((var (gensym)))
    (do ((tos args (cdr tos)) (step '(setq ,var (cdr ,var))) (result))
        ((null tos) '(let ((,var ,v)) ,@(nreverse result)))
      (cond ((not (eq tos args)) (push step result)))
      (cond ((ignorable-by-setf? (car tos)))
            ((cdr tos) (push '(setf ,(car tos) (car ,var)) result))
            (t  (push '(setf ,(car tos) ,var) result))))))
```

```
;;;; Array abstractions

;;; (ARRAYDEFS (name type) (name type) ...)
;;; or
;;; (ARRAYDEFS name name name ...)
;;;
;;;  This takes in a list of atom type pairs and sets things up to access
;;;  the atoms as arrays. The type info was for Maclisp and is now
;;;  obsolete.
;;;
;;;  It makes macros, so that the atom name can be used to refer to the
;;;  array, even though the array is the value of the atom and not the
;;;  array property of the atom.  The fact that it is the value makes
;;;  parameter passing easier.
;;;
;;;  Note: ARRAYDEFS does NOT allocate any arrays or initialize the atoms
;;;        in any way.  You must do that elsewhere.

(defmacro arraydefs (&rest pairs)
  '(progn 'compile
          ,@(mapcar #'(lambda (form)
                        (let ((name (cond ((symbolp form) form)
                                          (t (car form)))))
                          '(defmacro ,name (&rest args)
                             '(aref ',name ,@args))))
                    pairs)))
```

```
;;;; ALIST utility macros

;;; The following macros implement simple actions on a new data type the
;;; array with non-numeric indices.  This is actually implemented as an
;;; alist.  The intention is that it should be possible to write an
;;; expression just as if you were using an array i.e. (foo 'bar) and get
;;; the element in foo associated with 'bar.  The main macros are
;;; DEF-ALIST and DEF-ALISTQ which are like ARRAYDEFS above.  They generate
;;; a set of macros which can be used to fetch and store in an alist.
;;; you must also bind the name of the alist as some kind of variable and
;;; initialize it.  NIL is the empty alist and is an appropriate
;;; initialization in general.
;;;
;;; It is important to note that these macros work by side-effecting
;;; the alist (sometimes by a setq to the atom holding the alist).
;;; This is why they must be macros, and forces some care in their use.
;;; Particularly, if an alist is passed to a subroutine, then
;;; some side effects will reflect back to the caller and some won't,
;;; unless the alist name is a special variable which is used as the name
;;; of the alist in both the caller and called.
;;;
;;; More specifically two basic operations are supported by the
;;; macros produced by a call (DEF-ALIST[Q] FOO).  First an individual
;;; element of the alist (which is stored as the value of the atom FOO)
;;; can be referrenced as (FOO BAR).  this always returns a value no
;;; matter whether or not the alist actually has an explicit entry for
;;; BAR.  NIL is returned as the implicit value for all elements not in
;;; the alist.  there is thus no way to tell if an element is explicitly
;;; present unless you know that NIL is not a possible explicit value.
;;; (you can use ASSOC/ASSQ if you really want to know this.  the
;;; intention of this package is that an alist is an array-like object
;;; that implicitly has an infinite number of entries.)  Second, the
;;; form (setf (FOO BAR) BAZ) can be used to assign a value to an element
;;; of the alist.  this creates an explicit element of the alist if one
;;; does not already exist (to do this it does a setq to foo so beware).
```

```
;;; This is the defining form.  It must be used at the top level since it
;;; produces defprops that act like defuns.

(defmacro def-def-alist (name accessor)
  '(defmacro ,name (atom)
     (let ((temp (gensym 'g)))
       '(progn 'compile
               (defmacro ,atom (key)
                 '(cdr (,',accessor ,key ,',atom)))
               (define-setf-handler ,atom ((nil key) v)
                 '(let ((,',temp ,key))
                    (rplacd (or (,',accessor ,',temp ,',atom)
                                (car (setq ,',atom (cons (ncons ,',temp)
                                                         ,',atom))))
                            ,v)))
               ',atom))))

;;; This defines our def-alist guys...

(def-def-alist def-alist  assoc)
(def-def-alist def-alistq assq )
```

```
;;; (MEMOIZE var key . body)
;;;
;;;  This macro is usually wrapped around a function and used to memoize
;;;  its results in order to save recomputation.  It could also be used
;;;  when recomputation was undesirable for any reason.  VAR must be a
;;;  variable bound outside the scope of a set of calls on memoize.
;;;  memoize first looks (with assoc) on VAR to see if a result is
;;;  recorded under KEY.  If there is it is returned.  If not, BODY is
;;;  computed and the result stored under KEY on VAR.  This is obviously
;;;  related to the alist fns above

(defmacro memoize (var key &body original-body)
  (let ((body (cond ((cdr original-body)
                     '(progn ,@original-body))
                    (t (car original-body)))))
    '(cdr (or (assoc ,key ,var)
              (top (push (cons ,key ,body) ,var))))))

;;; (MEMOIZEQ var key . body)
;;;
;;;  The same as MEMOIZE except that it uses ASSQ

(defmacro memoizeq (var key &body original-body)
  (let ((body (cond ((cdr original-body) '(progn ,@original-body))
                    (t (car original-body)))))
    '(cdr (or (assq ,key ,var)
              (top (push (cons ,key ,body) ,var))))))
```

```
;;;; Process locking utility macros

;;; (WITH-LOCK (loc . options) . body)
;;;
;;;  Waits for the location given by lock-id to become nil, then siezes
;;;  the lock itself and runs body.

(defmacro with-lock ((loc . options) &body body)
  (let ((var  (gensym))
        (opts (locf options)))
    (let ((whostate (or (get opts ':whostate) "Lock")))
      '(let ((,var (locf ,loc)))
         (unwind-protect (progn (process-lock ,var nil ,whostate)
                                ,@body)
                         (process-unlock ,var nil nil))))))

;;; (DEFUN-LOCKED fn bvl . body)
;;;
;;;  This is like (DEFUN fn bvl . body) except that the function fn
;;;  is not re-entrant. If you try to invoke fn recursively or
;;;  simultaneously from some other process, it will block waiting
;;;  for the first invocation to terminate before starting a second.

(defmacro defun-locked (namespec bvl &body body)
  (let ((var (intern (format nil "*FUNCTION-~A-LOCK*" namespec)
                     (cond ((symbolp namespec) (symbol-package namespec))
                           (t package)))))
    '(local-declare ()
       (defvar ,var nil
         ,(format nil "Synchronization lock for ~S" namespec))
       (defun ,namespec ,bvl
         (with-lock (,var :whostate
                          ,(format nil "~A"
                                   (cond ((symbolp namespec)
                                          (string-capitalize-words namespec))
                                         (t namespec))))
           ,@body)))))
```

```
;;;; Noises

(defmacro define-noise (name &rest args-info)
  '(init-noise-for ',name ,@(mapcar #'(lambda (x) '',x) args-info)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; KVARS: This module contains declarations and simple initializations
;;;        for variables used by more than one module.

(defvar *KBE-package* (pkg-find-package "KBE")
  "Cache for the KBE package object.")

(defvar *Vanilla-package* (pkg-find-package "VANILLA")
  "Special purpose package for handling cliches. Not for KBE code!")

(defvar *KBE-user-package* *KBE-package*
  "Package KBE should build code in.")

(defvar *keyword-package* (pkg-find-package "")
  "Catch for the KEYWORD package object." )
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; KMAC: This module contains KBE-specific utility macros for use
;;;       by any other module in the KBE system except BASIC and MACRO.
;;;
;;;       It may also contain related variable definitions.
```

```
;;;; Interface Control

;;; These primitives are mainly for use in KBE bootstrapping and
;;; debugging. Except for special reasons (debugging, self-testing),
;;; they should not be called dynamically by the running KBE.
;;; Unless the interface is not broken, it shouldn't ever be disabled
;;; anyway.

;;; (ENABLE-INTERFACE)
;;; (DISABLE-INTERFACE)

(defvar *interface-enabled* nil ;Set to T after bootstrapping system.
  "If false, the KBE interface is disabled.")

(defun enable-interface  () (setq *interface-enabled* t))
(defun disable-interface () (setq *interface-enabled* nil))

;;; (WITH-INTERFACE-DISABLED ...forms...)
;;;
;;;   Runs FORMS in a context where the interface is bound off,
;;;   returning the value of the last form.

(defmacro with-interface-disabled (&body forms)
  '(let ((*interface-enabled* nil))
     ,@forms))

;;; (WHEN-INTERFACE-ENABLED ...forms...)
;;;
;;;   If executed in a context where the interface is enabled,
;;;   its effect and value is the same as for PROGN.
;;;
;;;   If executed in a context where the interface is disabled,
;;;   it has no effect and returns an undefined value.

(defmacro when-interface-enabled (&body forms)
  '(cond (*interface-enabled*
          ,@forms)
         (t ;We should never be being used for value in the
            ;interface, but in order to ease debugging, we'll
            ;return a distinctive marker just in case.
          '<interface-disabled>)))
```

```
;;;; Initializations

(defvar *system-initializations* '() "KBE initializations list.")

(defmacro define-system-initialization (name &body forms)
  (cond ((not forms)
         '(delete-initialization ',name '() '*system-initializations*))
        (t
         '(add-initialization ',name
                              ',(cond ((cdr forms) '(progn ,@forms))
                                      (t (car forms)))
                              '()
                              '*system-initializations*))))

(defun initialize-system (&optional redo?)
  (initializations '*system-initializations* redo?))

(define-system-initialization enable-interface
  (enable-interface))
```

```
;;;; KBE Option Set

;;; *KBE-switches*
;;;
;;;  An alist of switch names and variables.

(defvar *KBE-switches* '())

(defun set-switch (switch value)
  (declare (values success? comment-string))
  (let ((var-info (assq switch *KBE-switches*))
        (switch-name (string-downcase switch)))
    (cond ((not var-info)
           (values nil
                   (text "I don't know about any switch named /"~A/"."
                         switch-name)))
          (t
           (let ((old-value (symeval (cadr var-info))))
             (set (cadr var-info) value)
             (values t
                     (text (cond ((eq old-value value)
                                  "~:[Not~;Already~] in ~A mode.")
                                 (t
                                  "~:[Disabled~;Enabled~] ~A mode."))
                           value switch-name)))))))

;;; *KBE-Options*
;;; *KBE-Internal-Options*
;;; *KBE-Trace-Options*
;;;
;;;  Options defined in this set control the various parameterized
;;;  actions of the KBE which are intended for the user to be able
;;;  to control.

(defmacro defdef-option (name doc options-list)
  '(local-declare ()
     (define-user-option-alist ,options-list)
     (defprop ,name ,doc si:definition-type-name)
     (defmacro ,name (var doc default-val type &rest data)
       '(local-declare ()
          (define-user-option (,var ',options-list)
                              ,default-val ,type ,doc ,@data)
          ,(when (eq type ':boolean)
             (let ((tag (intern-keyword (string-trim "*" var))))
               '(if (not (memq ',tag *kbe-switches*))
                    (push (list ',tag ',var) *KBE-switches*))))
          ',var))))

(defdef-option define-KBE-option      "KBE option"
  *KBE-Options*)
(defdef-option define-internal-option "KBE internal option"
  *KBE-Internal-Options*)
(defdef-option define-trace-option    "KBE trace option"
  *KBE-Trace-Options*)

;;; For implementors debugging the system, this variable should be set T.
;;;
;;; For "production" (and, of course, for demos) it should be left NIL to
;;;  save the embarrassment and frustration of error breaks in at least
;;;  some of the places they might happen.

(define-internal-option *debug* "KBE debug mode (local)"
  t :boolean)

(define-system-initialization disable-debug-mode
  (setq *debug* nil))

(define-internal-option *bug-mail-address*
```

```
                        "Network address for KBE bug reports"
  "BUG-KBE@MIT-OZ" :sexp)

;;; For tracing modules during debugging

(defmacro format-trace (tracevar format-string &rest format-args)
  '(when ,tracevar
     (format trace-output "~&")
     (format trace-output ,format-string ,@format-args)))

(defmacro pp-trace (tracevar object)
  '(when ,tracevar (pp:pp ,object trace-output)))

(defmacro with-trace (tracevar &body forms)
  '(when ,tracevar
     (let ((standard-output trace-output))
       ,@forms)))

(defun edit-KBE-options (&optional (full? *debug*))
  (cond (full?
         (choose-user-options
           (append *KBE-Options*
                   *KBE-Internal-Options*
                   *KBE-Trace-Options*)))
        (t
         (choose-user-options *KBE-Options*))))

(defun odd-KBE-options ()
  (append (odd-options *KBE-Options*)
          (odd-options *KBE-Internal-Options*)))
```

```
;;; (WITH-AUTOMATIC-ERROR-RECOVERY . body)
;;;
;;;  Try to continue errors that can be easily proceeded.

(defmacro with-automatic-error-recovery-if (cond &body forms)
  '(condition-bind-if ,cond ((continuable-error-condition-mixin
                               #'(lambda (cond)
                                   (send cond ':proceed ':continue)))
                             (dbg:proceedable-ferror
                               #'(lambda (cond)
                                   (send cond ':proceed ':no-action))))
     ,@forms))

(defmacro with-automatic-error-recovery (&body forms)
  '(with-automatic-error-recovery-if t ,@forms))
```

```
;;; (DEFINE-CONDITION condition (arg1 arg2 ...))
;;;
;;;  This defines a way of signalling certain non-fatal conditions
;;;  internally in the system in a functional fashion. For example,
;;;
;;;   (DEFINE-CONDITION FOO ((X 3)))
;;;
;;;   (FOO)   ;signals a FOO condition, X defaults to 3
;;;   (FOO 4) ;signals a FOO condition, X defaults to 4
;;;
;;;  If body is supplied, it should take care of doing the
;;;  signalling. It may assume that the args are bound and
;;;  appropriately defaulted.

(defdefmacro define-condition "KBE condition" (name bvl . body)
  '(local-declare ()
     (defflavor ,name ,bvl (internal-condition)
       :initable-instance-variables
       :gettable-instance-variables)
     (compile-flavor-methods ,name)
     (defun ,name (&optional ,@bvl)
       ,@(or body
             '((signal
                 ',name
                 ':proceed-types '(:handled)
                 ,@(mapcan #'(lambda (x)
                               (cond ((atom x)
                                      (list '',(intern-keyword x) x))
                                     (t
                                      (list '',(intern-keyword (car x))
                                            (car x)))))
                           bvl)))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; INTERFACE-MACROS: Macros needed by the interface portion but
;;;                   not by the core system.
```

```
;;;; Packaging

;;; The concept of *KBE-user-package* is ill-formed, but we live with
;;; it for now because things tend to build symbols in the wrong
;;; packages and we haven't tracked down all the places that happens.
;;; As soon as we do, all references to the "user" package probably
;;; want to go away.
;;;
;;; I say probably because actually when communicating across machine
;;; boundaries, some interesting problems (like what package to use
;;; on the foreign host) may come up and the thing may have its place
;;; again.

(defmacro with-user-package (&body body)
  '(local-declare ((special *KBE-user-package*))
     (let ((package *KBE-user-package*))
       ,@body)))

(defmacro let-user-package (form &body body)
  '(let ((*KBE-user-package* ,form))
     (declare (special *KBE-user-package*))
     ,@body))

(defmacro with-kbe-package (&body body)
  '(local-declare ((special *KBE-package*))
     (let ((package *KBE-package*))
       ,@body)))
```

```
;;; The variable *DELEGATE* (defined in another file) is the
;;; rough analog of a self-variable in our message passing world.
;;; It wants to always hold a valid actor.

(defmacro with-delegate (handler &body forms)
  `(let ((*delegate* ,handler))
     (declare (special *delegate*))
     ,@forms))

(defvar *reply-to* nil
  "If non-NIL, a message to reply to instead of handling events locally.")

(defmacro with-reply-to (message &body forms)
  `(let ((*reply-to* ,message))
     (declare (special *reply-to*))
     ,@forms))

(defmacro tell (actor key &rest args)
  `(*tell ,actor ',key ,@args))

(defmacro ask (actor key &rest args)
  `(*ask ,actor ',key ,@args))

(defmacro delegate (actor key &rest args)
  `(*delegate ,actor ',key ,@args))

(defmacro reply (key &rest args)
  `(*reply ',key ,@args))
```

```
;;; -*- Mode: LISP; Package: KBE; Base: 10; Fonts: MEDFNB -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology.
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; MBOX: A mailbox implementation to support "real" message
;;;       passing between actors in the KBE.
```

```
;;;; Message

;;; A message has the following components:
;;;
;;;     DATE            Date the message was 'posted' (a universal time).
;;;     FROM            Sender's address (a string).
;;;     TO              Recipient's address (a string).
;;;     ID              An ID to reply to (a number).
;;;     RE              A string for self-documentation (or NIL).
;;;     TYPE            The type of this transaction (a symbol).
;;;     IN-REPLY-TO     An ID that this is a reply to (or NIL if none).
;;;     DATA            Any additional data needed for this transaction.
;;;     NOTIFY-UPON-TERMINATION         True if sender wants to know
;;;                                     when this has been fully processed.

(defflavor message ((date nil)
                    (from nil)
                    (to   nil)
                    (re   nil)
                    (id   nil)
                    (type nil)
                    (in-reply-to nil)
                    (data nil)
                    (notify-upon-termination nil))
          ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (message :after :init) (&rest ignore)
  (if (not date) (setq date (time:get-universal-time)))
  (if (not from) (ferror "No sender for ~S" self))
  (if (not to)   (ferror "No recipient for ~S" self))
  (if (not type) (ferror "No type for ~S" self)))

(defun make-message (message-spec)
  (lexpr-funcall #'make-instance 'message (cdr message-spec)))

(defmethod (message :print-self) (stream level prin1? &rest ignore)
  level  ;ignored
  (cond (prin1?
         (let ((package (pkg-find-package "")))
           (format stream "~%:(Message~@[ date ~S~]~@[ from ~S~]~
                                      ~@[ to ~S~]~@[ re ~S~]~@[ id ~S~]~
                                      ~@[ type ~S~]~@[ in-reply-to ~S~]~
                                      ~@[ data ~S~]~
                                      ~@[ notify-upon-termination ~S~])"
                   date from to re id type in-reply-to data
                   notify-upon-termination)))
        (t
         (format stream "{Message ~A^Y~A(~D)~@[ Re: ~A~]}"
                 from to id re))))

(defmethod (message :display) (&optional (stream standard-output))
  (format stream "~@[~
        ~&Date: ~\time\~]~@[~
        ~%From: ~A~]~@[~
        ~%To:   ~A~]~@[~
        ~%Re:   ~A~]~@[~
        ~%ID:   ~A~]~@[~
        ~%Type: ~A~]~@[~
        ~%Notify-Upon-Termination: ~A~]~@[~
        ~%In-Reply-To: Your message ~A~]~2%~{~A~%~}"
          date from to re id type
          notify-upon-termination
          in-reply-to data))
```

```
;;;; Mailbox

;;; Internally, a mailbox has the following state variables, none of
;;; which should be accesssed directly externally except for debugging:
;;;
;;;     TIMESTAMP       This is used internally by the :AWAIT message
;;;                     to make noticing changes to the mailbox more
;;;                     efficient.
;;;
;;;     READ-LOCK       A read lock is used by methods which intend
;;;                     to "read" the mail in the mailbox.
;;;
;;;     WRITE-LOCK      A write lock is used by methods which intend to
;;;                     modify the mailbox.
;;;
;;;     STORAGE         This is the actual stuff stored by the mailbox.

(defflavor mailbox ((owner      NIL)
                    (xmailbox   NIL)
                    (timestamp  0)
                    (read-lock  NIL)
                    (write-lock NIL)
                    (storage    NIL))
           ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defflavor xmailbox () (mailbox))

(defflavor inbox () (mailbox)
  (:default-init-plist :xmailbox t))

(defmethod (mailbox :after :init) (&rest ignore)
  (when (eq xmailbox t)
    (setq xmailbox (make-instance 'xmailbox ':owner self))))
```

```
;;;; For debugging

(defmethod (mailbox :print-self) (stream level prin1? &rest ignore)
  level prin1? ;ignored
  (cond (prin1?
         (format stream "#<~:[Anonymous~;~:*~A's~] ~A ~@[(~A) ~]~O>"
                 owner
                 (string-downcase (typep self))
                 (send self ':describe-storage)
                 (%pointer self)))
        (owner
         (format stream "~A's ~A" owner (string-downcase (typep self))))
        (t
         (format stream "~A[~O]"
                 (string-downcase (typep self))
                 (%pointer self)))))

(defmethod (mailbox :describe-storage) ()
  (let ((len (length storage)))
    (cond ((zerop len) "no messages")
          (t (format nil "~D message~:P" len)))))
```

```
;;;; Mailbox (exported methods)

;;; Externally, the only defined operations are:
;;;
;;;     :INSERT-MESSAGE message
;;;
;;;        Places MESSAGE in the mailbox's storage queue.
;;;
;;;     :AWAIT-MESSAGE msg-id [from] [type 'ANSWER]
;;;
;;;        Awaits a message with a given MSG-ID.
;;;        The FROM may be NIL or an actor name.
;;;         It is used only in generating a whostate if the message is
;;;        not already available.
;;;        The type (default RESULT) names the kind of result being
;;;        expected.
;;;
;;;     :NEXT-MESSAGE [timeout]
;;;
;;;        Reads the next message in the mailbox.
;;;        If timeout is not supplied, does not time out.
;;;        Otherwise, should be the number of 60ths of a second
;;;         until the wait should time out.

(defmethod (mailbox :insert-message) (msg)
  ;; No READ-LOCK needed to queue a message.
  (with-lock (write-lock :whostate "MBox Write Access")
    (incf timestamp)
    (setq storage (nconc storage (ncons msg)))
    (send msg ':id)))

(defmethod (mailbox :send-locked) (object key &rest args)
  (with-lock (read-lock :whostate "MBox Read Access")
    (lexpr-send object key args)))

 ;; This can only be called inside a :SEND-LOCKED contour
(defmethod (mailbox :await-message) (msg-id
                                     &optional from (type 'ANSWER)
                                     (process? t))
  (send self ':assure-read-lock)
  (do ((old-timestamp timestamp timestamp) ;May change asynchronously
       (state nil))
      (nil)
    (let ((message (send self ':extract-message msg-id type)))
      (when message
        (if process?
            (do ((m (send self ':extract-message msg-id)
                    (send self ':extract-message msg-id)))
                ((not m))
              (send owner ':process-message m))) ;process any other replies
        (return-list (send message ':data))))
    (let ((message (send self ':extract-message msg-id 'COMPLAINT)))
      (when message
        (if process?
            (do ((m (send self ':extract-message msg-id)
                    (send self ':extract-message msg-id)))
                ((not m)))) ;ignore any other replies
        (lexpr-funcall #'complain "~A" (send message ':data))))
    (process-wait (or state
                      (setq state
                            (cond ((not from) "Wait For Result")
                                  (t (format nil "Await ~A" from)))))
                  #'(lambda (loc val) (> (cdr loc) val))
                  (locf timestamp) old-timestamp)))


 ;; This can only be called inside a :SEND-LOCKED contour
(defmethod (mailbox :next-reply) (msg-id)
```

```
  (send self ':assure-read-lock)
  (with-lock (write-lock :whostate "MBox Write Access")
    (do ((p (locf storage) (cdr p))
         (s       storage  (cdr s)))
        ((null s) nil)
      (if (equal (send (car s) ':in-reply-to) msg-id)
          (return (prog1 (car s)
                         (incf timestamp)
                         (setf (cdr p) (cdr s))))))))

(comment ;tried this definition 4/26/84, but it failed.
         ;restored old definition 4/30/84 until this is
         ;better debugged -kmp

(defmethod (mailbox :next-message) (&optional (wait? t) prefer-fn)
  (do () (nil) ;forever
    (with-lock (read-lock :whostate "MBox Read Access")
      (if wait? (process-wait "Await Message" #'cdr (locf storage)))
      (cond (storage
             (with-lock (write-lock :whostate "MBox Write Access")
               (incf timestamp)
               (return (or (if prefer-fn
                               (dolist (msg storage)
                                 (when (funcall prefer-fn msg)
                                   (setq storage (delete msg storage))
                                   (return msg))))
                           (pop storage)))))
            ((not wait?) (return nil))
            (t
             (cbarf "Ignore ~A's problem and hope it's self correcting."
                    "Mailbox is inconsistent: ~S"
                    self))))))

);tnemmoc

(defmethod (mailbox :next-message) (&optional (wait? t) prefer-fn)
  (do ()
      (nil)
    (with-lock (read-lock :whostate "MBox Read Access")
      (if wait?
          (process-wait-with-timeout "Await Message"
                                     60. #'cdr (locf storage)))
      (cond (storage
             (with-lock (write-lock :whostate "MBox Write Access")
               (incf timestamp)
               (return (or (if prefer-fn
                               (dolist (msg storage)
                                 (when (funcall prefer-fn msg)
                                   (setq storage (delete msg storage))
                                   (return msg))))
                           (pop storage)))))
            ((not wait?) (return nil))))
    (process-sleep 10. "Avoid Deadlock")))
```

```
;;;; Mailbox (internal methods)

(defmethod (mailbox :assure-read-lock) ()
  (let ((lock read-lock))
    (if (neq lock current-process)
        (fsignal "Mailbox error: ~S had read lock on ~S."
                 lock self))))

(defmethod (mailbox :find-message) (msg-id &optional type)
  (send self ':assure-read-lock)
  (dolist (message storage)
    (if (and (equal (send message ':in-reply-to) msg-id)
             (or (not type)
                 (eq (send message ':type) type)))
        (return message))))

(defmethod (mailbox :extract-message) (msg-id &optional type)
  (send self ':assure-read-lock)
  (let ((message (send self ':find-message msg-id type)))
    (if message
        (with-lock (write-lock :whostate "MBox Write Access")
          (incf timestamp)
          (setq storage (delete message storage))
          message))))
```

```
;;; Request these flavors get fully compiled at compile time
;;; rather than on demand at runtime...

(compile-flavor-methods message mailbox xmailbox inbox)
```

```
;;; -*- Mode: LISP; Package: KBE; Base: 10; Fonts: MEDFNB -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology.
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; POST-OFFICE: A post office implementation to coordinate message
;;;              delivery between actors in the KBE.

(defflavor post-office ((mailing-lists (make-equal-hash-table ':size 50)))
           ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (post-office :add-to) (mailing-list mailbox)
  (let ((data (gethash mailing-list mailing-lists)))
    (when (not data)
      (ferror "No such mailing list: ~S" mailing-list))
    (when (not (memq mailbox mailing-lists))
      (push mailbox (cdr data)))))

(defmethod (post-office :remove-from) (mailing-list mailbox)
  (let ((data (gethash mailing-list mailing-lists)))
    (when (not data)
      (ferror "No such mailing list: ~S" mailing-list))
    (setf (cdr data) (delq mailbox (cdr data)))))

(defmethod (post-office :add) (mailing-list
                               &optional (initial-contents '())
                                         (err-if-exists? t))
  (let ((data (gethash mailing-list mailing-lists)))
    (when (and data err-if-exists?)
      (ferror "List already exists: ~S" mailing-list))
    (when (not data)
      (setq data (ncons mailing-list))
      (puthash mailing-list data mailing-lists))
    (dolist (mailbox initial-contents)
      (if (not (memq mailbox (cdr data)))
          (push mailbox (cdr data))))))

(defmethod (post-office :remove) (mailing-list)
  (remhash mailing-list mailing-lists))

(defmethod (post-office :resolve) (mailing-list)
  (cond ((typep mailing-list 'mailbox) (ncons mailing-list))
        (t
         (let ((data (gethash mailing-list mailing-lists)))
           (cond (data
                  (mapcan #'(lambda (address)
                              (send self ':resolve address))
                          (cdr data)))
                 (t
                  (send self ':resolve-remote mailing-list)))))))

(defmethod (post-office :resolve-remote) (mailing-list)
  (multiple-value-bind (actor-name host-name remote?)
      (parse-actor-full-name mailing-list)
    host-name ;ignored
    (cond ((not remote?)
           (when (string-equal actor-name mailing-list)
             (ferror "No such mailing list: ~S"
                     mailing-list))
           (prog1 (send self ':resolve actor-name)
```

```
                (send self ':add mailing-list
                      (list actor-name))))
          (t
           (let ((actor (parse-remote-actor mailing-list)))
             (prog1 (ncons (send actor ':inbox))
                    (let ((indirect-name (send actor ':name)))
                      (send self ':add mailing-list
                            (cond ((not (equal mailing-list indirect-name))
                                   (list indirect-name))
                                  (t ;Should never really happen...
                                   (list (send actor ':inbox))))))))))))


(compile-flavor-methods post-office)

(defvar *post-office* (make-instance 'post-office))

(defun deliver (address message)
  (dolist (mailbox (send *post-office* ':resolve address))
    (send mailbox ':insert-message message))
  (send message ':id))

(defun add-to-mailing-list (mailing-list &optional addresses (err? t))
  (send *post-office* ':add
        mailing-list
        (if (atom addresses) (ncons addresses) addresses)
        err?))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology.
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EVENT: A dictionary of events in the KBE actor world.

(defun not-a-delegate (message &rest args)
  (cbarf "Proceed quietly, ignoring the ~S message."
         "Attempt to send a message when there is no active delegate."
         message (copylist args)))

(defvar *delegate* #'not-a-delegate
  "An actor with all the default behaviors.")

;;; These are easily abused. Very sparing use is recommended.
;;; If you're not sure why you're using it, you're probably cheating.

(defun send-delegate (kwd &rest args)
  (lexpr-send *delegate* kwd args))

(defun lexpr-send-delegate (kwd &rest args)
  (lexpr-funcall #'lexpr-send *delegate* kwd args))

;;;

(defun window-delegate (&optional (window terminal-io))
  (or (send-any-superior-if-handles window ':delegate)
      *delegate*)) ;There must be something better -- might want to
                   ;create a *default-delegate* some day but this will do
                   ;for now. --kmp
```

```
;;;; Event Handler Mixin

;;; ACTOR-DEFAULT-BEHAVIOR-MIXIN
;;;
;;;  Adding this mixin allows a flavor to receive :EVENT messages
;;;  and handle events.

(defflavor actor-default-behavior-mixin () ()
  (:method-combination (:case :base-flavor-last :behavior)))

;;; :EVENT
;;;
;;;  This message is sent to a flavor instance when an event is
;;;  signalled. It just re-dispatches via a :BEHAVIOR message.
;;;  Other actors may want to redefine this method to conditionally
;;;  either forward the event notification to another actor, handle
;;;  it themselves (by sending self a :BEHAVIOR message), or
;;;  whatever.

(defmethod (actor-default-behavior-mixin :event) (&rest args)
  (lexpr-send self ':behavior args))

;;; :OTHERWISE :BEHAVIOR
;;;
;;;  This is the fall-through case for events that are not handled
;;;  by a :CASE :BEHAVIOR method.

(defmethod (actor-default-behavior-mixin :otherwise :behavior)
           (event &rest args)
  (ferror "~A has no defined behavior for the event ~S.~% Event Data: ~S"
          self event (copylist args)))

;;; (DEFINE-BEHAVIOR (flavor name) bvl . body)
;;;
;;;  This defines a :CASE method for event dispatching of FLAVOR on NAME
;;;  with the given bvl and body.

(defdefmacro define-behavior "KBE actor behavior" ((type event) bvl . body)
  '(defmethod (,type :case :behavior ,event) ,bvl
     (prog ()
           (progn ,@body)
           (return))))

(defprop define-behavior "KBE actor behavior" si:definition-type-name)
```

```
;;;; Event Definition

;;; (DEFINE-EVENT name bvl . body)
;;;
;;;  This declares an event type and creates a default behavior. If no
;;;  body is given, the default is to make a body which will type out
;;;  information about the event and prompt for any declared return
;;;  values.

(defdefmacro define-event "KBE event" (name bvl . body)
  (let ((declarations  nil)
        (documentation nil)
        (body-forms    nil))
    ;; Process body
    (do ((b body (cdr b)))
        ((null b) (setq body-forms (nreverse body-forms)))
      (let ((form (car b)))
        (cond ((and (cdr b) (stringp form))
               (setq documentation form))
              ((and (not (atom form)) (eq (car form) 'declare))
               (setq declarations (append declarations (cdr form))))
              (t
               (push form body-forms)))))
    ;; Hack defaulting
    (when (not body-forms)
      (let ((values (assq 'values declarations))
            (arglist (parse-argument-list bvl)))
        (setq body-forms
              '((let (prinlevel prinlength)
                  (format t
                          ,(text "~~&~A.~{~~% ~A: ~~S~}"
                                 (string-capitalize-words name)
                                 (mapcar #'string-capitalize-words
                                         arglist))
                          ,@arglist))
                (return ,@(mapcar #'(lambda (rv)
                                      '(prompt-and-read
                                        ':eval-form
                                        ,(text "Supply ~A return value: "
                                               rv)))
                                  (cdr values)))))))
    ;; Process body, returning result
    '(local-declare ()
       ;; default method handling for things which handle events
       (define-behavior (actor-default-behavior-mixin ,name) ,bvl
         ,@body-forms)
       ;; main function for signalling events
       (defun ,name (&rest args)
         (declare (arglist ,@bvl) ,@declarations)
         ,documentation
         (when-interface-enabled
           (lexpr-send-delegate ':event ',name args))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology.
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; EVENT: Default event descriptions.
```

```
;;; (ANSWER data)
;;;
;;;  This is the kind of event used for returning values.

(define-event answer (data)
  (cond ((atom data)
         (typeout (text "^Y ~S" data)))
        ((eq (car data) ':text)
         (typeout (get data ':text)))
        (t
         (typeout (let (prinlevel prinlength)
                    (text "~&The result was of type ~S.~{~% ~A:~11T ~S~}"
                          (car data)
                          (cdr data)))))))

;;; (COMPLAINT string)
;;;
;;;  This can be used in place of ANSWER to return error information.
;;;
;;;  Frequently, this behaves like REMARK (unsolicited typeout) but
;;;  its text is less likely to be less expected and is more important
;;;  than that of a REMARK, so it should be given better placement.

(define-event complaint (text)
  (remark (text "~&Error! ~A~&" text)))
```

```
;;;; Display Format Directives

;;; (TYPEOUT string)
;;;
;;;  Requests typeout of the given string. This is typeout typically
;;;  in direct response to a user query.

(defun typeout (string) (task (text "Typeout [~A]." string)))

(comment ;not needed? -kmp 4/8/84

(define-event typeout (string)
  (format t "~&~:A~&" string))

) ;tnemmoc

;;; (REMARK string)
;;;
;;;  Requests typeout of the given string. This is typically unsolicited
;;;  typeout which the user may find useful, but which s/he didn't
;;;  explicitly request.

(defun remark (string) (task (text "Remark [~A]." string)))

(comment ;not needed? -kmp 4/8/84

(define-event remark (string)
  (format t "~&* ~A~&" string))

);tnemmoc

;;; (CLEAR-DISPLAY)
;;;
;;;  If the customer has a display of some sort, suggests that it be
;;;  cleared, usually in preparation for other data yet to come.
;;;
;;;  This is probably a data abstraction violation since the apprentice
;;;  should not make assumptions about what is in view in the first place.

(defun clear-display () (task "Clear display."))
```

```
;;;; Beliefs

(define-event belief (belief-type belief-subject)
  (declare (values val))
  belief-type belief-subject ;ignored
  nil)

(define-event believe (belief-type belief-subject belief-val)
  (cbarf "Skip trying to assert this belief."
         "Cannot assert beliefs in this context.~% Data: ~S ~S ~S"
         belief-type belief-subject belief-val))
```

```
;;;; KBE-specific requests

;;; (TASK string)
;;;
;;;  This requests that an actor perform the action specified by
;;;  the KBE command given by the STRING.

(define-event task (request)
  (cbarf "Ignore this task request."
         "Task requests aren't handled in this context:~% Request: ~A"
         request))

;;; (CODER-CONFIRMATION function-name code)
;;;
;;;  Says that new code is available for the given function-name.

(define-event coder-confirmation (function-name code))

;;; (DOCUMENT-CODE function documentation)

(define-event document-code (function-name documentation))

;;; (SUGGEST-FOCUS focus-info)
;;;
;;;  Suggests that the focus be changed to item designated by the
;;;  given FOCUS-INFO.

(define-event suggest-focus (focus-info))

(define-event presentational-focus ()
  (declare (values focus))
  (return nil))

(define-event suggested-focus ()
  (declare (values focus))
  (return nil))

;;; (SUGGEST-CODE-HOME definition-name where)
;;;
;;;  Suggests that a home be found for the given definition, NAME,
;;;  in some location, WHERE. The location is in property list format
;;;  with possible keys of:
;;;    :BUFFER "...string name of buffer..."
;;;    :FILE   "...string name of file..."       ;unimplemented
;;;    :NEAR   symbolic-name
;;;  If more than one item is present, all are intended to be satisfied
;;;  (otherwise, a user should be queried for confirmation).

(define-event suggest-code-home (definition-name where))

(define-event suggested-code-home (definition-name)
  (declare (values where))
  definition-name ;ignored
  (return nil))

(define-event associated-apprentice ()
  (declare (values apprentice-or-NIL))
  (return nil))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
;;;
;;; (c) Copyright 1983, Massachusetts Institute of Technology.
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
;;;
;;; ACTOR: Primitive actor specification.

;;; For debugging.

(define-trace-option *actor-debug*
                     "Show trace of ACTOR internals"
  nil :boolean)
```

```
(defflavor actor-message-passing-mixin ((inbox nil)
                                        (current-msg-id 0)
                                        (debug-mode-default nil)) ()
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (actor-message-passing-mixin :after :init) (&rest ignore)
  (when (not inbox)
    (setq inbox (make-instance 'inbox ':owner self)))
  (add-to-mailing-list (send self ':short-name) inbox))

(defmethod (actor-message-passing-mixin :new-msg-id) ()
  (incf current-msg-id))
```

```
;;;; Tell

(defun *tell (actor key &rest args)
  (lexpr-send-delegate ':tell actor '(:notify-upon-termination t)
                       key args))

(defmethod (actor-message-passing-mixin :tell) (actor options key &rest args)
  (when actor
    (deliver actor
             (lexpr-funcall
               #'make-instance 'message
               ':to actor
               ':from (send self ':name)
               ':type key
               ':id (send self ':new-msg-id)
               ':data (copylist args)
               options))))
```

```
;;;; Ask

(defun *ask (actor key &rest args)
  (cond ((send-delegate ':name? actor)
         (lexpr-send-delegate ':event key args))
        (t
         (lexpr-send-delegate ':ask actor key args))))

(defmethod (actor-message-passing-mixin :ask) (actor key &rest args)
  (if (not actor) (ferror "No actor specified in (ASK -??- ~A~{ '~S~})"
                          key (copylist args)))
  (lexpr-send self ':send-locked self ':ask-internal actor key args))

(defmethod (actor-message-passing-mixin :ask-internal) (actor key &rest args)
  (send (send self ':inbox)
        ':await-message
        (lexpr-send self ':tell actor '(:notify-upon-termination t)
                    key args)
        actor))
```

```
;;;; Delegate

(defun *delegate (actor key &rest args)
  (cond ((send-delegate ':name? actor)
         (lexpr-send-delegate ':event key args))
        (t
         (lexpr-send-delegate ':delegate actor key args))))

(defmethod (actor-message-passing-mixin :delegate) (actor key &rest args)
  (if (not actor)
      (ferror "No actor specified in (DELEGATE -??- ~A~{ '~S~})"
              key (copylist args)))
  (lexpr-send self ':send-locked self ':delegate-internal actor key args))

(defmethod (actor-message-passing-mixin :delegate-internal) (actor key &rest args)
  (let ((msg-id (lexpr-send self ':tell actor
                            '(:notify-upon-termination t)
                            key args))
        (mbox (send self ':inbox)))
    (let ((result (multiple-value-list
                    (send mbox ':await-message msg-id actor))))
      (do ((m (send mbox ':next-reply msg-id)
              (send mbox ':next-reply msg-id)))
          ((null m))
        (send self ':process-message m))
      (answer (car result)))))
```

```
;;;; Message Support

(defmethod (actor-message-passing-mixin :send-locked) (&rest args)
  (lexpr-send (send self ':inbox) ':send-locked args))
```

```
;;;; Enqueueing Messages

(defmethod (actor-message-passing-mixin :queue-message) (message)
  (comment ;needed? -kmp 2/22/84
    (cond ((not (typep message 'message)) ;needed?
           (setq message (make-message message)))) )
  (send (send self ':inbox) ':insert-message message))
```

```
;;;; Dequeueing Messages

(comment ;need something more complex now. --kmp 4/9/84

(defun simple-request? (msg)
  (memq (send msg ':type) '(status-report)))

) ;tnemmoc

(defun simple-request? (ignore) nil)

(defmethod (actor-message-passing-mixin :read-messages) (&optional (wait? t))
  (let ((mbox (send self ':inbox)))
    (do ((m (send mbox ':next-message wait? #'simple-request?)
            (send mbox ':next-message wait? #'simple-request?)))
        ((not m))
      (send self ':process-message m))))

(define-condition end-of-command ())

(defvar *message-being-processed* nil)
(defvar *message-being-processed-terminated* nil)

(defmethod (actor-message-passing-mixin :process-message) (message)
  (with-defaults ((*debug* debug-mode-default))
    (with-trace *actor-debug*
      (format t "~&Processing...")
      (send message ':display))
    (let* ((*message-being-processed* message)
           (*message-being-processed-terminated* nil))
      (condition-bind ((end-of-command
                        ;; This should check for ending twice, but can't
                        ;; because that can happen in a benign way with
                        ;; when a proxy both forwards termination through
                        ;; and finishes that forwarding. I should think
                        ;; more about this sometime. --kmp
                        #'(lambda (cond)
                            (unless *message-being-processed-terminated*
                              (when (send *message-being-processed*
                                          ':notify-upon-termination)
                                (reply answer nil))
                              (setq *message-being-processed-terminated*
                                    t))
                            (send cond ':proceed ':handled))))
        (when-abnormal-exit
          (condition-case-if (not *debug*) (cond)
              (let ((result (lexpr-send self ':behavior
                                        (send message ':type)
                                        (send message ':data))))
                (if (send message ':notify-upon-termination)
                    (reply answer result)))
            (sys:error (reply complaint (text "~A" cond)))
            (sys:abort (reply complaint "Request aborted.")))
          (if (send message ':notify-upon-termination)
              (reply complaint "Request aborted."))))))))

(defmethod (actor-message-passing-mixin :current-message) ()
  *message-being-processed*)

(defmethod (actor-message-passing-mixin :current-client) ()
  (let ((message (send self ':current-message)))
    (if message (send message ':from))))

(defun *reply (key &rest args)
  (lexpr-send-delegate ':reply key args))

(defmethod (actor-message-passing-mixin :reply-to) (message key &rest args)
  (if (not message)
```

```
      (lexpr-send self ':event key args)
      (lexpr-send self ':tell (send message ':from)
                  (list ':in-reply-to (send message ':id))
                  key
                  args)))

(defmethod (actor-message-passing-mixin :reply) (key &rest args)
  (lexpr-send self ':reply-to (send self ':current-message) key args))

(defwhopper (actor-message-passing-mixin :event) (key &rest args)
  (cond (*reply-to*
         (lexpr-send self ':reply-to *reply-to* key args))
        (t
         (with-reply-to nil
           (lexpr-continue-whopper key args)))))
```

```
;;;; Actor Name Mixin

;;; This mixin allows actors to have both fully qualified names
;;; and short names. The semantics of these is as follows:
;;;
;;;  :NAME              A fully qualified name, necessary for use in
;;;                     communications between actors.
;;;
;;;  :SHORT-NAME        A possibly-abbreviated name, not suitable for
;;;                     use in inter-actor communications, but acceptable
;;;                     for use in text intended to immediately reach
;;;                     a human user (such as console output).

(defflavor actor-name-mixin ((short-name nil)) (name-mixin)
  :initable-instance-variables
  :gettable-instance-variables
  :settable-instance-variables)

(defmethod (actor-name-mixin :after :init) (&rest ignore)
  (let ((my-name (send self ':name)))
    (when (not (full-actor-name? my-name))
      (setq short-name my-name)
      (send self ':set-name (make-actor-full-name my-name)))
    (when (not short-name)
      (setq short-name my-name))))

(defmethod (actor-name-mixin :print-self) (stream level prin1?
                                           &rest ignore)
  level  ;ignored
  (cond (prin1?
         (format stream "#<~A>" (send self ':name)))
        (t
         (format stream "~A" (send self ':short-name)))))
```

```
;;;; Actor

;;; Anything which wants to be treated like an actor should be built
;;; out of this.

(defflavor actor () (actor-message-passing-mixin
                     actor-default-behavior-mixin
                     actor-name-mixin
                     property-list-mixin))

(compile-flavor-methods actor)
```

```
(defun parse-actor (actor)
  (cond ((not actor) nil)
        ((typep actor 'actor) actor)
        ((not (stringp actor))
         (barf "Cannot coerce ~S to an actor or actor designator." actor))
        ((send-any-that-handles ':name? actor))
        ((parse-remote-actor actor))
        (t (ferror "~A is not a known actor." actor))))

(defun full-actor-name? (name)
  (string-search "@" name))

(defun make-actor-full-name (name &optional (host si:local-host))
  (text "~A@~A" name host))

(defun parse-actor-full-name (name)
  (declare (values actor-name host-name remote?))
  (let ((pos (string-reverse-search "@" name)))
    (cond ((not pos)
           (values name (send si:local-host ':name) nil))
          (t
           (let ((name (substring name 0 pos))
                 (host (si:parse-host (substring name (1+ pos)))))
             (cond ((eq host si:local-host)
                    (parse-actor-full-name name))
                   (t
                    (values name (send host ':name) t)))))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; COMPLETE: Abstractions for computing the completion of a given
;;;           string from a table of possible completions.

;;; Probably the LispM provides a more efficient table lookup
;;; abstraction but we don't know what it is yet. This stuff will do
;;; for now...

;;; (ADD-TO-COMPLETION-TABLE completion-table item)
;;;
;;;  Adds a new item to an already-existing completion table.

(defun add-to-completion-table (completion-table new-entry)
  (let ((new-string (string new-entry)))
    (let ((alphabetic-case-affects-string-comparison t))
      (cond ((null completion-table) (list new-string))
            ((alphalessp new-string (car completion-table))
             (cons new-string completion-table))
            ((string-equal new-string (car completion-table))
             completion-table)
            (t
             (do ((l completion-table (cdr l))
                  (old-l nil l))
                 ((or (null l) (alphalessp new-string (car l)))
                  (setf (cdr old-l) (cons new-string l)))
               (if (string-equal new-string (car l))
                   (return)))
             completion-table)))))

;;; (MAKE-COMPLETION-TABLE initial-contents)
;;;
;;;  Coerces initial-contents from an arbitrary list (of chars, strings,
;;;  or symbols) into a well-formed completion-table for use in these
;;;  other functions.
;;;
;;;  eg,
;;;      (MAKE-COMPLETION-TABLE '(foo "bar" #/Q))
;;;        makes a table containing the entries "FOO", "bar", and "Q"

(defun make-completion-table (&optional list-of-strings)
  (let ((alphabetic-case-affects-string-comparison t))
    (sort (mapcar #'string list-of-strings) #'alphalessp)))

;;; (GET-COMPLETION-FROM-TABLE string completion-table)
;;;
;;;  Returns a string which is the characters which can be used as
;;;  a completion for string in completion-table.
;;;
;;;  eg,
;;;      (SETQ TABLE (MAKE-COMPLETION-TABLE '("FOOBAR" "FOOBAZ")))
;;;      (GET-COMPLETION-FROM-TABLE "FOO" TABLE) => "BA"
;;;

(defun get-completion-from-table (string completion-table)
  (*catch 'completion
    (do ((c completion-table (cdr c)))
        ((null c) "")
      (if (match-completion (car c) string)
          (let ((first-match (car c)))
```

```
          (do () (nil)
            (if (or (null (cdr c))
                    (not (match-completion (cadr c) string)))
                (*throw 'completion
                        (get-completion-from-range string
                                                   first-match
                                                   (car c))))
            (pop c)))))))

;;; (MATCH-COMPLETION pattern string)
;;;
;;;  Returns true if string is a substring of pattern beginning
;;;  with the first character.
;;;
;;;  eg,
;;;      (MATCH-COMPLETION "foobar" "foo") => T
;;;      (MATCH-COMPLETION "foo"    "foo") => T
;;;      (MATCH-COMPLETION "foobar" "bar") => NIL
;;;      (MATCH-COMPLETION "foobar" "abc") => NIL
;;;      (MATCH-COMPLETION "f"      "foo") => NIL
;;;

(defun match-completion (pattern string)
  (let ((alphabetic-case-affects-string-comparison t))
    (%string-equal pattern 0 string 0 (string-length string))))

;;; (GET-COMPLETION-FROM-RANGE seed lower upper)
;;;
;;;  Returns a string which is the set of characters which are common
;;;  to the string lower and upper in order and immediately following
;;;  seed, which is assumed to be common to both lower and upper.
;;;
;;;  eg,
;;;      (GET-COMPLETION-FROM-RANGE "foo" "foobar" "foobaz") => "ba"
;;;      (GET-COMPLETION-FROM-RANGE "foo" "foo" "foobar")
;;;      (GET-COMPLETION-FROM-RANGE "foo" "foo" "foo") => ""

(defun get-completion-from-range (seed lower upper)
  (let ((seed-length (string-length seed)))
    (substring lower
               seed-length
               (do ((i seed-length (1+ i))
                    (end (min (string-length lower)
                              (string-length upper))))
                   ((= i end) i)
                 (if (not (= (aref lower i) (aref upper i)))
                     (return i))))))

;;; (POSSIBLE-COMPLETIONS string table)

(defun possible-completions (string table)
  (do ((l table (cdr l)))
      ((null l) nil)
    (cond ((match-completion (car l) string)
           (return (do ((l l (cdr l))
                        (r nil (cons (car l) r)))
                       ((or (null l)
                            (not (match-completion (car l) string)))
                        (nreverse r))))))))
```

```
;;; Local Modes:;
;;; Mode:LISP;
;;; Lisp WHEN Indent:1;
;;; Lisp UNLESS Indent:1;
;;; End:;
```

```
;;; -*- Mode:LISP; Package: KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;THIS HAS THE STUFF FOR SUPPORTING THE READMACROS FOR LISP ROLES.
;THE ONLY FILES WHICH DEPEND ON THIS TO COMPILE ARE THE ACTUAL
;CLICHE LIBRARY FILES.

(defun kbe-comment? (x)
  (and (not (atom x)) (eq (car x) 'kbe-/#-comment-macro)))

(defun make-kbe-comment (code name)
  '(kbe-/#-comment-macro ,code ,name))

(defun kbe-comment-code (x) (cadr x))

(defun kbe-comment-name (x) (caddr x))

(defun kbe-comment-name->string (name)
  (if (stringp (car name))
      (car name)
      (format nil "~{~a~^ ~}"
              (mapcar #'string-downcase name))))

(defun string->kbe-comment-name (string)
  (let ((package *kbe-package*))
    (if (and (> (string-length string) 2)
             (member (substring string 0 2) '("a " "an")))
        (ncons string)
        (read-from-string (string-append "(" string ")")))))

(defun kbe-comment-type (comment)
  (let ((name (kbe-comment-name comment)))
    (cond ((null name) ':dummy)
          ((stringp (car name)) ':indefinite)
          ((memq 'input name) ':input)
          ((memq 'output name) ':output)
          (t ':segment))))

(defun kbe-comment-needs-implementation? (comment)
  (let ((code (kbe-comment-code comment)))
    (and (or (null code) (and (listp (car code)) (eq (caar code) '...)))
         (memq (kbe-comment-type comment) '(:input :segment)))))

;obsolete fn version

(defun part-marker? (x)
  (and (not (atom x)) (eq (car x) 'kbe-/#-comment-macro)))
```

```
;this has been hacked up yet again to add too new kinds of
;external sugar to the syntax.
;external {X, depending on Y} goes to
; internal (depending-on Y X)
;external {X, modified} goes to
; &IN-OUT X -- not this is spliced into the list.

;The following implements reading the
;new syntax for slot comment things in code.
;it has the following two forms:
;1 {<name>} ==> (semi-kbe-/#-comment-macro <name>)
;is any THE or A phrase or "".
;a phrases are left as strings.  the names are lists.
;2 {<code> , <name>} ==> (kbe-/#-comment-macro (<code>) <name>)
; <name> is any THE phrase in the command language.
;  It becomes a list of the words in the phrase
;  The name can be nil.
; <code> is any lisp sexpr.
;in order to indicate that a role takes parameters you write
;({<name>} . args)  You can put &IN-OUT in the arg list.
;input and output roles are indicated by the keywords INPUT and OUTPUT
;in their names.
; instances of SEMI-KBE-/#-COMMENT-MACRO get changed
;o the correct kbe-/#-comment-macro
;immediately after reading so that all the internal stuff can
;stay the same.

;;; This sets things up for the text editor so that it can do
;;; more or less the right thing with #{...} expressions.

(set-editor-list-syntax-from-character #/{ #/()
(set-editor-list-syntax-from-character #/} #/))

(set-syntax-macro-char #/{ 'KBE-/{-reader)

(set-syntax-from-description #/} 'si:single)

(defun kbe-/{-reader (list-so-far stream)
  (let* ((stuff (read-to-matching-/} stream))
         (comma (string-reverse-search #/, stuff)))
    (if (null comma)
        '(semi-kbe-/#-comment-macro ,(string->kbe-comment-name stuff))
      (let ((code (get-code-for-kbe-comment (substring stuff 0 comma)))
            (comment
              (string->kbe-comment-name (substring stuff (1+ comma)))))
        (cond ((and (eq (car comment) 'depending)
                    (eq (cadr comment) 'on)
                    (= (length comment) 3)
                    (car code) (null (cdr code)))
               '(depending-on ,(car (last comment)) ,(car code)))
              ((and (eq (car comment) 'depending)
                    (eq (cadr comment) 'on)
                    (eq (caddr comment) 'the)
                    (> (length comment) 4)
                    (car code) (null (cdr code)))
               '(depending-on (kbe-/#-comment-macro nil ,(cddr comment))
                              ,(car code)))
              ((and (equal comment '(modified))
                    (car code) (null (cdr code)))
               (values (nconc list-so-far '(&in-out ,(car code))) nil T))
              (T '(kbe-/#-comment-macro ,code ,comment)))))))

(defvar kbe-/#-comment-length 1000
 "The number of characters to read before complaining about no } after {")

;this will not work if there is a #/} or something like that in the way.

(defun get-code-for-kbe-comment (string)
```

```
  (let ((code (read-from-string string)))
    (if (equal code '(...)) (setq code nil))
    (ncons code)))

(defun read-to-matching-/} (stream)
  (do ((char (tyi stream) (tyi stream))
       (chars nil (cons char chars))
       (level 0)
       (i 0 (1+ i)))
      ((> i kbe-/#-comment-length)
       (complain "No '}' found where one expected"))
    (cond ((= char #/{) (incf level))
          ((= char #/}) (decf level)
           (if (minusp level)
               (return (apply #'string-append (nreverse chars))))))))

;this does output-inversion of depending-on

(pp:define-list-print-function depending-on (list stream &rest ignore)
  (format stream "~@!{~=, depending on ~A<}~."
          (caddr list)
          (if (kbe-comment? (cadr list))
              (kbe-comment-name->string (kbe-comment-name (cadr list)))
            (cadr list))))
```

```
;THE FOLLOWING OLD SYNTAX IS STILL SUPPORTED AT THE MOMENT, and
;it is still being used internally.

;#{{CODE,} {INPUT ! OUTPUT} name} ==>
;    (kbe-/#-comment-macro list-of-code list-of-keyword-and-names)
;INPUT specifies it is an input slot
;OUTPUT specifies it is an output slot
;otherwise it is a segment slot
;If code is of the form "(... . args)" then it specifies that
; the result of each arg is a data input to the slot
;If the code is of the form "sexpr" then it speicifies the code
; which fills the role.
;NAME is the slot name. the name can be omitted.
; The name may be compound of the form
; "name of the other-name" in which case it specifies that there
; is a higher level slot which this is part of.  (This only works
; for loop enumerators at the moment.)

(defun kbe-/#-comment-reader (ignore stream)
  (read-kbe-/#-stuff stream))

(set-syntax-/#-macro-char #/{ #'kbe-/#-comment-reader)

(defun read-kbe-/#-stuff (&optional (stream standard-input))
  (let* ((stuff (read-to-matching-/} stream))
         (comma (string-reverse-search #/, stuff)))
    (if comma
        '(kbe-/#-comment-macro
           ,(get-code-for-kbe-comment (substring stuff 0 comma))
           ,(string->kbe-comment-name (substring stuff (1+ comma))))
        '(kbe-/#-comment-macro
           () ,(string->kbe-comment-name stuff)))))
```

```
;this takes a piece of code and gets rid
;of every semi-kbe-/#-comment-macro thing
;in it in favor of kbe-/#-comment-macro things.
;It does the following transformations by side-effect
;((semi-kbe-/#-comment-macro <name>) . <args>)
;  => (kbe-/#-comment-macro ((... . <args>)) <name>)
;when not in fn position.
;   (semi-kbe-/#-comment-macro <name>) => (kbe-/#-comment-macro () <name>)

(defun fix-up-semi-kbe-/#-comment-macros (code)
  (cond ((not (listp code)))
        ((eq (car code) 'semi-kbe-/#-comment-macro)
         (rplacd code (list nil (cadr code)))
         (rplaca code 'kbe-/#-comment-macro))
        ((and (listp (car code))
              (eq (caar code) 'semi-kbe-/#-comment-macro))
         (mapc #'fix-up-semi-kbe-/#-comment-macros (cdr code))
         (let ((new-code (if (cdr code) '((... .,(cdr code))))))
           (rplacd code '(,new-code ,(cadar code)))
           (rplaca code 'kbe-/#-comment-macro)))
        ((eq (car code) 'cond)
         (mapc #'(lambda (c) (mapc #'fix-up-semi-kbe-/#-comment-macros c))
               (cdr code)))
        (T (do ((list code (cdr list)))
               ((not (listp list)))
             (fix-up-semi-kbe-/#-comment-macros (car list)))))
  code)

(defun read-hacking-/{ (&optional (stream standard-input))
  (fix-up-semi-kbe-/#-comment-macros (read stream)))
```

```
;this stuff prints out kbe-/#-comment-macro in either old
;or new style.

(define-internal-option *kbe-role-marker* "KBE role marker (internal)"
  "#{" :assoc '(("{"  . "{") ("#{" . "#{")))

(pp:define-list-print-function kbe-/#-comment-macro (x s)
  (cond ((equal *kbe-role-marker* "{")
         (new-style-kbe-/#-comment-macro-output x s))
        (T (old-style-kbe-/#-comment-macro-output x s))))

(defun new-style-kbe-/#-comment-macro-output (x s)
  (let* (((code name) (cdr x))
         (comment (kbe-comment-name->string name)))
    (cond ((null code) (format s "{~A}" comment))
          ((and (listp (car code)) (eq (caar code) '...))
           (format s "(~@!{~A}~^ ~@_~:I~{~=~^ ~_~}~.)"
                   comment (handle-&in-out (cdar code))))
          (T (format s "{~@!~{~=, ~:_~}~A~.}" code comment)))))

(defun old-style-kbe-/#-comment-macro-output (x s)
  (let* (((code name) (cdr x))
         (comment (kbe-comment-name->string name)))
    (format s "#{~@!~{~=, ~:_~}~A~.}" code comment)))

(pp:define-list-print-function |...| (x s)
  (format s "(~!~A~^ ~:I~@_~@{~=~^ ~}~.)" x))

;this looks for &in-out in a list and turns it into the right {} thing
;note we only do this in the new style, because the old style
;reader cannot hack this.

(defun handle-&in-out (list)
  (if (not (memq '&in-out list)) list
    (do ((l list (cdr l)) result) ((null l) (nreverse result))
      (if (not (eq (car l) '&in-out)) (push (car l) result)
        (pop l)
        (push '(kbe-#-comment-macro (,(car l)) ,(ncons 'modified))
              result)))))
```

```
;this defines how kbe-/#-comment-macro is evaluated.  It is important
;to note that there are three different ways in which it is evaluated
;1- in actual code (it tries just to be invisable)
;   Note that this is the only case that ever gets compiled.
;2- in constraints and comments in cliches where it computes
;   the segcase that it coressponds to.
;3- when a clichde is initially being instanciated where it
;   computes the parameter to the cliche it corresponds too.
;4- also A phrase comments are handled specially by CREATE-PLAN-FOR-CODE

(declare (special 2creation-time-plan *2focus-seg*
                  2role-fillers 2cliche-name))

(defmacro kbe-/#-comment-macro (code comment)
  (cond ((and code (not (and (listp (car code)) (eq (caar code) '...))))
         (car code))
        (2creation-time-plan
         '(2creation-time-kbe-comment ',comment))
        (*2focus-seg*
         '(2focus-seg-kbe-comment ',comment))
        (T '(kbe-/#-comment-complaint ',code ',comment))))

(defun kbe-/#-comment-complaint (ignore comment)
  (complain "The ~{~A~^ ~} has not been implemented yet" comment))

(defun 2creation-time-kbe-comment (name-list)
  (let* ((role-name (car (last name-list)))
         (primary-roles (get (get 2cliche-name 'cliche-comments)
                             'primary-roles))
         (spot (memq role-name primary-roles))
         (ordinal (- (length primary-roles) (length spot))))
    (cond ((and spot (> (length 2role-fillers) ordinal))
           (car (nth ordinal 2role-fillers)))
          (*2focus-seg* (2focus-seg-kbe-comment name-list)))))

(defun 2focus-seg-kbe-comment (name)
  (if (not (equal (car name) 'the))
      (push 'the name))
  (setq name (string-append (kbe-comment-name->string name)
                            " of the segment " *2focus-seg* "."))
  (car (eval-command (parse-command name #'parse-definite-reference))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;;; Readmacro Characters

;;; The following are used to implement a syntactically sugared list
;;; construction operation a la plasma.  This form is the dual of
;;; backquote.
;;;
;;; The characters [ ] and ! !! have special meaning in this context.
;;; For example,
;;;               [1 2 !'(3 4) !!'(5) 6 !'(7 8 9)]
;;;
;;;               evaluates to
;;;
;;;               (1 2 3 4 5 6 7 8 9).
;;;
;;; The only difference between ! and !! is that ! uses NCONC
;;; and !! uses APPEND.  there is no difference between ! and
;;; !! if they precede the last element in a [] list.
;;;
;;; The read macros on these characters transform the input
;;; into functional form. eg, the above example reads in as:
;;;
;;;      (/[ 1 2 |!!| '(3 4) |!!!| '(5) 6 |!!| '(7 8 9)).
;;;
;;; Note that ! and !! work only when reading in the KBE package
;;; because they are not readmacros.

 ;; Someday, if we flush the [...] syntax, we may need to retain
 ;; this readmacro for things like LOAD-DUMP. In that case, the number
 ;; 200. should become simply 2, since 2-lists are the largest that
 ;; occur in plan dumps.

(defun KBE-/[-readmacro (list-so-far stream)
  list-so-far ;ignored
  (do ((l (ncons '/[) (cons item l))
       (item (read stream) (read stream))
       (i 200. (1- i)))
      ((eq item '/]) (nreverse l))
    (if (zerop i)
        (barf "No /"]/" seen scanning a [...] expression.~% ~S"
              (reverse l)))))

(set-syntax-macro-char #/[ #'KBE-/[-readmacro)

; KMP 24-Jul-82 2:54am
; The following was wrong because si:single means intern on the current
; package. We need to have the ] come back always in the KBE package so
; that the [ reader will match it regardless of the binding of PACKAGE.

;(set-syntax-from-description #/] 'si:single)

; Here's the fix:

(defun KBE-/]-readmacro (list-so-far stream)
  list-so-far stream ;ignored
  '/]) ;not

(set-syntax-macro-char #/] #'KBE-/]-readmacro)
```

```
;;; This does the work of translating the construction to ordinary LISP
;;; code. eg, from the example above, you'd get back:
;;;
;;;     (LIST* 1 2 (NCONC '(3 4) (APPEND '(5) (LIST* 6 '(7 8 9)))))

(defun (/[ macro) (body)
  (prog (args items code fn)
        (setq args (cdr body))
     L  (setq fn (cdr (or (assq (car args) '((|!!| . nconc)
                                             (|!!!| . append)))
                          '(nil . list*))))
     M  (cond ((memq (car args) '(|!!| |!!!|))
               (setq args (cdr args))
               (go M)))
        (cond (args
               (setq items (cons (cons fn (car args)) items)
                     args  (cdr args))
               (go L)))
        (cond ((null items)
               (return (displace body ''nil))))
        (cond ((eq (caar items) 'list*)
               (setq fn 'list))
              ((setq fn (or (caadr items) 'list*)))) ;cute eh?
        (setq code (list (cdar items)))
     P  (setq items (cdr items))
        (cond ((null items)
               (return (displace body (cons fn code)))))
        (cond ((or (eq (caar items) fn)
                   (and (eq (caar items) 'list*)
                        (eq fn 'list)))
               (setq code (cons (cdar items) code)))
              ((setq code (list (cdar items) (cons fn code))
                     fn   (caar items))))
        (go P)))

;;; This sets things up for the text editor so that it can do
;;; more or less the right thing with [...] expressions.

(set-editor-list-syntax-from-character #/[ #/()
(set-editor-list-syntax-from-character #/] #/))

;;; This sets things up so that [...] expressions print out
;;; the way they were typed in.

(pp:define-list-print-function /[ (list s)
  (format s "[~@!~{~=~^ ~@_~}~.]" (cdr list)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This file contains a bunch of obsolete macros that many of DICK's files
;;; use, but which should not be used in new code. Functions defined in
;;; this file will be phased out as soon as possible.

;;; one of the gross things about this file is that you have to load it
;;; before you can compile it in maclisp.

(declare (special deffn pat assigned macros))

;;; This makes it possible to assign to an arbitrary pattern when this
;;; file is loaded.
;;;
;;; Note: I am claiming that the right thing is returned from the
;;;       pattern is important to make things work, even though it is
;;;       a lie --Dick

;;; Notes:
;;;
;;; BEGIN is not used externally to this file. It can go away any time.
;;;   -kmp
;;;
;;;

(define-setf-handler /[ (pat v) '(match ,v ,pat nocheck))
```

```
;THIS WAS MAINLY JUST USED FOR DESTRUCTURING AND SHOULD NOT BE NEEDED
;ANY MORE. IT CANNOT GO AWAY, HOWEVER, BECAUSE A FEW PLACES STILL USE IT.

;the following set of functions and macros implement a simple pattern
;matching facility. the main ideas behind this facility are:
;
; 1. the main use is intended to be picking out pieces of a list structure
; 2. the facility is intended to be efficient, to give you as much power
;    as can be gotten cheaply.
; 3. to this end all the pattern matching expands into inline code,
;    instead of being a call on a subroutine.
; 4. the basic matching is the inverse of the list construction described
;    above.
;               for example if v1 = 4 and v2 = 5 then:
;               ['foo v1 v2] evaluates to '(foo 4 5)
;               as an inverse of this:
;               (match '(foo a b) ['foo v1 v2]) returns T
;                  and sets v1 to 'a and v2 to 'b
;               with this simple type of pattern,
;               (equal list (progn (match 'list pattern) (eval 'pattern)))
; 5. a basic pattern consists of a series of "pattern-elements" in []s
;    each pattern-element is either:
;       a. an atom, this is assigned the value of the corresponding item
;             in the matched list.
;       b. a quoted atom, this matches an occurence of the atom.
;             otherwise the pattern fails returning NIL.
;       c. a pattern in []s, this matches recursively against a sublist
;             in the matched list.
;     the matching proceeds by matching the first element of the pattern
;     against the first element of the list, the second against the second
;     etc. if a pattern element is a pattern list then matching recurses
;     matching this pattern against the sublist at this point.
; 6. the basic pattern is extended in several ways:
;       a. a pattern element may be the special atom "_" which
;          matches anything.
;       b. a pattern element may be preceded by the special atom "!" which
;          indicates that the pattern applies to the CDR of the matched
;          list at this point rather than the CADR.  in addition when
;          an ! match succeeds it does not move the matching position
;          pointer.  for example [! x ! y] will assign exactly the same
;          thing to x and y.  it should be noted that !
;          is the inverse of the list constructor ! only if it is on the
;          last pattern element.  ! can not be used to match against a
;          non-terminal portion of a list.  the pattern never has to back
;          up, and anything which is assigned to a variable is an actually
;          part of the matched list rather than a copied out central
;          portion.  this is a clear example of getting all that can be
;          gotten from ! cheaply without going on to do the hard stuff.
;       c. a pattern element can be followed by arbitraryly many of the
;          following three things in any order
;       d. an unquoted sexpr, this is evaluated and the match fails if it
;          returns NIL
;       e. a quoted sexpr, this is evaluated and the value ignored
;       f. in both types of sexpr, an occurence of _ referres to the
;          current item being matched against unless it is quoted.
;       g. the special atom "=" followed by another atom, the atom is
;          assigned the value of the corresponding item in the matched
;          list.  (this is particularly usefull when the basic
;          pattern-element is a sub pattern)
;       h. several patern-elements can be set up as alternatives by
;          seperating them by "~"s. if the first one of these fails to
;          match, then instead of causing the entire pattern to fail,
;          the matcher attempts to match the next alternative against the
;          same place.  as soon as one of the alternatives succeedes,
;          control passes to after the last alternative.
;       7. if the match fails, it returns NIL but no attempt is made to
;          undo any of the side effects which may have happend before
;          the point of failure.
```

```
; 8. the parts of the pattern are evaluated in the exact order in which
;    they appear.
; 9. the matcher has a mode where it assumes that the pattern will match
;    and does not check that it does match (except in explicit
;    alternatives) this saves a lot of time when the mtching is being
;    used solely for the purpose of selecting parts of a list.
```

```
;consider the following examples of the use of the matcher:
;1. simple selection of parts of a list
;       (match foo [p1 p2])
;       if foo = '(a b) then
;       the match succeeds and p1 <- 'a, p1 <- p2
;       if foo = '(a b c) then
;       the match fails becasue foo is too long and p1 <- 'a, p2 <- 'b
;       if foo = 4 then
;       the match fails because foo is not a list and the values
;       of p1 and p2 are not changed
;2. more complex selection of parts of a list
;       (match foo [![p1 p2 ! _ = p3] = p0])
;       if foo = '(a b) then
;       the match succeeds and p0 <- '(a b), p1 <- 'a, p2 <- 'b, p3 <- nil
;       if foo = '(a b c) then
;       the match succeeds and
;          p0 <- '(a b c), p1 <- 'a, p2 <- 'b, p3 <- '(c)
;       if foo = 4 then
;       the match fails and p0, p1, p2, p3 are unchanged.
;       (note that (match foo [! p0 p1 p2 ! p3]) does the same thing
;       whenever it succeeds but that p0 will always be assigned to
;       no matter how it fails)
;3. an optional trailing argument
;       (match foo [p1 p2 ~ ! _ '(setq p2 'default)])
;       if foo = '(a b) then
;       the match succeeds and p1 <- 'a, p2 <- 'b
;       if foo = '(a) then
;       the match succeeds and p1 <- 'a, p2 <- 'default
;       if foo = '(a b c) then
;       the match succeeds and p1 <- 'a, p2 <- 'default
;       if foo = 4 then
;       the match fails and p1, p2 are unchanged
;4. an optional internal item. It is important to note that the match
;       does not look ahead and will not back up.  suppose we expect a
;       list to contain (p1 (optional p2) p3 p4)
;       there are several ways this type of match can be done:
;       a. it is important to note that this way will not work!
;          (match foo [p1 p2 ~ ! _ p3 p4])
;          if foo = '(a b c d) then
;          the match will succeed and
;              p1 <- 'a, p2 <- 'b, p3 <- 'c, p4 <- 'd
;          if foo = '(a c d) then
;          the match will fail and
;              p1 <- 'a, p2 <- 'c, p3 <- 'd, p4 unchanged
;          after p2 successfully matches 'c, the matcher goes on and
;            will not backup when there is nothing to match p4 against.
;       b. we can explicitly encode the backup.
;             (match foo [p1 ![p2 p3 p4] ~ ![p3 p4]])
;       c. we can explicitly look ahead.
;             (match foo [p1 ! _ (eq (length _) 2) ~ p2 p3 p4])
;       d. if there is some feature which p2 has and p3 can not have then
;          we can test that for example if p3 must be a list and p2
;          must be an atom then we can use
;             (match foo [p1 _ (atom _) = p2 ~ ! _ p3 p4])
;5. for a complex example of a pattern see the example in the macro deffn
```

```
;the following grammer gives the exact syntax of a pattern

;         pattern = p-list
;          p-list = [ {p-clause}*n ]
;        p-clause = {p-element ~}*n p-element
;       p-element = {!}
;                   atom v _ v ' atom v p-list
;                   {sexpr v = atom v ' sexpr}*n

;the pattern checking functions below check that a given pattern is
;syntactically correct, and return an internal form of the pattern which
;is described by the grammer below.  In the internal form a pattern
;element consists of three parts:
;       1. a flag specifying whether or not the element matches the
;               CADR of CDR of the current point.
;       2. a subpattern or nil if there is none.
;       3. a list of sexprs which will be evaluated in the order given.
;               if a sexpr is quoted then its value will be ignored,
;               otherwise a NIL value will cause the match to fail.
;in addition u1-p-check returns a list of all of the variables explicitly
;assigned too (ie cases 5.a and 6.g above) the if a is assigned to twice,
;it is in the list twice.

;  checked-pattern = checked-p-list
;   checked-p-list = ( {checked-p-clause}*n )
; checked-p-clause = ( {checked-p-element}*n>0 )
;checked-p-element = ( T v NIL
;                      checked-p-list v NIL
;                      ( { {'} sexpr }*n ) )
;
;---------------------------------------------------------------------
;
;as an example of the checking operation:
;
;
;       (u1-p-check '[! [CODE (MEMQ CODE '(A B C))  ~ ! _  '(SETQ CODE 'C)
;                       OPERATION
;                       ['OPANDS
;                         [_ ! _] = OPAND1
;                         OPAND2 (NOT (ATOM OPAND2))]
;                       ENV  ~ ! _  '(SETQ ENV NIL)]        = BODY])
;
;==>
;            ((((T (((NIL NIL ('(SETQ CODE _) (MEMQ CODE '(A B C))))
;                   (T NIL ('(SETQ CODE 'C))))
;                  ((NIL NIL ('(SETQ OPERATION _))))
;                  ((NIL
;                    (((NIL NIL ((EQ _ 'OPANDS))))
;                     ((NIL (((NIL NIL NIL)) ((T NIL NIL)))
;                           ('(SETQ OPAND1 _))))
;                      ((NIL NIL ('(SETQ OPAND2 _) (NOT (ATOM OPAND2))))))
;                     NIL))
;                  ((NIL NIL ('(SETQ ENV _))) (T NIL ('(SETQ ENV NIL)))))
;                 ('(SETQ BODY _)))))
;            BODY ENV OPAND2 OPAND1 OPERATION CODE)
```

```
;;; U1-P-CHECK
;
;                        BEHAVIOR
;INPUT  PATTERN      - a pattern
;OUTPUT RETURN VALUE - a checked pattern in internal form and a list
;                      of assigned variables
;
;                        PLAN
;VARS   ASSIGNED - a special variable used to collect the
;                  assigned variables
;
;METHOD the fns below exactly follow the structure of the grammers above
;       in order to check and parse the pattern

(defun u1-p-check (pattern)
  (let ((assigned nil))
    (declare (special assigned))
    (cons (u1-p-l-check pattern) (reverse assigned))))

;---------------------------------------------------------------------

;                        BEHAVIOR
;INPUT  PAT - pattern (this variable is special, and is used by the
;             following two functions as they step through the pattern.
;             it is rebound at each call of u1-p-l-check
;             as parsing recurses into a sub pattern)
;OUTPUT RETURN VALUE - a checked pattern

(defun u1-p-l-check (pat)
  (cond ((or (atom pat)
             (not (eq (car pat) '/[)))          ;a pattern must be in []s
         (ferror "u1-p-l-check1 pattern fails to be a list starting with '[;
 look up stack at special variable pat to see whole pattern:~%~S" pat))
        (t (setq pat (cdr pat))))
  (do ((p-l nil [(u1-p-c-check) ! p-l])) ;a pattern is a list of clauses
      ((null pat) (reverse p-l))))

;---------------------------------------------------------------------

(defun u1-p-c-check ()
  (cond ((atom pat)
         (ferror "u1-p-c-check1 pattern clause fails to be a list of one
 or more elements; look up stack at special variable pat to see more:~%~S" pat)))
  (do ((p-c [(u1-p-e-check)]                                    ;clause is one or more elements
            [(u1-p-e-check) ! p-c]))
      ((or (atom pat)
           (cond ((eq (car pat) '/:)
                  (ferror "/":/" in pattern")))
           (not (eq (car pat) '~)))                             ;seperated by ~s
       (reverse p-c))
      (setq pat (cdr pat))))
```

```
(defun u1-p-e-check ()

;                       PLAN
;VARS   BANG - flag set if ! is found
;       SUB-PAT - holds the sub pattern if any
;       SEXPRS - collects the sexprs derived from the pattern
;PURPOSE this is the main routine. it matches and parses a pattern element.
;       it just follows the grammer, lengthy but easy.

  (prog (bang sub-pat sexprs)
    (cond ((atom pat)
           (ferror "u1-p-e-check1 pattern element is fails to be list of items;
 look up the stack at special variable pat to see more:~%~S" pat))
          ((eq (car pat) '!)                                    ;OPTIONAL initial !
           (setq bang t)
           (setq pat (cdr pat))))
    (cond ((atom pat)
           (ferror "u1-p-e-check2 pattern element fails to have obligatory
 basic matching element; look up te stack at special variable pat to see more:~%~S" pat))
          ((eq (car pat) '_))                                   ;OBLIGATORY _
          ((atom (car pat))                                     ;or atom
           (setq sexprs
                 [['quote ['setq (car pat) '_]]                   ;which gets assigned to
                  ! sexprs])
           (setq assigned [(car pat) ! assigned]))                 ;remember was assigned to
          ((and (eq (caar pat) 'quote)                          ;or quoted atom
                (atom (cadar pat)))
           (setq sexprs
                 [['eq '_ (car pat)]                              ;which gets compared
                  ! sexprs]))
          ((eq (caar pat) '/[)                                  ;or subpattern
           (setq sub-pat (u1-p-1-check (car pat))))
          (t (ferror "u1-p-e-check3 subpattern fails to begin with '[;
 look up the stack at the special varible pat to see more:~%~S" pat)))
    (setq pat (cdr pat))
    (do ()                                                      ;0-n extra expressions
        ((or (atom pat)                                         ;if pat points to the start of
             (and (atom (car pat))                              ;a pattern element then we
                  (not (eq (car pat) '=)))                      ;are done
             (and (not (atom (car pat)))
                  (eq (caar pat) 'quote)
                  (not (atom (cdar pat)))
                  (atom (cadar pat)))
             (and (not (atom (car pat)))
                  (eq (caar pat) '/[))))
        (cond ((eq (car pat) '=)                                ;might be a = assignment
               (setq pat (cdr pat))
               (cond ((or (atom pat)
                          (not (atom (car pat))))
                      (ferror "u1-p-e-check = in pattern fails to be followed by on atom;
 look up the stack at special variable pat to see more:~%~S" pat)))
               (setq sexprs
                     [['quote [ 'setq (car pat) '_]]            ;in which case do assignment
                      ! sexprs])
               (setq assigned [(car pat) ! assigned]))          ;remember was assigned
              (t (setq sexprs [(car pat) ! sexprs])))           ;otherwise take sexpr unchanged
        (setq pat (cdr pat)))
    (return [bang sub-pat (reverse sexprs)])))                  ;return the checked pattern

;the macro MATCH produces code to do the mathcing. the code produced to match a pattern
;against a list has the form of a lambda binding up the list as U1-ITEM, whose body is an
;AND.  the sexprs whose value does not effect the success or failure of the match are
;hidden inside PROGNs.  a set of alternatives shows up as an OR.  MATCH tries to be smart
;enough so that simple cases produce simple code.  The following is an example of the code
;produced.

;       (MATCH FOO [! [CODE (MEMQ CODE '(A B C))  ~ ! _  '(SETQ CODE 'C)
;                      OPERATION
```

```
;                       ['OPANDS
;                         [_ ! _] = OPAND1
;                         OPAND2 (NOT (ATOM OPAND2))]
;                       ENV  ~ ! _  '(SETQ ENV NIL)]        = BODY])
; expands into ==>
;
;       ((LAMBDA (U1-ITEM)
;          (AND ((LAMBDA (U1-ITEM)
;                  (AND (OR (AND (NOT (ATOM U1-ITEM))
;                                (PROGN (SETQ CODE (CAR U1-ITEM))
;                                (MEMQ CODE '(A B C)))
;                                (PROGN (SETQ U1-ITEM (CDR U1-ITEM)) T))
;                           (PROGN (SETQ CODE 'C) T))
;                       (NOT (ATOM U1-ITEM))
;                       (PROGN (SETQ OPERATION (CAR U1-ITEM))
;                              (SETQ U1-ITEM (CDR U1-ITEM))
;                       (NOT (ATOM U1-ITEM)))
;                       ((LAMBDA (U1-ITEM)
;                           (AND (NOT (ATOM U1-ITEM))
;                                (EQ (CAR U1-ITEM) 'OPANDS)
;                                (PROGN (SETQ U1-ITEM (CDR U1-ITEM))
;                                (NOT (ATOM U1-ITEM)))
;                                ((LAMBDA (U1-ITEM) (NOT (ATOM U1-ITEM))) (CAR U1-ITEM))
;                                (PROGN (SETQ OPAND1 (CAR U1-ITEM))
;                                       (SETQ U1-ITEM (CDR U1-ITEM))
;                                (NOT (ATOM U1-ITEM)))
;                                (PROGN (SETQ OPAND2 (CAR U1-ITEM))
;                                (NOT (ATOM OPAND2)))
;                                (NULL (CDR U1-ITEM))))
;                        (CAR U1-ITEM))
;                        (PROGN (SETQ U1-ITEM (CDR U1-ITEM))
;                        (OR (AND (NOT (ATOM U1-ITEM))
;                                 (PROGN (SETQ ENV (CAR U1-ITEM))
;                                 (NULL (CDR U1-ITEM))))
;                            (PROGN (SETQ ENV NIL) T)))))
;                 U1-ITEM)
;                (PROGN (SETQ BODY U1-ITEM) T)))
;         FOO)
```

```
;MATCH takes an optional second argument which if eq to NOMATCH causes the code produced
;to assume that the pattern will match.  in this case, the lambda bodies do not need to
;be ANDs and all of the predicates are ignored. this leads to a significant reduction
;in the code produced as the following example shows.


;       (MATCH FOO [! [CODE (MEMQ CODE '(A B C))  ~ ! _  '(SETQ CODE 'C)
;                      OPERATION
;                      ['OPANDS
;                        [_ ! _] = OPAND1
;                        OPAND2 (NOT (ATOM OPAND2))]
;                      ENV  ~ ! _  '(SETQ ENV NIL)]        = BODY]  NOCHECK)
;
;expands into ==>
;
;       ((LAMBDA (U1-ITEM)
;          ((LAMBDA (U1-ITEM)
;              (OR (AND (NOT (ATOM U1-ITEM))
;                       (PROGN (SETQ CODE (CAR U1-ITEM))
;                       (MEMQ CODE '(A B C)))
;                       (PROGN (SETQ U1-ITEM (CDR U1-ITEM)) T))
;                  (PROGN (SETQ CODE 'C) T))
;              (SETQ OPERATION (CAR U1-ITEM))
;              (SETQ U1-ITEM (CDR U1-ITEM))
;              ((LAMBDA (U1-ITEM)
;                   (SETQ U1-ITEM (CDR U1-ITEM))
;                  (SETQ OPAND1 (CAR U1-ITEM))
;                  (SETQ U1-ITEM (CDR U1-ITEM))
;                  (SETQ OPAND2 (CAR U1-ITEM)))
;               (CAR U1-ITEM))
;              (SETQ U1-ITEM (CDR U1-ITEM))
;              (OR (AND (NOT (ATOM U1-ITEM))
;                       (PROGN (SETQ ENV (CAR U1-ITEM))
;                        (NULL (CDR U1-ITEM))))
;                  (PROGN (SETQ ENV NIL) T)))
;            U1-ITEM)
;          (SETQ BODY U1-ITEM))
;        FOO)
```

```
(defun (match macro) (body)

;                       BEHAVIOR
;INPUT  BODY = ( 'match obj pattern {check})
;OUTPUT RETURN VAL - the code to do the matching (this replaces the macro call itself)
;                       PLAN
;VARS   OBJ - the object to match the pattern against (this will be evaluated before the match)
;       PATTERN - the pattern
;       CHECK - this inhibits code to check matching if (third body) = NOCHECK
;       CODE - this holds the code to return
;METHOD u1-p-cgen does the real work

  (prog (obj pattern check code)
    (cond ((lessp (length body) 3)
           (ferror "Not enough arguments to function MATCH:~%~S" body)))
    (setq obj (cadr body)
          pattern (caddr body))
    (setq check (cond ((atom (cdddr body)))
                      ((not (eq (cadddr body) 'nocheck)))))
    (setq code (u1-p-cgen check obj
                          (car (u1-p-check pattern))))
    (cond ((atom code) (setq code ['progn code])))
    (rplaca body (car code))
    (rplacd body (cdr code))
    (return body)))
```

```
(defun u1-p-cgen (check item pattern)

;                       BEHAVIOR
;INPUT  CHECK - T/NIL to check or not
;       ITEM - the item to match the pattern against
;       PATTERN - the checked pattern

    (cond ((null pattern) 't)                                    ;if no pattern no matching
          ((and (null (cdr pattern))                             ;[![ ... ]] = [ ... ]
                (null (cdar pattern))                            ;a minor optimization to save
                (caaar pattern)                                  ;code
                (null (caddr (caar pattern))))
           (u1-p-cgen check item (cadaar pattern)))
          (t (u1-make-lambda item                                ;make a lambda of
               [(u1-make-body check
                  (do ((code (u1-p-c-cgen check t nil
                                          (car (last pattern)))
                             [!(u1-p-c-cgen check nil code       ;all the clause codes
                                 (car pat))
                              ! code])
                       (pat (cdr (reverse pattern))
                            (cdr pat)))
                      ((null pat) code)))]))))

;---------------------------------------------------------------

(defun u1-p-c-cgen (check last following p-c)

;                       BEHAVIOR
;INPUT  CHECK - T/NIL
;       LAST - T/NIL whether clause is last in the pattern (therefore will need to test
;              whether item matched against is too long)
;       FOLLOWING - ARB/NIL this says whether or not any of the clauses following this one
;              have produced any code at all, if not then this clause does not have to
;              produce any code to get ready for the next clause to exicute
;              (namely cdring down the list)
;OUTPUT RETURN VAL - the code for the clause

  (cond ((null (cdr p-c))
         (u1-p-e-cgen check last following (car p-c)))           ;if only one element no OR
        (t[(u1-make-or                                           ;an OR of
             (mapcar #'(lambda (x)
                         (u1-make-body t
                           (u1-p-e-cgen t last following x)))  ;the codes for the elements
                     p-c))])))
```

```
(defun u1-p-e-cgen (check last following p-e)

;                       BEHAVIOR
;INPUT  same as for u1-p-c-cgen
;OUTPUT RETURN VAL - the code for the element
;                       PLAN
;VARS   BANG - SUBP - SEXPRS the parts of the pattern
;       CURRENT - the thing being matched against. either U1-ITEM or (CAR U1-ITEM)

  (prog (bang subp sexprs current)
    (setq bang (car p-e)
          subp (cadr p-e)
          sexprs (caddr p-e))
    (setq current (cond (bang 'u1-item)
                        (t '(car u1-item))))
    (cond ((not check)                                          ;if not checking throw away
           (setq sexprs                                         ;the predicates
                 (mapcar #'(lambda (x)
                             (cond ((and (not (atom x))
                                         (eq (car x) 'quote))
                                    x)
                                   (t t)))
                         sexprs))))
    (setq sexprs                                                ;substitute CURRENT for _ in
          (mapcar #'(lambda (x)                                 ;the sexprs
                      (cond ((atom x) x)
                            ((eq (car x) 'quote)                ;outermost ' is different
                             ['quote ! (u1subst current '_ (cdr x))]) ;does not block subst
                            (t(u1subst current '_ x))))
                  sexprs))
    (setq sexprs [(cond ((and check (not bang))                 ;if checking and no !
                         '(not (atom u1-item)))                 ;must see that u1-item is list
                        (t t))
                  (cond (subp (u1-p-cgen check current subp))   ;recurse for subpattern if any
                        (t t))
                  ! sexprs                                      ;put sexprs here
                  (cond ((and following (not bang))             ;do cdr if needed
                         '(quote (setq u1-item (cdr u1-item))))
                        (t t))
                  (cond ((and check last (not bang))            ;check for end if needed
                         '(null (cdr u1-item)))
                        (t t))])
    (return (delq 't sexprs))))                                 ;throw away the Ts

;---------------------------------------------------------------

(defun u1subst (new old list)

;                       BEHAVIOR
;INPUT  NEW - object to replace old
;       OLD - an object usually an atom to be substituted for
;       LIST - the list to do the substitution in
;OUTPUT RETURN VAL - the list after substitution
;                       PLAN
;PURPOSE this does substitution just like SUBST except that it does not go done
;       past qoutes

  (cond ((eq list old) new)
        ((or (atom list)
             (eq (car list) 'quote))                            ;stop if hit quote
         list)
        (t (do ((rest list (cdr rest))
                (newl nil (cons (u1subst new old (car rest)) newl)))
               ((null rest) (reverse newl))
               (cond ((atom rest) (return (nreconc newl (u1subst new old rest)))))))))
```

```
;these little fns try to be reasonably intelligent about producing basic code structures

(defun u1-make-and (preds)

;make an AND of a list of preds

  (setq preds (u1-flatten-like-ops 'and preds))                ;expand out inner ands
  (setq preds (delq 't preds))                                 ;discard Ts
  (cond ((null preds) t)                                       ;no preds then T
        ((null (cdr preds)) (car preds))                       ;one pred the pred
        (t ['AND ! preds])))                                   ;else an and
;---------------------------------------------------------------

(defun u1-make-or (preds)

;this is just like u1-make-and but for ORs

  (setq preds (u1-flatten-like-ops 'or preds))
  (setq preds (delq 'nil preds))
  (cond ((null preds) nil)
        ((null (cdr preds)) (car preds))
        (t ['OR ! preds])))
;---------------------------------------------------------------

(defun u1-make-lambda (arg body)

;this makes a lambda out of a body which is a list of actions

  (setq body (u1-flatten-like-ops 'progn body))                ;expand out inner progns
  (cond ((equal body '(t)) t)                                  ;if only a (T) no lambda
        (t [['lambda '(u1-item) ! body] arg])))                ;lambda var is 'u1-item
;---------------------------------------------------------------

(defun u1-make-progn (actions)

;just like u1-make-and but makes a PROGN

  (setq actions (u1-flatten-like-ops 'progn actions))
  (cond ((null actions) t)
        ((null (cdr actions)) (car actions))
        (t ['progn ! actions])))

;---------------------------------------------------------------

(defun u1-flatten-like-ops (op actions)

;nconc in any action whose car is op this reduces the height of the list

  (mapcan #'(lambda (action)
              (cond ((or (atom action)
                         (not (eq (car action) op)))
                     [action])
                    (t (cdr action))))
          actions))
```

```
(defun u1-make-body (check code)

;                       BEHAVIOR
;INPUT  CHECK - T/NIL
;       CODE - a list of sexprs
;OUTPUT RETURN VAL - if check then an AND of the sexprs with the qouted sexprs
;               hidden in progns so that there values will be ignored.  As few
;               progns as possible are used.  if not check then just a progn of the
;               sexprs with the quotes removed
;                       PLAN
;METHOD the only difficulty comes in trying to get the progns just right

  (setq code (delq 't code))                                    ;get rid of Ts
  (cond (check
         (u1-make-and
           (do ((ocode (reverse code) (cdr ocode))              ;the original code in reverse
                (ncode nil)                                     ;the list of codes to be anded
                (pcode nil))                                    ;a set of actions to be progned
               (())
               (cond ((or (null ocode)                          ;if not a pred
                          (atom (car ocode))
                          (not (eq (caar ocode) 'quote)))
                      (cond (pcode                              ;if some pcode
                             (setq ncode
                                   [(u1-make-progn pcode) ! ncode])))  ;package them up
                             ;note if only one then no progn is used the one is a pred
                      (cond (ocode (setq pcode [(car ocode)])))) ;set to use pred to end progn
                     (t (setq pcode [(cadar ocode)              ;put action on pcode
                                     !(cond (pcode              ;if last thing is action
                                            (t [t]))]))))       ;must use T in progn
               (cond ((null ocode) (return ncode))))))
        (t (u1-make-progn                                       ;if not checking
             (mapcar #'(lambda (x)
                         (cond ((and (not (atom x))
                                     (eq (car x) 'quote))       ;just strip of quotes
                                (cadr x))
                               (t x)))
                     code)))))
```

```
;NOW THAT DEFUN& AND DEFMACRO EXIST, THIS IS OBSOLETE
;DO NOT USE IT ANY MORE.

;the following function is intended to replace defun as the basic function for defining
;functions.  its major improvements are:
;       1. it uses a pattern to specify what the inputs to the function being defined are.
;          they way the pattern is interpreted depends on the type of the function being
;          defined:
;               a. (FEXPR MACRO) the function is known to have only 1 arg. the pattern is
;                  matched against this arg.  if the function is a MACRO and the first
;                  element of the pattern is a literal match against the name of the
;                  macro, that part of the match is optimized away.
;               b. (LEXPR) here the pattern is matched against listify of the args.
;                  if the corespondence between parts of the pattern and the args can
;                  be determined at macro time (ie no top level ~s or !s in an initial portion
;                  of the pattern) then initial parts of teh listify wont actually be done.
;               c. (EXPR) here the pattern is decomposed and the first pattern element is
;                  matched against the first arg etc.  (there must not be any top level
;                  ~s or !s because it must be possible to tell how many args are intended.)
;       2. one of the major uses of the pattern is to pick out parts of an input argument
;          and put them in to variables.  to facilitate this, deffn binds every variable name
;          which is assigned to (by = or as first item in a pattern element).  If you do not
;          want a variable bound, do a  '(setq var _). if you do want it bound but it is not
;          logical to assign a part of the input to it assign some part to it and then override
;       NOTE that all of the variable binding set up in the defined function by deffn
;          is done with lambdas and not with progs. as a result the value of the function
;          is the value of its last form, there should not be a return unless the prog is
;          explicit in the body.
;       3. the pattern has two roles.  first it allows for selecting out parts of the input.
;          second it checks the syntactic correctness of the input.  if the function
;          being  defined is called with an input which does not match the pattern, the
;          function issues an error 'BAD-ARGS-TO-name and a list of  arguments.  For
;          efficiency, the checking feature is turned off in compiled code.  The CHECK
;          parameter to deffn will override this default causing the match to be checked
;          all the time, or (NOCHECK) never.
;       4. in general deffn makes up variable names (begining in U1) for the actual
;          arguments to the function. However it optimizes a bit, and uses names in the
;          pattern if it can.  This results in a simplyfication of the pattern.  as a
;          result a simple deffn like:
;               (deffn foo [a b c] body)
;          produces no more code than the corresponding defun:
;               (DEFUN foo (a b c) body)
;       5. deffn has features aimed at making macro definition easier:
;               a. if the MACROS switch is T then a macro is both known in the
;                  compiler and COMPILED (this is an extension of the basic compiler
;                  action). the keyword KEEP/NOKEEP will override the value of MACROS
;                  for a particular macro definition KEEP is the same as MACROS=T.
;               b. the macros are by default displacing
;                  the DISPLACE parameter will make them always displace
;                  or NODISPLACE never displace.
;       6. deffn takes an optional parameter after the name to specify the property
;          of the atom which the function should go on (this is similar to defuns
;          (name prop)).
;       7. deffn assigns its body to the global variable DEFFN to aid in debugging
;          at initial read in time.
```

```
(defun (deffn macro) (mbody)
;                       BEHAVIOR
;INPUT  MBODY - ('deffn
;               name            ;the name of the function
;               {pname}         ;the prop name (the default is based on the type)
;               {type}          ;expr,fexpr,macro or lexpr (expr is default)
;               {keep/nokeep}   ;says whether to compile a macro default nokeep
;               {displace/nodisplace} ;displacing is the default
;               {check/nocheck} ;default is nocheck
;               pattern         ;the pattern describing the inputs
;               ! body)         ;the body of the function
;OUTPUT RETURN-VALUE - a defun with the right body to do all the stuff discussed above.
;               basically binding the right variables and doing the matching in addition to
;               the body.  deffn always displaces itself.

  (setq deffn mbody)                                                ;to aid debugging
  (prog (name pname namecode type displace? keep? check? pat body fbody)
    (cond ((not (match mbody                                        ;parse the macro call
                  ['deffn
                    name (atom name)
                    _ (memq _ '(expr fexpr macro)) = type = pname
                     ~ 'lexpr = type '(setq pname 'expr)
                     ~ _ (atom _)
                         (not (memq _ '(keep nokeep displace nodisplace check nocheck)))
                         = pname '(setq type 'expr)
                     ~ ! _ '(setq type 'expr pname 'expr)
                    _ (memq _ '(keep nokeep)) '(setq keep? (eq _ 'keep))
                     ~ ! _ '(setq keep? nil)
                    _ (memq _ '(displace nodisplace))
                     '(setq displace? (eq _ 'displace))
                     ~ ! _ '(setq displace? t)
                    _ (memq _ '(check nocheck)) '(setq check? (eq _ 'check))
                     ~ ! _ '(setq check? nil)
                    pat (not (atom pat))
                  ! body (not (atom body))]))
           (ferror "Bad args to DEFFN:~%~S" mbody)))
    (setq namecode                                                  ;construct name code for deffn
          (cond ((eq pname 'expr) [name])                           ;if EXPR omit keyword EXPR
                ((memq pname '(fexpr macro))                        ;if FEXPR or MACRO
                 [name pname])                                      ;one after the other
                (t [[name pname]])))                                ;otherwise sublist format
    (setq fbody (u1-make-fn name type displace? check? pat body)) ;call this to do the work
    (setq fbody
          (cond ((and (eq pname 'macro) keep?) ;if a kept macro
                 '(progn 'compile
                    (declare (defun ,name macro ,.fbody))           ;declare a defun
                    (defun (,name macro) ,.fbody)))                 ;and compile the macro
                (t ['defun !! namecode !! fbody])))                 ;otherwise just normal defun
    (return (displace mbody fbody))))                               ;displace yourself
```

```
(defun u1-make-fn (name type displace? check pat body)

;                       BEHAVIOR
;INPUT  name - the name of the fn being defined
;       type - either (macro fexpr expr or lexpr) says how to parse the parameters
;       displace? - either T or nil says whether a macro
;               being defined will displace itself or what
;       check - (t/nil) says whether argument matching should be checked
;               at runtime or just assumed
;       pat - the pattern for the parameters to the fn
;       body - the body of the function
;OUTPUT return value - the cdr of a lambda definition for the fn being defined
;                       PLAN
;VARS   c-pat - the checked form of the pattern
;       vars - the variables assigned too in the pattern (variables which become args
;               are deleted from this list).
;       variable-part - the tail of cpat begining with the first top level pattern
;               element which is either optional or led.  Ie the first place where
;               it is not clear how many arguments there are.
;       args - the list of arguments to the top level lambda expression.
;       pat-code - the code which does the pattern matching connected with the args.
;       i - used to count the args to a lexpr.
;       exprl - expression list used with u1findelligablesetq below.
;       assign - a var which can be used directly as an argument name.
;METHOD most of the work here involves deciding how to pattern match to get the vars
;       assigned too there are three seperate segments one for macros and fexprs, one
;       for exprs and one for lexprs.  then the results are consed together and returned
```

```
  (prog (c-pat vars variable-part args pat-code i exprl assign)
    (match (u1-p-check pat) [c-pat ! vars] nocheck)            ;check pattern
    (setq variable-part                                        ;get variable part
          (do ((pat c-pat (cdr pat)))
              ((null pat) nil)
              (cond ((or (cdar pat)                            ;there is if an alternative
                         (caaar pat))                          ;or a !
                     (return pat)))))
;for macros and fexprs
    (cond ((memq type '(macro fexpr))
           (cond (nil)
;this stuff is patched out untill JONL fixes the bug where
;displace messes up when compiled, if you also use the var name
;it is displacing for other things. !!!!!
;                 ((and (match c-pat [[['t _ ! exprl]] ! _])    ;if there is a variable to use
;                       (setq assign (u1findelligablesetq (car exprl) vars))) ;for the arg
;                  (setq args [(cadadr assign)])                          ;use it
;                  (rplaca exprl (delq assign (car exprl)))     ;take it out of pattern
;                  (setq vars (delq (car args) vars)))          ;take it out of vars
                  (t(setq args ['u1-body])))                   ;otherwise use U1-BODY
           (setq pat-code
                 [(cond ((match c-pat                          ;if first part of pattern is
                                [[['nil 'nil
                                   [['eq '_ [ 'quote _ (eq _ name)]]]]] ;fn name don't bother
                                 ! _ (not (atom _))])
                         (u1-p-cgen check ['cdr (car args)] (cdr c-pat))) ;to match against it
                        (t(u1-p-cgen check (car args) c-pat)))]) ;otherwise match whole thing
           (cond ((and (eq type 'macro) displace?)
                  (cond ((cdr body) (setq body [['progn ! body]]))) ;make body progn if list
                  (setq body [['displace (car args) ! body]])))))
;for exprs
          ((eq type 'expr)
           (cond (variable-part (ferror "u1-make-fn1 fn is an expr but the pattern does not
 specify an unconditional number of arguments:~%~S" deffn)))
           (mapc #'(lambda (p-c)                               ;for each arg position
                     (cond ((and (match p-c [['nil _ ! exprl]])  ;do same as for macros
                                 (setq assign (u1findelligablesetq (car exprl) vars)))
                            (setq args [(cadadr assign) ! args])
                            (rplaca exprl (delq assign (car exprl)))
                            (setq vars (delq (car args) vars)))
                           (t(setq args [(gensym) ! args])))
                     (rplaca (car p-c) 'T)                     ;set up match for this arg
                     (setq pat-code [(u1-p-cgen check (car args) [p-c]) ! pat-code]))
                 c-pat))
```

```
;for lexprs
        ((eq type 'lexpr)
         (setq args 'u1-nargs)
         (setq i 0)
         (prog ()                                              ;so return will work in map
          (map #'(lambda (p-cl)
                 (cond ((eq variable-part p-cl)                ;do listify for variable part
                        (setq pat-code
                              [(u1-p-cgen check
                                          ['listify ['- i 'u1-nargs]]
                                          p-cl)
                               ! pat-code])
                        (return t))
                       (t(rplaca (caar p-cl) 't)               ;match others seperately
                         (setq i (1+ i))
                         (setq pat-code
                               [(u1-p-cgen check
                                           ['arg i]
                                           [(car p-cl)])
                                ! pat-code])))))
               c-pat))
         (cond ((and check (not (= i 0)))                      ;check for right number of args
                (setq pat-code
                      [!! pat-code ['not ['lessp 'u1-nargs i]]]))))))
;now for all types
    (cond ((not (atom args))(setq args (reverse args))))       ;args were in wrong order
    (setq pat-code (reverse pat-code))                         ;also pat code
    (cond (check                                               ;if checking set up for
           (setq pat-code (u1-make-and pat-code))              ;error message if fail
           (setq pat-code
                 (cond ((eq pat-code t) nil)
                       (t[['cond [['not pat-code]
                                  ['ferror nil
                                           (format nil "Bad args to ~S: ~~S"
                                                   name)
                                           (cond ((atom args) ['listify args])
                                                 (['list ! args]))]]]]))))
          (t(setq pat-code                                     ;otherwise just do neccesary
                  (delq 't                                     ;matching
                        (u1-flatten-like-ops 'progn
                                             pat-code)))))
    (setq body [!! pat-code ! body])                           ;add patcode to body
    (do ((v vars (cdr v)))                                     ;remove duplicat vars
        ((null v))
        (rplacd v (delq (car v) (cdr v))))
    (cond (vars                                                ;if vars set up a lambda for
           (setq body [[['lambda vars ! body]                  ;them with appropriate initial
                       ! (mapcar #'(lambda (ignore) nil)
                                 vars)]])))
    (setq body [args ! body])                                  ;put the args in front
    (return body)))                                            ;and return
```

```
;                        BEHAVIOR
;INPUT  actions - the action list of a pattern element
;       vars - the list of all assignments in the whole pattern
;OUTPUT return value - if there is an assignment in the action list to a var which
;               ocurs exactly once in the var list, the assignment is returned.
;               it is assumed that if this is the case then it is alright to assign
;               the correct value to this variable before the match starts. I think
;               that it may be possible to think up an example where this isn't quite true.

(defun u1findelligablesetq (actions vars)
  (do ((action actions (cdr action))
       (var)(mem))
      ((null action) nil)
      (cond ((and (match (car action) ['quote ['setq var ! _]])
                  (setq mem (memq var vars))
                  (not (memq var (cdr mem))))
             (return (car action))))))
```

```
;OBSOLETE USE LET* INSTEAD

;this is a much more fancy substitute for an inline lambda.  the main
;differences are:
;1. the arguments are not evaluated in parallel.
;2. arbitrary patterns can be used instead of a variable as the
;destination of a value.  begin figures what variables have to be lambda
;bound in the same way as deffn.
;3. the correct lambda binding value {nil/0/0.0} is used so that things
;will complie right.
;4. begin does not allow the archaic form of setting values.
;
;the input is of the form:
;(begin (
;   var                OR
;   var <- arg
;each as many times as desired
;   ) body)
;a var can be either: a variable, or a pattern (recognized by the []s)
;the resulting lambda is:
;((lambda (vars ..) (<- var arg ...) body) {nil/0/0.0} ...)
;
;for example a valid begin is:
;(begin (x  y <- foo  z <- '(1)  [a b c] (caddr y)) body)
;which becomes:
;((lambda (x y z a b c) (<- y foo  z '(1)  [a b c] (caddr y)) body)
; nil nil nil nil nil nil)
;
;this function iterates matching pieces off the front of the
;initializations and then conses up the result.

(deffn begin macro keep displace check ['begin inits ! body]
  (prog (vars assigns)
    (do ()
        ((null inits))
        (cond ((atom inits)
               (ferror "BEGIN inits not a list~% ~S" '(,inits . ,body)))
              ((atom (car inits))                                  ;atom var is ok
               (setq vars [(car inits) ! vars]))                   ;remember var
              ((eq (caar inits) '/[)                               ;pattern var is ok
               (setq vars [!!(cdr (u1-p-check (car inits))) ! vars]))   ;remember vars in pat
              (t (ferror "Unrecognizable init in BEGIN; ~
                          probably a missing /"<-/"~%Source: ~S"
                         '(BEGIN ,inits ,@body))))
        (cond ((or (atom (cdr inits))
                   (not (eq (cadr inits) '<-)))                    ;if <-
               (setq inits (cdr inits)))
              ((atom (cddr inits))
               (ferror "A trailing /"<-/" in a BEGIN "
                       '(BEGIN ,inits ,@body)))
              (t (setq assigns [(caddr inits) (car inits) ! assigns])   ;remember to assign
                 (setq inits (cdddr inits)))))                    ;(note assigns is reversed)
    (cond (assigns
           (setq body [['<- !! (reverse assigns)] ! body])))      ;put assigns in body
    (setq vars (reverse vars))                                    ;put vars in order
    (do ((v vars (cdr v)))
        ((null v))
        (rplacd v (delq (car v) (cdr v))))                        ;kill duplicates
    (return [['lambda vars ! body]                                ;cons up result
             !!(mapcar
                 #'(lambda (ignore) nil)
                 vars)])))
```

```
;THIS IS GOING TO BE OBSOLETE AS SOON AS I GET TC TO WORK

;the next pages define a looping construct OVER which is particularly convenient in
;a fairly large subset of cases where looping is required.  It is
;inspired by the map functions in lisp.  however it differes from them in the fact
;that instead of a functional argument, it has a literal body which is
;expanded inline into the code and repetatively executed for example:
;       (mapc #'(lambda (x) (print x)) foo)      is the same as
;       (over foo (print _))
;note that the basic order of arguments is reversed (on the theory that
;the functional specification is usually much longer than the data
;specification).  also, "_" is the default name for the variable which
;holds the successive sublists.
;    in addition, over is a bit like do in that it allows
;the user to explicitly specify how to step over the initial value, and
;how to accumulate a result, this leads to a great deal more flexability
;than the six forms of map give you. a use of the macro over (or for) has
;three basic parts:
;       (enumeration-specifications
;        accumulation-specification
;        body)
;there must be at least one enumeration-specification there can be
;several.  each one has six parts:
;       "over"
;       overvar "<-"    ;which defaults (see below) if omitted
;       start-expr      ;which must be present
;       "by" byexp      ;which defaults to (cdr _) or (1+ _) (see below) if omitted
;       "until" untilexp ;which defaults to (null _) or nil (see below) if omitted
;       source-expr     ;which defaults to (car _) if omitted
;       "->" source-var ;which defaults to "_" if there is no overvar
;       there can be any number of additional 'source-expr -> source-var' specifications
;for example the following are all valid enumeration specifications:
;       over [a x] <- foo by (cadr x) until (hunkp x)  -> w  (nreverse (cadar x)) -> [y z]
;note that any fns can be used in the source expression and the variables to assign to
;can actually be patterns.  any variable assigned in the overvar can be referrenced in
;the by, until, and source expresions; the variables assigned to by a source expression
;cannot be referred to in any start by or until expression, but can be refered to by
;any source expression to the right of it.
;    the power of the macro is increased by the fact that there are a lot of defaults available
;for example in the by, until, and source expressions "_" can be used to refer to the
;overvar even if the user has not specified the overvar (! this is not usually compatable with
;using a pattern for the overvar because the entire overvar expression is substituted for
;each instance of "_"). as an example of the use of "_" consider:
;       over x <- foo by (cdr x) until (null x) (car x) -> y            is the same as
;       over x <- foo by (cdr _) until (null _) (car _) -> y            is the same as
;       over foo by (cdr _) until (null _) (car _) -> y
;in the last case an internal name is generated for the overvar and used, it cannot
;be refferenced by the user except as _ in the by until and source expressions.
;     in addition, the by, untill, and source expresions have default values
;specified in terms of "_".  what they default to depends on the start-expression.
;if the start-expr is a number then the by defaults to "(1+ _)" the until defaults
;to "nil" which is to say the enumeration will not stop and the source defaults to
;"_". the following two specifications are the same:
;       over 1 by (1+ _) until nil _ -> i
;       over 1 -> i
;if the start-expression is not a number then the by defaults to "(cdr _)", the until
;defaults to "(null _)" and the source defaults to "(car _)" as a result the example
;prior to the last could be specified as:
;       over foo -> y
```

```
;this makes it very easy to specify the enumeration used by mapc, mapcar, and mapcan
;     the enumeration used by map, maplist, and mapcon can be abtained by specifying a
;overvar and no source var e.g.:
;       over x <- foo
;if neither an overvar or a source var is specified e.g.:
;       over foo
;then _ is used as the source var and an internally generated name is used as the
;overvar.  (note that the "_" used in the by, until, and
;source expresions has a local scope and therefor has nothing to do with the "_" used as
;a default for the source var).  clearly if there is more than
;one enumeration-specification then at most one of them can be allowed to generate "_"
;as a default source var or there will be trouble.
;
;consider now an accumulation-specification, it specifies how a result is accumulated and
;returned.  it has four parts
;       "select" select   ;which defaults to T (ie no selection)
;       "return" return   ;which defaults to nil or over-acc (if there is a step)
;       "step" step       ;which defaults to nil (ie no step)
;       "init" init       ;which defaults to a value of nil
;in writing the select, and step, all of the variables defined in the enumeration-
;specifications can be used (the init and return cannot refer to any source vars).  in addition
;there are several other special variables which can be referred to (they are defined only
;if they are reffered to somewhere).  these variables are
;  1. over-acc which is assigned the value init to start out and is assigned the result
;       of the step at each iteration.
;  2. over-body this can appear only in the step and only once!  the entire body of the over
;       (see below) is substituted for this in the step.
;  3. first-start this is assigned the initial value of the start expression of the first
;       enumeration-specification (it is usefull for sideffecting this start expression)
;before discussing how the parts of the accumulation specification interact it should be said
;that the body of a call on over is simply the remainder of the call after the above parts
;are matched at the front.  if the body consists of more than one sexpr it is wrapped in
;a progn.  they body can contain an explicit return in order to stop the iteration.
;if it does, it must also specify what is to be returned, the return expression only
;applies to the normal termination of the over.
;the best way to see how the accumulation specification works is through a generalized
;schema (the schema does not show how simple cases are optimized to promote efficiency)
;       (over ov <- start by (A _) until (B _) (C _) -> sv
;            select (D ov sv over-acc first-start)
;            return (E ov over-acc first-start)
;            step (F ov sv over-acc first-start over-body)
;            init (G ov first-start)
;            (H ov sv over-acc first-start))
;expands into:
;       (KBE-obsolete-loop-macro (first-start <- start
;              ov <- first-start
;              sv
;              over-acc <- (G ov first-start))
;         (if (B ov) (return (E ov over-acc first-start)))
;         (<- sv (C ov))
;         (if (D ov sv over-acc first-start)
;             (<- over-acc (F ov sv over-acc first-start
;                              (H ov sv over-acc first-start))))
;         (<- ov (A ov)))
```

```
;the body says what to do on each iteration, the select says which iterations to do something
;on, the step says how to combine the results of a series of iterations, the init says what
;value to start with, and the return says what to return.  for example the following sums
;the numbers in a list foo:
;       (over foo -> x
;             select (numberp x)
;             return over-acc
;             step (+ over-acc over-body)
;             init 0
;         x)
;the return does not actually have to be specified because the return defaults to over-acc
;if there is a step and nil otherwise.
;
;in order to increase the convenience there is a way of quickly specifying what the return,
;step, and init are for a bunch of common cases.  this is by the use of a keyword.  the keyword
;is placed between the select and the return and specifies defaults for the return step and
;init.  a keyword is any atom which has a u1overstep property and optionaly a u1overreturn and
;u1overinit property.  this use of plists allows for the easy extension of the number
;of keywords.  it is expected that keywords will be used 95% of the time.  individual
;keywords already defined are discussed below where the defprops are done.  for example
;one of them "sum" allows you to accumulate a sum and enables you to write the above as:
;       (over foo -> x select (numberp x) sum x)
;
;finally to give a better feel for this construct consider a couple of comparison examples:
;       (mapcar #'+ l1 l2 l3)
;would be expressed as
;       (over l1 -> c1 over l2 -> c2 over l3 -> c3
;            list (+ c1 c2 c3))
;
;
;       (do ((l list (cdr l))
;            (k nil)
;            (i 1 (1+ i)))
;           ((null l) (nreverse k))
;           (setq k (cons (symbolconc (car l) i) k)))
;would be expressed as
;       (over list
;        for i <- 1
;        list (symbolconc _ i))
;
;
;       (memq x list)
;could be expressed as
;       (over l <- list -> item
;         (if (eq item x) (return l)))
```

```
(deffn over macro keep displace [! macrocall]
  (begin (e initpart bypart exitpart assignpart
          step return init body keyword select)
    (do ((overvar)
         (startexpr)
         (byexpr)
         (untilexpr)
         (sourceexpr)
         (assignableexpr))
        (nil)
      (if (not (match macrocall
                 ['over ! [overvar '<- ! e] '(<- _ e) ~ ! _ '(<- overvar nil)
                        startexpr
                        ! ['by byexpr ! e] '(<- _ e)
                          ~ ! _ '(<- byexpr (cond ((numberp startexpr)
                                                   '(1+ _))
                                                  (t '(cdr _))))
                        ! [_ (memq _ '(until untill)) untilexpr ! e]
                          '(<- _ e)
                          ~ ! _ '(<- untilexpr (cond ((numberp startexpr)
                                                      nil)
                                                     (t '(null _))))
                        ! [sourceexpr ! e '-> ! _] '(<- _ e)
                          ~ ! _ '(<- sourceexpr (cond ((numberp startexpr)
                                                       '_)
                                                      (t '(car _))))
                        ! ['-> assignableexpr ! e] '(<- _ e)
                          ~ ! _ '(<- assignableexpr (if (null overvar) '_))
                        ! macrocall]))
          (return nil))
      (if (null overvar)
          (setq overvar (symbolconc 'overvar assignableexpr)))
      (push overvar   initpart)
      (push '<-       initpart)
      (push startexpr initpart)
      (cond (untilexpr
             (push (u1oversubst overvar '_ untilexpr) exitpart)
             (when (and (not (atom overvar))
                        (not (equal (car exitpart) untilexpr)))
               (ferror "OVER: Non-atomic overvar in UNTIL expression:~
                        ~% data: ~S"
                       (car exitpart)))))
      (cond (byexpr (push overvar                          bypart)
                    (push (u1oversubst overvar '_ byexpr) bypart)))
      (do () (nil)
        (if (null assignableexpr) (return nil))
        (push assignableexpr initpart)
        (push assignableexpr assignpart)
        (push (u1oversubst overvar '_ sourceexpr) assignpart)
        (match macrocall
               [! [sourceexpr ! e '-> ! _] '(<- _ e)
                  ~ ! _ '(<- sourceexpr (cond ((numberp startexpr) '_)
                                              (t '(car _))))
                ! ['-> assignableexpr ! e] '(<- _ e)
                  ~ ! _ '(<- assignableexpr nil)
                ! macrocall])))
      (match macrocall
        [ ! ['select select ! e] '(<- _ e) ~ ! _ (<- select t)
         keyword (atom keyword)
                 (<- step (get keyword 'u1overstep))
                 '(<- return (get keyword 'u1overreturn))
                 '(<- init (get keyword 'u1overinit))
          ~ ! _
         ! ['return return ! e] '(<- _ e) ~ ! _
         ! ['step step ! e] '(<- _ e)
                            '(if (not return) (<- return 'over-acc)) ~ ! _
         ! ['init init ! e] '(<- _ e) ~ ! _
         ! body])
```

```
(setq body (cond ((cdr body) '(progn ,@body))
                 (t (car body))))
(if step (setq body (u1oversubst body 'over-body step)))
(cond ((u1overusedin 'over-acc [step return (cdr macrocall)])
       (setq body '(<- over-acc ,body))
       (push 'over-acc initpart)
       (cond (init (push '<-  initpart)
                   (push init initpart)))))
(if (not (eq select t)) (<- body '(if ,select ,body)))
(setq exitpart (cond ((cdr exitpart) '(or ,@(reverse exitpart)))
                     (t (car exitpart))))
(setq initpart (reverse initpart))
(if (u1overusedin 'first-start [step return (cdr macrocall) init])
    (setq initpart '(first-start       <- ,(third initpart)
                     ,(first initpart) <- first-start
                     ,@(cdddr initpart))))
'(KBE-obsolete-loop-macro
       ,initpart
       ,@(if exitpart   '((if ,exitpart (return ,return))))
       ,@(if assignpart '((<- ,@(reverse assignpart))))
       ,@(if body       '(,body))
       ,@(if bypart     '((<- ,@(reverse bypart)))))))
```

```
(defun u1oversubst (new old list)
  (cond ((eq list old) new)
        ((or (atom list)
             (memq (car list) '(quote over for)))              ;stop if hit quote or over
         list)
        (t (mapcar #'(lambda (x) (u1oversubst new old x))
                   list))))

(defun u1overusedin (item list)
  (cond ((eq list item) t)
        ((or (atom list)
             (memq (car list) '(quote over for)))
         nil)
        (t (do ((l list (cdr l)))
               ((atom l) nil)
               (cond ((u1overusedin item (car l)) (return t)))))))

;---------------------------------------------------------------

(defmacro def-over-step (name code)
  '(defprop ,name ,code u1overstep))

(defmacro def-over-return (name code)
  '(defprop ,name ,code u1overreturn))

(defmacro def-over-init (name code)
  '(defprop ,name ,code u1overinit))

;;; LIST
;;;  This conses up a list of the results of the steps in the order they
;;;  are produced. (this is like MAPCAR, or MAPLIST)

(def-over-step   LIST (cons over-body over-acc))
(def-over-return LIST (nreverse over-acc))

;;; NLIST
;;;  This produces a list of the results of the steps but reuses the list
;;;  cells in the first argument.  the number of results must be the same
;;;  as the length of the first argument for this to work right.

(def-over-step   NLIST (cdr (rplaca over-acc over-body)))
(def-over-return NLIST first-start)
(def-over-init   NLIST first-start)

;;; REVLIST
;;;  This CONS's up a list like "list" above but it is faster because
;;;  it does not reverse the result in order to get the right order.

(def-over-step   REVLIST (cons over-body over-acc))
(def-over-return REVLIST over-acc)

;;; BAG
;;;  This is the same thing with a better name for it. It is used when
;;;  the order of the object return doesn't matter.

(def-over-step   BAG     (cons over-body over-acc))
(def-over-return BAG     over-acc)

;;; NCONC
;;;  This NCONC's the results of the steps together like MAPCAN or MAPCON.

(def-over-step   NCONC   (nreconc over-body over-acc))
(def-over-return NCONC   (nreverse over-acc))

;;; APPEND
;;;  This is like the above but does an APPEND.

(def-over-step   APPEND  (nconc (reverse over-body) over-acc))
```

```
(def-over-return APPEND  (nreverse over-acc))

;;; AND
;;;  This AND's together the results of the steps.

(def-over-step   AND     (and over-body over-acc))
(def-over-return AND     over-acc)
(def-over-init   AND     t)

;;; FASTAND
;;;  This does an AND, but aborts on the first null step.

(def-over-step   FASTAND (or over-body (return nil)))
(def-over-return FASTAND over-acc)
(def-over-init   FASTAND t)

;;; OR
;;;  This OR's together the results of the steps.

(def-over-step   OR      (or over-body over-acc))
(def-over-return OR      over-acc)

;;; FASTOR
;;;  This does an OR, but aborts on the first non-null step.

(def-over-step   FASTOR  (let ((u1u1**temp over-body))
                           (if u1u1**temp (return u1u1**temp))))
(def-over-return FASTOR  nil)

;;; SUM
;;;  This adds together the results of the steps.

(def-over-step   SUM     (+ over-body over-acc))
(def-over-return SUM     over-acc)
(def-over-init   SUM     0)
```

```
;;; CLASSIFYQ
;;;  This is significantly more complex than some of the others. It sorts
;;;  the results of each step (which must be a cons of a key and a value
;;;  ASSQ style) into an assq structure which is returned.  it has one
;;;  bucket for each destinct key encountered. The items are in the buckets
;;;  in reverse temporal order

(def-over-step   CLASSIFYQ (let ((pair over-body))
                             (begin ([key ! obj] <- pair
                                     buck <- (assq key over-acc))
                                    (cond (buck
                                           (rplacd buck [obj ! (cdr buck)])
                                           over-acc)
                                          (t [[key obj] ! over-acc])))))

(def-over-return CLASSIFYQ over-acc)

;;; CLASSIFY
;;;  The same as CLASSIFYQ except that ASSOC is used, not ASSQ.

(def-over-step   CLASSIFY  (let ((pair over-body))
                             (begin ([key ! obj] <- pair
                                     buck <- (assoc key over-acc))
                                    (cond (buck
                                           (rplacd buck [obj ! (cdr buck)])
                                           over-acc)
                                          (t [[key obj] ! over-acc])))))

(def-over-return CLASSIFY  over-acc)

;;; MAKEQSET
;;;  This is used to construct QSET's of the results.

(def-over-step   MAKEQSET  (let ((item over-body))
                             (cond ((memq item over-acc) over-acc)
                                   (t (cons item over-acc)))))

(def-over-return MAKEQSET  over-acc)

;;; MAKESET
;;;  This is used to construct SET's of the results.

(def-over-step   MAKESET   (let ((item over-body))
                             (cond ((member item over-acc) over-acc)
                                   (t (cons item over-acc)))))

(def-over-return MAKESET   over-acc)


;;; MAKEFSET
;;;  This is used to construct FSET's of the results.

(def-over-step   MAKEFSET  (cons over-body over-acc))
(def-over-return MAKEFSET  (makefset over-acc))
```

```
;;; This provided an alternative looping construct. Its main virtues were:
;;;
;;; 1. It is a PROG not a do it has no syntactic place for a termination
;;;    expression you just do a return when you want to. RETURN works,
;;;    of course.
;;;
;;; 2. The inits are used in a LET* (they are helpful to set up the
;;;    right environment) note that things are not evaluated in parallel
;;;    in the inits.
;;;
;;; For compatability with the past, if there is a <- in the inits, then
;;; a BEGIN is used instead of a LET*
;;;
;;; Syntax:  (KBE-obsolete-loop-macro inits . body)

(defmacro KBE-obsolete-loop-macro (inits . body)
  (setq body '(do () (nil) ,@ body))
  (cond (inits '(,(cond ((memq '<- inits) 'begin) (t 'let*))
                 ,inits ,body))
        (t body)))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; Notes:
;;;
;;;  This file uses ordinary Lisp control primitives only.
;;;
;;;  It does not depend on (OVER ...), keyworded (IF ...), (LOOP ...),
;;;  (BEGIN ...), (REPEAT ...), (REPEAT* ...), (MATCH ...), or (<- ...).
;;;
```

```
;;;; Tracing DB transactions

;;; This section implements a tracing facility that lets a user get
;;; get notified when transactions occur with the database. The key
;;; idea is that the trace package is woken up every time a fact is
;;; processed by FIND-FACTS, ADD-FACT, ASSERT-FACT, or ERASE-FACTS. The
;;; action taken then depends on trace packets which can be set up by
;;; the user. Such packets are set up by calling DB-TRACE. Calling
;;; DB-UNTRACE removes all trace information.

(defvar *db-trace* nil "If non-nil, information about active db traces.")

;;; (WHEN-DB-TRACED . forms)
;;;
;;;  This macro should be wrapped around any code which references
;;;  DB-DISPLAY-TRACE-INFO so that such code will not end up getting
;;;  run in the case where *DB-TRACE* has no trace packets in it.

(defmacro when-db-traced (&body body) '(cond (*db-trace* ,@body)))

;;; (DB-TRACE pattern codes predicate action)
;;;
;;;  pattern     tracing occurs only if this pattern matches the fact
;;;              being processed. * means match every fact. Any other
;;;              form in this position is EVAL'd at DB-TRACE time and
;;;              the result is placed in the trace packet as a pattern.
;;;
;;;  code-list   if NIL, trace always. otherwise, a list of the times
;;;              that tracing should occur (one or more of ADD, FIND, ERASE)
;;;
;;;  predicate   an arbitrary piece of code that is executed iff the
;;;              given pattern and code-list match correctly. tracing
;;;              occurs only if this returns t.
;;;
;;;  action      an arbitrary piece of code that is executed when tracing
;;;              has occurred. at the point of tracing
;;;
;;;  The fact being processed will be bound to the free variable
;;;  *DB-FACT* during the predicate and action in case user code needs
;;;  to access it.

(defmacro db-trace (pattern codes predicate &optional action)
  (if (eq pattern '*) (setq pattern ''',pattern))
  '(progn (push '(,,pattern ,',codes ,',predicate ,',action) *db-trace*)
          t))

(defun db-untrace () (setq *db-trace* nil))

;;; (DB-DISPLAY-TRACE-INFO code fact)
;;;
;;;  CODE is not evaluated. It is a symbol which names the kind of trace.
;;;
;;;  FACT is evaluated. This is the fact which may need trace info
;;;       displayed
;;;
;;;  This is a macro and not a function since tracing is likely to be
;;;  slow and we need all the help we can get...

(defmacro db-display-trace-info (code fact)
  '(let ((*db-fact* ,fact))
     (declare (special *db-fact*))
     (dolist (packet *db-trace*)
       (let (((pattern codes predicate action) packet))
         (cond ((and (or (null codes) (memq ',code codes))
                     (or (eq pattern '*)
                         (matches-fact? pattern *db-fact*))
                     (or (eq predicate T) (eval predicate)))
                (format t "~%;Tracing - ~S - ~:=" ',code *db-fact*)
```

```
(if action (eval action))
(return)))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; (SEL01 facts)  n = 0,1
;;; (SEL1  facts)  n = 1
;;; (SEL1+ facts)  n > 0
;;;
;;; These functions are intended to provide convenience and error
;;; checking in database accesses. They accomplish the following:
;;;
;;;  (1) Checking that the proper number of facts were found.
;;;  (2) De-listing; Given a list of facts, they return the first in
;;;      the list, or NIL if no facts were given and that is not an error.
;;;
;;; The conditions for the number of facts is shown above beside the
;;; syntax information.
;;;
;;; A variety of macros are available for combining the use of these
;;; functions with FIND-FACTS and erase-facts.
;;;
;;; Note: The special variable *DB-LAST-PATTERN-FOUND*
;;;       is used to increase the usefulness of the error messages
;;;       produced. It should be noted that the error messages will be
;;;       very confusing if you use these functions anywhere but wrapped
;;;       directly around calls on FIND-FACTS or ERASE-FACTS, since the
;;;       error messages they type may otherwise reference stale data.

(defun sel1 (facts)
  (cond ((null facts)
         (barf "No facts found when 1 expected.~
              ~% Pattern: ~:="
               *db-last-pattern-found*))
        ((cdr facts)
         (cbarf "Let SEL1 return ~2*~:="
                "More than 1 fact found when 1 expected.
               ~% Pattern: ~:=~
               ~% Facts found: ~:="
                *db-last-pattern-found* facts (car facts))
         (car facts))
        (t
         (car facts))))

(defun sel01 (facts)
  (cond ((not (cdr facts))
         (car facts))
        (t
         (cbarf "Let SEL01 return ~2*~:="
                "More than one fact found when 0 or 1 expected.~
               ~% Pattern: ~:=~
               ~% Facts found: ~:="
                *db-last-pattern-found* facts (car facts))
         (car facts))))

(defun sel1+ (facts)
  (cond ((not (null facts))
         (car facts))
        (t
         (barf "No facts found when one or more expected.~
              ~% Pattern: ~:="
               *db-last-pattern-found*))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; DMAC: Macros supporting DB

;;; Notes:
;;;
;;;  This file uses ordinary Lisp control primitives only.
;;;
;;;  It does not depend on (OVER ...), keyworded (IF ...), (LOOP ...),
;;;  (BEGIN ...), (REPEAT ...), (REPEAT* ...), (MATCH ...), or (<- ...).

;;;; DB memory management

;;; Notes:
;;;
;;;  This file uses ordinary Lisp control primitives only.
;;;
;;;  It does not depend on (OVER ...), keyworded (IF ...), (LOOP ...),
;;;  (BEGIN ...), (REPEAT ...), (REPEAT* ...), (MATCH ...), or (<- ...).
;;;
;;; Overview:
;;;
;;;  Because so much temporary consing is done in the current system,
;;;  we would, without this system, end up scattering our permanent
;;;  data onto pages far from each other and end up paging ourselves to
;;;  death. What this code does is to help keep live data clustered
;;;  together in a place called *DB-AREA*. The database is frequently
;;;  compacted into a new area, and *DB-AREA* is updated to point at this
;;;  new area. To make the compaction fast, we assume we know everything
;;;  in the world that might conceivably be pointing at our database.
;;;  If new variables, plist slots, etc. are added to the system which
;;;  point at objects consed in *DB-AREA*, the function DB-GC must be
;;;  updated appropriately to know about those slots or total randomness
;;;  will start to occur later on when pointers into an old db area start
;;;  to get referenced.

(defvar *db-area*              default-cons-area "cons-area used by db")
(defvar *db-size*              nil               "last known size of db")
(defvar *db-area-count*        0                 "# of db areas created")
(defvar *db-areas*             nil               "list of all db areas")
(defvar *db-free-areas*        nil               "list of free db areas")
(defvar *db-vars*              nil               "who points at db area")
```

```
;;; The macros DB-CONS, DB-NCONS, DB-LIST*, etc. work just like the
;;; functions CONS, NCONS, LIST*, and so on, but they cons their
;;; structure in *DB-AREA* rather than the system default cons area.

(defmacro db-cons  (x y) '(cons-in-area  ,x ,y *db-area*))
(defmacro db-xcons (x y) '(xcons-in-area ,x ,y *db-area*))
(defmacro db-ncons (x)   '(ncons-in-area ,x    *db-area*))

(defmacro db-list* (&rest args) '(list*-in-area *db-area* ,@args))
(defmacro db-list  (&rest args) '(list-in-area  *db-area* ,@args))

(defmacro db-make-list (n) '(make-list ,n ':area *db-area*))

(defmacro db-copylist (l) '(copylist ,l *db-area*))

;;; The macro DB-PUSH is just like PUSH, but the cons it uses comes
;;; from *DB-AREA*.
;;;
;;; Note: This uses the cheapy expansion for push which allows the
;;;       potential screw of double evaluation of some of its subforms.
;;;       So be careful how you use it.

(defmacro db-push (x y) '(setf ,y (db-cons ,x ,y)))
```

```
;;;; Declarations

;;; (DEF-DB-VAR sym [ value [ documentation ]])
;;;
;;;  Works just like DEFVAR, but also declares that this symbol will
;;;  point to something in *DB-AREA*, so that the DB-GC routine will
;;;  know to move it to the new area when it makes a new *DB-AREA*.

(defmacro def-db-var (sym &rest maybe-value-and-doc)
  '(progn 'compile
          (if (not (memq ',sym *db-vars*)) (push ',sym *db-vars*))
          (defvar ,sym ,@maybe-value-and-doc)))

;;; (DEF-DB-CONST sym [ value [ documentation ]])
;;;
;;;  Works just like DEFCONST, but also declares that this symbol will
;;;  point to something in *DB-AREA*, so that the DB-GC routine will
;;;  know to move it to the new area when it makes a new *DB-AREA*.

(defmacro def-db-const (sym &rest maybe-value-and-doc)
  '(progn 'compile
          (if (not (memq ',sym *db-vars*)) (push ',sym *db-vars*))
          (defconst ,sym ,@maybe-value-and-doc)))

;;; (DEF-DB-STRUCT name . slots)
;;;
;;;  A super-simple defstruct. Allocates its storage in the DB area.
;;;  Defines functions MAKE-name, name?, and accessors with the names
;;;  of each slot. Note also that the constructor function does not
;;;  take keyword pairs. It takes exactly one required arg per slot.

(defmacro def-db-struct (name &rest slots)
  (let ((pred (symbolconc name "?"))
        (make (symbolconc "MAKE-" name))
        (tag  (symbolconc "^^" name)))
    '(progn 'compile
            (defmacro ,make ,slots
              '(db-list ',',tag ,,@slots))
            (defun ,pred (obj)
              (and ;this test may lose if asynchronous GC occurs midway
                (= (%area-number obj) *db-area*)
                (eq (car obj) ',tag)))
            ,@(do ((i 1 (1+ i))
                   (s slots (cdr s))
                   (d nil))
                  ((null s) d)
                (let ((slot (car s)))
                  (push '(defmacro ,slot (,name)
                           '(nth ,,i ,,name))
                        d))))))
```

```
;;; This could relatively easily be changed to be some other form
;;; of data storage besides properties. The only functions currently
;;; relying on things to still be properties are DB-GC and DB-INIT
;;; which for efficiency use plist mappings... All other functions should
;;; be referencing the appropriate abstractions.

;; NOTE IT IS ESSENTIAL THAT A DB-PROPERTY ONLY BE PUT ON AN ATOM IN
;; THE LIST *DB-SEGS* AND THAT IF YOU TAKE IT OFF THAT LIST YOU CALL
;; DB-INIT-SYM FIRST.

(defvar *db-properties* '()
  "List of symbol properties dynamically hacked by DB")

(defmacro db-property? (tag) '(memq ,tag *db-properties*))

(defun db-init-sym (sym)
  (dolist (prop *db-properties*) (remprop sym prop)))

;;; (DEF-DB-PROPERTY prop-name)
;;;
;;;  Defines (prop-name sym) as an abstract accessor to a property about
;;;  some symbol. (setf (prop-name sym) ...value...) assigns that property.
;;;  It is only coincidence that these are stored on the plist of sym; it
;;;  should not be depended upon.

(defmacro def-db-property (name &optional (comment ""))
  '(progn 'compile
          (push ',name *db-properties*)
          (defmacro ,name (x) '(get ,x ',',name))
          (defprop ,name ,comment db-property)))

;these two properties are essential for speed, there shouldn't
;be any others.
(def-db-property DB-INDEX)
(def-db-property DB-NAMES)
```

```
;;;; LAYER

;;; A layer is a structure (currently a simple list) of contexts in which
;;; a fact is known to be in.

;;; Note: This abstraction doesn't really exist uniformly everywhere,
;;;       but we're creating it gradually so we won't have random calls
;;;       to LIST, CAR, CDR, etc. floating around like we have now.

(defmacro build-layers-list (&rest args) '(db-list ,@args))

;;; (ADD-LAYER layer list)
;;;
;;;  Adds a layer to a list of layers. If all you do is assert the largest
;;;  layer, the layer lists will remain sorted.

(defun add-layer (layer list)
  (cond ((not (memq layer list)) (db-cons layer list))
        (t list)))

;;; (DEL-LAYER layer list)
;;;
;;;  Deletes a layer from a list of layers.

;;;  Implementation notes:
;;;
;;;    Tries to be very cautious about consing. Conses only when it
;;;    has to, building cdr-coded lists in the database's special area.
;;;
;;;    Assumes that the argument will be in the right area already, so
;;;    just returns it if the layer does not occur in the list.

(defun del-layer (layer list)
  (let ((size 0))
    (dolist (element list) (if (not (= element layer)) (incf size)))
    (let ((result (db-make-list size)))
      (do ((l result (cdr l)))
          ((null l) result)
        (do () ((not (= (car list) layer))) (pop list))
        (setf (car l) (car list))
        (pop list)))))
```

```
;;;; CMARKER

;;; A cmarker is a context marker for a fact.

;;; A cmarker has the following fields:
;;;
;;;   CMARKER-IS-IN?  true if the cmarker is IN in the current context.
;;;   IN-LAYERS       a list of contexts in which the fact is IN.
;;;   OUT-LAYERS      a list of contexts in which the fact is OUT.
;;;   NEW-CMARKER     the cmarker for the surrounding context.

(defmacro build-cmarker (? i o n)  ;note this defines evaluation order
  '(db-list '*cmarker* ,? ,i ,o ,n))

(defmacro cmarker-is-in? (f) '(cadr        ,f))
(defmacro in-layers      (f) '(caddr       ,f))
(defmacro out-layers     (f) '(cadddr      ,f))
(defmacro new-cmarker    (f) '(car (cddddr ,f)))

(pp:define-list-print-function *cmarker* (cmarker s)
  (format s "cmarker{~@!~@{~=~^ ~:_~}~.}"
          (cmarker-is-in? cmarker)
          (in-layers cmarker)
          (out-layers cmarker)
          (if (new-cmarker cmarker) '-new-)))
```

```
;;;; FACT

;;; A fact is an object which expresses some proposition about the world
;;; that is true in some (possibly null) context(s), given by its
;;; cmarkers.

;;; Note the way it goes through a fair bit of work in order to
;;; get the whole fact into one little bit of cdr coding
;;; rather than having the fields part be in a separate bit of memory.

;;; FACT

(defmacro build-fact (cmarker . fields)
  '(db-list ,cmarker ,@fields))

(defmacro cmarker (f) '(car (cr      ,f)))
(defmacro field0  (f) '(car (cdr     ,f)))
(defmacro fieldk  (f) '(car (cddr    ,f)))
(defmacro field1  (f) '(car (cdddr   ,f)))
(defmacro field2  (f) '(car (cddddr  ,f)))
(defmacro field3  (f) '(car (cdddddr ,f)))

(defmacro cmarker? (obj)
  (once-only (obj)
    '(and (listp ,obj)
          (eq (car ,obj) '*cmarker*))))

;;; (FACT? x)
;;;
;;;  FACT? is a predicate which is responsible for assuring:
;;;
;;;    * Its success must ensure that it is ok to use all of the
;;;      field selectors
;;;
;;;    * It must be able to discriminate between a fact and a segcase.
;;;
;;;  Nothing else is expected of it.
;;;
;;;  Note: Currently, we recognize facts by assuming they are lists
;;;        whose car is a cmarker. The only object the system
;;;        intentionally creates which would answer T to this and not
;;;        be a fact is *CMARKERS*, and it should never be geting
;;;        involved in fact tests anyway.

(defmacro fact? (thing)
  (once-only (thing)
    '(and (listp ,thing) (cmarker? (car ,thing)))))

(pp:define-generalized-print-function fact (fact s)
    (fact? fact)
  (local-declare ((*expr audit-fact compress-fact))
    (cond ((fact? fact)
           (cond ((audit-fact fact t)
                  (format s "{~@!~{~=~^ ~:_~}~.}" (compress-fact fact)))
                 (T (format s "fact{~@!~{~=~^ ~:_~}~.}" fact)))))))

;;; (FACT-IS-IN? fact)
;;;
;;;  Tests to see whether a fact (as opposed to a cmarker)
;;;  is IN. Note: It must test if there is a cmarker first!

(defmacro fact-is-in? (fact)
  '(let ((cmarker (cmarker ,fact)))
     (and cmarker (cmarker-is-in? cmarker))))

;;; (OVER-IN-FACTS (var list) . body)
;;;
;;;  Iterates across list, executes body with var bound to each fact
```

```
;;;  which FACT-IS-IN? returns true for.

(defmacro over-in-facts ((var list) &body body)
  `(dolist (,var ,list)
     (cond ((fact-is-in? ,var) ,@body))))
```

```
;;;; Fact Keywords

;;; Overview:
;;;
;;;  This section will be filled with a lot of abstraction macros.
;;;  At some point, when everything is properly abstracted to call these
;;;  macros, they may do something radically different.

;;; (KEYWORD-NAME keyword)
;;;  Returns the symbol which is the name of a given keyword.
;;;
;;;  Note: Since keywords are currently symbols, this is a no-op.
;;;        If keywords are changed to be represented another way
;;;        (eg, fixnums), this will no longer be a no-op.

(defsubst keyword-name (keyword) keyword)

;;; (DEF-FACT-PREDICATE name keyword-name-list)
;;;
;;;  This defines a predicate on facts which returns T if the symbolic
;;;  name of the fact is in the given keyword-name-list
;;;
;;;  Note: Right now we just do a MEMQ of the fact keyword in this
;;;        list, but someday we'll do something smart like a bit-test
;;;        to improve our speed by a ton.

(defmacro def-fact-predicate (name list)
  (cond ((cdr list)
         '(defmacro ,name (fact)
           '(memq (~keyword ,fact) ',',list)))
        (t
         '(defmacro ,name (fact)
           '(eq (~keyword ,fact) ',',(car list))))))

(def-fact-predicate        input-fact? (input                                ))
(def-fact-predicate       output-fact? (      output                         ))
(def-fact-predicate       tinput-fact? (             tinput                  ))
(def-fact-predicate      toutput-fact? (                    toutput         ))

(def-fact-predicate  input-port-fact? (input        tinput                  ))
(def-fact-predicate output-port-fact? (      output        toutput         ))

(def-fact-predicate        port-fact? (input output tinput toutput         ))

(def-fact-predicate          io-fact? (input output                         ))
(def-fact-predicate         tio-fact? (             tinput toutput         ))

(def-fact-predicate       cflow-fact? (cflow                                ))
(def-fact-predicate       dflow-fact? (      dflow                          ))
(def-fact-predicate        flow-fact? (cflow dflow                          ))

(def-fact-predicate flow-or-port-fact? ( input  output cflow dflow
                                         tinput toutput                    ))

(def-fact-predicate        @seg-fact? (             @seg                    ))
(def-fact-predicate         @io-fact? (                  @io                ))
(def-fact-predicate  @createvar-fact? (                       @createvar   ))

(def-fact-predicate          @-fact? (             @seg @io @createvar      ))
(def-fact-predicate     io-or-@-fact? (input output @seg @io @createvar      ))

(def-fact-predicate        plan-fact? (plantype recursive role subseg
                                       cflow    dflow                      ))

(def-fact-predicate        slot-fact? (slot                                 ))

(def-fact-predicate    wildcard-fact? (~*                                   ))
```

```
;;; (FACT-KEY-DISPATCH fact . clauses)
;;;
;;;  key is an evaluable form which must yield a fact. The clauses
;;;  are in the same form as a CASEQ, where the things being cased
;;;  for are the symbolic names of fact keywords.

(defmacro fact-key-dispatch (val &body clauses)
  '(caseq (~keyword ,val) ,@clauses))

;;; (SAME-FACT-KEY fact1 fact2)
;;;
;;;  Returns true iff fact1 and fact2 have identical keyword fields.

(defmacro same-fact-key? (fact1 fact2)
  '(eq (~keyword ,fact1) (~keyword ,fact2)))
```

```
;;; Standard constructors/selectors for facts.
;;;
;;;  For each field "name" there is a selector "~name" that picks that
;;;  field out, and a modifier &~name that changes that field
;;;  (non-destructively).

;;; (FACTDEF . specifiers)
;;;
;;;  This macro is used for building selector functions for the
;;;  assertions. It receives the list of field specifiers in order.
;;;  Each specifier is a list (SELECTOR-NAME TYPE IDX . REST)
;;;     The type must be one of:
;;;
;;;    S   meaning that it is a segcase
;;;    P   meaning that it is a port assertion
;;;          (an INPUT, OUTPUT, TINPUT, TOUTPUT),
;;;    SP  meaning that it is either a port or a segcase
;;;    C   meaning that it is a list of pieces of code
;;;    K   meaning that it is an atomic keyword which takes on
;;;          one of a list of values (supplied as rest)
;;;    NP  meanig that it is the name of a port
;;;    L   meaning the thing is some lisp sexpr
;;;
;;;  The first field must be either S, P, or SP.
;;;  The second field is the keyword, and must be a K with one value.
;;;
;;;  if the field is S P or SP then it can be indexed on.  If you want
;;;  it to be then you supply the index as a number in IDX.  Note it
;;;  ABSOLUTLEY MUST be tha case that every fact is indexed exactly
;;;  once in buckets 0-10 and that it NEVER be indexed twice in the
;;;  same bucket!
;;;
;;;  The above information drives all of the major functions in the DB.
;;;
;;;  Note that now fact can have more than 3 fields (after the keyword)
;;;  and if two facts use the same field name, the fields must be in the
;;;  same positions in the facts.

(eval-when (eval compile load) ;must have these when compiling too.

(defconst *db-keywords* nil
    "List of possible fillers for the ~keyword slot of a fact")

(defconst *db-fields* nil
    "List of all of the possible field selector names in facts"))
```

```
(defmacro factdef (&body specs)
  (let* ((default-cons-area working-storage-area)
         (selectors (mapcar #'car specs))
         (types     (mapcar #'cadr specs))
         (idxs      (mapcar #'caddr specs))
         (rests     (mapcar #'cdddr specs))
         (keyword   (caadr rests)))
    (cond ((memq keyword *db-keywords*)
           (grumble "Redefining fact keyword: ~A" keyword)))
    '(eval-when (eval compile load)
       (push ',keyword *db-keywords*)
       (defprop ,keyword ,types DB-KEYWORD-TYPES)
       (defprop ,keyword ,idxs  DB-KEYWORD-IDXS)
       (defprop ,keyword ,rests DB-KEYWORD-RESTS)
       ,@(mapcan
           #'(lambda (fieldsym get-n put-n)
               (let* ((old-def (cdr (assq fieldsym *db-fields*))))
                 (cond ((or (null old-def) (eq get-n old-def))
                        ;always redefining is robust when patching!
                        (let* ((field  (get-pname fieldsym))
                               (get-fn (intern (string-append "~"  field)
                                               (symbol-package fieldsym)))
                               (put-fn (intern (string-append "&~" field)
                                               (symbol-package fieldsym))))
                          '((push ',(cons fieldsym get-n) *db-fields*)
                            (defmacro ,get-fn (s)   '(,',get-n ,s))
                            (defmacro ,put-fn (s x) '(,',put-n ,s ,x)))))
                       (T
                        (barf "Redefining field in different location~
                               ~%~A" fieldsym)))))
           selectors
           '( field0  fieldk  field1  field2  field3)
           '(&field0 &fieldk &field1 &field2 &field3)))))

;;; These define additional macros that
;;; non-destructively modify parts of a fact.

(defmacro &field0 (s x)
  '(let ((h (db-copylist ,s)))
     (rplaca (cdr h) ,x)
     (rplaca h *null-cmarker*)))

(defmacro &fieldk (s x)
  '(let ((h (db-copylist ,s)))
     (rplaca (cddr h) ,x)
     (rplaca h *null-cmarker*)))

(defmacro &field1 (s x)
  '(let ((h (db-copylist ,s)))
     (rplaca (cdddr h) ,x)
     (rplaca h *null-cmarker*)))

(defmacro &field2 (s x)
  '(let ((h (db-copylist ,s)))
     (rplaca (cddddr h) ,x)
     (rplaca h *null-cmarker*)))

(defmacro &field3 (s x)
  '(let ((h (db-copylist ,s)))
     (rplaca (cdr (cddddr h)) ,x)
     (rplaca h *null-cmarker*)))
```

```
;;; These define the actual facts now in use.

(factdef (segcase S 8.) (keyword K nil INPUT)
  (kind L nil NIL ID) (obj NP nil) (id NP nil))

(factdef (segcase S 9.) (keyword K nil OUTPUT)
  (kind L nil NIL ID PART) (obj NP nil) (id NP nil))

(factdef (segcase S 8.) (keyword K nil TINPUT)
  (lower-env S 14.) (obj NP nil) (lower-io P 15.))

(factdef (segcase S 9.) (keyword K nil TOUTPUT)
  (lower-env S 15.) (obj NP nil) (lower-io P 14.))

(factdef (segcase S 10.) (keyword K nil EXPECT) (wff L nil))

(factdef (segcase S 10.) (keyword K nil ASSERT) (wff L nil))

(factdef (segcase S 10.) (keyword K nil WHEN) (wff L nil))

(factdef (segcase S 2.) (keyword K nil SEGTYPE)
  (type K nil SPLIT JOIN SIMPLE))

(factdef (segcase S 0.) (keyword K nil SUBSEG) (subseg S 11.))

(factdef (segcase S 6.) (keyword K nil DFLOW) (from P 12.) (to P 13.))

(factdef (segcase S 6.) (keyword K nil CFLOW) (from S 14.) (to S 15.))

(factdef (segcase S 1.) (keyword K nil PLANTYPE)
  (type K nil NEW-VIEW EXPR AND COMP XOR PRED JOIN SSR TEMPCOMP
             SSR-BODY CONSTANT BUILTIN))

(factdef (segcase S 7.) (keyword K nil ROLE) (roleseg S 11.)
  (rolename K nil ACTION INIT PRED JOIN OP BODY REC DFJOIN
                  BSSR GEN TRN ACC TERM FILTER))

(factdef (segcase S 7.) (keyword K nil SLOT) (slotseg S 11.)
        (slotname L nil) (slottype L nil INPUT OUTPUT SEGMENT))

(factdef (segcase S 10.) (keyword K nil CLICHE) (clichename L nil))

(factdef (segcase S 10.) (keyword K nil CLICHE-DEF)
        (cliche-def-name L nil) (cliche-def-comments L nil))

(factdef (segcase S 7.) (keyword K nil RECURSIVE) (subseg S 11.))

(factdef (segcase S 3.) (keyword K nil @SEG)
  (kind K nil JOIN FUNCALL CONSTANT LAMBDA-EXP OPENCODE RECURSION)
  (name L nil) (code C nil))

(factdef (deep P 4.) (keyword K nil @IO)
  (kind K nil LAMBDA-ARG ACTUAL-ARG FREE-VAR RETURNVAL SIDE-EFFECT DUMMY)
  (name L nil) (argno L nil))

(factdef (deep P 5.) (keyword K nil @CREATEVAR) (varname L nil))
```

```
;;; These macros decompose and compose segcases.

(defmacro ~seg  (s) '(car ,s))
(defmacro ~case (s) '(cdr ,s))

(defmacro &~seg  (s x) '(db-xcons (cdr ,s) ,x))
(defmacro &~case (s x) '(db-cons (car ,s) ,x))

;;; The following are compound names for directly accessing the seg out
;;; of a field which is a segcase.

(defmacro  ~seg~segcase (s) '(~seg (~segcase ,s)))
(defmacro  ~seg~deep    (s) '(~seg (~deep    ,s)))
(defmacro  ~seg~subseg  (s) '(~seg (~subseg  ,s)))
(defmacro  ~seg~roleseg (s) '(~seg (~roleseg ,s)))

(defmacro &~seg~segcase (s x)
  (once-only (s) '(&~segcase ,s (&~seg (~segcase ,s) ,x))))

(defmacro &~seg~deep (s x)
  (once-only (s) '(&~deep    ,s (&~seg (~deep    ,s) ,x))))

(defmacro &~seg~subseg (s x)
  (once-only (s) '(&~subseg  ,s (&~seg (~subseg  ,s) ,x))))

(defmacro &~seg~roleseg (s x)
  (once-only (s) '(&~roleseg ,s (&~seg (~roleseg ,s) ,x))))
```

```
;;; (~SET ~<field> ~<field> ... <list-of-facts>)
;;;  Extracts from a group of assertions. It is used in conjunction
;;;    with other macros.
;;;  Returns list of elements extracted from statements in indicated field.
;;;
;;; eg, (~SET ~x ~y facts) => (MAPCAR #'(LAMBDA (S) (~x (~y S))) facts)

(defmacro ~set (&rest args)
  (let* ((reversed-args              (reverse args))
         ((facts . reversed-fields) reversed-args)
         (selector                  (do ((fl reversed-fields (cdr fl))
                                         (s 's (list (car fl) s)))
                                        ((null fl) s))))
    '(mapcar #'(lambda (s) ,selector) ,facts)))

;;; (~NSET ~<field> ~<field> ... <list-of-facts>)
;;;
;;;  Same as ~SET but destructively and faster. Bashes the results of
;;;  the accesses back into the list structure which held the input
;;;  list of facts, returning that list.
;;;
;;; eg, (~NSET ~x ~y facts) => (LET ((TEMP facts))
;;;                              (MAP #'(LAMBDA (S) (SETF (CAR S)
;;;                                                       (~X (~Y (CAR S)))))
;;;                                   TEMP)
;;;                              TEMP)

(defmacro ~nset (&rest args)
  (let* ((reversed-args              (reverse args))
         ((facts . reversed-fields) reversed-args)
         (selector                  (do ((fl reversed-fields (cdr fl))
                                         (s '(car s) (list (car fl) s)))
                                        ((null fl) s))))
    (cond ((symbolp facts)
           '(progn (map #'(lambda (s) (setf (car s) ,selector)) ,facts)
                   ,facts))
          (t '(let ((temp ,facts))
                (map #'(lambda (s) (setf (car s) ,selector)) temp)
                temp)))))

;;; (~1ST ~<field> ~<field> ... <list-of-facts>)
;;;
;;;  Selects a field from the first of a list of facts.
;;;  Returns NIL if there were no facts in the list.
;;;
;;;  Functionally equivalent to (CAR (~SET ...)), but more efficient.

(defmacro ~1st (&rest args)
  (let* ((reversed-args              (reverse args))
         ((facts . reversed-fields) reversed-args)
         (selector                  (do ((fl reversed-fields (cdr fl))
                                         (s 's (list (car fl) s)))
                                        ((null fl) s))))
    (cond (reversed-fields
           '(let ((s (car ,facts)))
              (cond (s ,selector) (t nil))))
          (t
           '(car ,facts)))))

;;; (~01 ~<field> ~<field> ... <list-of-facts>)
;;;
;;;  Same as ~1ST but gives an error message if there is more than one
;;;  item in the given list of facts.

(defmacro ~01 (&rest args)
  (let* ((reversed-args              (reverse args))
         ((facts . reversed-fields) reversed-args)
         (selector                  (do ((fl reversed-fields (cdr fl))
```

```
                                        (s 's (list (car fl) s)))
                                       ((null fl) s))))
    (cond (reversed-fields
           '(let ((s (sel01 ,facts)))
              (cond (s ,selector) (t nil))))
          (t
           '(sel01 ,facts)))))
```

```
;;; (& ...)
;;;
;;;  Basic segcase/fact/pattern constructor
;;;
;;;  (& seg case)                          Creates a seg with a given case.
;;;  (& seg)                               Creates a seg with a null case;
;;;                                         same as (& seg NIL)
;;;
;;;
;;;  (& field0 keyword field1 field2 ...) Creates a fact with given info.
;;;
;;;   Note that it uses the number of arguments it is passed in order
;;;   to decide whether it is creating a segcase or a fact.
;;;
;;;   Also note that if keyword is one of the standard fact keywords
;;;   (as defined by FACTDEF), it will be implicitly quoted. Otherwise,
;;;   it will be evalauted normally. This of course implies that variables
;;;   with the same name as one of these keywords will not work in that
;;;   position.

(defmacro & (&rest body)
  (if (memq '* body)
      (ferror "Cannot use * in an (& ...) expression."))
  (let* (((a . as) body)
         ((b . bs) as))
    (cond ((null as) '(db-ncons ,a))  ;Redundant clause - just for clarity
          ((null bs) '(db-cons ,a ,b))
          (t
           (cond ((atom b)
                  (let ((len (length (get b 'db-keyword-types))))
                    (cond (len
                           (if (not (= len (+ 2 (length bs))))
                               (ferror "Wrong number of fields.~% ~S"
                                 '(& ,@body)))
                           (setq b ',b))))))
           '(build-fact *null-cmarker* ,a ,b ,@bs)))))
```

```
;;; (&* ...)
;;;
;;;  Pattern builder.
;;;
;;;  As a convenience, the macro looks for occurences of * and converts
;;;  them to '~*, so that you don't have to. This means that you cannot
;;;  ever use a variable named * in a pattern.
;;;
;;;  There is nothing nothing that you can do with &* that you cannot do
;;;  with a vanilla &, but often &* will make things much more compact.
;;;
;;;  It is very similar to & except that it requires a literal keyword
;;;  which has been defined by FACTDEF and if given less than the correct
;;;  number of fields for that keyword, it will supply '~*'s to fill
;;;  omitted fields.
;;;
;;;  As a result of its argument defaulting properties, the two argument
;;;  case is assumed to be a fact, not a case (as it would for  &). This
;;;  is because, for example,
;;;      (&* FOO INPUT) = (& FOO INPUT '~* '~* '~*)
;;;  so the & macro sees more than 2 args.
;;;
;;;  By special case, (&* seg) = (&* seg '~*)
;;;
;;;  &* also recognizes the special pseudo-keyword PORT.  It replaces
;;;  this with '~*, and treats it as having the number of fields
;;;  required of an INPUT/OUTPUT/TINPUT/TOUTPUT fact.  eg,
;;;      (&* foo port * 'obj) = (& foo '~* '~* 'obj '~*).

(defmacro &* (&rest body)
  (map #'(lambda (l) (if (eq (car l) '*) (setf (car l) ''~*)))
       body)
  (cond ((null (cdr body)) '(& ,(car body) '~*))
        (t
         (let ((keyword    (cadr body))
               (slot1      (car  body))
               (slot2      (cadr body))
               (more-slots (cddr body)))
           (cond ((eq keyword 'PORT)
                  (setq slot2 '~*)
                  (setq keyword 'INPUT)))
           (let ((len (length (get keyword 'db-keyword-types))))
             (if (< len (length body))
                 (ferror "Too many args to &*~% Source: ~S" '(&* . ,body)))
             '(& ,slot1 ',slot2 ,@more-slots
                 ,@(nthcdr (- 3 (- len (length body)))
                           '('~* '~* '~*))))))))

;;; These macros combine the use of & with calls on the basic db fns.

(defmacro  +& (&rest body) '(add-fact    (&  . ,body)))
(defmacro  >& (&rest body) '(assert-fact (&  . ,body)))
(defmacro ??& (&rest body) '(find-facts  (&* . ,body)))
(defmacro --& (&rest body) '(erase-facts (&* . ,body)))

;;; These add in calls on the checkers

(defmacro  ?& (&rest body) '(sel1  (find-facts  (&* . ,body))))
(defmacro a?& (&rest body) '(sel1+ (find-facts  (&* . ,body))))

(defmacro  -& (&rest body) '(sel1  (erase-facts (&* . ,body))))
(defmacro a-& (&rest body) '(sel1+ (erase-facts (&* . ,body))))
```

```
;;;; Named DB contexts

;;; This is allows some convenient notation for referring to contexts.

(defmacro db-context-is (context)
  '(local-declare ((special ,context *db-context* *db-context-names*))
     (cond ((not (memq ',context *db-context-names*))
            (push ',context *db-context-names*)))
     (setq ,context *db-context*)))

(defmacro db-context-was (context)
  '(local-declare ((special ,context *db-context* *db-context-names*))
     (cond ((not (memq ',context *db-context-names*))
            (push ',context *db-context-names*)))
     (setq ,context (cdr *db-context*))))

;;; The context named by old-name is given an additional name, new-name.

(defmacro remember-db-context (old-name new-name)
  '(local-declare ((special ,new-name *db-context-names*))
     (cond ((not (memq ',new-name *db-context-names*))
            (push ',new-name *db-context-names*)))
     (setq ,new-name ,old-name)))
```

```
;;;; DBHUNKs: Database hunk structures

;;; This page of code defines the DBHUNK data structure. In Maclisp,
;;; these were hunks (hence the name). Ideally, they could be simple
;;; vectors, but we'll use cdr-coded lists here instead.

;;; Note: Because we use %MAKE-POINTER-OFFSET below in the definition
;;;       of the DBHUNK accessors, the cdr-coding of these lists must
;;;       be maintained or total randomness will start to occur. Any
;;;       data copying algorithms (eg, copying garbage collector) must
;;;       preserve cdr-coding in these structures.

;;; (MAKE-DBHUNK)
;;;
;;;  Creates an empty dbhunk.
;;;
;;;  Note: It is essential that the object be cdr-coded. We'll rely on
;;;        that in the accessors.

(defmacro make-dbhunk ()
  '(rplaca (db-make-list 17.) '*DBHUNK*))

;;; (DBX n dbhunk)
;;;
;;;  Accesses the nth slot of a dbhunk.
;;;
;;;  Implementation note: Exploits cdr-coding.
;;;
;;;  Warning: For historical reasons, this takes its arguments in the
;;;           reverse of the order it will evaluate them in. Code written
;;;           using it should not rely on side-effects in the arg
;;;           positions occurring in any particular order.

(defmacro dbx (num hunk)
  '(car (%make-pointer-offset dtp-list ,hunk (1+ ,num))))

;;; pretty printer format for dbhunks

(defsubst dbhunk-name (hunk)
  (local-declare ((special *db-segs*)) ;not defvar'd yet in this file
    (dolist (seg *db-segs*)
      (if (eq (db-index seg) hunk) (return seg)))))

(pp:define-list-print-function *dbhunk* (hunk s)
  (let ((seg (dbhunk-name hunk)))
    (cond (seg (format s "dbH-~A" seg))
          (T   (format s "~\block\" hunk)))))
```

```
;;;; Locking the DB
;;;
;;; Any "toplevel" function that accesses the DB should assure
;;; it has a consistent view of the database using this...

(defmacro with-db-locked (&body forms)
  (let ((lock? (gensym)))
    `(let ((,lock? (neq *db-lock* current-process)))
       (unwind-protect (progn (if ,lock? (process-lock (locf *db-lock*)
                                                       nil "Await DB"))
                              ,@forms)
         (if ,lock? (process-unlock (locf *db-lock*) nil nil))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-
```

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;; All Rights Reserved.
;;; Permission to copy all or part of this material is granted,
;;; provided that the copies are not made or distributed for resale,
;;; the MIT copyright notice and reference to the source file and
;;; the software distribution version appear, and that notice is
;;; given that copying is by permission of Massachusetts Institute
;;; of Technology.

```
;;;; Declarations for the DB

(defvar *db-lock* nil
  "Lock cell to assure only one consumer of the DB at a time.")

(def-db-var *db-segs*            '())
(def-db-var *cmarkers*           '())

(def-db-var *null-cmarker*       (build-cmarker nil nil nil nil))

(defvar     *initial-db-context-number* 1)

(def-db-var *initial-db-context* (build-layers-list
                                   *initial-db-context-number*))

(def-db-var *db-context*         *initial-db-context*)
(defvar     *db-context-count*   *initial-db-context-number*)

(defvar *db-last-pattern-found* nil "Last pattern seen by FIND-FACTS")

(defvar *db-context-names*       '())

(define-internal-option *db-check-facts*
                        "Dynamically audit DB facts"
  t :boolean)

 ;; It doesn't work to run with this set to T yet because there
 ;; are TOO many bugs caught by it...
(define-internal-option *db-check-fact-ports*
                        "Warn if a removed fact is still referred to"
  nil :boolean)

(defvar QF                       nil)
(defvar QS                       nil)
(defvar QP                       nil)
(defvar QPAS                     nil)
(defvar QPP                      nil)
(defvar QPWP                     nil)

(defvar *db-top-fn*              nil)
(defvar *db-audit*               nil)

(defun db-init-vars ()
  ;; This used to init the variables
  ;;  *DB-CHECK-FACTS*
  ;;  *DB-CHECK-FACT-PORTS*
  ;; but I don't see the point. I think they are not the sort of
  ;; thing that want to reset when the DB is reset. --kmp
  (setq *db-segs*            '()
        *cmarkers*           '()
        *null-cmarker*       (build-cmarker nil nil nil nil)
        *db-context*         *initial-db-context*
        *db-context-count*   *initial-db-context-number*
        *db-context-names*   nil
        QF                   nil
        QS                   nil
        QP                   nil
        QPAS                 nil
        QPP                  nil
        QPWP                 nil
        *db-top-fn*          nil
        *db-trace*           nil
        *db-audit*           nil)
  'DONE)
```

```
;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;; This is a concise listing of each kind of fact that can exist
;;; in the database, annotated with the fields it has and notes about
;;; what those fields can contain.

;;; (segcase INPUT kind obj id)  kind={NIL,ID}
;;; (segcase OUTPUT kind obj id)  kind={NIL,ID,PART}
;;;
;;; (segcase TINPUT lower-env obj lower-io)
;;; (segcase TOUTPUT lower-env obj lower-io)
;;;
;;; (segcase EXPECT wff)
;;; (segcase ASSERT wff)
;;; (segcase WHEN wff)
;;;
;;; (segcase SEGTYPE type)  type={SPLIT, JOIN, SIMPLE}
;;;
;;; (segcase SUBSEG subseg)
;;;
;;; (segcase DFLOW from to)
;;; (segcase CFLOW from to)
;;;
;;; (segcase PLANTYPE type)   type={EXPR,AND,COMP,XOR,PRED,SSR,SSR-BODY,
;;;                                 TEMPCOMP,JOIN,CONSTANT,BUILTIN,NEW-VIEW}
;;;
;;; (segcase ROLE roleseg rolename)  rolename={ACTION,INIT,PRED,JOIN,REC,
;;;                                            OP,BODY,DFJOIN,BSSR,GEN,
;;;                                            ACC,TRN,TERM,FILTER}
;;;
;;; (segcase SLOT slotseg slotname slottype)  slottype={INPUT,OUTPUT,SEGMENT}
;;;
;;; (segcase CLICHE clichename)
;;; (segcase CLICHE-DEF cliche-def-name cliche-def-comments)
;;;
;;; (segcase RECURSIVE subseg)
;;;
;;; (segcase @SEG kind name code)  kind={OPENCODE,FUNCALL,CONSTANT,
;;;                                      LAMBDA-EXP,JOIN,RECURSION}
;;;
;;; (deep @IO kind name argno)  kind={LAMBDA-ARG,ACTUAL-ARG,FREE-VAR,
;;;                                   RETURNVAL,SIDE-EFFECT,DUMMY}
;;;
;;;
;;; (deep @CREATEVAR varname)
```

```
;;; DB: Database for the Knowledge-Based Editor

;;; Implementation Notes:
;;;
;;;  This file is now free of 8names!
;;;
;;;  This file contains no explicit (<- ...) calls.
;;;
;;;  All (LOOP ...)'s have been updated to (REPEAT ...) or (REPEAT* ...)
;;;
;;;  This file contains no (BEGIN ...)
;;;
;;;  This file seems to no longer depend on (MATCH ...).
;;;  This file looks clear of all but 2-arg IF.
;;;  This file DOES still depend on (OVER ..).
```

```
;;;; ...

;;; (Re)Initializes the DB

(defun db-init ()
  (mapc #'db-init-sym *db-segs*)
  (db-init-vars))

;;; (MAKE-INTERNED-CMARKER in? in out)
;;;
;;;  Returns a cmarker whose IN-LAYERS, OUT-LAYERS, and CMARKER-IS-IN?
;;;  fields are the values given by the arguments. If possible, returns
;;;  an existing cmarker, otherwise conses a fresh one.
;;;
;;;  Note: Although we attempt here to eliminate duplicate cmarkers,
;;;        this notion is not rigidly enforced elsewhere.

(defsubst maybe-find-existing-cmarker (in out)
  (dolist (cmarker *cmarkers*)
    (if (and (equal in  (in-layers cmarker))
             (equal out (out-layers cmarker)))
        (return cmarker))))

(defun make-interned-cmarker (in? in out)
  (cond ((null in) *null-cmarker*)
        ((maybe-find-existing-cmarker in out))
        (t (top (db-push (build-cmarker in? in out nil) *cmarkers*)))))

;;; These functions manipulate contexts.
;;;
;;; Notes:
;;;
;;;  The functions have to work a bit because pre-computation is done
;;;  on the cmarkers.
;;;
;;;  The NEW-CMARKER field is just set to nil in order to save time
;;;  here, and in order to save creating cmarkers that may never be used.
;;;
;;;  PUSH-DB-CONTEXT cannot change whether something is IN or OUT and is
;;;  therefore particularly fast.

(defsubst invalidate-new-cmarkers ()
  (setf (new-cmarker *null-cmarker*) nil)    ;invalidate *null-cmarker*
  (dolist (cmarker *cmarkers*)               ; and all the real cmarkers'
    (setf (new-cmarker cmarker) nil)))

(defun push-db-context ()
  (incf *db-context-count*)
  (setq *db-context* (db-cons *db-context-count* *db-context*))
  (invalidate-new-cmarkers)
  *db-context*)

(defun pop-db-context ()
  (set-db-context (cdr *db-context*)))

(defun set-db-context (new-context)
  (setq *db-context* (sort (db-copylist new-context) #'>))
  (setf (new-cmarker *null-cmarker*) nil) ;Note this is essential
  (let ((in-layers) (out-layers))
    (dolist (cmarker *cmarkers*)
      (setq in-layers  (in-layers  cmarker))
      (setq out-layers (out-layers cmarker))
      (setf (cmarker-is-in? cmarker)
            (dolist (layer *db-context*)
              (cond ((member layer  in-layers) (return t  ))
                    ((member layer out-layers) (return nil)))))
      (setf (new-cmarker cmarker) nil))
    *db-context*))
```

```
;;;; Making facts be IN

;;; Overview:
;;;
;;;  These are the fns for making something IN in the db. Each of these
;;;  functions returns the fact which was made IN. In the case of
;;;  ASSERT-FACT, this may not necessarily be EQ to the fact given as
;;;  an argument.
;;;
;;;  There are also abstraction macros, +& and >& which compose the
;;;  behavior of ADD-FACT or ASSERT-FACT (respectively) and the & macro.


;;;  (MAKE-FACT-BE-IN fact)
;;;
;;;   Makes a fact that is already indexed in the database be IN.

(defun make-fact-be-in (fact)
  (let ((c (cmarker fact)))
    (setf (cmarker fact)
          (cond ((cmarker-is-in? c) c)
                ((new-cmarker c))
                (t (setf (new-cmarker c)
                         (make-interned-cmarker T
                           (add-layer (car *db-context*) (in-layers  c))
                           (del-layer (car *db-context*) (out-layers c))))
                   (new-cmarker c)))))
  (when-db-traced (db-display-trace-info ADD fact))
  fact)

;;;  (ADD-FACT fact)            ;see also +&
;;;
;;;   Indexes a fact into the data base, and then makes it be IN.
;;;   Note that it is imperative that an equal fact not already be
;;;   indexed in the db whether it is IN or not!

(defun add-fact (fact)                                          ;+&
  (if (and *db-check-facts* (null (audit-fact fact nil)))
      (barf "ADD-FACT: Syntactically incorrect fact.~
           ~% Notes: ~:=~
           ~% Offending datum: ~:="
            *db-audit* fact))
  (dolist (hunk-and-num (index-fact fact))
    (db-push fact (dbx (cdr hunk-and-num) (car hunk-and-num))))
  (make-fact-be-in fact))

;;;  (ASSERT-FACT fact)         ;see also >&
;;;
;;;   Takes a fact, checks to see if it is already indexed, indexes it
;;;   if it isn't, and then makes it be IN.
;;;
;;;   Note:  ASSERT-FACT is the safe function to call if you have any
;;;          doubts at all about whether the fact is already indexed in
;;;          the database or not.

(defun assert-fact (fact)                                       ;>&
  (if (and *db-check-facts* (null (audit-fact fact nil)))
      (barf "ASSERT-FACT: Syntactically incorrect fact.~
           ~% Notes: ~:=~
           ~% Offending datum: ~:="
            *db-audit* fact))
  (let* ((idxs (index-fact fact))
         (db-fact (dolist (f (dbx (cdar idxs) (caar idxs)))
                    (if (matches-fact? fact f) (return f)))))
    (cond (db-fact (setq fact db-fact))
          (t (dolist (hunk-and-num idxs)
               (db-push fact (dbx (cdr hunk-and-num)
                                  (car hunk-and-num)))))))
```

```
(make-fact-be-in fact))
```

```
;;;; Finding DB facts

;;; (FIND-FACTS pattern)
;;;
;;;  This function finds facts in the database.  It takes in a pattern,
;;;  and finds all of the facts that are IN, and which match the pattern.
;;;  A key complexity is dealing with inadequately indexed patterns.
;;;  A list of all the facts found is returned.
;;;
;;;  The macros ??&, ?&, and A?& are available for composing the actions
;;;  of FIND-FACTS and &* with varying kinds of error checking.

(defun find-facts (pattern)
  (setq *db-last-pattern-found* pattern)
  (if (and *db-check-facts* (null (audit-fact pattern t)))
      (barf "FIND-FACTS: Syntactically incorrect pattern.
            ~% Notes: ~:=~
            ~% Pattern: ~:="
             *db-audit* pattern))
  (let* (((hunk . num) (car (index-fact pattern))) facts)
    (setq facts
          (cond (num
                 (cond (hunk (collect (fact (dbx num hunk))
                                      (and (fact-is-in? fact)
                                           (matches-fact? pattern fact))))
                       (t
                        (mapcan #'(lambda (seg)
                                    (collect
                                      (fact (dbx num (db-index seg)))
                                      (and (fact-is-in? fact)
                                           (matches-fact? pattern fact))))
                                *db-segs*))))
                (t (grumble "FIND-FACTS: Very slow retrieval pattern:
                            ~% Pattern: ~:="
                            pattern)
                   (dolist (seg *db-segs*)
                     (let ((hunk (db-index seg)))
                       (over num <- 10. by (1- num) until (minusp num)
                        nconc (collect
                                (fact (dbx num hunk))
                                (and (fact-is-in? fact)
                                     (matches-fact? pattern fact)))))))))
    (when-db-traced (dolist (fact facts)
                      (db-display-trace-info FIND fact)))
    facts))

;this fn gives you all of the facts that have the given seg in their deep.
;it is called by facts? below.  it is needed so that you can get all these
;facts fast without doing a bunch of finds with all the different keywords.

(defun seg-facts (seg)
  (let* ((hunk (db-index seg)) facts)
    (setq facts
          (if hunk
              (over num <- 10. by (1- num) until (minusp num)
               nconc (collect (fact (dbx num hunk)) (fact-is-in? fact)))))
    (when-db-traced (dolist (fact facts)
                      (db-display-trace-info FIND fact)))
    facts))
```

```
;;;; ...
;this gives you a list of all the facts that refer to the given port.
;ie @io @createvar dflow tinput toutput input output.
;note that with inputs, outputs, the ref could be indirect through
;the id field, and with a dflow it is remotely possible (though an
;error) that it could refer to the same port twice, in this case
;the dflow will be in the list twice.

(defun port-facts (port)
  (let* ((seg (~seg~segcase port))
         (obj (~obj port))
         (hunk (db-index seg)) facts)
    (cond (hunk
           (over-in-facts (f (dbx 4 hunk))
             (if (eq (~deep f) port) (push f facts)))
           (over-in-facts (f (dbx 5 hunk))
             (if (eq (~deep f) port) (push f facts)))
           (over-in-facts (f (dbx 8 hunk))
             (if (eq (~id f) obj) (push f facts)))
           (over-in-facts (f (dbx 9 hunk))
             (if (eq (~id f) obj) (push f facts)))
           (over-in-facts (f (dbx 10. hunk))
             (if (eq (~deep f) port) (push f facts)))
           (over-in-facts (f (dbx 12. hunk))
             (if (eq (~from f) port) (push f facts)))
           (over-in-facts (f (dbx 13. hunk))
             (if (eq (~to f) port) (push f facts)))
           (over-in-facts (f (dbx 14. hunk))
             (if (and (toutput-fact? f) (eq (~lower-io f) port))
                 (push f facts)))
           (over-in-facts (f (dbx 15. hunk))
             (if (and (tinput-fact? f) (eq (~lower-io f) port))
                 (push f facts)))))
    (when-db-traced (dolist (fact facts)
                      (db-display-trace-info FIND fact)))
    facts))
```

```
;these are the fns that make facts out in the data base.  MAKE-FACT-BE-OUT
;takes an actual fact indexed in the db and makes it be out.  ERASE-FACTS
;takes a pattern as input, finds all of the facts that match that pattern,
;and then makes each of those facts be out.  Each fn returns the facts it
;made out.  as a collateral effeciency, MAKE-FACT-BE-OUT keeps count of
;all of the
;facts it makes out which are then not IN in any layer, and from time
;to time it flushes all of them out of the db all together in order to
;save space.  There are macros available: --&, -&, and a-& combining
;the actions of &* and ERASE-FACTS.

(defun make-fact-be-out (fact)
  (cond ((cmarker fact)
         (if (and *db-check-facts* *db-check-fact-ports*)
             (if (port-fact? fact)
                 (let ((facts (port-facts fact)))
                      (if facts
                          (barf "MAKE-FACT-BE-OUT: Deleting a port ~
                                   that is referred to.~
                                ~% Fact deleted: ~:=~
                                ~% Relevant Data: ~:="
                                 fact facts)))))
         (let ((c (cmarker fact)))
           (setf (cmarker fact)
                 (cond ((not (cmarker-is-in? c)) c)
                       ((new-cmarker c))
                       (t
                        (setf
                          (new-cmarker c)
                          (make-interned-cmarker nil
                            (del-layer (car *db-context*) (in-layers  c))
                            (add-layer (car *db-context*) (out-layers c))))
                        (new-cmarker c)))))
         (when-db-traced (db-display-trace-info ERASE fact))))
  fact)

(defun erase-facts (pattern)
  (let ((facts (find-facts pattern)))
    (dolist (fact facts) (make-fact-be-out fact))
    facts))

(defun erase-fact (pattern)
  (sell (erase-facts pattern)))

;;; (KILL-SEG seg)
;;;
;;;  This function gets rid of every fact refering to the indicated
;;;  segment in any way. It is needed so that you can wipe out all
;;;  those facts conveniently without doing dozens of seperate fetches.
;;;  It does not return the facts killed.
;;;
;;;  Note: This might have some far reaching effects if somebody
;;;        refers indirectly to the ports through a tinput or something.

(defun kill-seg (seg)
  (let* ((hunk (db-index seg)))
    (if hunk
        (dolist (num '( 0.  1.  2.  3.  4.  5.  6.  7.   ;up to ports
                       10. 11. 12. 13. 14. 15.           ;after ports
                        8.  9.))                         ;ports last!
          (over-in-facts (f (dbx num hunk))
            (cond ((port-fact? f)
                   (kill-port f))
                  (t
                   (make-fact-be-out f))))))))
```

```
;this kills every fact refering to a given port.
;ie @io @createvar dflow tinput toutput input output.
;note that this can have some far reaching transitive effects if
;some tinput or something refers to the port. (note that if there
;was a loop in this refering (a gross bug), this will loop)

(defun kill-port (port)
  (let* ((seg (~seg~segcase port))
         (obj (~obj port))
         (hunk (db-index seg)))
    (when hunk
      (over-in-facts (f (dbx  4 hunk))
        (if (eq (~deep f) port) (make-fact-be-out f)))
      (over-in-facts (f (dbx  5 hunk))
        (if (eq (~deep f) port) (make-fact-be-out f)))
      (over-in-facts (f (dbx  8 hunk))
        (if (eq (~id   f)  obj) (kill-port f)))
      (over-in-facts (f (dbx  9 hunk))
        (if (eq (~id   f)  obj) (kill-port f)))
      (over-in-facts (f (dbx 10. hunk))
        (if (eq (~deep f) port) (make-fact-be-out f)))
      (over-in-facts (f (dbx 12. hunk))
        (if (eq (~from f) port) (make-fact-be-out f)))
      (over-in-facts (f (dbx 13. hunk))
        (if (eq (~to   f) port) (make-fact-be-out f)))
      (over-in-facts (f (dbx 14. hunk))
        (if (and (toutput-fact? f)
                 (eq (~lower-io f) port))
            (kill-port f)))
      (over-in-facts (f (dbx 15. hunk))
        (if (and (tinput-fact? f)
                 (eq (~lower-io f) port))
            (kill-port f)))
      (make-fact-be-out port))))
```

```
;delete all assertions in the data base which are not present in any
;subset of the specified context.  This is the function you must use
;in order to restore the db to a prior state.  It makes the assumption
;that only the layers in the specified context are being saved
;anyplace.  all references to any other layers are removed,
;and *db-context-count* is reduced to reflect this fact.

(defun restore-db-context (context)
  (set-db-context context)
  (setq *db-context-count* (car *db-context*))
  (setq *db-context-names*
        (over *db-context-names* -> name
              (symeval name) -> con
         select (cond ((over c <- context fastor (equal con c)))
                      (t (set name nil)))
         bag name))
  (dolist (cmarker *cmarkers*)
    (setf (in-layers cmarker)
      (over (in-layers cmarker)  select (member _ *db-context*) list _))
    (setf (out-layers cmarker)
      (over (out-layers cmarker) select (member _ *db-context*) list _)))
  (push-db-context)
  (prog1 (db-expunge)
         (dolist (seg *db-segs*) (set-up-db-names seg))))
```

```
;;;; Unique name generation

;;; This cluster of functions supports the generation of unique names.
;;;
;;; There are three key requirments of the database in this regard:
;;;
;;;   (1) There must not be two segs with the same ~SEG name in the
;;;       db at once.
;;;
;;;   (2) There must not be two ports with the same ~OBJ on the same seg.
;;;
;;;   (3) There must not by two cases with the same ~case name on the
;;;       same seg.
;;;
;;; An additional desideratum is that it should be possible to
;;; back up the naming process, when the db is backed up, so that things
;;; will go along again just as they did before. This helps debugging
;;; an amazing amount.
;;;
;;; Also, symbols take up lots of storage, so we don't want to have any
;;; more names then we need. [This was true in Maclisp. Do we care in
;;; LispM lisp? --kmp]
;;;
;;; The following functions meet all of these criteria.
;;;
;;; The key idea is that every seg in the db has a DB-NAMES property that
;;; holds a list of object names used on it, and the number of the largest
;;; case number used so far.  To get a new
;;; name, you must specify what seg you want it on, and it makes a
;;; name from the root you gave that doesn't clash with any of the names
;;; already in existence. (The criterion for non-clashing of seg names
;;; is that the symbol not already appear in the list held by *DB-SEGS*.)
;;; To get restoration to an earlier time, the DB-NAMES properties are
;;; simply recreated as of that time.
;;;
;;; The way things are set up, they assume that you use
;;; GENSEG, GENOBJ, and GENCASE -always-.

(defun genseg (atom)
  (let* ((seg (genatom-not-in *db-segs* atom)))
    (db-push seg *db-segs*)
    (setf (db-names seg) (db-cons nil 0))
    (setf (db-index seg) (make-dbhunk))
    seg))

;This makes the actual name.

(defun genatom-not-in (list atom)
  (prog (prefix-string new i)
     (if (not (memq atom list)) (return atom))
     (setq prefix-string (string-right-trim ".0123456789" (string atom)))
     (when (not (string-equal prefix-string (string atom)))
       (setq new (intern prefix-string *kbe-package*))
       (if (not (memq new list)) (return new)))
     (setq i 1)
   L (setq new (intern (string-append prefix-string
                         (let ((*nopoint T)) (format nil "~A" i)))
                       *kbe-package*))
     (if (not (memq new list)) (return new))
     (setq i (1+ i))
     (go L)))
```

```
;this works for objects.  (note it assumes that the seg is created by
;genseg)

(defun genobj (seg atom)
  (let* ((entry (db-names seg))
         (name (genatom-not-in (car entry) atom)))
    (push name (car entry))
    name))

;this works for cases, however, it runs on different assumptions.  there
;are two ways cases are used.  either they are mnemonic things like
;yes-no or rec-body, in which case they never clash, and are not checked.
;or they are just generated numbers.  All this does for you is give
;you a new unique integer. you can also copy the cases from a join to
;another that has
;none, which is always ok, and you don't call gencase in that situation.

(defun gencase (seg)   ;always uses the root CE
  (let* ((entry (db-names seg))
         (num (1+ (cdr entry))))
    (setf (cdr entry) num)
    num))

;this sets up the db-names lists non-incrementally.
;(see restore-db-context)
;it just looks everywhere an assertion could be using a name, and lists
;them.

(defun set-up-db-names (seg)
  (let* ((entry (db-names seg)))
    (when entry
      (setf (car entry)
            (let ((hunk (db-index seg)) obj objs)
              (dolist (num '(8 9))
                (dolist (port (dbx num hunk))
                  (setq obj (~obj port))
                  (if (not (memq obj objs)) (push obj objs))))
              (db-copylist objs)))
      (setf (cdr entry)
            (let* ((hunk (db-index seg)) (num 0) case)
              (dolist (num '(8 9 10.))
                (dolist (fact (dbx num hunk))
                  (setq case (if (not (fact? (~deep fact)))
                                 (~case (~deep fact))))
                  (if (and (numberp case) (> case num))
                      (setq num case))))
              (dolist (fact (dbx 14. hunk))
                (setq case
                  (cond ((cflow-fact?  fact) (~case (~from      fact)))
                        ((tinput-fact? fact) (~case (~lower-env fact)))))
                (if (and (numberp case) (> case num))
                    (setq num case)))
              (dolist (fact (dbx 15. hunk))
                (setq case
                  (cond ((cflow-fact?   fact) (~case (~to        fact)))
                        ((toutput-fact? fact) (~case (~lower-env fact)))))
                (if (and (numberp case) (> case num))
                    (setq num case)))
              num)))))
```

```
;;; (index-fact fact)
;;;
;;;  This indexing function is the heart of the database.
;;;
;;;  Notes: It is important that we know that no fact can ever index
;;;         into the same bucket twice, because it never has the same
;;;         number twice.
;;;
;;;         This function is overloaded, which is good because it means
;;;         both FIND-FACTS and ASSERT-FACT can call it, but it is bad
;;;         because it always does the work necessary for both functions,
;;;         which is likely to be inefficient.

(defun index-fact (fact)
  (let* ((idxs (get (~keyword fact) 'db-keyword-idxs))
         (indecies))
    (over idxs -> idx
     over (cdr fact) until nil -> field
     select idx
      (setq indecies (nconc (cond ((fact? field) (idx-port field idx))
                                  (t (idx-seg field idx)))
                            indecies)))
    (or indecies (over idxs select _ (return [[nil ! _]])))))

;this takes in a segcase, and a bucket idx.
;it returns [[hunk ! idx]] or nil if the segcase is ~*

(defun idx-seg (segcase idx)
  (cond ((or (eq segcase '~*)
             (eq (~seg segcase) '~*)) nil)
        (T (let ((hunk (db-index (~seg segcase))))
             (if hunk (ncons (cons hunk idx)))))))

;this indexes a port fact

(defun idx-port (field idx)
  (cond ((eq field '~*) nil)
        (t (idx-seg (~deep field) idx))))
```

```
;this fn compares a pattern and a fact (both hunks) to see if they match.

(defsubst mempat (x y)
  (cond ((and (listp x)
              (or (eq (car x) '*code-is-car*)
                  (eq (car x) '*code-is-cdr*)))
         (dolist (item y)
           (if (and (eq (car x) (car item))
                    (eq (cdr x) (cdr item)))
               (return t))))
        (t (memq x y))))

(defun matches-fact? (pattern fact)
  (cond ((not (and (fact? pattern) (fact? fact))) nil)
        ((not (eq (cmarker pattern) *null-cmarker*))
         (eq pattern fact))
        (T (over (get (~keyword fact) 'db-keyword-types) -> type
            over (cdr fact) until nil -> field
            over (cdr pattern) until nil -> pat
             fastand
              (or (eq pat '~*)
                  (caseq type
                    (S  (match-segcase pat field))
                    (P  (matches-fact? pat field))
                    (SP (cond ((fact? field) (matches-fact? pat field))
                              (t (match-segcase pat field))))
                    (C (or (eq pat field)
                           (and (= (length pat) (length field))
                                (over pat fastand (mempat _ field)))))
                    ((K NP) (eq pat field))
                    (L  (equal pat field))))))))


;this matches segcases

(defun match-segcase (pat sc)
  (and (not (fact? pat))
       (not (fact? sc))
       (or (eq (~seg  pat) '~*) (eq (~seg  pat) (~seg  sc)))
       (or (eq (~case pat) '~*) (eq (~case pat) (~case sc)))))
```

```
;the following routine audits facts.  this simple audit just checks for
;syntactic correctness, and does as much as it can fast without doing
;any searches in the data base.  In a sense these routines are the real
;definition of what is correct in a fact.  the routines are used by
;ASSERT-FACT in order to check it isn't putting garbage in the db.

;Audit one fact/pattern.  the tests are basically the same for the
;two, except that only patterns can have ~*s in them.  If the check is
;supposed to allow this, then the second argument must be T. (note we
;check that each pattern has the right number of fields.)  The special
;variable *db-audit* is set to a list of all the errors
;that were found.

(defun audit-fact (fact pattern?)
  (let (types)
    (setq *db-audit* nil)
    (cond ((not (fact? fact))
           (push "Not a fact//pattern at all" *db-audit*))
          ((eq (~keyword fact) '~*)
           (push 'fact-has-* *db-audit*))
          ((null (setq types (get (~keyword fact) 'db-keyword-types)))
           (push "Bad keyword" *db-audit*))
          ((not (= (length types) (length (cdr fact))))
           (push "wrong number of fields" *db-audit*))
          ((over types -> type
            over (get (~keyword fact) 'db-keyword-rests) until nil -> rest
            over (cdr fact) until nil -> field
            (cond
              ((eq field '~*)
               (push 'fact-has-* *db-audit*))
              ((caseq type
                 (S  (audit-segcase field))
                 (P  (if field (audit-fact field nil)))
                 (SP (cond ((fact? field) (audit-fact field nil))
                           (field (audit-segcase field))))
                 (K  (cond ((not (memq field rest))
                            (push "bad flag field value" *db-audit*))))
                 (NP (cond ((fact? field)
                            (push "NP field is fact" *db-audit*))
                           ((not (symbolp field))
                            (push "NP field not symbol" *db-audit*))
                           ((null field))
                           ((not (or (eq (~deep fact) '~*)
                                     (eq (~seg~segcase fact) '~*)
                                     (memq field
                                           (car (db-names
                                                  (~seg~segcase fact))))))
                            (push "NP not made with GENOBJ" *db-audit*))))
                 ((L C) (cond ((fact? field)
                               (push "field is fact" *db-audit*)))))))))
    (when *db-audit*
      (if pattern? (setq *db-audit* (delq 'FACT-HAS-* *db-audit*)))
      (setq *db-audit* (nreverse *db-audit*)))
    (null *db-audit*)))

(defun audit-segcase (sc)
  (let* (seg case)
    (cond ((fact? sc)
           (push "segcase is fact" *db-audit*))
          ((not (and (listp sc)
                     (setq case (~case sc) seg  (~seg sc))
                     (symbolp seg)
                     (or (symbolp case) (numberp case))))
           (push "Segcase is ill-formed" *db-audit*))
          ((or (eq seg '~*) (eq case '~*))
           (push 'fact-has-* *db-audit*))
          ((not (db-index seg))
           (push "Seg not created with genseg" *db-audit*)))))
```

```
;Audit every fact in the db (usefull if you haven't been auditing
;for a while.)

(defun db-audit ()
  (let ((count 0))
    (over *db-segs* -> seg
          (db-index seg) -> hunk
      (over num <- 10 by (1- num) until (minusp num)
        (over (dbx num hunk)
          select (and (fact-is-in? _) (null (audit-fact _ nil)))
          (incf count)
          (grumble "DB-AUDIT found a bad fact.~
                  ~% Notes: ~:=~
                  ~% Offending fact: ~:=" *db-audit* _))))
    count))
```

```
;the following fns compress and expand facts.  the goal if the compressed
;form is to get something that is at the same time more compact, and
;easier to read.  The key properties of the compression
;are as follows:
;1. a fact is compressed, by converting it to a list, and compressing
;   each of its fields.
;2. a segcase (SEG . CASE) becomes the list (SEG CASE) and
;   a seg (SEG) is converted to simply the atom SEG.
;3. the keyword field is left unchanged.
;4. the other fields are printed in order as a list.
;5. if a field is a port (eg input output tinput toutput) then it is
;   abbreviated to just [(abbreviated (~segcase field)) (~object field)]
;   note that this is printed out by a grind macro, from an internal form
;   (/[ segcase obj), and that in order to read it back in, you
;   have to have an appropriate read macro defined
;6. if the initial field is a port, then the abbreviation of 5 is done,
;   but in addition, it is spliced into the top level result, and the
;   keyword is inserted before the object.  IE
;      if the deep is a segcase then each field is just compresed:
;    (C-segcase KEY C-field1 C-field2 ...)
;      if the deep is a port then this is broken apart into two pieces:
;    (C-segcase-dep KEY obj-deep C-field1 C-field2 ...)
;   [NOTE that one of the goals of the compaction is that the segcase
;   of the fact will always be the first field of the fact, and the keyword
;   the second]
;7. the most complex compression happens to the code field of an @seg,
;   the problem is that the EQ structure relating various assertions is
;   vital in this situation, and is hard to recover across printing
;   and reading.  when several facts are being compressed, the variable
;   *DB-TOP-FN* is bound to information describing the code of the top
;   level fn.
;   (at other times, this variable is NIL).  Each code element is
;    represented as either itself (if it is not in the top-fn), or a
;    number.  This number
;   is the mouse scan position order of the the piece of code.
;   (counting from the root as 0.)  What *DB-TOP-FN* actually contains is
;    an assq list where the ith entry is a cons of the code with mouse
;    scan position i and i
;8. fields which are lists are copied so that there will be no sharing
;   between the compressed form and the data base.  It is VITAL
;   that there never be any sharing.

;note that compress is designed so that it doesn't crash if you give
;it a pattern.

;here are some examples of the compression:
;{CM (join1) output nil obj1 nil} => (join1 output nil obj1 nil)
;{CM (join1 . 1) segtype simple} => ((join1 1) segtype simple)
;{CM (ssr1) tinput (expr6) foo {CM (expr6) input nil foo  nil}}
;   => (ssr1 tinput expr6 foo [expr6 foo])
;{CM (ssr1) toutput (pred1 . yes) bar
;    {CM (ssr1) tinput (pred1) tuz {CM (pred1) input nil x nil}}}
;   => (ssr1 toutput (pred1 yes) bar [ssr1 tuz])
```

```
(defun compress-fact (fact)
  (let ((cfact
         (over (get (~keyword fact) 'db-keyword-types) -> type
          over (cdr fact) -> field
          list (caseq type
                 (S  (compress-segcase field))
                 (P  (compress-port field))
                 (SP (cond ((fact? field) (compress-port field))
                           (t (compress-segcase field))))
                 (C  (compress-code-list field))
                 ((K NP)  field)
                 (T  (copytree field))))))
    (if (and (listp (car cfact)) (eq (caar cfact) '/[))
        (setq cfact '(,(cadar cfact) ,(cadr cfact)
                      ,(caddar cfact) .,(cddr cfact))))
    cfact))

(defun compress-segcase (segcase)
  (cond ((eq segcase '~*) segcase)
        ((~case segcase) [(~seg segcase) (~case segcase)])
        (t (~seg segcase))))

(defun compress-port (port)
  (cond ((eq port '~*) port)
        ((null port) nil)
        ((not (and (fact? port) (fact-is-in? port)))
         (grumble "COMPRESS-FACT: Compressing non-existent subport.~
                   ~% Port: ~:="
                  port)
         nil)
        (t '(/[ ,(compress-segcase (~segcase port)) ,(~obj port)))))
```

```
;this expands a compressed fact back out into an actual fact and puts
;it in the DB.  Note that unique seg names are generated so that
;there will be no collision between these new facts and any old ones.
;(NOTE that it is critical that any port fact referred to by the compressed
;fact, must be IN the data base before expansion is attempted.)

;It is imperative that the facts created share no list structure
;with the arguments passed in.


;;; This interacts with plan-def in order to rename segs.
;;; Note: *DB-NEW-NAMES* is bound by PLAN-DEF.
(declare (special *db-new-names*))

(defun expand-fact (cfact)
  (let*  ((types (get (cadr cfact) 'db-keyword-types))
          (cseg (cond ((atom (car cfact)) (car cfact))
                      (T (caar cfact))))
          (fact (db-make-list (1+ (length types)))))
    (if (> (length cfact) (length types))
        (setq cfact '((/[ ,(car cfact) ,(caddr cfact))
                      ,(cadr cfact) ., (cdddr cfact))))
    (rplaca fact *null-cmarker*)
    (over types -> type
     over cfact -> field
     over f <- (cdr fact)
      (rplaca f
        (caseq type
          (S  (expand-segcase field))
          (P  (expand-port field))
          (SP (cond ((and (listp field) (eq (car field) '/[))
                     (expand-port field))
                    (t (expand-segcase field))))
          (C  (expand-code-list field))
          (NP (if field
                  (let ((seg (~seg~segcase fact))
                        (key (cons cseg field)))
                    (cond ((cdr (assoc key *db-new-names*)))
                          (T (let ((o (genobj seg field)))
                               (push (cons key o) *db-new-names*)
                               o))))))
          (K  field)
          (L  (copytree field)))))
    (add-fact fact)))

(defun expand-segcase (csc)
  (let* ((seg  (cond ((atom csc) csc) (t (car csc))))
         (case (cond ((atom csc) nil) (t (cadr csc))))
         (seg-to-use
           (cond ((cdr (assq seg *db-new-names*)))
                 (T (let ((s (genseg seg)))
                      (push (cons seg s) *db-new-names*)
                      s))))
         (case-to-use
           (let ((key (cons seg case)))
             (cond ((symbolp case) case)
                   ((cdr (assoc key *db-new-names*)))
                   (T (let ((c (gencase seg-to-use)))
                        (push (cons key c) *db-new-names*)
                        c))))))
      (& seg-to-use case-to-use)))

(defun expand-port (port)
  (cond ((null port) nil) ;special case for null lower-ios
        (T (let* (((csc old-obj) (cdr port))
                  (old-seg (cond ((atom csc) csc) (T (car csc))))
                  (segcase (expand-segcase csc))
                  (new-obj (cdr (assoc (cons old-seg old-obj)
```

```
                                *db-new-names*))))
        (car (or (??& segcase   input * new-obj)
                 (??& segcase  output * new-obj)
                 (??& segcase  tinput * new-obj)
                 (??& segcase toutput * new-obj)
                 (barf "EXPAND-PORT: Can't find deep io~
                     ~% Port: ~:=" port)))))))
```

```
;this takes in a set of facts, and compresses them together in a block.
;it assumes that all of the facts are about the same seg, and it is only
;really usefull if the facts have been sorted first.
; initially, each individual fact is compressed as above.  after this each
;one is of the form [segcase keyword ! rest].  these are then factored
;together
;according to their cases, and keywords.  each group produced is a list of
;a keyword, and a sequence of rests.  a new group is started every time
;the keyword, or the case changes.  whenever the case changes, a group
;consisting of just the case is produced.  (if the first case is nil, then
;no initial nil is produced.)

(defun compress-facts (facts)
  (let* (case result group key
         (cases (and facts (all-fact-cases (car facts)))))
    (over facts -> fact
          (compress-fact fact) -> [sc k ! rest]
          (if (not (atom sc)) (cadr sc)) -> c
      (when (or (not (eq k key)) (not (eq c case)))
        (if group (push (nreverse group) result))
        (setq group [k]
              key   k))
      (push rest group)
      (when (not (eq c case))
        (push c result)
        (setq cases (delq c cases))
        (setq case c)))
    (push (nreverse group) result)
    (setq result (nconc cases result))
    (nreverse result)))

;takes a fact and gives all the cases on the seg of the fact.
;kludge just called above.

(defun all-fact-cases (fact)
  (let* ((seg (~deep fact)))
    (cond ((fact? seg) (setq seg (~deep seg))))
    (setq seg (~seg seg))
    (over [!!(~nset ~case ~to (??& * cflow * (&* seg)))
           !!(~nset ~case ~segcase ~to (??& * dflow * (&* (&* seg) port)))
           !!(~nset ~case ~segcase ~from (??& * dflow (&* (&* seg) port)))
           !!(~nset ~case ~from (??& * cflow (&* seg)))]
          select _ makeqset _)))

;this is the inverse of compress-facts.  it takes in the shared compressed
;form, and expands them all out.

(defun expand-facts (info)
  (let* (((seg . lines) info)
         (csc seg))
    (dolist (line lines)
      (cond ((atom line) (setq csc [seg line]))
            (T (dolist (rest (cdr line))
                 (expand-fact [csc (car line) ! rest])))))))
```

```
;;; (BUILD-*DB-TOP-FN* list)
;;;
;;;  This is used both on compression and reading in in order to construct
;;;  a *DB-TOP-FN*. It will globally assign *DB-TOP-FN*; if that's
;;;  not desirable, you should bind that variable before calling this
;;;  function.
;;;
;;;  See PLAN-DEF which calls this.

(defvar *db-top-fn-idx* nil
  "Communication var for build-*DB-TOP-FN* and build-*DB-TOP-FN*-aux")

(defun build-*DB-TOP-FN* (list)
  (let* ((*db-top-fn-idx* 0.))
    (setq *db-top-fn* nil)                          ;global assignment
    (build-*DB-TOP-FN*-aux list)
    (setq *db-top-fn* (reverse *db-top-fn*))))  ;global assignment

(defun build-*DB-TOP-FN*-aux (obj)
  (push (cons obj *db-top-fn-idx*) *db-top-fn*) ;global assignment
  (incf *db-top-fn-idx*)
  (cond ((not (atom obj))
         (build-*DB-TOP-FN*-aux (car obj))
         (build-*DB-TOP-FN*-aux (cdr obj)))))

(defun compress-code-list (list)
  (if (listp list) (mapcar #'compress-code list)
      list))

(defun compress-code (code)
  (cond ((and (listp code)
              (memq (car code) '(*code-is-car* *code-is-cdr*)))
         [(car code) (compress-code (cdr code))])
        ((cdr (assq code *db-top-fn*)))
        (t (copy code)))) ;prevents pointers into DB.

(defun expand-code-list (list)
  (mapcar #'expand-code list))

(defun expand-code (code)
  (cond ((null *db-top-fn*) nil)
        ((numberp code) (car (nth code *db-top-fn*)))
        ((and (listp code)
              (memq (car code) '(*code-is-car* *code-is-cdr*)))
         [(car code) ! (expand-code (cadr code))])))
```

```
(defun construct-DEFFACTS (seg)
  '(deffacts ,seg ,@(compress-facts (sort-facts (seg-facts seg)))))

(defun construct-DEFSPECS (seg)
  '(defspecs ,seg
             ,@(compress-facts
                 (sort-facts
                   (collect (fact (seg-facts seg))
                            (not (plan-fact? fact)))))))

(defun construct-DEFPLAN (seg)
  '(defplan ,seg
            ,@(compress-facts
                (sort-facts
                  (collect (fact (seg-facts seg)) (plan-fact? fact))))))

;note that the division into DEFFACTS, DEFSPECS, and DEFPLAN is only for
;the convenience of the user and means nothing in actuality.

;these are not ever evaled.  You MUST nest then inside a plan-def.

(defmacro deffacts (name &body stuff)
  (ferror "Not defined except inside PLAN-DEF: ~S"
          '(DEFFACTS ,name ,@stuff)))

(defmacro defplan (name &body stuff)
  (ferror "Not defined except inside PLAN-DEF: ~S"
          '(DEFPLAN ,name ,@stuff)))

(defmacro defspecs (name &body stuff)
  (ferror "Not defined except inside PLAN-DEF: ~S"
          '(DEFSPECS ,name ,@stuff)))

;this prints out a deffacts prettily.

(declare (special deffacts-latch))

(pp:define-list-print-function deffacts (list s)
  (let ((deffacts-latch T))
    (format s "~1!~=~^ ~@_~=~^ ~@{~\kbe:deffacts-form\~^~}~." list)))

(pp:define-printing-directive deffacts-form (item s)
  (cond ((or (atom item) deffacts-latch) (format s "~%")))
  (setq deffacts-latch T)
  (cond ((atom item) (prin1 item s) (setq deffacts-latch nil))
        ((memq (car item) '(dflow @io))
         (format s " ~4,0T(~!~S~^ ~:I~@_~@{~=~^ ~0,40T~:_~}~.)" item))
        ((memq (car item) '(cflow))
         (format s " ~4,0T(~!~S~^ ~:I~@_~@{~=~^ ~0,20T~:_~}~.)" item))
        (T (format s " ~4,0T(~!~S~^ ~:I~@_~@{~=~^ ~:_~}~.)" item))))

(pp:define-list-print-function defspecs (list s)
  (let ((deffacts-latch T))
    (format s "~1!~=~^ ~@_~=~^ ~@{~\kbe:deffacts-form\~^~}~." list)))

(pp:define-list-print-function defplan (list s)
  (let ((deffacts-latch T))
    (format s "~1!~=~^ ~@_~=~^ ~@{~\kbe:deffacts-form\~^~}~." list)))

(pp:define-list-print-function plan-def (list s)
  (format s "~1!~1I~=~^ ~@_~=~^~%  ~=~^~@{~%~=~^~}~."
          list))
```

```
;this reconstructs a dumped piece of data base.  Note the
;way it works to prevent there from being any name clashes with
;what ever else is in the db, and the way it does everything
;in reverse order so that the db finds will work out ok
;during expansion.  It returns the seg defined.

(defvar *plan-def-segcases* nil "the segcases plan-defs eval to")

(defun plan-def (&quote &rest body)
  (let* (((result-seg code . defns) body)
         *db-new-names* *db-top-fn*)
    (build-*DB-TOP-FN* (copytree code)) ;copying is vital!!
    (dolist (def (reverse defns))
      (when (eq (car def) 'deffacts)
        (expand-facts (cdr def))))
    (setq result-seg (cdr (assq result-seg *db-new-names*)))
    (push (& result-seg) *plan-def-segcases*)
    (& result-seg)))

;this dumps out a set of plans into a file that can be loaded back
;in again by doing a LOAD.

(defun dump-plans (segcases filename)
  (with-open-file (file (file-default-fn2 filename 'PLAN ':newest) ':out)
    (let ((package *KBE-package*))
      (format file ";;; -*- Mode:LISP; Package:KBE; Base:10; -*-~%")
      (dolist (segcase segcases)
        (multiple-value-bind (plan pas)
            (construct-plan-def (~seg segcase))
          (pl plan file)
          (format file "~/|~%/"")
          (pl pas file)
          (format file "/"~%~/|")))))))

;this reads in a file that contains 1 or more plan-defs, and
;returns a list of the top level segs asserted.
;THIS IS OBSOLETE. JUST LOAD THE FILE INSTEAD SEE ABOVE.

(defun load-dump (filename)
  (with-open-file (file (file-default-fn2 filename 'PLAN ':newest))
    (let ((package *KBE-package*))
      (repeat (result item)
        (setq item (read file t))
        (when (eq item t) ;returns t on eof
          (push-db-context)
          (db-context-was >LOAD)
          (return (nreverse result)))
        (cond ((and (not (atom item))
                    (eq (car item) 'plan-def))
               (push (eval item) result)))))))
```

```
;this conses up a list representation of all of the facts
;about a seg and its subsegs.  As a side-effect, it also creates the
;PAS print structure.  It returns them as two values.  if trim?
;is T then terminal segments are not included in the output.

(defun construct-plan-def (seg &optional trim?)
 (prog ()
  (let* ((*db-top-fn* nil)
         (external-rep)
         (code (~01 ~code (??& (&* seg) @seg)))
         (waiting [[seg]])
         (pasindex ['*ROLE-FORMAT* nil ! (car waiting)]))
    (build-*DB-TOP-FN* code)
    (repeat* ((todo [seg])
              done info subs roles)
      (cond ((null todo) (return nil)))
      (cond ((memq (top todo) done) (pop todo))
            (t (setq info (construct-DEFFACTS (pop todo)))
               (setq subs
                     (mapcar #'car
                             (cdr (assq-carefully 'subseg (cddr info))))
                     roles
                     (or (over (cdr (assq-carefully 'role (cddr info)))
                               -> [! e s n]
                               select
                               (cond ((eq n 'body)
                                      (not (over (cdr (assq-carefully 'role
                                                      (cddr info)))
                                                 fastor (eq (cadr _) 'op))))
                                     (t (not (memq n '(join rec dfjoin)))))
                               list e)
                         (over (cdr (assq-carefully 'subseg (cddr info)))
                               list [(car _) nil])))
               (cond ((or subs (not trim?)) (push info external-rep)))
               (setq todo (append subs todo))
               (let ((spot (assq (cadr info) waiting)))
                 (if (and spot subs)
                     (rplacd spot
                             [(caadr (assq-carefully 'plantype(cddr info)))
                              !(over roles -> [s n]
                                list ['*ROLE-FORMAT* n
                                      !(top (push [s] waiting))])]))))))
    (return '(plan-def ,seg ,(copytree code) ,@(nreverse external-rep))
            pasindex))))

;the purpose of this function is to produce a new segment which is an
;exact copy of another segment.  It just uses compress and expand in
;order to get all the right unique names generated and everything.
;the second arg used to be a prefix to ues for the new seg but is
;now obsolete.  This would be faster if it specially integrated everything
;together without doing any consing of compressed facts and leaving out
;most of the sorting.  But, this is simpler and more obviously bug free.

(defun copyseg (oldseg ignore)
  (eval (construct-plan-def oldseg)))
```

```
;these things just make the pas print out prettier.

(pp:define-list-print-function *role-format* (list s)
  (let* (((role seg ptype . subsegs) (cdr list))
         (>1level (over subsegs -> sub
                    fastor (and (not (atom sub))
                                (over sub fastor (not (atom _)))))))
    (setq role  (nametran role)
          ptype (nametran ptype))
    (if >1level
        (format s "~@!~2I~@[~A ~]~S~@[{~A}~]~{~%~=~^~}~."
                role seg ptype subsegs)
      (format s "~@!~@[~A ~]~S~@[{~A}~] ~:I~{~:_~=~^~}~."
              role seg ptype subsegs))))

(defun nametran (name)
  (or (cdr (assq name
             '((action   . "act" ) (init     . "init") (pred     . "pred"  )
               (join     . "join") (op       . "op"  ) (body     . "body"  )
               (rec      . "rec" ) (dfjoin   . "dfj" ) (bssr     . "bssr"  )
               (term     . "term") (filter   . "fltr") (trn      . "trn"   )
               (acc      . "acc" ) (gen      . "gen" ) (expr     . "expr"  )
               (and      . "and" ) (comp     . "comp") (xor      . "xor"   )
               (pred     . "pred") (ssr      . "ssr" ) (ssr-body . "ssrb"  )
               (aug      . "aug" ) (term     . "term") (filter   . "filter")
               (tempcomp . "tc"  ) (join     . "join") (constant . "const" )
               (builtin  . "bi"  ) (new-view . "nv"  ) )))
      name))
```

```
;Note that these funny fexprs are ment to be called by the user, they
;should not be called from code!!

(defun basic-qfn (seg var body)
  (when (or (setq seg (car seg)) (null (symeval var)))
    (eval (subst (get-seg seg 1) 'seg body)))
  (pp:pp (symeval var))
  (values))

(defun get-seg (seg cnt)
  (cond ((minusp cnt) (grumble "Bad seg designator.~% seg: ~S" seg) nil)
        ((atom seg)
         (cond ((db-index seg) seg)
               (t (get-seg (eval seg) (1- cnt)))))
        ((atom (car seg))
         (cond ((db-index (car seg)) (car seg))
               (t (get-seg (eval seg) (1- cnt)))))
        ((atom (caar seg))
         (if (db-index (car seg)) (caar seg)))))

(defun f fexpr (seg)
  (basic-qfn seg 'qf '(setq qf (construct-DEFFACTS 'seg))))

(defun s fexpr (seg)
  (basic-qfn seg 'qs '(setq qs (construct-DEFSPECS 'seg))))

(defun p fexpr (seg)
  (basic-qfn seg 'qp '(setq qp (construct-DEFPLAN  'seg))))

(defun pas fexpr (seg)
  (basic-qfn seg 'qpas
    '(multiple-value (qpp qpas) (construct-plan-def 'seg t))))

(defun pp fexpr (seg)
  (basic-qfn seg 'qpp
    '(multiple-value (qpp qpas) (construct-plan-def 'seg t))))

(defun pwp fexpr (seg)
  (basic-qfn seg 'qpwp
    '(multiple-value (qpwp qpas) (construct-plan-def 'seg nil))))
```

```
(def-alistq f-by-k)

(defvar *SO* nil "...???...")
(defvar *PO* nil "...???...")

(declare (special fo role-order))

(defvar *key-order*
        '(segtype plantype recursive when input output tinput toutput
          expect assert @seg @io @createvar role subseg cflow dflow))

;this takes in a set of facts that must be all about the same segment, and
;sorts them in a useful order.

(defun sort-facts (facts)
 (if facts
  (let* ((seg (cond ((fact? (~deep (car facts)))
                     (~seg (~deep (~deep (car facts)))))
                    (t (~seg (~deep (car facts)))))))
    (over (sortcar (over facts -> fact
                     classifyq [(cond ((fact? (~deep fact))
                                       (~case (~deep (~deep fact))))
                                      (t (~case (~deep fact))))
                                ! fact])
                   #'(lambda (x y)
                       (cond ((null x) t)
                             ((null y) nil)
                             ((alphalessp x y))))) -> case-fs
          (sortcar (over (cdr case-fs) -> f classifyq [(~keyword f) ! f])
                   #'(lambda (x y)
                       (let ((xtail (memq x *key-order*))
                             (ytail (memq y *key-order*)))
                         (cond (xtail (or (null ytail)
                                          (memq y (cdr xtail))))
                               (ytail nil)
                               (t (alphalessp x y)))))) -> f-by-k
          (if (f-by-k '@io)
              (~set ~deep
                (sort (append (f-by-k '@io) nil) ;don't kill entry
                  #'(lambda (x y)
                      (cond ((not (same-fact-key? (~deep x) (~deep y)))
                             (input-fact? (~deep x)))
                            ((memq (~kind x) '(lambda-arg actual-arg))
                             (or (not (memq (~kind y)
                                            '(lambda-arg actual-arg)))
                                 (< (~argno x) (~argno y))))
                            ((memq (~kind y) '(lambda-arg actual-arg)) nil)
                            ((eq (~kind x) 'returnval) t)
                            ((eq (~kind y) 'returnval) nil)
                            (t (alphalessp (~name x) (~name y))))))))
          -> *po*
          (enumerate-flows seg (~set ~seg~subseg (f-by-k 'subseg))
             (over (f-by-k 'cflow)
               bag [(~seg (~to _)) ! (~seg (~from _))])
             (over (f-by-k 'dflow)
               bag [(~seg~segcase (~to _)) ! (~seg~segcase (~from _))]))
          -> fo
          (if (f-by-k 'plantype)
              (get (~type (car (f-by-k 'plantype))) 'role-order))
          -> role-order
          (over (f-by-k 'role)
            bag [(~seg~roleseg _)!(~rolename _)]) -> role-info
          (mapcar #'car
            (sort [!(over (f-by-k 'subseg)
                      select (null (assq (~seg~subseg _) role-info))
                      bag [(~seg~subseg _)])
                   ! role-info]
              #'(lambda (x y)
```

```
                (car (or (in-order role-order (cdr x) (cdr y))
                         (in-order fo (car x) (car y))
                         [(alphalessp (car x) (car y))])))) -> *so*
        nconc (setq *so* [seg ! *so*])
              (over f-by-k nconc (sort-key-facts (cdr _)))))))
```

```
;this sorts a set of facts that all have the same key, and all are in
;the same case of the same seg.

(defun sort-key-facts (facts)
  (fact-key-dispatch (car facts)
    ((input output tinput toutput)
     (sort facts
       #'(lambda (x y)
           (car (or (in-order *po* x y)
                    [(alphalessp (~obj x) (~obj y))]))))))
    ((@io @createvar)
     (sort facts
       #'(lambda (x y)
           (car (or (in-order *po* (~deep x) (~deep y))
                    [(alphalessp (~obj (~deep x)) (~obj (~deep y)))]))))))
    (subseg
     (sort facts
       #'(lambda (x y)
           (in-order? *so* (~seg~subseg x) (~seg~subseg y)))))
    (role
     (sort facts
       #'(lambda (x y)
           (in-order? *so* (~seg~roleseg x) (~seg~roleseg y)))))
    (cflow
     (sort facts
       #'(lambda (x y)
          (let* ((xfrom (~seg (~from x)))
                 (xto (~seg (~to x)))
                 (yfrom (~seg (~from y)))
                 (yto (~seg (~to y))))
            (cond ((in-order? *so* xto xfrom)       ;x is back pointer
                   (cond ((in-order? *so* yfrom yto);y is forward pointer
                          (in-order? *so* xfrom yto));x start before y ends
                         (t ;(in-order? *so* yto yfrom) ;both back pointers
                          (in-order? *so* xfrom yfrom)))) ;x start before y
                  (t ;(in-order? *so* xfrom xto)     ;x is forward pointer
                   (cond ((in-order? *so* yfrom yto) ;both are fwd pointers
                          (cond ((eq xto yto)       ;both to same seg
                                 (in-order? *so* yfrom xfrom)) ;x goes less
                                                    ;    distance than y
                                (t (in-order? *so* xto yto)))) ;x end first
                         (t ;(in-order? *so* yto yfrom) ;y is back pointer
                          (not (in-order? *so* yfrom xto))))))))))
                                                    ; not start
                                                    ;  before x ends
    (dflow
     (sort facts
       #'(lambda (x y)
           (car (or (in-order *so*
                              (~seg~segcase (~from x))
                              (~seg~segcase (~from y)))
                    (cond ((not (equal (~from x) (~from y)))
                           [(alphalessp (~obj (~from x)) (~obj (~from y)))])
                          (t (in-order *so*
                                       (~seg~segcase (~to x))
                                       (~seg~segcase (~to y))))))))))
    (t facts)))

;the order of the roles in a seg is controled by the ROLE-ORDER
;property of the plantype atom.  for simplicity the ROLE-ORDER is
;only specified if there is more than one role for the plantype.

(defprop xor (init pred action join) role-order)
(defprop pred (init pred join) role-order)
(defprop ssr (init op body rec) role-order)
(defprop tempcomp (init) role-order)

;;; (IN-ORDER? order x y)
```

```
;;; (IN-ORDER  order x y)
;;;
;;;  If x precedes y in the given order, return true.
;;;
;;;  These two functions are the same except that IN-ORDER? returns truth
;;;  values, while IN-ORDER returns either NIL for unsure or a list whose
;;;  car is the answer.

(defun in-order? (order x y)
  (if (not (eq x y))
      (dolist (w order)
        (cond ((eq x w) (return t))
              ((eq y w) (return nil))))))

(defun in-order (order x y)
  (if (not (eq x y))
      (dolist (w order)
        (cond ((eq x w) (return '(t)))
              ((eq y w) (return '(nil)))))))
```

```
(def-alistq graph)
(def-alistq cgraph)

;head   - seg to start going from (not in segs or old) [until head is
;         added to old, no segs should be available as candidates]
;old    - list of segs selected already
;cut    - a list of segs which may follow the current old. [these
;         are immediately removed from segs and only later added to old]
;segs   - list of segs to consider (old intersect segs = nil)
;graph  - partial order on segs
;result - answer returned in reverse order

(declare (special SEGS))

(defun enumerate-flows (head SEGS cpairs dpairs)
  (setq cpairs (over cpairs makeset _)
        dpairs (over (append dpairs cpairs) makeset _)
        SEGS   (append SEGS nil))
  (repeat* ((graph (over dpairs classifyq _))
            (cgraph (cond (cpairs (over cpairs classifyq _)) (t graph)))
            result)
    (setq result (nconc (enumerate-flows-aux head result graph) result))
    (cond ((null SEGS) (return (nreverse result)))
          (t ;must be a cycle pick a seg to tag on.
           (setq head
                 (over SEGS -> seg
                       (cgraph seg) -> need
                       (length need) -> need-num
                       (over need select (memq _ result) sum 1) -> got-num
                  revlist [(- (* 100. got-num) need-num) ! seg]))
           (setq head
                 (cdar (sort head
                             #'(lambda (x y)
                                 (or (> (car x) (car y))
                                     (and (= (car x) (car y))
                                          (alphalessp (cdr x) (cdr y))))))))
           (setq SEGS (delq head SEGS))))))

(defun enumerate-flows-aux (head old graph)
  (repeat (cut (result [head]))
    (setq cut (over SEGS -> s
                    select (over (graph s)
                                 fastand (or (memq _ old) (memq _ result)))
                    bag (setq SEGS (delq s SEGS)) s))
    (cond ((null cut) (return result))
          ((null (cdr cut))
           (setq result (cons (car cut) result)))
          (t
           (let ((new-old [!! result !! old]))
             (over (sort (over cut
                               bag (enumerate-flows-aux _ new-old graph))
                         #'(lambda (x y)
                             (let ((lx (length x)) (ly (length y)))
                               (or (> lx ly)
                                   (and (= lx ly)
                                        (alphalessp (car x) (car y))))))))
                   (setq result (append _ result)))))))
```

```
; The following fns implement a whole new set of access fns into the
;data base.  They are modeled on the structure of the data, and ignore
;the fact that the db happens to be made up of assertions.  In general,
;they do not return facts, but parts of facts directly.  [among
;other things this leaves open the possibility of another great
;improvement in the db, if they are adopted.]  More importantly, they
;make a lot of things a lot easier to do.  Also some of the real
;common (ie executed often) ones are specially coded for increased
;speed.
; Each one is based on a name that is built up in general from an old
;db keyword, and/or a field name (for example, SEGTYPE or INPUT-OBJ)
;and a prefix.  these prefixes have the following general meanings:
; ?   find it - there is zero or one facts, return the one, or nil.
; ?1  find it - there is exactly one of them, return it. [the only
;               advantage of this is that it does error checking, and
;               immediately complains if there isn't one.]
; ??  find them - there are zero or more return a list of them.
; a?  find one - there are one or more, give me one at random. [this is
;                different from (car (?? )) because it checks!]

; -   delete it - analogous to ?, but deletes as well.
; -1  delete it - analogous to ?1.
; --  delete them - analogous to ??.
; a-  delete one - analogous to a?.

; +   add it - make it true in addition to other similar
;               things that may be true. (different from +&)
; >   assert it - make it be true replacing any similar
;                previous facts. (different from >&)  [Note that this
;                always carries the cannotation, that something is
;                probably getting deleted.]

; note that not all of these are defined, for each kind of fact, but
;rather only the ones I thought would be really usefull.  this stuff is
;in a state of flux at the current time.
; another change from the db as a whole, is that access fns that are
;related to facts that demanded a case of nil in their segcases, just
;take seg atoms, rather than taking segcases as arguments.

;PORTS - these are still handled as facts, because the port fact itself
;is the embodiment of the port.

(defun ??input (seg) (??& (&* seg) input))

(defun --input (seg) (--& (&* seg) input))

(defun ??output (seg) (??& (&* seg) output))

(defun --output (seg) (--& (&* seg) output))

;this implements the common operation that you need to know what
;the actual id port of a port is, and forshadows the fact that the
;actual port instead of the obj name is what should be in the
;port fact id field in the first place.

(defun ~id-port (port)
  (if (~id port)
      (sel1 (find-facts (& (& (~seg~segcase port))
                           (cond ((input-fact? port) 'output)
                                 (t 'input))
                           nil (~id port) nil)))))

;this kills unimportant commentary on a port and then kills the port.
;you must take care of any important stuff pointing to it
;(eg dflows) yourself first..

(defun kill-port-and-commentary (port)
  (--& port @io)
```

```
  (--& port @createvar)
  (make-fact-be-out port))

;TPORTS - these are also handled as facts that embody the ports themselves
```

```
;SEGTYPE

(defun ?segtype (seg) (~01 ~type (??& (& seg) segtype)))

(defun >segtype (seg type)
  (--& (& seg) segtype)
  (>& (& seg) segtype type)
  type)

(defun -segtype (seg) (~01 ~type (--& (& seg) segtype)))


;SUBSEG

(defun ??subseg (seg) (~nset ~seg ~subseg (??& (& seg) subseg)))

(defun all-subsegs (seg)
  [seg ! (over (??subseg seg) nconc (all-subsegs _))])

(defun +subseg (seg subseg)
  (>& (& seg) subseg (& subseg))
  subseg)

(defun -subseg (seg subseg)
  (-& (& seg) subseg (& subseg))
  subseg)

(defun --subseg (seg) (~nset ~seg ~subseg (--& (& seg) subseg)))

(defun ?supseg (seg) (~01 ~seg ~segcase (??& * subseg (& seg))))

(defun -supseg (seg) (~01 ~seg ~segcase (--& * subseg (& seg))))

(defun >supseg (seg subseg)
  (--& * subseg (&* subseg))
  (>& (& seg) subseg (& subseg)))
```

```
;DFLOW

(defun ?dflow-source (port)
  (~01 ~from (??& * dflow * port)))

(defun ??dflow-use (port)
  (~nset ~to (??& * dflow port)))

;note this takes note of the fact that the superseg field
;is redundant, and just computes it.

(defun +dflow (from to)
  (let* ((segf (~seg~segcase from))
         (segt (~seg~segcase to))
         (supf (?supseg segf))
         (supt (?supseg segt))
         (sup (cond ((eq segf segt)
                     (cond ((input-port-fact? from) segf)
                           (t supf)))
                    ((eq supt supf) supt)
                    ((eq segf supt) supt)
                    ((eq segt supf) supf)
                    (t
                     (barf "+DFLOW - no compatible supseg.~
                          ~% segf: ~S~
                          ~% segt: ~S"
                           segf segt)))))
    (>& (& sup) dflow from to)))


;CFLOW

;the next two things are commented out until (and I don't think I can)
;I think of some way to make them unambiguous.  IT is a big mistake
;that the input and output side cases of a seg are the same!!!

;(defun ?cflow-source (segcase)
;  (~01 ~from (??& * cflow * segcase)))

;(defun ?cflow-use (segcase)
;  (~01 ~to (??& * cflow segcase)))

;note this takes note of the fact that the superseg field
;is redundant, and just computes it.
;NOTE THAT THIS CANNOT BE USED TO CREATE A CFLOW INSIDE A SEG
;FROM ITS INPUT SIDE TO ITS OUTPUT SIDE!!  there is no way for it
;to tell this!

(defun +cflow (from to)
  (let* ((segf (~seg from))
         (segt (~seg to))
         (supf (?supseg segf))
         (supt (?supseg segt))
         (sup (cond ((eq supt supf) supf)
                    ((eq segf supt) supt)
                    ((eq segt supf) supf)
                    (t
                     (barf "+CFLOW - no compatible supseg~
                          ~% From: ~S~
                          ~% To:   ~S"
                           from to)))))
    (>& (& sup) cflow from to)))
```

```
;PLANTYPE

(defun ?plantype (seg) (~01 ~type (??& (& seg) plantype)))

(defun >plantype (seg type)
  (--& (& seg) plantype)
  (>& (& seg) plantype type)
  type)

(defun -plantype (seg) (~01 ~type (--& (& seg) plantype)))


;ROLE

;these first two are quite idiosyncratic, the return an alist
;describing all of the roles.

(defun ??roles (seg)
  (over (??& (& seg) role) -> role
    classifyq [(~rolename role) ! (~seg~roleseg role)]))

(defun --roles (seg)
  (over (--& (& seg) role) -> role
    classifyq [(~rolename role) ! (~seg~roleseg role)]))

(defun ??role (seg name) (~nset ~seg~roleseg (??& (& seg) role * name)))

(defun ?role (seg name) (~01 ~seg~roleseg (??& (& seg) role * name)))

(defun ?1role (seg name) (~seg~roleseg (?& (& seg) role * name)))

(defun --role (seg name) (~nset ~seg~roleseg (--& (& seg) role * name)))

(defun -role (seg name) (~01 ~seg~roleseg (--& (& seg) role * name)))

(defun +role (seg roleseg name) (>& (& seg) role (& roleseg) name))

;RECURSIVE

(defun ?recurrence (supseg)
  (~01 ~seg ~subseg (??& (&* supseg) recursive)))

(defun ?recursive-supseg (subseg)
  (~01 ~seg ~segcase (??& * recursive (&* subseg))))

(defun +recursive (supseg subseg)
  (>& (& supseg) recursive (& subseg)))
```

```
;@SEG

(defun code-for-defun (name)
  (let ((segcase (~01 ~segcase (or (??& * @seg 'lambda-exp name)
                                   (??& * cliche-def name)))))
    (cond (segcase (car (~01 ~code (??& segcase @seg)))))))

(defun ?lambda-exp (name)
  (~01 ~seg ~segcase (??& * @seg 'lambda-exp name)))

(defun ?@seg-kind (seg) (~01 ~kind (??& (&* seg) @seg)))

(defun ?@seg-name (seg) (~01 ~name (??& (&* seg) @seg)))

(defun ?@seg-code (seg) (~01 ~code (??& (&* seg) @seg)))

;the copying of the code list is VITAL to avoid bugs.

(defun >@seg-code (seg code)
  (let ((@seg (-& (&* seg) @seg)))
    (>& (& seg) @seg (~kind @seg) (~name @seg) (append code nil))))

;@IOs

(defun ?@io-kind (port) (~01 ~kind (??& port @io)))

(defun ?@io-name (port) (~01 ~name (??& port @io)))

;;; (?@IO-CODE port)
;;;
;;;  Do the best we can to figure out what code goes with a port
;;;  since that information is not explicitly represented in the DB.
;;;  Things only work really well for return values and arguments.
;;;
;;;  Returns a list of code pieces (just like ?@SEG-CODE), or NIL if
;;;  no good guess can be made.

(defun ?@io-code (port)
  (cond ((tio-fact? port)
         (cond ((~lower-io port) (?@io-code (~lower-io port)))))
        (T (let ((@seg (~01 (??& (& (~seg~segcase port)) @seg)))
                 (@io (~01 (??& port @io))))
             (cond ((not (and @io
                              @seg
                              (~code @seg)
                              (listp (car (~code @seg)))))
                    nil)
                   ((and (eq (~kind @io) 'actual-arg)
                         (eq (~kind @seg) 'funcall))
                    '((*CODE-IS-CAR* . ,(nthcdr (~argno @io)
                                                (car (~code @seg))))))
                   ((and (eq (~kind @io) 'lambda-arg)
                         (eq (~kind @seg) 'lambda-exp)
                         (eq (caar (~code @seg)) 'define))
                    '((*CODE-IS-CAR* . ,(nthcdr (1- (~argno @io))
                                                (caddar (~code @seg))))))
                   ((eq (~kind @io) 'returnval)
                    (~code @seg))))))))
```

```
;this checks to see if a slot is empty.

(defun ?empty-slot (seg)
  (let ((slotF (~01 (??& * slot (&* seg))))
        (unfilled (??& (&* seg) @seg 'funcall nil)))
    (cond ((null slotF) nil)
          (T (caseq (~slottype slotF)
               (input (if unfilled 'feed))
               (output
                 (if (no-dflow-from
                       (~deep (?& (&* (&* seg) output) @io 'returnval)))
                     'use))
               (segment
                 (if unfilled 'expand)))))))

(defun no-dflow-from (port)
  (and (null (??& * dflow port))
       (Rand-fast
         (no-dflow-from
           (Elist (??& * toutput * * port))))))
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.

;;;; Sanitation Engineering (GC)

(defvar *gc-target-area*  nil "The area which is the target for GC moves")
(defvar *gc-next-address* nil "current-insertion point in area (fixnum)")
(defvar *gc-last-address* nil "last free cell in area (fixnum)")


;;; (DB-REAP)
;;;
;;;  This function goes through the database deleting any fact that
;;;  is not IN in the current context
;;;
;;;         Backing up to anything is impossible
;;;         once this runs, since facts about pushed contexts are
;;;         OUT at the time this runs, and hence destroyed.
;;;
;;;         If you are not sure what this function does, you probably
;;;         don't want to be calling it. Its effects are reasonably
;;;         devastating.


(def-db-var *db-context-after-reap*
            (db-list (+ 1 *initial-db-context-number*)
                     *initial-db-context-number*)
  "Context to be instantiated after a reap has been done.")

(defvar *db-context-count-after-reap* (+ 1 *initial-db-context-number*))

(defun db-reap ()
  (let ((context *db-context*))
    (setq *db-context-count* *db-context-count-after-reap*)
    (setq *db-context*       *db-context-after-reap*)
    (dolist (name *db-context-names*)
      (let ((con (symeval name)))
        (cond ((equal con context) (set con (cdr *db-context*)))
              (t (set name nil)))))
    (dolist (cmarker *cmarkers*)
      (setf (in-layers  cmarker)
            (cond ((cmarker-is-in? cmarker) (cdr *db-context*))
                  (t nil))) ;facts with this marker will be expunged below
      (setf (out-layers cmarker) nil))
    (db-expunge)))
```

```
;;; (DB-EXPUNGE)
;;;
;;;  This function goes through the database deleting any fact which
;;;  could not be IN in any context (and which could therefore not ever
;;;  be IN again unless explicitly re-asserted by the user).
;;;  It does this by removing every fact which has a null IN-LAYERS field.
;;;  Also, every seg hunk that doesn't point to any facts at
;;;  all is cast adrift so that it will be GCed.
;;;  It returns the number of facts removed.
;;;
;;;  Notes: The function has to take extra care because it visits many
;;;         facts more than once and it is vital that it set a fact's
;;;         cmarker field to *null-cmarker* before it casts it adrift.
;;;
;;;         This operation may take a while to run, but is reasonably
;;;         harmless otherwise and thus safe to run just about any time
;;;         that the database is idle.

(defun db-expunge ()
  (let ((segs (cons nil *db-segs*)) (header (ncons nil)) (count 0))
    (repeat ((hsegs segs) hunk)
      (cond ((null (cdr hsegs)) (setq *db-segs* (cdr segs)) (return)))
      (setq hunk (db-index (cadr hsegs)))
      (cond ((over num <- 15. by (1- num) until (minusp num)
              and (repeat ((list (rplacd header (dbx num hunk))))
                    (cond ((null (cdr list))
                           (setf (dbx num hunk) (cdr header))
                           (return (null (cdr header)))))
                    (cond ((cond ((eq (cmarker (cadr list))
                                      *null-cmarker*))
                                 ((null (in-layers (cmarker (cadr list))))
                                  (setf (cmarker (cadr list))
                                        *null-cmarker*)
                                  (setq count (1+ count))))
                           (setf (cdr list) (cddr list)))
                          (t (pop list)))))
             (db-init-sym (cadr hsegs))       ;flush KBE properties...
             (setf (cdr hsegs) (cddr hsegs))) ;splice
            (t (pop hsegs))))
    (setq *cmarkers*
          (db-copylist (collect (cmarker *cmarkers*) (in-layers cmarker))))
    count))
```

```
;;; (GET-DB-AREA)
;;;
;;;  This returns three values:
;;;    (1) a new area for use by the database. If the flag
;;;        *RECYCLE-OLD-DB-AREAS* is non-nil, it will try to recycle an
;;;        old area if one is free. Otherwise, it will always cons fresh
;;;        areas.
;;;    (2) A fixnum address of the next free cell in the area.
;;;    (3) A fixnum address of the last free cell in the area.
;;;
;;;  Note: Many of the other routines work, for efficiency, on the
;;;        assumption that these DB areas are always only one region
;;;        long. As such, the region size we choose for the DB area
;;;        has to be large so that we don't overflow that region size.

(define-internal-option *recycle-old-db-areas*
                        "GC may recycle used DB areas"
  t :boolean)

(defun get-db-area ()
  (let* ((area
          (or (if *recycle-old-db-areas* (pop *db-free-areas*))
              (car
                (push
                  (make-area
                    ':name (intern-KBE
                             (format nil "*DB-AREA-~D*"
                                     (incf *db-area-count*)))
                    ':region-size #o400000
                    ':representation ':list
                    ':gc #+symbolics ':temporary
                         #-symbolics ':static)
                  *db-areas*))))
         (next (%pointer (make-list #o2000 ':area area)))
         (last (+ next #.(- #o2000 2))))
    (values area next last)))

;;; (FREE-DB-AREA area)
;;;
;;; This gets rid of an old area by setting it up for re-use.
;;;
;;; Note: It is not defined what will happen if one calls this function
;;;       on an area which is still pointed into by live structure.

(defun free-db-area (area)
  (cond ((memq area *db-areas*)
         (sys:reset-temporary-area area)
         (push area *db-free-areas*))
        ((eq area default-cons-area) ;we cheat and use this area initially
         ;(format t "~&;FREE-DB-AREA: Ignoring arg of DEFAULT-CONS-AREA.")
         nil) ;This seems debugged, so the warning is disabled for now...
        (t
         (barf "Area ~D., ~S, is not a DB area." area (area-name area)))))

;;; (FORWARD-DB-VALUE sym)
;;;
;;;  Using this 'function' will cause the value cell of a given
;;;  symbol, sym, to be forwarded to the new db area.

(defmacro forward-db-value (sym)
  (once-only (sym)
    '(set ,sym (move-to-new-db-area (symeval ,sym)))))

;;; (FORWARD-DB-PROPERTIES sym)
;;;
;;;  Using this form will cause all the interesting db-properties of
;;;  a given symbol, sym, to be forwarded to the new db area.
;;;
```

```
;;;  Note: Right now, we have to check and make sure that we don't copy
;;;        properties twice.  Actually, we are making here the assumption
;;;        that one property is copied iff all properties have been.

(defmacro forward-db-properties (sym)
  (let ((p (gensym)) (f (gensym)))
    '(do ((,p (plist ,sym) (cddr ,p)) (,f nil))
         ((null ,p))
       (when (db-property? (car ,p))
         (unless ,f
           (setq ,f t)
           (cond ((= (%area-number (cadr ,p)) *gc-target-area*)
                  (return nil))))
         (cond ((eq (car ,p) 'DB-INDEX) ;treat cdr-coding specially...
                (let ((new (copylist (cadr ,p) *gc-target-area*)))
                  (do ((l new (cdr l)))
                      ((null l))
                    (setf (car l) (move-to-new-db-area (car l))))
                  (setf (cadr ,p) new)))
               (t
                (setf (cadr ,p)
                      (move-to-new-db-area (cadr ,p)))))))))
```

```
;;; (MOVE-TO-NEW-DB-AREA cell)
;;;
;;;  Takes an object, obj, and moves it to the indicated area.
;;;
;;;  It assumes that any cell which is in the area cannot point to a
;;;  list structure cell outside the area and preserves this property.
;;;
;;;  It can deal with and preserve any kind of sharing and circularity
;;;  in the object.  It makes maximal use of cdr coding.
;;;
;;;  Other Assumptions:
;;;
;;;  (1) Only list cells are being copied, so if x isn't a list we need
;;;      not copy it. (and non-lists are never forwarded to lists)
;;;  (2) This is probably the only program in the world that puts DTP-FREE
;;;      markers for forwarding.  (Note we get the side benefit of rapidly
;;;      finding buggy pointers into the DB stuff).
;;;  (3) No cell in the thing to copy was in the target region to start
;;;      with. Therefore it can only be in it now if it is forwarded there.
;;;      Also there can never be two levels of forwarding.  By assumption
;;;      2 above, a list cell is already copied to the target if and only
;;;      if it is DTP-FREE.
;;;  (4) We assume that what MAKE-LIST returns is cdr coded and hence
;;;      contiguous in memory.
;;;  (5) It assumes that the pointer you give it does not point inside the
;;;      target region.  If it does, it will just copy it anyway.
;;;
;;;  NOTE: Once something is copied it is destroyed, and you can never
;;;        look at any part of it again.  You must from then on refer only
;;;        to the copy.
;;;
;;; Inputs:
;;;
;;;  CELL (formal parameter) -- The current object to be copied.
;;;
;;;  *GC-NEXT-ADDRESS* (special var) -- A fixnum which is the
;;;      current free spot.  We make sure that there are always
;;;      three cells available starting at this point. One each for
;;;      a car, a cdr, and a pointer to the next block of cells.
;;;
;;;  *GC-LAST-ADDRESS* (special var) -- The spot where the current block
;;;      runs out.
;;;
;;;  *GC-TARGET-AREA* (special var) -- The area we are moving to.
;;;
;;; Notes on variables used internally:
;;;
;;;  DO-CARS -- A fixnum which points to the first cell whose car hasn't
;;;       been copied yet.
;;;
;;;  COPY -- a list which holds the final result.

(defmacro compiler-add (&rest args)
  (eval '(+ ,@args)))

(defun move-to-new-db-area (cell) #+3600 cell #-3600
  (prog (copy do-cars)
        (cond ((not (listp cell)) (return cell))
              ((= (%p-data-type cell) dtp-free)
               (return (%make-pointer dtp-list (%p-pointer cell)))))
        (setq copy (%make-pointer dtp-list *gc-next-address*)
              do-cars *gc-next-address*)
   CDRS (%p-store-contents *gc-next-address* (car cell))
        (let ((cdr (cdr cell)))
          (%p-store-tag-and-pointer cell dtp-free *gc-next-address*)
          (setq cell cdr))
        (cond ((null cell) (%p-store-cdr-code *gc-next-address* cdr-nil))
              ((not (listp cell))
```

```
            (%p-store-cdr-code *gc-next-address* cdr-normal)
            (incf *gc-next-address*)
            (%p-store-contents *gc-next-address* cell)
            (%p-store-cdr-code *gc-next-address* cdr-error)
            (setq cell nil))
           ((= (%p-data-type cell) dtp-free)
            (%p-store-cdr-code *gc-next-address* cdr-normal)
            (incf *gc-next-address*)
            (%p-store-tag-and-pointer *gc-next-address*
                                      (compiler-add (lsh cdr-error 6)
                                                    dtp-list)
                                      (%p-pointer cell))
            (setq cell nil)))
      (incf *gc-next-address*)
      (cond ((not (< *gc-next-address* *gc-last-address*))
             (let ((b (%pointer (make-list #o400
                                           ':area *gc-target-area*))))
               (cond (cell
                      (%p-store-cdr-code (1- *gc-next-address*)
                                         cdr-normal)
                      (%p-store-tag-and-pointer *gc-next-address*
                                                (compiler-add
                                                  (lsh cdr-error 6)
                                                  dtp-list)
                                                b)
                      (incf *gc-next-address*)))
               (%p-store-tag-and-pointer *gc-next-address* dtp-free b)
               (setq *gc-next-address* b
                     *gc-last-address* (+ *gc-next-address*
                                          #.(- #o400 2))))))
      (cond (cell (go CDRS)))
CARS  (cond ((not (< do-cars *gc-next-address*)) (return copy))
            ((= (%p-data-type do-cars) dtp-free)
             (setq do-cars (%p-pointer do-cars)) (go CARS))
            ((or (= (%p-cdr-code do-cars) cdr-error)
                 (not (= (%p-data-type do-cars) dtp-list)))
             (incf do-cars) (go CARS)))
      (let ((ptr (%p-pointer do-cars)))
        (cond ((= (%p-data-type ptr) dtp-free)
               (%p-store-pointer do-cars (%p-pointer ptr))
               (incf do-cars) (go CARS)))
        (setq cell (%make-pointer dtp-list ptr))
        (%p-store-pointer do-cars *gc-next-address*)
        (incf do-cars)
        (go CDRS))))
```

```
;;; (DB-GC)
;;;
;;;  This function is called to do a garbage-collection of the database,
;;;  leaving the data compacted. The return value of DB-GC,
;;;  which is saved in the variable *DB-SIZE*, is the number of words
;;;  of storage in use in the newly compacted db area.
;;;
;;;  As noted in the overview above, this function assumes it knows
;;;  the names of all variables, plist slots, etc. which may point to
;;;  things consed in *DB-AREA*. If you make a new variable or other
;;;  slot pointing into that area, you should update this function.
;;;
;;;  See also the function DB-REAP. If you call that function, you
;;;  want to do so before calling this one.

(define-internal-option *gc-enabled*
                        "Garbage collector enabled"
  t :boolean)

#+3600 ;GC isn't going to work on the 3600
(setq *gc-enabled* nil)

(defun db-gc ()
  (if *gc-enabled*
      (multiple-value-bind (*gc-target-area*
                            *gc-next-address*
                            *gc-last-address*)
          (get-db-area)
        (let ((start-p *gc-next-address*))
          (dolist (sym *db-vars*) (forward-db-value sym))
          (dolist (sym *db-context-names*) (forward-db-value sym))
          (dolist (sym *db-segs*) (forward-db-properties sym))
          (free-db-area (prog1 *db-area*
                               (setq *db-area* *gc-target-area*)))
          (setq *db-size* (- *gc-next-address* start-p))))))
```

```
;;;; GC debugging tools

;;;  If obj is of pointer-type DTP-FREE, prints it as
;;;
;;;      #<DTP-Free address> = ...obj...
;;;
;;;  Where object will be a DTP-LIST pointer to the same object.
;;;
;;;  The intended use of this function is for debugging the KBE
;;;  special-purpose garbage collector, which uses DTP-FREE pointers
;;;  as forwarding pointers from one list-structure region to another.
;;;

(comment

(pp:define-generalized-print-function dtp-free (obj s)
    (and (listp obj) (= (%p-data-type obj) dtp-free))
  (format s "#<~@!DTP-Free ~S~.> = ~="
          (%pointer obj)
          (%make-pointer dtp-list (%p-pointer obj))))  )
```

```
;;; -*- Mode:LISP; Package:KBE; Base:10; Fonts:MEDFNB; -*-

;;; (c) Copyright 1983, Massachusetts Institute of Technology
;;;     All Rights Reserved.
;;;     Permission to copy all or part of this material is granted,
;;;     provided that the copies are not made or distributed for resale,
;;;     the MIT copyright notice and reference to the source file and
;;;     the software distribution version appear, and that notice is
;;;     given that copying is by permission of Massachusetts Institute
;;;     of Technology.
```

```
;;;; For Debugging Only

;;; (SHOW-DB-CONTEXT)
;;;
;;;  Calls to SHOW-DB-CONTEXT type out info about labeled points in the
;;;  database context stack. These labels are assigned by the special
;;;  forms DB-CONTEXT-IS and DB-CONTEXT-WAS.
;;;
;;;  Note that all names are in fact just normal special variables, and
;;;  are bound to the context they indicate.

(defun context-names (context)
  (do ((l *db-context-names* (cdr l)) (result))
      ((null l) result)
    (cond ((equal (symeval (car l)) context) (push (car l) result)))))

(defun show-db-context ()
  (format t "~&;Context stack:")
  (map #'(lambda (context)
           (format t "~%;~2D " (car context))
           (mapc #'(lambda (n) (format t "~A " n))
                 (context-names context)))
       *db-context*)
  '(:Gunterpri))

(setf #'sc #'show-db-context) ;short name
```

```
;;; (DB-STATS)
;;;  This computes some rather copious statistics about the data base
;;;  and returns them to you in an alist.  Hopefully each individual
;;;  statistic is self explanatory.

(def-alistq facts-by-k)
(def-alistq facts-by-cm)
(def-alist  facts-by-idx)

(defun db-stats ()
  (let* ((facts-by-k   (mapcar #'(lambda (x) (cons x 0)) *db-keywords*))
         (facts-by-cm  (cons (cons *null-cmarker* 0)
                             (mapcar #'(lambda (x) (cons x 0))
                                     *cmarkers*)))
         (facts-by-idx (over n <- 15. by (1- n) until (minusp n)
                             revlist (cons n 0))))
    (dolist (s *db-segs*)
      (let ((hunk (db-index s)))
        (over num <- 15. by (1- num) until (minusp num)
              (dbx num hunk) -> b
          (setf (facts-by-idx num) (+ (facts-by-idx num) (length b)))
          (if (< num 11.)
              (dolist (f b)
                (setf (facts-by-k (~keyword f))
                      (1+ (or (facts-by-k (~keyword f)) 0)))
                (setf (facts-by-cm (cmarker f))
                      (1+ (facts-by-cm (cmarker f)))))))))
    (terpri standard-output)
    (format T "~\vertical\"
     (let ((segnum (length *db-segs*))
           (fnum   (over facts-by-k sum (cdr _))))
       '((*db-context* ,*db-context*)
         (number-of-segments ,segnum)
         (number-of-facts ,fnum)
         ,@(if (not (zerop segnum))
               '((average-facts-per-segment
                  ,(//$ (float (// (* fnum 10.) segnum)) 10.0))))
         (facts-per-cmarker
           ,(mapcar #'(lambda (x) '(,(in-layers (car x))
                                    ,(out-layers (car x))
                                    ,(cdr x)))
                    facts-by-cm))
         (facts-per-keyword
           ,(mapcar #'(lambda (x) '(,(car x) ,(cdr x)))
                    facts-by-k))
         ,@(if (not (zerop segnum))
               '((average-facts-per-index
                  ,(mapcar #'(lambda (x)
                               '(,(car x)
                                 ,(//$ (float (// (* (cdr x) 10.)
                                                  segnum))
                                       10.0)))
                           facts-by-idx)))))))))

(pp:define-list-print-function facts-per-cmarker (x s)
  (format s "(~@!~{~=~%~\tabular\~}~.)" x))

(pp:define-list-print-function facts-per-keyword (x s)
  (format s "(~@!~{~=~%~\tabular\~}~.)" x))

(pp:define-list-print-function average-facts-per-index (x s)
  (format s "(~@!~{~=~%~\tabular\~}~.)" x))
```